📚 AWS SAP-C02 — Solutions Architect Professional Study Book

> **Author:** Claude AI | **Target Exam:** AWS Certified Solutions Architect – Professional (SAP-C02)
> **Level:** Professional (assumes Associate-level knowledge)
> **Language:** English
> **Started:** 2026-10-28

> 💡 **How to use this book:**
> Each chapter covers a service or topic in depth. Every chapter includes:
> service overview · comparison with alternatives · reliability & security · Well-Architected Framework · 4 exam domains · limitations
> Questions are accumulated in the **Test Your Knowledge** chapter at the end.

📋 Table of Contents

- 📚 AWS SAP-C02 — Solutions Architect Professional Study Book
  - 📋 Table of Contents
  - 🌐 Chapter 1: AWS Direct Connect (DX)
  - 🏗️ Chapter 2: Private and Public Subnets
  - 🚪 Chapter 3: Internet Gateway and Alternatives
  - 🔀 Chapter 4: AWS Transit Gateway (TGW)
  - 📊 Chapter 5: VPC Flow Logs
  - 🔐 Chapter 6: Remote Network to VPC Connectivity
  - 🌍 Chapter 7: AWS Resource Scope — Global, Regional, Zonal and Edge
  - 🗄️ Chapter 8: Amazon Aurora
  - 🐘 Chapter 9: Amazon RDS
  - ⚡ Chapter 10: Amazon DynamoDB
  - 📊 Chapter 11: Amazon Redshift
  - 🔗 Chapter 12: Amazon Neptune
  - 🚀 Chapter 13: Amazon ElastiCache
  - 📄 Chapter 14: Amazon DocumentDB
  - 📒 Chapter 15: Amazon QLDB
  - 🔀 Chapter 16: VGW vs Direct Connect Gateway vs Transit Gateway — On-Premises Connectivity
  - 🔒 Chapter 17: Traffic Segmentation in AWS
  - 💾 Chapter 18: AWS Backup
  - 🔄 Chapter 19: AWS DataSync
  - 🌍 Chapter 20: Cross-Region Replication and Multi-Region Architecture
  - 🔌 Chapter 21: VPC Endpoints
  - 🪣 Chapter 22: Amazon S3
  - 🔐 Chapter 23: AWS IAM — Identity and Access Management
  - 📋 Chapter 24: AWS CloudTrail
  - 🏛️ Chapter 25: AWS Organizations & Service Control Policies (SCPs)
  - 🔑 Chapter 26: AWS IAM Identity Center (formerly AWS SSO)
  - 🔥 Chapter 27: Network Security — Defense in Depth
  - 📊 Chapter 28: Additional Services — Athena, Federation, KMS, Aurora Serverless, EKS, Timestream, ALB, EC2 Purchasing, Placement Groups, Lambda, App Runner, Step Functions, API Gateway
  - 🔒 Chapter 29: Security Add-Ons
  - 📨 Chapter 30: Amazon SQS — Simple Queue Service

- 📣 Chapter 31: Amazon SNS — Simple Notification Service
- ⚡ Chapter 32: Amazon EventBridge
- 💰 Chapter 33: AWS Cost Management & Budgets
- 💉 Test Your Knowledge

🌐 Chapter 1: AWS Direct Connect (DX)

📖 Overview

AWS Direct Connect (DX) is a **dedicated, private network connection** between your on-premises data center (or colocation facility) and AWS. Unlike VPN over the internet, DX bypasses the public internet entirely — traffic travels over a private fiber link to an AWS Direct Connect location (DX PoP), then into the AWS backbone network.

> 🔑 **Exam key:** DX = consistent latency, high bandwidth, private connection. VPN = encrypted, over internet, variable latency. They are complementary, not mutually exclusive.

Scope: DX is a **regional** service. DX Gateways extend it globally.

🕯️ Physical and Logical Setup

```
On-Premises DC → (your fiber/MPLS) → DX Location (PoP) → AWS Region
```

1. **Cross-connect** — physical fiber between your router and AWS DX router at the colocation facility
2. **Virtual Interface (VIF)** — logical 802.1Q VLAN on top of the physical connection
3. **BGP session** — routing protocol between your router and AWS

You do NOT need to be physically present at a DX location — you can use a **Direct Connect Partner** (Equinix, AT&T, Verizon, etc.) who has existing cross-connects with AWS.

🔗 Connection Types

Type	Bandwidth Options	Who Provisions	Port Ownership
Dedicated Connection	1 Gbps, 10 Gbps, 100 Gbps	AWS directly	You own the port
Hosted Connection	50 Mbps – 10 Gbps (via partner)	DX Partner	Partner owns port
Hosted VIF	Shared port, single VIF	DX Partner	Partner owns port, shares capacity

> 🔑 **Exam key:** Dedicated = SLA from AWS. Hosted = SLA from partner. Sub-1Gbps = must use hosted. 100Gbps = only dedicated.

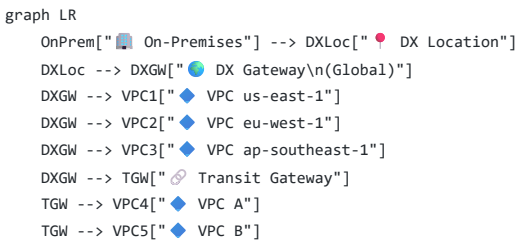
🔀 Virtual Interfaces (VIFs)

VIF Type	Connects To	Use Case
Private VIF	VPC via Virtual Private Gateway (VGW) or DX Gateway	Access private resources in VPC
Public VIF	AWS public services (S3, DynamoDB, EC2 public IPs)	Access AWS public endpoints without internet
Transit VIF	Transit Gateway via DX Gateway	Access multiple VPCs, Transit Gateway networks

> ⚠️ **Limitation:** One Private VIF per VGW per DX connection. Use DX Gateway for multi-VPC.

🌍 Direct Connect Gateway (DXGW)

DX Gateway is a **globally scoped** resource that lets you use a **single DX connection to reach VPCs in multiple AWS Regions**.



> 🔑 **Exam key:** DX Gateway does NOT allow VPC-to-VPC routing. It only provides on-prem → VPC connectivity. For VPC-to-VPC, add Transit Gateway.

Limits:

- Up to 10 VGWs (or 3 TGWs) per DX Gateway
- Up to 20 DX connections per DX Gateway

⚡ Link Aggregation Groups (LAG)

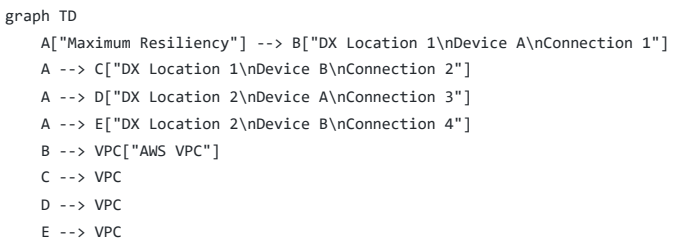
LAG bundles **multiple DX connections** into one logical connection for:

- Increased bandwidth (e.g., 4 × 10 Gbps = 40 Gbps logical)
- Active/active load sharing (LACP protocol)

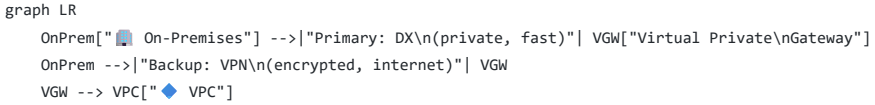
> ⚠️ All connections in a LAG must be: same bandwidth, same DX location, same AWS device. Max 4 connections per LAG (Dedicated), 2 for Hosted.

🛡️ Resiliency Models

Model	Setup	Use Case
No Resiliency	Single connection, single location	Dev/test only
Development	Two connections, same location	Better than nothing
High Resiliency	Two connections, two locations	Production workloads
Maximum Resiliency	Two connections at two locations + two separate devices	Mission-critical (financial, healthcare)



🔄 DX + Site-to-Site VPN as Backup



BGP failover mechanism:

- Use **MED (Multi-Exit Discriminator)** to prefer DX over VPN

- Use **AS Path Prepending** to make VPN less preferred from AWS side
- DX fails → BGP withdraws routes → VPN takes over automatically

> 🔑 **Exam key:** DX does NOT encrypt traffic by default. For encryption over DX, run **IPSec VPN over the DX connection** (using a Public VIF to the VGW). This gives you private connectivity + encryption.

📡 BGP Routing Details

Concept	Description
BGP ASN	You need a private or public ASN for your router
MED	Lower MED = more preferred path. Set on DX to prefer it over VPN
AS Path Prepending	Append your ASN multiple times to make a path less preferred
BGP Communities	AWS uses communities to control route propagation scope
Route Summarization	Advertise aggregated prefixes to reduce BGP table size

AWS advertises up to **1,000 routes** to your router per BGP session. Exceeding this drops the BGP session.

💰 Pricing Model

Component	Cost
Port hour	Per hour for dedicated connection (1G/10G/100G)
Data transfer OUT	Per GB (lower than internet rates)
Data transfer IN	Free
VIF creation	Free
DX Gateway	Free (pay for data transfer)

> 💡 DX data transfer OUT is ~50-70% cheaper than internet. Break-even at high sustained transfer volumes.

🆚 Direct Connect vs Alternatives

Feature	DX	Site-to-Site VPN	Client VPN
Connection type	Physical/dedicated	IPSec over internet	TLS over internet
Bandwidth	Up to 100 Gbps	Up to 1.25 Gbps	Per-user
Latency	Consistent, low	Variable	Variable
Encryption	No (add VPN on top)	Yes (IPSec)	Yes (TLS)
Setup time	Weeks–months	Minutes–hours	Minutes–hours
Use case	Hybrid cloud, migrations	Backup, small offices	Remote users
Cost	High fixed + transfer	Low fixed + transfer	Per-user hour
SLA	99.99% (max resiliency)	No AWS SLA	No AWS SLA

☁️ Azure Equivalent: ExpressRoute

Feature	AWS Direct Connect	Azure ExpressRoute
Service	Direct Connect	ExpressRoute
Private peering	Private VIF → VGW	Private Peering → VNet Gateway
Microsoft peering	Public VIF	Microsoft Peering
Global reach	DX Gateway	ExpressRoute Global Reach
Encryption	Optional (MACsec on Dedicated)	Optional (MACsec)
Max bandwidth	100 Gbps	100 Gbps (ExpressRoute Direct)
Redundancy	LAG + multiple locations	Redundant circuits required
Pricing model	Port + data transfer	Circuit + gateway + data

> **MACsec (Layer 2 encryption):** Available on DX Dedicated connections (10G/100G). Encrypts at L2, before traffic even enters AWS network. Azure also supports MACsec on ExpressRoute Direct.

🐧 Open Source / Linux Equivalent

Concept	AWS DX	Real World
Physical layer	Cross-connect fiber	Dark fiber, MPLS circuit
Routing protocol	BGP	BGP (Quagga, FRRouting, BIRD)
Logical multiplexing	802.1Q VLANs (VIFs)	802.1Q trunk ports
Redundancy	LAG (LACP)	Linux bonding (mode 4 = LACP)
Traffic engineering	MED, AS prepend	BGP communities, local preference

---

🏛️ Well-Architected Framework — Direct Connect

Pillar	Consideration
Operational Excellence	Use CloudWatch metrics for DX (BpsIngress, BpsEgress, ConnectionState). Set alarms on BGP flaps.
Security	DX traffic is NOT encrypted by default — use MACsec (Dedicated) or IPSec VPN over DX for encryption. Use Private VIF, never Public VIF for sensitive workloads unless needed.
Reliability	Use maximum resiliency model (2 locations × 2 devices). Add Site-to-Site VPN as tertiary backup. Test failover regularly.
Performance Efficiency	Use LAG for bandwidth aggregation. Choose DX location geographically close to your DC. Use 100G for high-throughput migrations.
Cost Optimization	Compare DX vs VPN break-even (usually >1 TB/month justifies DX). Use Hosted connections for lower initial commitment. Data IN is free.
Sustainability	DX reduces reliance on internet routing, fewer hops = less energy. Consolidate multiple connections via DX Gateway instead of individual circuits.

📋 SAP-C02 Domain Coverage — Direct Connect

Domain 1: Design Solutions for Organizational Complexity

- Multi-account access via DX Gateway + Transit Gateway
- DX shared across AWS Organizations using Resource Access Manager (RAM)
- Centralized network account pattern with DX terminating in shared VPC

Domain 2: Design for New Solutions

- Choosing DX vs VPN for hybrid architecture based on bandwidth/latency SLA
- MACsec for encryption compliance requirements
- BGP design for deterministic routing (MED, communities)

Domain 3: Continuous Improvement for Existing Solutions

- Upgrading from VPN to DX for production workloads
- Adding DX as backup to existing DX (resiliency improvements)
- Moving from Hosted to Dedicated connection as bandwidth grows

Domain 4: Accelerate Workload Migration and Modernization

- DX 100G for large-scale data migration (DataSync over DX)
- Snow family + DX as complementary migration strategies
- Direct Connect for VMware Cloud on AWS connectivity

🔌 Connections — Deep Dive

Provisioning Flow

```
1. Request connection in AWS Console (choose Dedicated or Hosted)
   ↓
2. AWS issues LOA-CFA (Letter of Authorization – Connecting Facility Assignment)
   within 72 hours (Dedicated only)
   ↓
3. You (or your colocation provider) arrange physical cross-connect at DX facility
   using the LOA-CFA
   ↓
4. Connection state: ordering → pending → available
   ↓
5. Create Virtual Interface (VIF) on the connection
   ↓
6. Configure BGP on your router using IPs/ASN from VIF creation
   ↓
7. BGP session comes up → routes exchanged → traffic flows
```

Connection Types — Full Comparison

Feature	Dedicated Connection	Hosted Connection	Hosted VIF
Provisioned by	AWS directly	DX Partner	DX Partner
Port ownership	You own the port	Partner owns the port	Partner owns the port
Bandwidth options	1G, 10G, 100G only	50 Mbps – 10 Gbps (flexible)	Shared partner port capacity
Sub-1G possible	❌ No	✅ Yes	✅ Yes (best-effort)
100G possible	✅ Yes	❌ No	❌ No
LOA-CFA	✅ AWS issues to you	❌ Partner handles	❌ Not applicable
SLA	AWS SLA	Partner SLA (not AWS)	Partner SLA (not AWS)
MACsec support	✅ Yes (10G/100G only)	❌ No	❌ No
VIFs allowed	Up to 50 VIFs	Exactly 1 VIF	Exactly 1 (it IS the VIF)
Bandwidth guaranteed	✅ Yes (dedicated port)	✅ Yes (dedicated bandwidth tier)	❌ No (shared, best-effort)
Physical presence at colo	Required (or via agent)	❌ Not required	❌ Not required
Setup time	Weeks–months	Days–weeks	Days
Best for	High bandwidth, full control, compliance	Quick start, sub-1G, no colo presence	Low-cost testing, non-critical

> 🔑 **Exam key traps:**
>
> 1. **100G = Dedicated only** — no hosted 100G exists
> 2. **MACsec = Dedicated only** (10G/100G) — L2 wire-speed encryption
> 3. **Sub-1G = Hosted only** — e.g. 100Mbps, 500Mbps require a DX Partner
> 4. **Hosted = 1 VIF max** — cannot create multiple VIFs on a hosted connection
> 5. **SLA source differs** — AWS SLA on Dedicated; partner SLA on Hosted — critical for compliance requirements
> 6. **Hosted VIF ≠ Hosted Connection** — Hosted VIF has NO guaranteed bandwidth (shared); Hosted Connection has a dedicated bandwidth tier

Hosted VIF explained separately: A Hosted VIF is the most constrained option — the DX Partner gives you a single VIF on their already-shared port. You get ONE VIF, bandwidth is not guaranteed (best-effort shared), and you cannot add more VIFs. Use only for initial testing or non-critical workloads. For production: use at minimum a Hosted Connection.

Connection States

State	Meaning
`ordering`	Request submitted, AWS processing
`pending`	LOA-CFA issued, waiting for cross-connect
`available`	Physical link up, ready for VIF creation
`down`	Physical layer issue
`deleted`	Connection removed

MACsec (Layer 2 Encryption)

- Encrypts at **Layer 2** — before traffic enters AWS network
- Available only on **Dedicated 10G and 100G** connections
- Uses **CAK (Connectivity Association Key)** and **CKN (Connectivity Association Key Name)** pair
- Must be configured **before data flows** — enable on connection + configure on router
- Provides wire-speed encryption with no throughput penalty
- Not available on Hosted connections or 1G Dedicated

> 🔑 **Exam key:** MACsec = L2 encryption = Dedicated 10G/100G only. For encryption on Hosted or 1G, use IPSec VPN over Public VIF. For sub-1G bandwidth requirement = must use Hosted connection.

> ⚠️ **Exam trap:** 100G connections = Dedicated only. If question mentions 100G DX → only Dedicated applies.

🛡️ Resiliency Models — Deep Dive

AWS provides the **Resiliency Toolkit** in the DX console — a wizard that guides you through selecting and validating your resiliency model. It checks your configuration against best practices.

Four Resiliency Models

Model	Connections	Locations	Devices	Survives	SLA
No Resiliency	1	1	1	Nothing	None
Development	2	1	2	Single device failure	None
High Resiliency	2	2	2	Full location failure	99.9%
Maximum Resiliency	4	2	4 (2 per location)	Location + device failure simultaneously	99.99%

```
graph TD
    subgraph "No Resiliency"
        NR_OP["On-Prem"] --> NR_L1["DX Loc 1\nDevice A\nConn 1"]
        NR_L1 --> NR_VPC["VPC"]
    end

    subgraph "Development"
        DEV_OP["On-Prem"] --> DEV_L1A["DX Loc 1\nDevice A\nConn 1"]
        DEV_OP --> DEV_L1B["DX Loc 1\nDevice B\nConn 2"]
        DEV_L1A --> DEV_VPC["VPC"]
        DEV_L1B --> DEV_VPC
    end

    subgraph "High Resiliency"
        HR_OP["On-Prem"] --> HR_L1["DX Loc 1\nConn 1"]
        HR_OP --> HR_L2["DX Loc 2\nConn 2"]
        HR_L1 --> HR_VPC["VPC"]
        HR_L2 --> HR_VPC
    end

    subgraph "Maximum Resiliency"
        MR_OP["On-Prem"] --> MR_L1A["Loc 1\nDevice A\nConn 1"]
        MR_OP --> MR_L1B["Loc 1\nDevice B\nConn 2"]
        MR_OP --> MR_L2A["Loc 2\nDevice A\nConn 3"]
        MR_OP --> MR_L2B["Loc 2\nDevice B\nConn 4"]
        MR_L1A --> MR_VPC["VPC"]
        MR_L1B --> MR_VPC
        MR_L2A --> MR_VPC
        MR_L2B --> MR_VPC
    end
```

Adding VPN as Tertiary Backup

```
graph LR
    OP["On-Prem"] -->|"Primary: DX (MED=100)"| VGW["VGW/TGW"]
    OP -->|"Secondary: DX location 2 (MED=200)"| VGW
    OP -->|"Tertiary: VPN (MED=300 + AS prepend)"| VGW
    VGW --> VPC["VPC"]
```

BGP preference order (lower MED = more preferred):

1. DX Location 1 — MED 100
2. DX Location 2 — MED 200

3. Site-to-Site VPN — MED 300 + AS path prepended

> 🔑 **Exam key:** Maximum Resiliency = 99.99% SLA = 4 connections, 2 locations, 2 devices per location. High Resiliency = 99.9% = 2 connections at 2 locations. Development = 2 connections SAME location = no location-level HA.

🔀 Virtual Interfaces (VIFs) — Deep Dive

VIFs are **logical 802.1Q VLAN** interfaces on top of a physical DX connection. Each VIF has its own VLAN ID, BGP session, and routing configuration.

Private VIF

Connects your on-premises network to **private resources inside a VPC**.

Property	Value
Connects to	Virtual Private Gateway (VGW) or DX Gateway
IP type	Private IPs only
BGP peers	Private or link-local IPs
Max prefixes advertised to AWS	100
Max prefixes received from AWS	200
VLAN ID	1–4094 (unique per connection)
Access	VPC private resources only
Cannot access	S3 public endpoints, public AWS services

> ⚠️ **BGP prefix limit:** Exceeding 100 advertised prefixes → BGP session drops. Use route summarization on your router.

Public VIF

Connects to **all AWS public IP ranges** globally — not tied to a VPC.

Property	Value
Connects to	AWS public network (all regions)
IP type	Public IPs on both sides
BGP peers	Public IPs
Access	S3, DynamoDB, EC2 public IPs, CloudFront, all AWS public services
Scope	ALL AWS regions (not just the region where DX is)
Use case	S3 access without VPC, IPSec VPN over DX (for encryption)

> 🔑 **Exam key:** Public VIF ≠ internet. It's a private path to AWS public endpoints. Use it for S3/DynamoDB without a VPC, or to establish VPN over DX for encryption.

> ⚠️ **BGP table size:** AWS advertises thousands of public prefixes via Public VIF — ensure your router can handle a large BGP table.

Transit VIF

Connects to a **DX Gateway associated with Transit Gateway**.

Property	Value
Connects to	DX Gateway → Transit Gateway
Requires	DX Gateway (cannot connect directly to TGW)
Enables	Multi-region, multi-VPC access from single DX
Replaces	Multiple Private VIFs to multiple VGWs
VIF limit	1 Transit VIF per DX connection per DX Gateway

VIF Comparison Table

Feature	Private VIF	Public VIF	Transit VIF
Destination	VGW / DX GW	AWS public network	DX GW → TGW
IPs	Private	Public	Private
Multi-VPC	No (1 VGW)	N/A	Yes (via TGW)
Multi-region	Via DX GW	Yes (all regions)	Yes (via TGW)
AWS services	VPC resources only	All public services	VPC resources via TGW
Cross-account VIF	✅	✅	✅
Max per Dedicated	50	50	50
Max per Hosted	1	1	1
Encryption	No (add MACsec/VPN)	No	No

> 🔑 **Exam key:** Hosted connection = exactly 1 VIF. Dedicated = up to 50 VIFs. Cross-account VIF allocation: owner creates VIF in their account but assigns it to another AWS account ID.

⚡ Link Aggregation Groups (LAGs) — Deep Dive

LAG bundles multiple physical DX connections into one **logical interface** using **LACP (802.3ad)** protocol.

Requirements — All Must Match

Requirement	Detail
Bandwidth	All connections must be same speed (e.g., all 10G)
Location	All connections must be at same DX location
AWS device	All connections must terminate on same AWS device
Connection type	All Dedicated OR all Hosted (cannot mix)

> ⚠️ **Critical limit:** LAG = same DX location = SPOF for location failure. LAG provides bandwidth + device-HA only, NOT location HA.

LAG Limits

Parameter	Dedicated	Hosted
Max connections	4	2
Min active links	Configurable (1–max)	Configurable

Active/Active Behavior

- All links carry traffic **simultaneously** (not active/standby)
- LACP negotiates which ports participate
- Traffic hashed across links (by src/dst IP, ports — ECMP-style)

- One link fails → traffic redistributes to remaining links automatically

Min Links Setting

Set **minimum number of active links** for LAG to remain up. If active links drop below minimum → entire LAG goes down:

- Prevents split-brain where LAG operates at reduced capacity
- Example: 4-link LAG with min=2 → if 3 links fail, LAG goes down rather than limping on 1 link

VIFs on LAG

VIFs are created **on the LAG**, not on individual connections. When a link fails, VIFs remain up (still on LAG) — no BGP reconvergence needed.

```
graph LR
    OP["On-Prem Router"] -->|"10G fiber 1"| DEV["AWS DX Device\n(same location)"]
    OP -->|"10G fiber 2"| DEV
    OP -->|"10G fiber 3"| DEV
    OP -->|"10G fiber 4"| DEV
    DEV -->|"40G LAG (logical)"| VIF["VIF (on LAG)"]
    VIF --> VGW["VGW → VPC"]
```

LAG vs Multiple Independent Connections

Aspect	LAG	Independent Connections
Bandwidth	Combined (e.g., 4×10G = 40G logical)	Per-connection (use BGP ECMP)
Management	Single logical interface	Multiple VIFs to manage
Failover	Automatic (within LAG)	BGP reconvergence required
Location HA	❌ (same location)	✅ (can span locations)
Device HA	✅ (different devices allowed)	✅
BGP sessions	One per VIF (on LAG)	One per VIF per connection

> 🔑 **Exam key:** LAG = bandwidth aggregation + device HA. Does NOT give location HA. For location HA + bandwidth: use LAG at each location + multi-location setup.

🌍 Direct Connect Gateway (DXGW) — Deep Dive

DX Gateway is a **globally scoped** resource — it exists outside any region and can connect DX locations to VPCs in **any AWS region**.

The Problem DXGW Solves

Without DXGW — each VPC needs its own Private VIF:

```
DX connection → Private VIF → VGW → ONE VPC (one region only)
```

5 VPCs in 3 regions = 5 separate Private VIFs + 5 VGWs. Complex and expensive.

With DXGW — one Private VIF reaches everything:

```
DX connection → Private VIF → DXGW → VGW-1 (VPC-A, us-east-1)
                                   → VGW-2 (VPC-B, eu-west-1)
                                   → VGW-3 (VPC-C, ap-southeast-1)
                                   → TGW   (hundreds of VPCs via Transit VIF)
```

Key Properties

Property	Value
Scope	Global (no region)
Cost	Free (pay data transfer only)
Max DX connections	20
Max VGW associations	10
Max TGW associations	3
VIF type required	Private VIF or Transit VIF
Cross-account	✅ (share via RAM or manual proposal/accept)

Association Types — Mutually Exclusive

> ⚠️ **Critical limitation:** Cannot mix VGW and TGW associations on the same DXGW. Choose one model.

Mode	VIF Type	Use Case
DXGW + VGW	Private VIF	Single VPC per VGW, up to 10 VPCs
DXGW + TGW	Transit VIF	Multiple VPCs per TGW, multi-region

Routing Behavior

- DXGW routes **on-prem → VPC only**
- Does NOT route VPC → VPC (inter-VPC traffic)
- For VPC-to-VPC: add Transit Gateway behind DXGW
- **Allowed prefixes**: explicitly configure which VPC CIDRs DXGW advertises to on-prem — prevents accidental route leakage

```
graph LR
    OP["🏢 On-Prem"] --> DX["DX Connection"]
    DX --> DXGW["🌍 DX Gateway\n(Global)"]
    DXGW --> VGW1["VGW → VPC-A\nus-east-1"]
    DXGW --> VGW2["VGW → VPC-B\neu-west-1"]
    DXGW --> TGW["TGW\nap-southeast-1"]
    TGW --> VPC_C["VPC-C"]
    TGW --> VPC_D["VPC-D"]
```

> 🔑 **Exam key:** DXGW does NOT allow VPC-A to reach VPC-B via DXGW. Add TGW for VPC-to-VPC routing. DXGW is NOT regional — one DXGW serves all regions.

DXGW alone vs DXGW + TGW

Scenario	Use DXGW alone	Use DXGW + TGW
Access ≤10 VPCs, no VPC-to-VPC	✅ Simpler	Overkill
Access many VPCs	Limited (10 VGW max)	✅ Scale
VPC-to-VPC via DX path	❌ Not supported	✅ TGW handles it
Multi-region VPC access	✅	✅
SD-WAN/Connect attachment	❌	✅

🔒 Virtual Private Gateways (VGW) — Deep Dive

VGW is the **AWS-side termination point** for VPN tunnels and DX Private VIFs — attached to a single VPC.

Key Properties

Property	Value
Scope	Regional (per VPC)
Attachment	One VPC at a time
ASN	Custom private ASN, set at creation — **cannot change**
Default ASN	64512 (if not specified)
Cost	Free resource (pay for VPN/DX transfer)
VPN tunnels	Supports Site-to-Site VPN
DX	Supports Private VIF (direct or via DXGW)

Route Propagation

VGW can **auto-propagate routes** into VPC route tables:

- Must enable "Route Propagation" on each route table
- Routes learned via BGP (DX or VPN) appear automatically
- Without propagation: must add static routes manually

> ⚠️ **Exam trap:** Route propagation must be **explicitly enabled per route table**. A new route table does NOT inherit propagation from the main route table.

VGW vs DXGW vs TGW

Feature	VGW	DX Gateway	Transit Gateway
Scope	Single VPC	Global, multi-VPC	Regional, multi-VPC
VPCs supported	1	10 (via VGW) / many (via TGW)	5000 attachments
VPN support	✅	❌ (use TGW or VGW)	✅
DX support	✅ (Private VIF)	✅ (Private/Transit VIF)	✅ (Transit VIF via DXGW)
VPC-to-VPC routing	❌	❌	✅
Cross-region	❌	✅ (via DXGW)	✅ (via TGW peering)
Cost	Free	Free	Per attachment-hour + data

> 🔑 **Exam key:** VGW = single VPC gateway for VPN/DX. DXGW = global hub for DX to multi-VPC. TGW = regional hub for any-to-any routing. VGW ASN cannot be changed after creation.

🤝 AWS Direct Connect — Partner Interconnect (APN)

AWS does not own the "last mile" — a network of **AWS Partner Network (APN) members** provides physical access to DX locations.

How the Partner Model Works

```
Customer DC → (last mile circuit) → Partner PoP/Exchange → (partner cross-connect) → AWS DX Device
```

- Partners have **existing physical ports** at AWS DX locations
- They sub-divide their capacity into **Hosted Connections** or **Hosted VIFs** for customers
- Customers never need physical presence at a DX facility

Major DX Partners

Partner	Type	Notes
Equinix Fabric	Exchange	Global, multi-cloud exchange
Megaport	Exchange	SDN-provisioned, multi-cloud
PacketFabric	Exchange	Automated circuit provisioning
AT&T, Verizon, BT	Telco	Traditional MPLS/carrier circuits
Tata, Lumen, NTT	Telco	Global enterprise carriers

Benefits of Partners

- Sub-1G connections (not available direct from AWS)
- Faster provisioning (days vs weeks for Dedicated)
- Last-mile managed service (partner provides DC-to-DX-location circuit)
- Some partners sell DX via **AWS Marketplace** — consolidated billing

Multicloud via Exchange Partners

Equinix Fabric, Megaport, PacketFabric act as **neutral interconnection exchanges**:

```
graph TD
    CustomerDC["🏢 Customer DC"] -->|"Last mile MPLS/fiber"| Exchange["🔄 Exchange PoP\n(Equinix/Megaport)"]
    Exchange -->|"Cross-connect"| AWSDX["☁️ AWS Direct Connect"]
    Exchange -->|"Cross-connect"| AzureER["☁️ Azure ExpressRoute"]
    Exchange -->|"Cross-connect"| GCPCI["☁️ GCP Cloud Interconnect"]
```

- Single physical cross-connect at exchange → reach multiple clouds
- Consistent private latency to all clouds
- Single billing point (the exchange provider)

> 🔑 **Exam key:** AWS has no native multicloud product. Multicloud connectivity = exchange partners (Equinix/Megaport). AWS Outposts/Local Zones = latency-sensitive hybrid but NOT multicloud.

🔗 Last Mile & Multicloud Connectivity

Last Mile — What It Is

The **last mile** is the physical circuit from your data center to the DX colocation facility. **AWS does not provide it** — you arrange it separately.

Last Mile Option	Description	Typical Provider
MPLS circuit	Managed Layer 3 VPN from telco	AT&T, Verizon, BT, Tata
Dark fiber	Unlit fiber — you run your own optics/routing	Zayo, Lumen, local providers
Ethernet WAN	Point-to-point Ethernet circuit	Various carriers
SD-WAN over broadband	Software-defined WAN using internet links	Various managed SD-WAN providers
Partner managed	DX Partner provides end-to-end circuit	Equinix, Megaport (managed)

> ⚠️ **Latency note:** Last mile latency dominates total DX latency. Choose DX location **geographically close** to your DC. A 10ms last-mile circuit negates DX's latency benefits.

Last Mile Provisioning Timeline

Step	Owner	Time
DX Dedicated connection request	Customer	Day 0
LOA-CFA issued	AWS	≤72 hours
Last-mile circuit provisioning	Customer/telco	2–8 weeks
Cross-connect installation at facility	Colocation	1–5 business days
BGP configuration	Customer	Hours
Total		**4–12 weeks typical**

Multicloud Architecture Pattern

```
graph LR
    DC["🏢 Customer DC"] -->|"MPLS/dark fiber\n(last mile)"| PoP["🔄 Exchange PoP\n(Equinix/Megaport/PacketFabric)"]
    PoP -->|"Private VIF"| AWS["☁️ AWS\nDirect Connect"]
    PoP -->|"Private Peering"| Azure["☁️ Azure\nExpressRoute"]
    PoP -->|"Partner Interconnect"| GCP["☁️ GCP\nCloud Interconnect"]
    AWS --> AWSVPC["AWS VPCs"]
    Azure --> AzureVNet["Azure VNets"]
    GCP --> GCPProject["GCP Projects"]
```

AWS Hybrid Alternatives (not multicloud)

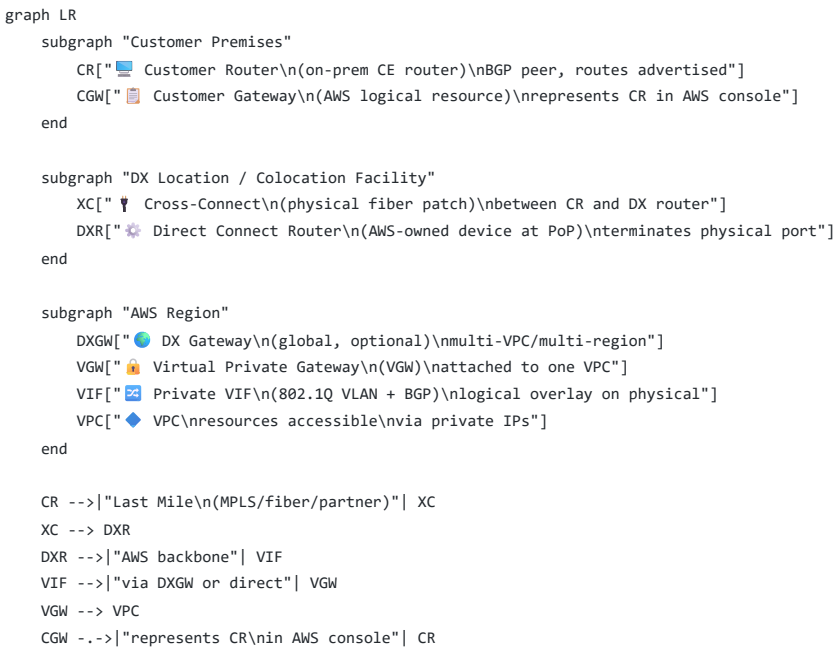
Service	What it does	Use case
AWS Outposts	AWS rack in your DC	Ultra-low latency to AWS services
AWS Local Zones	AWS infrastructure near major cities	Low latency for specific geographies
AWS Wavelength	AWS at 5G carrier edge	Mobile/edge workloads

> 🔑 **Exam key:** Outposts/Local Zones = AWS extending into your premises/edge. They do NOT connect to other clouds. Multicloud = exchange provider pattern only.

🗺️ Full DX Architecture: All Components End-to-End

This section maps every physical and logical component in a Direct Connect deployment and explains what each one is, where it sits, who owns it, and how it connects to the next component.

🔭 Component Map



📦 Component-by-Component Breakdown

1. 🖥️ Customer Router (CR) — Customer-Owned

Attribute	Detail
What it is	Physical or virtual router at your data center or branch office
Owner	You (customer)
Location	Your premises (or your cage at a colocation facility)
Role	Terminates the last-mile circuit; establishes BGP session with AWS DX Router
Protocol	BGP (eBGP with AWS) — must use a private or public ASN
Config required	BGP neighbor config pointing to AWS VIF BGP peer IP; advertise your on-prem prefixes
Examples	Cisco ASR/ISR, Juniper MX/SRX, Palo Alto, pfSense, VyOS, Arista
Key requirement	Must support 802.1Q VLAN tagging (for VIF VLAN) and BGP

> ⚠️ **Your router is entirely your responsibility** — AWS does not manage, monitor, or have access to it.

---

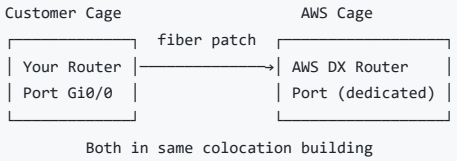
2. 📋 Customer Gateway (CGW) — AWS Logical Resource

Attribute	Detail
What it is	An AWS resource that represents your on-prem router in the AWS console
Owner	You (created in AWS console/CLI)
Location	AWS (logical only — no physical component)
Role	Stores the public IP and BGP ASN of your Customer Router for AWS to reference
Used with	Site-to-Site VPN connections (NOT required for DX — DX uses VIF BGP config directly)
Contains	Public IP of your router, BGP ASN (optional for static VPN)

> 🔑 **Exam key:** CGW is a **VPN concept**, not a DX concept. For Direct Connect, your router is configured directly in the VIF BGP settings — there is no "CGW" resource for DX. CGW is only used when setting up Site-to-Site VPN as DX backup.

3. 🔌 Cross-Connect — Physical Layer

Attribute	Detail
What it is	Physical fiber patch cable at the DX colocation facility
Owner	The colocation facility arranges it; customer requests via LOA-CFA
Location	Inside the DX colocation facility (cage/suite)
Role	Physically connects YOUR router port to the AWS DX Router port
How to get one	AWS issues LOA-CFA (Letter of Authorization - Connecting Facility Assignment) → you give it to the colo facility → they install the cross-connect
Timeline	AWS issues LOA within 72h; colo facility installs in days–weeks
If using a partner	Partner already has cross-connect to AWS — you connect to partner, not AWS directly



4. ⚙️ Direct Connect Router (AWS-Owned)

Attribute	Detail
What it is	Physical router owned and operated by AWS at the DX colocation PoP
Owner	AWS
Location	DX location (colocation facility — Equinix, CyrusOne, Interxion, etc.)
Role	Terminates the physical port; presents 802.1Q VLANs for VIFs; runs BGP with your router
Visibility	You do not manage or access this device — AWS abstraction
What you see	DX connection state (available/down), BGP session status, optical light levels via CloudWatch
Redundancy	For maximum resiliency: two connections at same location terminate on DIFFERENT AWS devices

> 🔑 **Exam key:** "Development resiliency" = 2 connections, same location — they MAY be on different devices but AWS doesn't guarantee it. Only "Maximum resiliency" model explicitly requires 2 locations × 2 separate AWS devices.

5. 🔀 Private VIF — Logical Overlay

Attribute	Detail
What it is	Logical 802.1Q VLAN + BGP session on top of the physical DX connection
Owner	You (created in DX console)
Location	Logical — spans from your router to VGW/DXGW
VLAN ID	1–4094, must be unique per DX connection
BGP peers	Your router IP ↔ AWS VIF peer IP (both in /30 or /31 subnet)
What it carries	Private IP traffic to/from your VPC(s)
Prefix limits	Max 100 prefixes advertised TO AWS; max 200 prefixes received FROM AWS
Connects to	VGW (single VPC) OR DX Gateway (multi-VPC/multi-region)
Per connection limit	Dedicated: up to 50 VIFs. Hosted: exactly 1 VIF.
MTU	Default 1500 bytes; jumbo frames (9001 bytes) supported — must enable on VIF and configure router

```
Physical Port (1G/10G/100G)
├── VLAN 100 → Private VIF → VGW → VPC-Production
├── VLAN 200 → Private VIF → DXGW → VPC-Development (another region)
└── VLAN 300 → Public VIF  → AWS public services
```

> 🔑 **Exam key:** Jumbo frames (MTU 9001) improve throughput for large data transfers. Must be enabled on BOTH the VIF (in DX console) AND your router interface. Mismatched MTU causes fragmentation or drops.

---

6. 🔒 Virtual Private Gateway (VGW)

Attribute	Detail
What it is	AWS-managed gateway attached to a single VPC — terminates DX Private VIF or VPN
Owner	You (created and attached in VPC console)
Location	AWS Region — attached to one VPC
Scope	Regional, single-VPC
ASN	Configured at creation time (private or public); default varies; CANNOT change after creation
Route propagation	Can auto-propagate learned BGP routes into VPC route tables (must enable per route table)
Connects to	Private VIF (DX) AND/OR Site-to-Site VPN — same VGW can serve both simultaneously
Limitation	Only one VGW per VPC; only one VPC per VGW

> ⚠️ **Route propagation must be explicitly enabled** on each VPC route table. Without it, DX/VPN learned routes are NOT automatically inserted — traffic won't flow even if BGP is up.

---

7. 🌍 DX Gateway (DXGW) — Optional Global Hub

Attribute	Detail
What it is	Global resource that aggregates DX connections and distributes to multiple VGWs or TGWs
Owner	You
Location	Global (not tied to any region)
Purpose	Use ONE DX connection to reach VPCs in multiple regions
Associates with	Up to 10 VGWs OR up to 3 TGWs (not both on same DXGW)
Routing	On-prem → VPC only. Does NOT route VPC-to-VPC.
Allowed prefixes	Explicitly configured list of VPC CIDRs advertised to on-prem

---

🔁 Full Traffic Flow: Packet Walk

```
1. On-prem host (192.168.1.10) sends packet to VPC resource (10.0.1.50)

2. On-prem host → Customer Router
   Customer Router has BGP route: 10.0.0.0/16 via DX Private VIF

3. Customer Router → (last mile) → DX Location
   Packet tagged with VLAN ID of Private VIF (e.g., VLAN 100)

4. DX Location → Cross-Connect → AWS DX Router
   AWS DX Router sees VLAN 100, maps to Private VIF BGP session

5. AWS DX Router → Private VIF → VGW
   VGW receives packet destined for 10.0.1.50

6. VGW → VPC local routing → EC2 in subnet 10.0.1.0/24
   VPC route table: 10.0.0.0/16 local → delivered to EC2 ENI

Return path is symmetric — EC2 → VGW → DX Router → cross-connect → Customer Router → on-prem host
```

🔁 Component Ownership Summary

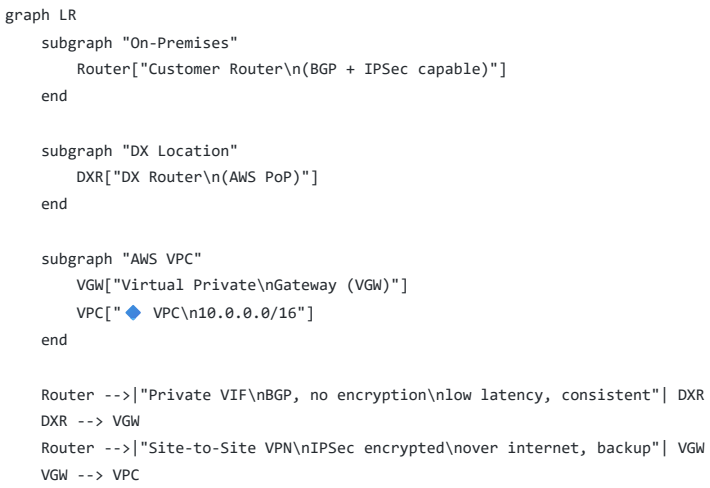
Component	Owner	Managed by	Physical?
Customer Router	You	You	Yes (or VM)
Customer Gateway	You	AWS console	No (logical)
Last Mile Circuit	You / Partner	Telco / Partner	Yes
Cross-Connect	Colo Facility	Colo Facility	Yes
DX Router (PoP)	AWS	AWS	Yes
Private VIF	You	AWS console	No (logical)
DX Gateway	You	AWS console	No (logical)
Virtual Private Gateway	You	AWS console	No (logical)
VPC Route Tables	You	AWS console	No (logical)

⚡ BGP Session Details

```
Your Router (BGP Peer)          AWS DX Router (BGP Peer)
──────────────────────────────────────────────────────────
IP: 169.254.x.x/30 (link-local) ↔  IP: 169.254.x.x/30 (AWS assigned)
ASN: your private ASN            ↔  ASN: VGW/DXGW ASN (e.g. 64512)
Advertise: your on-prem CIDRs        Advertise: VPC CIDR(s)
Auth: MD5 optional                   Auth: MD5 optional
BFD: recommended (fast failure)      BFD: supported
Hold timer: 90s default             Hold timer: 90s default
```

> 🔑 **Exam key:** AWS uses **link-local IPs (169.254.x.x)** for BGP peering on VIFs — these are NOT routable, only used for the BGP session itself. Your actual traffic uses your real private IPs.

🔗 DX + VPN Side-by-Side (Same VGW)



> Same VGW can terminate both DX Private VIF AND Site-to-Site VPN simultaneously. BGP MED makes DX preferred. VPN activates automatically on DX failure.

🪜 Chapter 2: Private and Public Subnets

📖 Overview

A **subnet** is a range of IP addresses within a VPC. Subnets are **zonal resources** — each subnet lives in exactly one Availability Zone. They subdivide a VPC's CIDR block for network segmentation and routing control.

> 🔑 **Exam key:** What makes a subnet "public" is NOT the IP addresses — it's having a **route to an Internet Gateway** in its route table. Remove that route → subnet becomes private.

Scope: Subnets are **AZ-scoped (Zonal)**.

🔢 CIDR and Subnet Sizing

Concept	Detail
VPC CIDR	/16 to /28
Subnet CIDR	Must be within VPC CIDR, /16 to /28
Reserved IPs	AWS reserves **5 IPs** per subnet
IPv6	/56 for VPC, /64 for subnets (mandatory size)

The 5 reserved IPs in each subnet (example: 10.0.1.0/24):

Address	Reserved For
`10.0.1.0`	Network address
`10.0.1.1`	VPC router
`10.0.1.2`	DNS server (VPC+2)
`10.0.1.3`	Future AWS use
`10.0.1.255`	Broadcast (not used, but reserved)

> 🔑 **Exam key:** /28 subnet = 16 IPs − 5 reserved = **11 usable**. Always subtract 5 for AWS reserves.

🌐 Public vs Private Subnet

```
graph TB
    Internet["🌍 Internet"] --> IGW["🚪 Internet Gateway\n(attached to VPC)"]
    IGW --> PublicRT["Public Route Table\n0.0.0.0/0 → IGW"]
    PublicRT --> PubSubnet["🟢 Public Subnet\n(has route to IGW)"]
    PubSubnet --> EC2Pub["EC2 with Public IP\nor Elastic IP"]

    PrivRT["Private Route Table\n0.0.0.0/0 → NAT GW"] --> PrivSubnet["🔴 Private Subnet\n(no route to IGW)"]
    PrivSubnet --> EC2Priv["EC2 — no public IP"]
    PubSubnet --> NATGW["🔁 NAT Gateway\n(in public subnet)"]
    NATGW --> PrivRT
```

Feature	Public Subnet	Private Subnet
Route to IGW	✅ Yes	❌ No
Instances can have public IPs	✅ Yes	❌ No
Direct inbound from internet	✅ Possible	❌ Not possible
Outbound to internet	✅ Direct via IGW	Via NAT Gateway/Instance
Typical resources	Load balancers, bastion hosts, NAT GW	App servers, databases, Lambda in VPC

🗺️ Route Tables

- Every subnet must be associated with **one route table** (default = main route table)
- Multiple subnets can share one route table
- Route table is evaluated **most-specific prefix first**

```
# Public subnet route table
10.0.0.0/16   →  local       (VPC local routing)
0.0.0.0/0     →  igw-xxx     (internet gateway)

# Private subnet route table
10.0.0.0/16   →  local
0.0.0.0/0     →  nat-xxx     (NAT gateway)
10.1.0.0/16   →  pcx-xxx     (VPC peering to another VPC)
172.16.0.0/12 →  vgw-xxx     (VPN/Direct Connect)
```

🔒 Network ACLs vs Security Groups

Feature	Network ACL (NACL)	Security Group
Scope	Subnet level	Instance/ENI level
State	**Stateless** — must allow inbound AND outbound	**Stateful** — return traffic auto-allowed
Rules	Allow AND Deny	Allow only
Rule evaluation	Numbered order (lowest first), stops at first match	All rules evaluated
Default	Default NACL allows all	Default SG allows no inbound
Use case	Broad subnet-level blocks (deny IP ranges)	Fine-grained instance-level control

> 🔑 **Exam key:** NACL is stateless — if you allow HTTP inbound (port 80), you must also allow **ephemeral ports outbound** (1024–65535). Security Groups don't have this problem.

🏢 Multi-Tier Architecture Pattern

```
graph TB
    Internet["🌍 Internet"] --> ALB["Application Load Balancer\n(Public Subnet)"]
    ALB --> App["App Servers\n(Private Subnet)"]
    App --> DB["RDS Database\n(Private Subnet - DB tier)"]
    App --> NATGW["NAT Gateway\n(Public Subnet)"]
    NATGW --> Internet
    DB --> |"No internet access"| X["🚫"]
```

3-tier VPC design:

Tier	Subnet Type	Resources
DMZ / Edge	Public	ALB, NLB, NAT GW, Bastion
Application	Private	EC2, ECS, Lambda, EKS nodes
Data	Private (isolated)	RDS, ElastiCache, DynamoDB VPC Endpoint

---

🔵 IPv6 Subnets

- VPC: optional /56 IPv6 CIDR (from Amazon's pool or your own BYOIP)
- Subnet: fixed /64 (cannot change)
- IPv6 addresses are globally unique — **no NAT for IPv6**
- Egress-only for IPv6 = **Egress-Only Internet Gateway**
- Dual-stack: subnet can have both IPv4 and IPv6

🏛️ Well-Architected Framework — Subnets

Pillar	Consideration
Operational Excellence	Use consistent CIDR naming convention. Document subnet purpose via tags. Use VPC Flow Logs at subnet level for traffic visibility.
Security	Separate tiers into distinct subnets. Use NACLs as defense-in-depth. Never put databases in public subnets. Use dedicated isolated subnets for data tier.
Reliability	Deploy subnets across **minimum 2 AZs** (3 recommended). Size subnets generously — you cannot resize without recreating.
Performance Efficiency	Place resources in same AZ as dependent services to minimize cross-AZ data transfer costs and latency.
Cost Optimization	Cross-AZ data transfer costs $0.01/GB each direction. Co-locate tightly coupled services in same AZ.
Sustainability	Use IPv6 to eliminate NAT layer for IPv6-only workloads, reducing gateway resource consumption.

📋 SAP-C02 Domain Coverage — Subnets

Domain 1: Design Solutions for Organizational Complexity

- Shared VPC (Resource Access Manager) — subnets shared to member accounts
- Centralized egress pattern: one private subnet per account, NAT GW in shared account

Domain 2: Design for New Solutions

- CIDR planning for multi-region, no overlapping addresses (VPC peering, TGW requirement)
- IPv6 dual-stack design for future-proofing

Domain 3: Continuous Improvement for Existing Solutions

- Adding AZs to existing deployment (add subnets per AZ)
- Migrating from single-AZ to multi-AZ subnet design

Domain 4: Accelerate Workload Migration and Modernization

- Subnet sizing for EKS node groups (large IP consumption per pod with VPC CNI)
- Subnet design for containers vs VMs (containers need more IPs)

🚪 Chapter 3: Internet Gateway and Alternatives

📖 Internet Gateway (IGW)

An Internet Gateway is a **horizontally scaled, redundant, highly available VPC component** that allows communication between instances in a VPC and the internet. It performs **NAT (Network Address Translation)** for instances with public IPv4 addresses.

- **Scope:** Regional (attached to VPC, not tied to specific AZ)
- **Cost:** Free (no hourly charge, no data transfer charge — pay for data transfer at VPC level)
- **Limits:** One IGW per VPC

> 🔑 **Exam key:** IGW itself is not the bottleneck — it scales automatically. You cannot control its bandwidth. The bottleneck is your EC2 instance's network capacity.

How IGW works:

1. EC2 in public subnet sends packet to internet
2. IGW translates private IP → Elastic IP (one-to-one NAT)

3. Response comes back, IGW translates EIP → private IP
4. For instances with auto-assigned public IPs: same process, but IP is not persistent

---

🔄 NAT Gateway

NAT Gateway allows **private subnet instances to initiate outbound internet connections** while blocking inbound initiated connections from the internet.

```
graph LR
    EC2["🖥️ EC2\nPrivate Subnet\n10.0.2.5"] -->|"1. Packet to 8.8.8.8"| NATGW["🔄 NAT Gateway\nPublic Subnet\nEIP: 1.2.3.4"]
    NATGW -->|"2. Translated:\nSrc=1.2.3.4"| IGW["🚪 IGW"]
    IGW -->|"3. To internet"| Internet["🌍 Internet"]
    Internet -->|"4. Response to 1.2.3.4"| IGW
    IGW --> NATGW
    NATGW -->|"5. Translated back\nto 10.0.2.5"| EC2
```

Feature	NAT Gateway	NAT Instance
Management	Managed by AWS	Self-managed EC2
Availability	Highly available within AZ	Single point of failure unless clustered
Bandwidth	Up to 100 Gbps (auto-scales)	Limited by instance type
Security Groups	Cannot attach	Can attach
Port forwarding	❌ Not supported	✅ Supported
Bastion host	❌ Not possible	✅ Can double as bastion
Cost	Per hour + per GB	EC2 cost only
IPv6	❌ Not supported	✅ Can configure

> 🔑 **Exam key:** NAT Gateway is **AZ-scoped** for high availability. Deploy one NAT GW **per AZ** to avoid cross-AZ traffic charges and eliminate AZ as SPOF. Each private subnet's route table should point to NAT GW in the same AZ.

NAT Gateway types:

Type	Use Case
Public	Private subnet → internet (via EIP)
Private	Private subnet → other VPCs/on-prem (no internet, no EIP needed)

🔵 Egress-Only Internet Gateway

Egress-Only IGW is for **IPv6 traffic only** — allows outbound IPv6 connections, blocks inbound initiated IPv6 traffic. It is the IPv6 equivalent of NAT Gateway.

Feature	NAT Gateway	Egress-Only IGW
Protocol	IPv4	IPv6
Direction	Outbound only (private subnets)	Outbound only
Cost	Per hour + per GB	Free
Scope	AZ-scoped	Regional (VPC-level)
Inbound blocking	✅ Yes	✅ Yes

🔗 VPC Endpoints

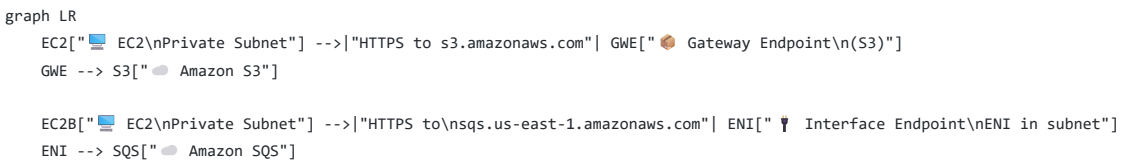
VPC Endpoints allow private connectivity to AWS services **without traversing the internet**, no IGW, NAT GW, or public IP required.

Gateway Endpoints

Property	Value
Supported services	Amazon S3 and DynamoDB only
Cost	Free
Implementation	Route table entry (prefix list → endpoint)
Scope	Regional
DNS	No DNS change required

Interface Endpoints (powered by AWS PrivateLink)

Property	Value
Supported services	100+ AWS services + partner services
Cost	Per hour per AZ + per GB data processed
Implementation	ENI in your subnet with private IP
Scope	AZ-specific ENI, Regional service
DNS	Private DNS overrides service DNS to resolve to private IP

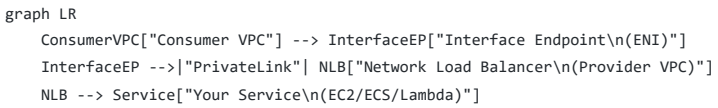


> 🔑 **Exam key:** Gateway endpoints add a **prefix list route** to your route table. Interface endpoints use **DNS resolution** — enable "Private DNS" on the endpoint so the standard service FQDN resolves to the private ENI IP.

🔒 AWS PrivateLink

AWS PrivateLink is the underlying technology for Interface Endpoints. It also enables you to **expose your own services** to other VPCs or AWS accounts without VPC peering or internet.

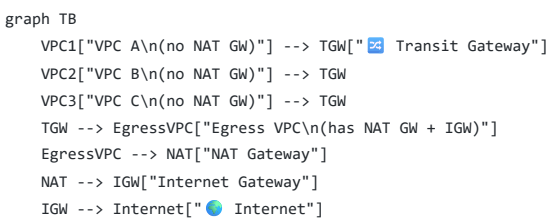
Use case: You have a service (SaaS) running in your VPC. Other customers can consume it via PrivateLink without seeing your VPC or needing peering.



Feature	PrivateLink	VPC Peering
Traffic direction	One-way (consumer → provider)	Two-way
Overlapping CIDRs	✅ Works	❌ Does not work
Scalability	Scales to many consumers	Complex at scale
Routing	No route table changes for consumer	Requires route table entries
Use case	SaaS, shared services	Full VPC connectivity

🔀 Transit Gateway (TGW) as Internet Egress Alternative

For multi-VPC architectures, Transit Gateway enables **centralized internet egress** through a single NAT Gateway instead of one per VPC.



> 🔑 **Exam key:** Centralized egress via TGW reduces NAT GW costs for many VPCs. However, TGW has per-attachment and per-GB charges — do the math. For 3+ VPCs with significant egress, centralized is usually cheaper.

📊 Complete Egress/Ingress Options Comparison

Service	Direction	IPv4/IPv6	Cost	Use Case	Internet?
Internet Gateway	Both (inbound + outbound)	Both	Free	Public-facing resources	✅ Yes
NAT Gateway (Public)	Outbound only	IPv4	$0.045/hr + 0.045/GB	Private subnet internet access	✅ Yes
NAT Gateway (Private)	Outbound only	IPv4	$0.045/hr + 0.045/GB	VPC-to-VPC via TGW	❌ No
NAT Instance	Outbound only	IPv4	EC2 cost	Legacy, custom routing	✅ Yes
Egress-Only IGW	Outbound only	IPv6	Free	IPv6 private subnet egress	✅ Yes
Gateway Endpoint	Outbound to S3/DynamoDB	IPv4	Free	Private S3/DynamoDB access	❌ No
Interface Endpoint	Outbound to AWS services	IPv4	$0.01/hr/AZ + 0.01/GB	Private access to 100+ services	❌ No
PrivateLink	Outbound to partner/own service	IPv4	Same as Interface EP	SaaS, shared services	❌ No
Transit Gateway	Both (between VPCs/on-prem)	Both	$0.05/hr/attach + 0.02/GB	Hub-spoke multi-VPC	Optional

🏛️ Well-Architected Framework — Internet Gateway & Alternatives

Pillar	Consideration
Operational Excellence	Use VPC Flow Logs to monitor NAT GW traffic. Set CloudWatch alarms on NAT GW ErrorPortAllocation (port exhaustion). Tag all endpoints for cost allocation.
Security	Prefer VPC Endpoints over NAT GW for AWS service access — no internet exposure. Never put databases in public subnets. Use Security Groups on Interface Endpoints.
Reliability	Deploy NAT Gateway in each AZ used by private subnets. NAT GW is AZ-scoped — AZ failure removes NAT for that AZ. Use one per AZ.
Performance Efficiency	NAT GW auto-scales to 100 Gbps. Gateway Endpoints have no bandwidth limit. Interface Endpoints scale with NLB behind them.
Cost Optimization	Gateway Endpoints are free — always prefer over NAT GW for S3/DynamoDB. NAT GW charges per GB — optimize with S3 Transfer Acceleration or endpoints. Centralize egress with TGW for many VPCs.
Sustainability	VPC Endpoints eliminate internet routing for AWS service calls, reducing network hops and energy. Gateway Endpoints have no additional infrastructure footprint.

📋 SAP-C02 Domain Coverage — IGW & Alternatives

Domain 1: Design Solutions for Organizational Complexity

- Centralized egress VPC pattern with TGW for large AWS Organizations
- Shared VPC endpoints via RAM to reduce Interface Endpoint costs

Domain 2: Design for New Solutions

- Choosing between IGW, NAT GW, PrivateLink based on direction and service type
- IPv6-only VPC design with Egress-Only IGW

Domain 3: Continuous Improvement for Existing Solutions

- Replacing NAT instances with NAT Gateways
- Adding VPC Endpoints to reduce NAT GW data processing costs
- Migrating from per-VPC NAT GW to centralized TGW egress

Domain 4: Accelerate Workload Migration and Modernization

- PrivateLink for exposing legacy services to new microservices without VPC peering sprawl
- Gateway Endpoints for S3-heavy migration workloads (DataSync, Storage Gateway)

🔀 Chapter 4: AWS Transit Gateway (TGW)

📖 Overview

AWS Transit Gateway (TGW) is a **regional, fully managed network transit hub** that connects VPCs and on-premises networks through a central gateway. Instead of creating a full mesh of VPC peering connections (which grows as N*(N-1)/2), Transit Gateway provides a hub-and-spoke model where every network attaches to the TGW and routing is controlled centrally.

> 🔑 **Exam key:** TGW solves the VPC peering mesh problem. 10 VPCs = 45 peering connections. With TGW = 10 attachments. At scale, TGW always wins on manageability.

Scope: TGW is a **Regional** resource. Cross-region connectivity is via TGW Peering (separate attachments).

Default Limits:

Limit	Value
Attachments per TGW	5,000
VPCs per TGW	5,000
Bandwidth per VPN attachment	10 Gbps
Aggregate bandwidth burst	50 Gbps
Route tables per TGW	20
Routes per TGW route table	10,000

🔗 Attachment Types

Attachment Type	Connects To	Notes
VPC	VPC (one subnet per AZ recommended)	Most common; VPC route tables need updating
VPN	Site-to-Site VPN connection	Supports ECMP when enabled
Direct Connect Gateway	DX Gateway → on-premises	Requires Transit VIF on DX side
TGW Peering	Another TGW (same or different region)	Static routes only, no BGP between TGWs
Connect	SD-WAN appliance via GRE tunnel	Supports BGP, higher bandwidth than VPN

> 🔑 **Exam key:** Connect attachment is for SD-WAN solutions (Cisco, Fortinet). It uses GRE + BGP and supports up to 20 Gbps per Connect peer (vs 10 Gbps for VPN).

🛣️ Route Tables and Routing Domains

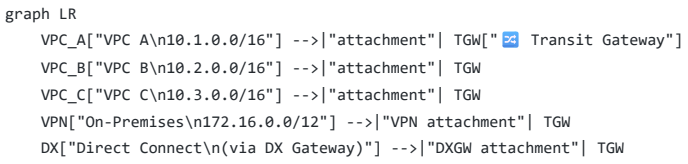
TGW has its **own route tables** — completely separate from VPC route tables.

Key concepts:

Concept	Description
Association	Links an attachment to ONE TGW route table. Attachment uses that RT to make routing decisions.
Propagation	Attachment advertises its CIDR routes INTO a TGW route table.
Blackhole route	Route that drops traffic (explicit drop, like a /dev/null route).
Static route	Manually added route in TGW RT.

How routing works:

1. Packet arrives at TGW from Attachment A
2. TGW looks at the route table **associated** with Attachment A
3. Matches destination to a route → forwards to target attachment
4. Target attachment receives packet and delivers to its VPC/network



🔀 Default vs Custom Route Tables — Network Segmentation

Default (flat network) — both flags ON:

```
graph TB
    subgraph "Default TGW Route Table"
        R1["10.1.0.0/16 → VPC-A-attach"]
        R2["10.2.0.0/16 → VPC-B-attach"]
        R3["10.3.0.0/16 → VPC-C-attach"]
        R4["172.16.0.0/12 → VPN-attach"]
    end
    VPC_A -->|"associated + propagates"| R1
    VPC_B -->|"associated + propagates"| R2
    VPC_C -->|"associated + propagates"| R3
    VPN_attach -->|"associated + propagates"| R4
```

All VPCs see each other = no segmentation.

Custom route tables — network segmentation:

```
graph TB
    subgraph "Prod Route Table"
        PR1["10.1.0.0/16 → Prod-VPC-attach"]
        PR2["172.16.0.0/12 → VPN-attach"]
        PR3["10.100.0.0/16 → SharedSvc-attach"]
    end
    subgraph "Dev Route Table"
        DR1["10.2.0.0/16 → Dev-VPC-attach"]
        DR2["10.100.0.0/16 → SharedSvc-attach"]
    end
    subgraph "Shared Services Route Table"
        SR1["10.1.0.0/16 → Prod-attach"]
        SR2["10.2.0.0/16 → Dev-attach"]
    end

    ProdVPC["Prod VPC\n10.1.0.0/16"] -->|"associated with Prod RT"| PR1
    DevVPC["Dev VPC\n10.2.0.0/16"] -->|"associated with Dev RT"| DR1
    SharedSvc["Shared Svc VPC\n10.100.0.0/16"] -->|"associated with Shared RT"| SR1
```

Prod and Dev cannot reach each other. Both can reach Shared Services. Shared Services can reach both.

> 🔑 **Exam key:** Network segmentation = turn OFF default association + propagation → create custom route tables per domain → manually control which attachments associate/propagate to which tables.

⚙️ Key Feature Flags — Detailed

These are TGW-level settings configured at creation or modifiable afterward (except Multicast):

1. Default Route Table Association

Default	**Enabled**
Effect ON	Every new attachment automatically associates with the TGW's default route table
Effect OFF	New attachments have no route table association — you must manually associate
When to turn OFF	When implementing network segmentation — you want full control over which RT each attachment uses
Exam trap	When OFF + you forget to manually associate an attachment, that attachment cannot route any traffic

2. Default Route Table Propagation

Default	**Enabled**
Effect ON	Every new attachment automatically propagates its CIDRs into the default route table
Effect OFF	New attachments do not propagate routes anywhere — you must manually configure propagation
When to turn OFF	Network segmentation — you don't want prod VPC routes appearing in dev route tables
Exam trap	You can propagate the same attachment into MULTIPLE route tables simultaneously

3. Multicast Support

Default	**Disabled**
Can change after creation?	❌ **NO — must enable at TGW creation time**
Protocol	IGMPv2 (group membership), static multicast group membership also supported
Use cases	Media streaming, financial market data feeds, gaming, video conferencing infrastructure
Cost	Additional per-GB charge for multicast traffic
Exam trap #1	If asked how to enable multicast on existing TGW → must create NEW TGW
Exam trap #2	Multicast does NOT work across TGW peering (intra-TGW only)

4. DNS Support

Default	**Enabled**
Effect	Enables resolution of public DNS hostnames of EC2 instances to private IPs when queried from VPCs attached to TGW
Requirement	VPC must also have "Enable DNS Hostnames" and "Enable DNS Support" enabled
Example	ec2-1-2-3-4.compute-1.amazonaws.com resolves to 10.x.x.x when queried from another attached VPC
When to turn OFF	Only if you don't need cross-VPC DNS resolution and want to reduce complexity

5. Security Groups Referencing Support

Default	**Disabled**
Effect	Allows Security Group rules in VPC-A to reference Security Group IDs from VPC-B connected via TGW
Scope	Same region only (does not work across TGW peering or inter-region)
Use case	Micro-segmentation in multi-VPC architectures — app SG in VPC-A allows traffic only from specific SG in VPC-B
Without this	You must use CIDR ranges in SG rules for cross-VPC traffic (less precise)
Exam note	This is a newer feature; older exam questions may not include it

6. VPN ECMP Support

Default	**Disabled**
ECMP	Equal-Cost Multi-Path routing
Effect ON	Traffic load-balanced across multiple VPN tunnels advertising the same prefixes
Effect OFF	BGP picks one active tunnel (active/standby)
Requirement	Must use **BGP** on VPN connections — static routing does NOT participate in ECMP
Bandwidth	Each VPN tunnel = up to 1.25 Gbps. With ECMP across 8 tunnels = ~10 Gbps aggregate
Exam trap	Static VPN + ECMP enabled = ECMP is ignored; static routes are distributed but not load-balanced

---

🔢 Private ASN in BGP Context

ASN (Autonomous System Number) identifies routing domains in BGP.

Range Type	Values
16-bit Public	1 – 64511
16-bit Private	**64512 – 65534**
32-bit Public	65536 – 4199999999
32-bit Private	**4200000000 – 4294967294**
Reserved	0, 65535, 4294967295

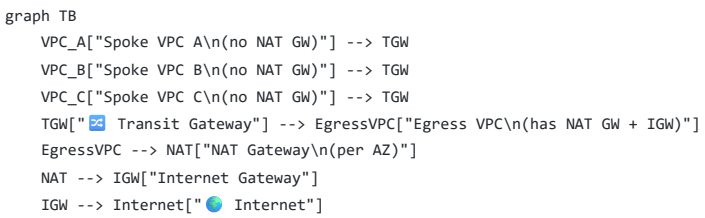
In TGW context:

Setting	Default	Notes
TGW Amazon Side ASN	**64512**	Configurable at creation; change requires TGW recreation
On-prem ASN	Your choice	Must be DIFFERENT from TGW ASN
VPN connection ASN	Per-connection	Can override per VPN attachment

> 🔑 **Exam key:** BGP requires unique ASNs on each side of a peering session. If TGW uses 64512, your on-prem router must use a different ASN (e.g., 65000). Private ASNs are not routable on the internet — fine for private BGP sessions.

🏗️ Centralized Architecture Patterns

Pattern 1: Centralized Egress



Route table setup: Spoke VPC TGW route tables have `0.0.0.0/0 → TGW`. TGW route table for spoke VPCs has `0.0.0.0/0 → Egress VPC attachment`.

Pattern 2: Centralized Ingress

Internet → ALB/NLB in Ingress VPC → TGW → Spoke VPC app servers. Reverse hairpinning via TGW.

Pattern 3: Centralized Security Inspection (East-West)

```
graph LR
    VPC_A["VPC A"] -->|"to VPC B"| TGW["TGW"]
    TGW -->|"0.0.0.0/0 route"| InspVPC["Inspection VPC\n(AWS Network Firewall\nor GWLB + appliance)"]
    InspVPC -->|"inspected traffic"| TGW
    TGW --> VPC_B["VPC B"]
```

Pattern 4: Shared Services VPC

DNS, Active Directory, monitoring, patch management in Shared Services VPC. All other VPCs access via TGW. Shared Services RT propagates to all; spoke RTs only propagate to Shared Services.

🆚 TGW vs VPC Peering vs PrivateLink

Feature	Transit Gateway	VPC Peering	PrivateLink
Transitive routing	✅ Yes	❌ No	N/A
Overlapping CIDRs	❌ No (same as peering)	❌ No	✅ Yes
Cross-account	✅ Yes (via RAM)	✅ Yes	✅ Yes
Cross-region	✅ TGW Peering	✅ Yes	✅ (Inter-region PrivateLink)
Bandwidth	50 Gbps burst	No limit (uses VPC bandwidth)	NLB-limited
Cost	$0.05/hr/attach + 0.02/GB	$0.01/GB (cross-AZ)	$0.01/hr/AZ + 0.01/GB
Routing control	Central RT	Per-VPC RT	N/A (service-level)
Use case	Many VPCs, on-prem, complex routing	Simple 1:1 VPC connectivity	Service exposure
Complexity	Medium	Low	Low-medium
Max connections	5,000 attachments	125 per VPC	Many endpoints

> 🔑 **Exam key decision tree:**
>
> - Simple 2-3 VPC peering, no overlap issues → **VPC Peering** (cheaper)
> - 4+ VPCs, need transitive routing, on-prem → **TGW**
> - Expose one service across accounts/CIDRs → **PrivateLink**

🌍 Inter-Region TGW Peering

```
graph LR
    TGW_US["TGW us-east-1"] -->|"Peering attachment\n(encrypted, AWS backbone)"| TGW_EU["TGW eu-west-1"]
    TGW_US --> VPC_US["VPC us-east-1"]
    TGW_EU --> VPC_EU["VPC eu-west-1"]
```

Feature	Value
Routing	**Static routes only** — no BGP between TGWs
Encryption	Automatic — traffic encrypted on AWS backbone
Bandwidth	Not guaranteed, subject to underlying inter-region capacity
Cost	TGW peering attachment + inter-region data transfer charges
DNS	DNS support flag does NOT work across TGW peering
SG referencing	Does NOT work across TGW peering
Multicast	Does NOT work across TGW peering

> 🔑 **Exam key:** Inter-region TGW peering = static routes only. If you need dynamic routing across regions, use separate BGP sessions to on-premises as the routing exchange point.

💰 Pricing

Component	Cost
Attachment hour	$0.05/hr per attachment (VPC, VPN, DX GW, peering)
Data processed	$0.02/GB in + $0.02/GB out through TGW
Peering attachment	$0.05/hr (each side pays)
Free tier	None

> 💡 **Cost tip:** 1 VPC attachment running 24/7 = $0.05 \times 24 \times 30$ = $36/month just for the attachment, before any data transfer. For a small number of VPCs with low traffic, VPC Peering (pay only for cross-AZ data) may be cheaper.

☁️ Azure Equivalent

AWS	Azure	Notes
Transit Gateway	**Azure Virtual WAN (vWAN)**	Both are managed hub-and-spoke transit services
VPC Peering	Azure VNet Peering	Both non-transitive, require non-overlapping CIDRs
TGW Connect	Azure SD-WAN integration	Both support third-party SD-WAN appliances

Key differences:

- Azure vWAN is a global resource; AWS TGW is regional (use TGW peering for multi-region)
- Azure vWAN Standard Hub supports routing, firewall, VPN in one hub; AWS uses separate services (TGW + Network Firewall + VPN)
- Azure vWAN has built-in BGP peering capability; AWS TGW static routes for inter-TGW peering

---

🐧 Open Source / Linux Equivalent

Concept	AWS TGW	Linux Equivalent
Virtual router	Transit Gateway	Linux network namespaces + ip route
Route tables	TGW route tables	VRF (Virtual Routing and Forwarding) tables
BGP routing	VPN/DX BGP sessions	FRRouting (FRR) or BIRD daemon
VRF isolation	Route domains	`ip vrf exec <vrf>` per routing domain
Multicast	TGW multicast groups	`smcroute` / `pimd` (IGMPv2)

Conceptually similar to **MPLS L3VPN**: route distinguishers (RD) per VRF = TGW route table per segment. Route targets (RT) control import/export = TGW propagation rules.

🏛️ Well-Architected Framework — Transit Gateway

Pillar	TGW Consideration
Operational Excellence	Use TGW Network Manager for centralized network visibility. Enable CloudWatch metrics (BytesIn/Out, PacketDropCountNoRoute). Use consistent tag naming for all TGW resources. Automate route table management via CloudFormation/CDK.
Security	Use custom route tables to isolate prod/dev/shared networks. Enable VPC Flow Logs on all TGW-attached VPCs. Use centralized Inspection VPC with AWS Network Firewall for east-west inspection. Use RAM to share TGW securely across accounts within AWS Organizations only.
Reliability	Deploy TGW in multiple AZs (attach VPC subnets across all AZs). Use redundant VPN attachments with ECMP for on-premises connectivity. Use DX + VPN for maximum reliability to on-premises. TGW is a regional service — for cross-region HA, use TGW peering with routing redundancy.
Performance Efficiency	Use Connect attachment (GRE) for SD-WAN — higher bandwidth than VPN. Enable VPN ECMP to aggregate VPN tunnel bandwidth. Place TGW subnet attachments in same AZ as heavy-traffic workloads to minimize cross-AZ hops. Use TGW for centralized egress to reduce number of NAT Gateways.
Cost Optimization	Evaluate TGW vs VPC Peering break-even: TGW is cost-effective for 4+ VPCs. Centralize NAT Gateways via TGW to reduce per-VPC NAT costs. Monitor TGW data processing costs in Cost Explorer. Use VPC Endpoints in spoke VPCs to reduce TGW data processing for AWS service traffic.
Sustainability	Centralized egress reduces infrastructure footprint (fewer NAT GWs). Shared Inspection VPC via TGW is more sustainable than per-VPC firewalls. TGW Connect attachment for SD-WAN can consolidate multiple physical appliances.

📋 SAP-C02 Domain Coverage — Transit Gateway

Domain 1: Design Solutions for Organizational Complexity

- TGW shared via AWS RAM across AWS Organizations accounts
- Centralized Network account pattern (TGW + DX + firewalls in shared account)
- TGW route table isolation for multi-tenant network segmentation (prod/dev/shared)
- TGW Network Manager for multi-region, multi-account network topology visualization

Domain 2: Design for New Solutions

- Hub-and-spoke replacing VPC peering mesh (N*(N-1)/2 → N attachments)
- Centralized egress, ingress, and inspection architectural patterns
- TGW + DX Gateway for hybrid connectivity at enterprise scale
- Multicast architecture for media/financial services (plan at creation time)

Domain 3: Continuous Improvement for Existing Solutions

- Migrating from VPC peering mesh to TGW (non-disruptive: add TGW, test, remove peering)
- Adding centralized security inspection to existing TGW topology
- Upgrading VPN attachments with ECMP for additional bandwidth
- Transitioning from per-VPC NAT GW to centralized egress via TGW

Domain 4: Accelerate Workload Migration and Modernization

- TGW as connectivity backbone for lift-and-shift migrations (on-prem → VPCs)
- TGW + DX for high-throughput migration paths
- Connecting legacy on-premises networks to cloud-native services via TGW
- SD-WAN integration via Connect attachment for branch office modernization

📊 Chapter 5: VPC Flow Logs

📖 Overview

VPC Flow Logs captures **IP traffic metadata** flowing through Elastic Network Interfaces (ENIs) in your VPC. It records connection-level information — source, destination, ports, bytes, action — but **does NOT capture packet payloads**.

> 🔑 **Exam key:** Flow Logs = metadata, not packets. For full packet capture, use **VPC Traffic Mirroring**.

Scope: Regional service. Flow log data is stored in the destination you choose (CloudWatch Logs, S3, or Kinesis Firehose) in the same region.

Can be enabled at three levels:

Level	Captures
VPC	All ENIs in the entire VPC
Subnet	All ENIs in that subnet
ENI	Single network interface

> 💡 Multiple flow logs can be created per resource — e.g., one to S3 for long-term retention and one to CloudWatch for real-time alerting on the same ENI.

Performance: Captured **out-of-band** — zero impact on network throughput or latency.

📋 What Flow Logs Capture

Field	Description	Example
`srcaddr`	Source IP address (post-NAT at ENI)	`10.0.1.45`
`dstaddr`	Destination IP address	`52.94.1.100`
`srcport`	Source port	`54321`
`dstport`	Destination port	`443`
`protocol`	IANA protocol number	`6` (TCP), `17` (UDP), `1` (ICMP)
`packets`	Packets transferred in aggregation window	`15`
`bytes`	Bytes transferred	`9842`
`start`	Unix timestamp of first packet	`1690000000`
`end`	Unix timestamp of last packet	`1690000600`
`action`	ACCEPT or REJECT	`ACCEPT`
`log-status`	OK / NODATA / SKIPDATA	`OK`

Log status meanings:

- `OK` — records captured normally
- `NODATA` — no traffic in the aggregation interval (normal, informational)
- `SKIPDATA` — internal capacity issue; some records were dropped — investigate if persistent

---

📐 Log Record Format Versions

Default v2 fields (original format):

```
version account-id interface-id srcaddr dstaddr srcport dstport protocol packets bytes start end action log-status
```

Version	Fields Added	Key Additions
v2	Base 14 fields	Core connection metadata
v3	`vpc-id`, `subnet-id`, `instance-id`, `tcp-flags`, `type`, `pkt-srcaddr`, `pkt-dstaddr`	Instance context, NAT tracing, TCP flag analysis
v4	`region`, `az-id`, `sublocation-type`, `sublocation-id`	AZ-level visibility, Outposts/Wavelength support
v5	`pkt-src-aws-service`, `pkt-dst-aws-service`, `flow-direction`, `traffic-path`	AWS service identification, ingress/egress, path tracing

Key v3+ fields explained:

- `pkt-srcaddr` vs `srcaddr`: `srcaddr` = IP at ENI (may be NAT-translated); `pkt-srcaddr` = original source IP before NAT — use `pkt-srcaddr` to trace true origin through NAT Gateway
- `tcp-flags`: bitmask — `2`=SYN, `18`=SYN-ACK, `1`=FIN, `4`=RST — useful for detecting scan patterns (many SYNs, no SYN-ACK)
- `instance-id`: EC2 instance ID associated with the ENI — simplifies correlation without knowing ENI→instance mapping

Custom format: When creating a flow log, choose any subset of fields in any order — only those fields appear in records. Recommended: always include at minimum v5 fields for maximum exam and operational value.

🎯 Destinations

Destination	Latency	Query Method	Best For
CloudWatch Logs	~2–5 min	CloudWatch Insights, Metric Filters	Real-time alerting, dashboards
S3	~5–15 min	Athena, S3 Select, Macie	Long-term retention, cost-effective at scale
Kinesis Data Firehose	Near real-time	OpenSearch, Splunk, S3 downstream	SIEM integration, streaming analytics

Choosing a destination:

- **CloudWatch Logs** — set metric filters to count REJECT records and alarm on security incidents
- **S3 + Athena** — query terabytes cheaply; use Parquet format for best Athena performance
- **Kinesis Firehose** — integrate with existing SIEM (Splunk, OpenSearch Service); Firehose buffers and batches before writing downstream

> ⚠️ **TGW Flow Logs limitation:** TGW flow logs support only **S3 and CloudWatch Logs** — Kinesis Firehose is NOT supported for TGW flow logs.

📁 Log File Format (S3 Destination)

S3 path structure:

```
s3://bucket/AWSLogs/{account-id}/vpcflowlogs/{region}/{year}/{month}/{day}/
```

File name:

```
{account-id}_vpcflowlogs_{region}_{flow-log-id}_{date}_{time}_{hash}.log.gz
```

Format options:

Option	Description	Use Case
Text (default)	Space-delimited, gzip compressed	Simple parsing, grep
Parquet	Columnar binary format	Athena queries (~20% smaller, faster scans)

Aggregation interval:

Setting	Interval	Cost	Use Case
Default	10 minutes	Lower	General logging
1-minute	1 minute	Higher	Incident response, real-time troubleshooting

Hive-compatible prefix: Optional setting that adds partition-style prefix:

```
s3://bucket/AWSLogs/year=2024/month=01/day=15/
```

Enables automatic partition discovery in Athena — reduces scan costs significantly for large datasets.

🌐 VPC Context

Capture levels and their scope:

```
VPC Flow Log (VPC level)
└── Subnet Flow Log (Subnet level)
    └── ENI Flow Log (ENI level)
```

Traffic filter options (set per flow log):

- `ALL` — both accepted and rejected traffic
- `ACCEPT` — only traffic allowed by SG/NACL
- `REJECT` — only traffic blocked by SG/NACL

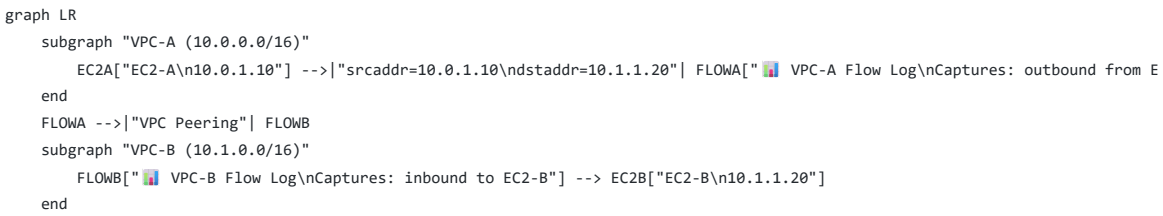
> 🔑 **Exam key:** Flow logs are NOT retroactive — they only capture traffic after the flow log is created.

Traffic NOT captured by VPC Flow Logs:

Traffic Type	Why Excluded	Alternative
DNS queries to VPC+2 resolver	Internal AWS DNS	Route 53 Resolver Query Logs
DHCP requests/responses	AWS DHCP service	CloudTrail (DHCP option sets)
Instance metadata (169.254.169.254)	Link-local, internal	CloudTrail
Amazon Time Sync (169.254.169.123)	Link-local NTP	N/A
Windows license activation	AWS internal KMS	N/A
VPC router traffic	Internal	N/A

🔗 VPC Peering Context

Flow logs capture traffic at the **ENI level** — each VPC sees only its own side:



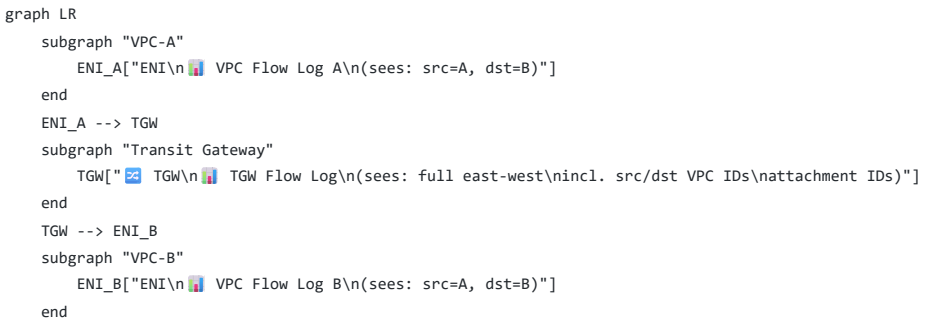
Key points:

- VPC-A flow log: sees `srcaddr=10.0.1.10, dstaddr=10.1.1.20, action=ACCEPT`
- VPC-B flow log: sees `srcaddr=10.0.1.10, dstaddr=10.1.1.20, action=ACCEPT`
- Neither side logs "through the peering" — each logs its own ENI traffic
- **For full visibility: enable flow logs on BOTH VPCs**
- `pkt-srcaddr` = original source IP (useful when traffic passes through NAT before peering)

> 🔑 **Exam key:** To prove VPC-A is communicating with VPC-B, you need flow logs on BOTH sides — one side alone gives incomplete evidence.

🔀 Transit Gateway Flow Logs

TGW has **its own separate flow log capability** — independent from VPC flow logs.



TGW-specific flow log fields:

Field	Description
`tgw-id`	Transit Gateway ID
`tgw-attachment-id`	Source attachment ID
`tgw-src-vpc-account-id`	AWS account owning source VPC
`tgw-dst-vpc-account-id`	AWS account owning destination VPC
`tgw-src-vpc-id`	Source VPC ID
`tgw-dst-vpc-id`	Destination VPC ID
`tgw-src-subnet-id`	Source subnet ID
`tgw-dst-subnet-id`	Destination subnet ID
`tgw-pair-attachment-id`	Return-path attachment ID

Destinations for TGW flow logs: S3 or CloudWatch Logs only (**no Kinesis Firehose**).

Use case: Audit which VPC is communicating with which VPC via TGW — essential for security and compliance in multi-account environments.

🛤️ Traffic Path Field (v5)

The `traffic-path` field (v5+) identifies how traffic exited the VPC:

Value	Path
1	Through Internet Gateway (IGW)
2	Through Virtual Private Gateway (VGW) — DX or VPN
3	Through VPC Interface Endpoint (PrivateLink)
4	Through NAT Gateway
5	Through Transit Gateway
6	Through VPC Peering
7	Through Egress-Only Internet Gateway (IPv6)
8	Through Gateway VPC Endpoint (S3/DynamoDB)

> 🔑 **Exam key:** `traffic-path` only populated for **egress** traffic. Not set for traffic staying within the VPC.

⚠️ Limitations

Limitation	Detail
No payload capture	Metadata only — use Traffic Mirroring for packets
Not real-time	2–15 min delay depending on destination
Not retroactive	Only captures traffic after flow log creation
SKIPDATA records	Internal capacity issue — records dropped (investigate if persistent)
Regional destination	CloudWatch Log Group must be in same region as VPC
IAM role required	Flow logs service needs `logs:CreateLogGroup`, `logs:PutLogEvents`, or S3 `PutObject` permissions
S3 bucket policy	Must allow `delivery.logs.amazonaws.com` as principal
TGW: no Firehose	TGW flow logs → S3 or CloudWatch only
Cannot capture DNS	VPC+2 resolver queries not logged — use R53 Resolver Query Logs

---

🔒 Security and Operations Use Cases

Use Case	How	Flow Log Field
Detect port scanning	Many distinct dstports from same srcaddr in short window	`srcaddr`, `dstport`, `action=REJECT`
Rejected connection audit	Filter `action=REJECT` — identifies SG/NACL blocks	`action`
Identify unused SG rules	No `ACCEPT` records for a specific port over 30 days	`dstport`, `action`
Compliance evidence	Log all connections to sensitive resources	`srcaddr`, `dstaddr`, `action`
Troubleshoot connectivity	REJECT = network block; ACCEPT + no data = app issue	`action`, `bytes`
Top bandwidth consumers	Sort by `bytes` DESC — find traffic hogs	`bytes`, `srcaddr`
Lateral movement detection	Unusual east-west ACCEPT flows between subnets	`srcaddr`, `dstaddr`, `vpc-id`

CloudWatch Metric Filter example — alert on REJECT spikes:

```
# Metric filter pattern
[version, account, eni, src, dst, srcport, dstport, proto, packets, bytes, start, end, action="REJECT", logstatus]

# Create metric: VPCFlowLogRejects
# Alarm: VPCFlowLogRejects > 100 per 5 minutes → SNS notification
```

🔍 Athena Query Examples (S3 Destination)

First, create the Athena table:

```
CREATE EXTERNAL TABLE vpc_flow_logs (
  version int, account string, interfaceid string,
  sourceaddress string, destinationaddress string,
  sourceport int, destinationport int, protocol int,
  numpackets int, numbytes bigint, starttime int, endtime int,
  action string, logstatus string
)
PARTITIONED BY (year string, month string, day string)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ' '
LOCATION 's3://my-bucket/AWSLogs/123456789/vpcflowlogs/us-east-1/'
TBLPROPERTIES ('skip.header.line.count'='1');
```

1. Top source IPs by bytes transferred:

```
SELECT sourceaddress, SUM(numbytes) AS total_bytes
FROM vpc_flow_logs
WHERE year='2024' AND month='01' AND day='15'
GROUP BY sourceaddress
ORDER BY total_bytes DESC
LIMIT 20;
```

2. All REJECT traffic in last 24h:

```
SELECT sourceaddress, destinationaddress, destinationport, protocol, COUNT(*) AS reject_count
FROM vpc_flow_logs
WHERE action = 'REJECT'
  AND year='2024' AND month='01' AND day='15'
GROUP BY sourceaddress, destinationaddress, destinationport, protocol
ORDER BY reject_count DESC;
```

3. Traffic between specific CIDR ranges:

```
SELECT *
FROM vpc_flow_logs
WHERE (sourceaddress LIKE '10.0.1.%' AND destinationaddress LIKE '10.1.0.%')
   OR (sourceaddress LIKE '10.1.0.%' AND destinationaddress LIKE '10.0.1.%')
  AND year='2024' AND month='01' AND day='15';
```

4. Find all traffic on a specific port:

```
SELECT sourceaddress, destinationaddress, action, COUNT(*) AS connection_count
FROM vpc_flow_logs
WHERE destinationport = 22
  AND year='2024' AND month='01' AND day='15'
GROUP BY sourceaddress, destinationaddress, action
ORDER BY connection_count DESC;
```

🆚 Flow Logs vs Other Network Monitoring Tools

Tool	Captures	Latency	Payload	Cost	Use Case
VPC Flow Logs	Connection metadata	2–15 min	❌ No	Low	Audit, troubleshoot, compliance
VPC Traffic Mirroring	Full packets (pcap)	Real-time	✅ Yes	Higher (target instance)	Deep packet inspection, forensics
AWS Network Firewall Logs	Rule matches, domain hits	Near real-time	Alert only	Medium	Firewall audit trail
CloudWatch Agent	OS metrics + app logs	Real-time	N/A	Medium	Host-level visibility
Route 53 Resolver Query Logs	DNS queries	Minutes	N/A	Low	DNS auditing, data exfil detection
AWS WAF Logs	HTTP requests	Near real-time	Headers only	Low	Web request auditing

> 🔑 **Exam key:** Customer asks "we need to capture full packets for forensics" → **Traffic Mirroring**, NOT Flow Logs. "We need to log all DNS queries" → **Route 53 Resolver Query Logs**, NOT Flow Logs.

🏛️ Well-Architected Framework — VPC Flow Logs

Pillar	Consideration
Operational Excellence	Enable flow logs at VPC level on all VPCs by default via AWS Config rule or Service Control Policy. Use Athena + QuickSight for traffic dashboards. Automate flow log creation via CloudFormation/Terraform.
Security	Enable REJECT-only flow logs to CloudWatch for real-time security alerting. Use metric filters + alarms for port scan detection. Send to Security Lake (OCSF format) for centralized SIEM. Use GuardDuty (which uses flow logs internally) for automated threat detection.
Reliability	Flow logs themselves are highly available — AWS manages the capture infrastructure. Monitor for SKIPDATA records which indicate log delivery issues. Use multiple destinations (S3 + CloudWatch) for redundancy.
Performance Efficiency	Use Parquet format + Hive partitioning for Athena queries — dramatically reduces scan cost and time. Use 1-minute aggregation only when needed (higher cost). Filter to REJECT-only when ACCEPT logs aren't needed.
Cost Optimization	Flow logs to S3 are cheapest for long-term storage. Use S3 lifecycle policies to move old logs to Glacier. REJECT-only filter cuts volume ~90% in most environments. Parquet is ~20% smaller than text.
Sustainability	Minimize log volume with targeted filters (REJECT-only, specific subnet) rather than blanket VPC-level ALL-traffic logging. Use Parquet to reduce storage and compute for queries.

📋 SAP-C02 Domain Coverage — VPC Flow Logs

Domain 1: Design Solutions for Organizational Complexity

- Centralized flow log aggregation across AWS Organizations using S3 with cross-account bucket policies
- AWS Security Lake for OCSF-normalized flow log data across all accounts
- RAM sharing of flow log destinations (Kinesis Firehose) across accounts

Domain 2: Design for New Solutions

- Architect real-time security alerting using Flow Logs → CloudWatch → Metric Filter → Alarm → SNS → Lambda
- Choose between CloudWatch vs S3 vs Firehose based on latency, query, and retention requirements
- Design TGW + VPC flow log strategy for full east-west traffic visibility in multi-VPC architectures

Domain 3: Continuous Improvement for Existing Solutions

- Add Parquet format and Hive partitioning to existing text-format flow logs for Athena cost reduction
- Upgrade from v2 to v5 format to gain `traffic-path` and `flow-direction` fields
- Implement 1-minute aggregation during incident investigations, revert to 10-minute after resolution

Domain 4: Accelerate Workload Migration and Modernization

- Use flow logs during migration to validate traffic patterns (which on-prem IPs are talking to which AWS IPs)
- Identify unused connections and firewall rules by analyzing REJECT vs ACCEPT patterns
- Enable TGW flow logs post-migration to audit cross-VPC communication in new landing zone architecture

---

☁️ Azure / Linux Equivalent

Concept	AWS	Azure	Linux
Flow log capture	VPC Flow Logs	NSG Flow Logs	iptables LOG target / nftables
Destination	S3 / CloudWatch / Firehose	Storage Account	syslog / rsyslog / journald
Packet capture	Traffic Mirroring	Network Watcher packet capture	tcpdump / Wireshark
Query	Athena	Traffic Analytics (via Log Analytics)	grep / awk / ELK stack
Netflow export	N/A (managed)	N/A (managed)	ntopng / pmacct / softflowd
DNS query logs	R53 Resolver Query Logs	Azure DNS diagnostic logs	dnsmasq logs / bind9 querylog

> **Azure NSG Flow Logs** store in Storage Account in JSON format with v1/v2 versions (v2 adds byte/packet counts). Azure Traffic Analytics (built on Log Analytics) provides pre-built dashboards similar to CloudWatch Insights with flow log data.

🔐 Chapter 6: Remote Network to VPC Connectivity

📖 Overview

This chapter covers **network-to-network (site-to-site)** connectivity options — connecting entire remote networks (on-premises data centers, branch offices, partner networks) to AWS VPCs. This is distinct from **AWS Client VPN**, which connects individual users to a VPC.

Main options covered:

- AWS Managed Site-to-Site VPN (via VGW or TGW)
- AWS VPN CloudHub (multi-site hub-and-spoke)
- Transit Gateway + Site-to-Site VPN (with Accelerated VPN)
- Software VPN on EC2 (OpenVPN, StrongSwan, pfSense)
- Third-party appliances from AWS Marketplace

Scope: All options are **regional** unless noted (Accelerated VPN uses Global Accelerator global PoPs).

> 🔑 **Exam key:** Site-to-Site VPN = network-to-network, IPSec over internet. Client VPN = user device-to-VPC, TLS. Direct Connect = private fiber, no internet. These are complementary options.

🕯️ AWS Site-to-Site VPN (Managed)

What It Is

AWS-managed IPSec VPN over the public internet between a **Customer Gateway (CGW)** on your side and a **Virtual Private Gateway (VGW)** or **Transit Gateway** on the AWS side.

Key characteristics:

- Fully managed by AWS — no VPN endpoint instances to patch
- Two tunnels per VPN connection (each tunnel in a different AZ on the AWS side)
- Active/active or active/standby tunnel modes
- Encrypted by default (IPSec)
- Works over any internet connection

Components

Component	What It Is	Key Detail
Customer Gateway (CGW)	AWS resource representing your on-prem VPN device	Stores public IP + BGP ASN of your router
Customer Gateway Device	Actual physical/virtual router at your site	Cisco, Juniper, pfSense, StrongSwan, etc.
VPN Connection	Logical IPSec connection between CGW and VGW/TGW	Contains 2 tunnels + routing config
Virtual Private Gateway (VGW)	AWS-side termination point	Attached to single VPC
Transit Gateway (TGW)	AWS-side termination (multi-VPC)	Enables access to all TGW-attached VPCs

Technical Specifications

Parameter	Value
IKE versions	IKEv1, IKEv2
Encryption	AES-128, AES-256
Integrity	SHA-1, SHA-256, SHA-384, SHA-512
DH Groups	2, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24
Tunnel bandwidth	Up to 1.25 Gbps per tunnel
Max bandwidth (2 tunnels + ECMP)	Up to 2.5 Gbps
Dead Peer Detection (DPD)	Enabled by default
NAT-Traversal (NAT-T)	Supported (UDP 4500)
Routing	Static or BGP (dynamic)
BGP hold timer	90 seconds (30s keepalive)

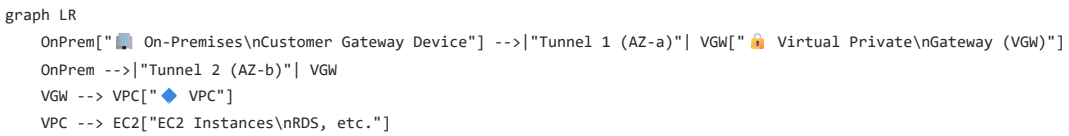
Static vs BGP Routing

Feature	Static Routing	BGP Dynamic Routing
Configuration	Manual CIDR entries	Automatic route exchange
Failover between tunnels	Manual	Automatic
ECMP support	No	Yes (with TGW ECMP enabled)
Route changes propagation	Manual update required	Automatic
VPN CloudHub support	❌ No	✅ Yes
Complexity	Low	Medium
Use case	Simple, small-scale	Production, multi-site, HA

> 🔑 **Exam key:** BGP is required for VPN CloudHub and for TGW VPN ECMP. Static routing works but lacks automatic failover between the two tunnels.

VPN Configuration Download

AWS console generates ready-to-use config files for 50+ device types (Cisco IOS, Cisco ASA, Juniper, Palo Alto, Fortinet, pfSense, StrongSwan, OpenVPN, Yamaha, etc.). Includes: pre-shared keys, IKE Phase 1/2 settings, tunnel IPs, BGP peer IPs.



> ⚠️ **Limitation:** VGW-based VPN connects to ONE VPC only. For multi-VPC access, use TGW-based VPN.

🔗 AWS VPN CloudHub

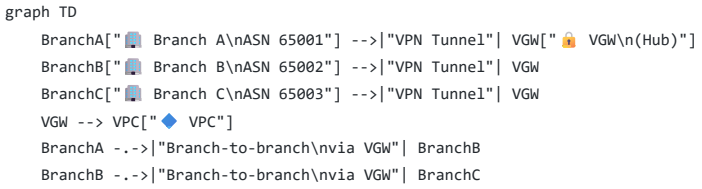
What It Is

VPN CloudHub is a **pattern** (not a separate AWS service) — it uses a single VGW with multiple CGWs attached to create a **hub-and-spoke WAN** where branch offices can communicate with AWS AND with each other through the VGW.

> 🔑 **Exam key:** CloudHub = VGW acting as hub. Branches talk to each other through AWS. Requires BGP. NOT a separate service — it's a VGW configuration pattern.

How It Works

1. Each branch office has its own CGW and VPN connection to the **same VGW**
2. Each branch uses a **unique BGP ASN** — required for route re-advertisement
3. VGW learns Branch-A routes via BGP, re-advertises to Branch-B → branch-to-branch routing
4. All branch-to-branch traffic transits through AWS VGW



Requirements

- BGP (dynamic routing) — static routing **cannot** enable CloudHub
- Each branch must have a **unique BGP ASN**
- Each CGW must have a **unique public IP address**
- Maximum **10 CGWs per VGW**

CloudHub vs TGW for Multi-Site

Feature	VPN CloudHub (VGW)	TGW + Multi-Site VPN
Max sites	10 CGWs per VGW	5000 attachments per TGW
VPC access	Single VPC	All TGW-attached VPCs
ECMP support	No	Yes (TGW ECMP)
Accelerated VPN	No	Yes
Routing granularity	Limited	Full TGW route tables
Cost	VPN connection + data	TGW attachment + VPN + data
Complexity	Low	Medium-High
Use case	Small branch networks	Enterprise multi-VPC

🔀 AWS Transit Gateway + Site-to-Site VPN

What It Is

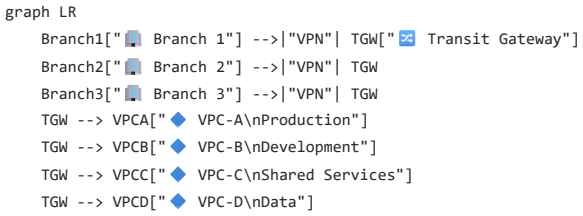
Attach S2S VPN connections **directly to TGW** instead of VGW. This gives VPN-connected sites access to **all VPCs attached to TGW** — not just one.

Advantages Over VGW-Based VPN

- One VPN connection → access to hundreds of VPCs via TGW route tables
- **VPN ECMP**: enable on TGW → load-balance traffic across multiple VPN tunnels (doubles bandwidth per added tunnel)
- BGP routes propagate directly into TGW route tables
- Network segmentation via TGW route table domains (keep VPN traffic in its own RT)
- Supports **Accelerated VPN** (not available with VGW)

Accelerated Site-to-Site VPN

- Integrates with **AWS Global Accelerator** — traffic enters AWS network at nearest edge PoP (not at the VPN endpoint region)
- Reduces latency significantly for geographically distant sites
- Traffic: Remote Site → nearest AWS PoP (GA edge) → AWS backbone → TGW
- Only available with **TGW attachment** (not VGW)
- Additional cost: ~$0.025/hr per accelerator + data transfer fees



VGW-Based VPN vs TGW-Based VPN

Feature	VGW-Based VPN	TGW-Based VPN
VPCs reachable	1 (attached VPC)	All TGW-attached VPCs
ECMP	No	Yes (enable on TGW)
Accelerated VPN	No	Yes
Network segmentation	No (all or nothing)	Yes (TGW route tables)
Max VPN connections	10 per VGW	5000 per TGW
Cost	Lower (no TGW)	Higher (TGW attachment fee)
Route propagation	VPC route table only	TGW route tables
BGP required for ECMP	N/A	Yes

💻 AWS Software VPN (Unmanaged)

What It Is

Running open-source or commercial VPN software on **EC2 instances** in your VPC. You own and manage everything — installation, config, HA, patching, capacity.

When to Use

- Need features unavailable in AWS Managed VPN (WireGuard, custom cipher suites, SSL VPN on port 443)
- Cost optimization: small EC2 instance < hourly VPN connection cost at low traffic
- Compliance: specific cipher suites, key management, audit logging requirements
- Existing vendor licensing (Cisco, Palo Alto, Fortinet)
- Protocol flexibility (GRE, L2TP, WireGuard, SSL)

EC2 Requirements (All Software VPN)

```
# Enable IP forwarding
echo "net.ipv4.ip_forward = 1" >> /etc/sysctl.conf
sysctl -p

# Disable source/destination check (required!)
# In AWS Console: EC2 → Actions → Networking → Change Source/Dest Check → Disable
# Or via CLI:
aws ec2 modify-instance-attribute --instance-id i-xxxx --no-source-dest-check
```

Security Group rules for IPSec VPN:

Protocol	Port	Purpose
UDP	500	IKE (key exchange)
UDP	4500	NAT-T (IKE over NAT)
Protocol 50	—	ESP (encrypted data)
Protocol 51	—	AH (optional, auth only)

VPC Route Table: Add static route: `<remote-site-CIDR> → <VPN EC2 ENI ID>`

Architecture Patterns

Pattern	HA	Use Case
Single EC2	None (SPOF)	Dev/test only
Two EC2 + health check script (ENI reassign)	Active/standby	Simple HA
Two EC2 + NLB	Active/active	Production
ASG (1 min, 1 max) + NLB	Auto-recovery	Automated recovery

OpenVPN on EC2 (Deep Dive)

What OpenVPN is:

- SSL/TLS-based VPN — uses **TCP 443** or **UDP 1194**
- NOT natively IPSec — uses its own TLS tunnel protocol (OpenSSL)
- Two editions: OpenVPN Community (open source, free) and OpenVPN Access Server (commercial, web UI)
- Operates in: site-to-site mode (routing between networks) OR client-to-site mode (remote users)
- Cross-platform: Linux, Windows, macOS, iOS, Android

```
graph LR
    Remote["🏢 Remote Site\nOpenVPN Client"] -->|"TLS/UDP 1194\nor TCP 443"| EC2["🖥️ EC2\n(OpenVPN Server)\nPublic Subnet"]
    EC2 -->|"Routing"| VPC["🔷 VPC\n10.0.0.0/16"]
    VPC --> App["App Servers\nRDS, etc."]
```

Setup on EC2:

```
# Amazon Linux 2
yum install -y openvpn easy-rsa

# Generate PKI
/usr/share/easy-rsa/3/easyrsa init-pki
/usr/share/easy-rsa/3/easyrsa build-ca
/usr/share/easy-rsa/3/easyrsa gen-req server nopass
/usr/share/easy-rsa/3/easyrsa sign-req server server
/usr/share/easy-rsa/3/easyrsa gen-dh
```

server.conf (site-to-site):

```
mode server
tls-server
dev tun
proto udp
port 1194
ca   /etc/openvpn/pki/ca.crt
cert /etc/openvpn/pki/server.crt
key  /etc/openvpn/pki/server.key
dh   /etc/openvpn/pki/dh.pem

server 10.8.0.0 255.255.255.0        # VPN tunnel address pool
push "route 10.0.0.0 255.255.0.0"   # advertise VPC CIDR to remote
route 192.168.1.0 255.255.255.0      # route to remote site network

cipher AES-256-GCM
tls-version-min 1.2
keepalive 10 120
persist-key
persist-tun
```

HA with NLB:

```
graph LR
    Remote["🏢 Remote Site"] --> NLB["⚖️ NLB\n(UDP 1194)"]
    NLB --> OVA["🖥️ OpenVPN EC2-A\nAZ-1"]
    NLB --> OVB["🖥️ OpenVPN EC2-B\nAZ-2"]
    OVA --> VPC["🔷 VPC"]
    OVB --> VPC
```

> ⚠️ **Note:** OpenVPN NLB HA requires stateless design or sticky sessions — NLB UDP doesn't guarantee session persistence without source IP affinity.

StrongSwan on EC2 (Deep Dive)

What StrongSwan is:

- Open-source **IKEv1/IKEv2 + IPSec** implementation for Linux
- Uses Linux kernel **XFRM** framework for IPSec (high performance, no userspace overhead)
- Industry standard — interoperates with Cisco, Juniper, Palo Alto, AWS Managed S2S VPN, Azure VPN Gateway
- Supports: PSK auth, certificate auth, EAP, X.509, RSA, ECDSA

Key configuration files:

File	Purpose
`/etc/ipsec.conf`	Connection definitions (peers, subnets, transforms)
`/etc/ipsec.secrets`	Pre-shared keys or cert private key references
`/etc/strongswan.conf`	Daemon settings (charon process config)

```
graph LR
    OnPrem["🏢 On-Prem\nCisco/Juniper/pfSense"] -->|"IKEv2/IPSec\nESP"| SW["🖥️ EC2 StrongSwan\nPublic Subnet"]
    SW -->|"Routing"| VPC["🔷 VPC\n10.0.0.0/16"]
```

/etc/ipsec.conf example:

```
config setup
    charondebug="ike 1, knl 1"

conn aws-vpc
    left=%defaultroute
    leftid=54.1.2.3                    # EC2 Elastic IP
    leftsubnet=10.0.0.0/16             # VPC CIDR
    right=203.0.113.1                  # On-prem public IP
    rightsubnet=192.168.0.0/24         # On-prem LAN CIDR
    keyexchange=ikev2
    authby=psk
    ike=aes256-sha256-modp2048!        # Phase 1: AES256, SHA256, DH14
    esp=aes256-sha256-modp2048!        # Phase 2 (ESP)
    auto=start
    dpdaction=restart
    dpddelay=30s
    dpdtimeout=120s
    ikelifetime=8h
    lifetime=1h
    margintime=9m
    rekeymargin=3m
```

/etc/ipsec.secrets:

```
54.1.2.3 203.0.113.1 : PSK "YourStrongPreSharedKey!"
```

EC2 setup:

```
# Install
amazon-linux-extras install -y epel
yum install -y strongswan

# Enable IP forwarding
echo "net.ipv4.ip_forward=1" >> /etc/sysctl.conf
sysctl -p

# Start
systemctl enable strongswan
systemctl start strongswan

# Check status
ipsec statusall
ipsec status
```

HA with FRRouting (BGP failover):

```
# Install FRRouting for BGP
yum install -y frr
# Configure BGP to advertise/withdraw routes based on tunnel state
# Route monitor script: if ipsec tunnel down → withdraw BGP prefix → traffic fails to standby
```

> 🔑 **Exam key:** StrongSwan is the recommended open-source IPSec solution for EC2-based VPN. It interoperates natively with AWS Managed S2S VPN (both speak IKEv2). Use StrongSwan when you need standard IPSec without the cost of hardware appliances.

🛒 Third-Party VPN Appliances (AWS Marketplace)

AWS Marketplace offers EC2-based VPN/firewall AMIs from major vendors:

Vendor	Product	Key Feature
Cisco	CSR 1000v	Full IOS-XE, SD-WAN, BGP, MPLS
Palo Alto	VM-Series	NGFW + VPN, App-ID
Fortinet	FortiGate VM	NGFW + SD-WAN + VPN
Juniper	vSRX	Full JunOS routing + VPN
pfSense	pfSense CE	Open source, low cost
Check Point	CloudGuard	NGFW + VPN + threat prevention
Barracuda	CloudGen Firewall	SD-WAN + VPN

Deployment pattern:

- Launch in **public subnet**
- Disable source/destination check
- Assign Elastic IP
- Manage via vendor web console or SSH
- HA: vendor-native (VRRP, HA pairs) or AWS-native (NLB + health checks)
- Licensing: BYOL or license-included (higher EC2 cost)

🆚 Comparison: All Remote Network to VPC Options

Option	Managed By	Max Bandwidth	HA Built-in	BGP	Encryption	Cost Model	Best For
S2S VPN (VGW)	AWS	2.5 Gbps (2 tunnels)	2 tunnels/AZ	Optional	Yes (IPSec)	$0.05/hr + 0.09/GB	Single VPC, simple hybrid
S2S VPN (TGW)	AWS	2.5 Gbps/conn + ECMP	2 tunnels/AZ	Optional	Yes (IPSec)	TGW + VPN + data	Multi-VPC, enterprise
VPN CloudHub	AWS	2.5 Gbps per site	2 tunnels	Required	Yes (IPSec)	Per VPN connection	Multi-branch without WAN
Accelerated VPN	AWS + GA	2.5 Gbps/conn	2 tunnels/AZ	Optional	Yes (IPSec)	VPN + GA fees	Distant sites, low latency
OpenVPN on EC2	You	EC2 NIC limit (~10Gbps)	NLB + 2 EC2	No	TLS	EC2 + NLB	SSL VPN, port 443, flexibility
StrongSwan on EC2	You	EC2 NIC limit	EC2 pair + routing	Via FRRouting	IPSec	EC2	Standard IPSec, enterprise interop
Cisco CSR/Marketplace	You (vendor)	Up to 10 Gbps	Vendor HA	Yes	IPSec/SSL	EC2 + license	Existing Cisco/vendor shops
Direct Connect	AWS + partner	Up to 100 Gbps	LAG + multi-location	Required	No (add MACsec)	Port/hr + data	Production, consistent latency
DX + VPN	AWS	DX bandwidth	DX primary + VPN backup	Yes	VPN for encryption	DX + VPN	Encrypted high-bandwidth hybrid

👤 Site-to-Site VPN vs AWS Client VPN

Feature	Site-to-Site VPN	AWS Client VPN
Use case	Network-to-network	User device-to-VPC
Initiator	VPN router/device	End-user computer/phone
Protocol	IPSec (IKEv1/v2)	OpenVPN (TLS)
AWS endpoint	VGW or TGW	Client VPN Endpoint (ENI in subnet)
Authentication	PSK or certificates	AD/LDAP, SAML, mutual cert
Routing	Static or BGP	Static (split tunnel or full tunnel)
Scaling	Fixed 2 tunnels	Auto-scales to ~20,000 concurrent
Cost	$0.05/hr + 0.09/GB	$0.10/hr endpoint + 0.05/hr per connection
Split tunneling	N/A	Supported
MFA	Not natively	Yes (via IdP + SAML)
When to use	Office/DC connectivity	Remote workers, contractors

> 🔑 **Exam key:** Client VPN = per-user TLS (think "VPN app on laptop"). Site-to-Site VPN = router-to-router IPSec (think "office to cloud"). Wrong VPN type = wrong answer on exam.

🏛️ Well-Architected Framework — Remote Network Connectivity

Pillar	Consideration
Operational Excellence	Use CloudWatch metrics for VPN tunnel state (TunnelState = 0 means down). Set alarms. Download AWS-generated config files to reduce misconfiguration. Use AWS Network Manager for global VPN visibility.
Security	S2S VPN encrypts traffic in transit by default (IPSec). Use IKEv2 over IKEv1. Use certificate auth over PSK where possible. Software VPN: rotate PSKs/certs regularly.
Reliability	Always provision 2 tunnels (both active). Use BGP for automatic failover. For critical workloads: DX primary + VPN backup. For Software VPN: deploy HA pair in 2 AZs behind NLB.
Performance Efficiency	TGW VPN ECMP for bandwidth aggregation. Accelerated VPN for geographically distant sites. Choose appropriate EC2 instance size for Software VPN (network-optimized: c5n, m5n).
Cost Optimization	Compare: S2S VPN hourly vs EC2-based hourly — at low traffic, EC2 t3.micro StrongSwan can be cheaper. For high bandwidth/steady traffic, DX is cheaper per GB than VPN. CloudHub eliminates dedicated WAN circuits.
Sustainability	Consolidated hub-and-spoke (CloudHub or TGW) reduces total number of tunnel endpoints and associated compute. DX reduces internet routing hops = fewer devices = lower energy per GB.

📋 SAP-C02 Domain Coverage — Remote Network Connectivity

Domain 1: Design Solutions for Organizational Complexity

- Multi-account VPN strategy: TGW shared via RAM → all accounts access on-prem via single TGW+VPN
- Network segmentation: separate TGW route tables per business unit, VPN routes only in appropriate RTs
- Centralized network account pattern: DX + VPN terminate in network account, spoke VPCs connect via TGW

Domain 2: Design for New Solutions

- Choosing VPN vs DX based on bandwidth, latency SLA, budget, timeline
- CloudHub for branch office connectivity without SD-WAN investment

- Accelerated VPN for latency-sensitive workloads connecting to AWS from distant locations
- Software VPN (StrongSwan) for custom cipher compliance requirements

Domain 3: Continuous Improvement for Existing Solutions

- Migrating from VGW-based VPN to TGW-based VPN for multi-VPC access
- Adding DX as primary, demoting VPN to backup (BGP MED manipulation)
- Upgrading from static to BGP routing to enable ECMP and auto-failover
- Replacing Software VPN with AWS Managed VPN to reduce operational overhead

Domain 4: Accelerate Workload Migration and Modernization

- VPN as first step in hybrid migration (fast to provision vs DX 4-12 week lead time)
- Multiple VPN connections with ECMP during migration for bandwidth
- Transitioning from VPN to DX as workloads stabilize in AWS
- SD-WAN integration via TGW Connect attachment (GRE) for modern branch networking

---

☁️ Azure / Open Source Equivalents

AWS Service/Concept	Azure Equivalent	Open Source Equivalent
Site-to-Site VPN (VGW)	Azure VPN Gateway (S2S)	StrongSwan, OpenVPN, WireGuard
VPN CloudHub	Azure Virtual WAN (Basic tier)	FRRouting BGP hub
TGW + VPN	Azure Virtual WAN (Standard)	FRRouting + Linux VRF + IPSec
Accelerated VPN	Azure ExpressRoute + VPN (no direct equiv)	WireGuard (lower overhead)
AWS Client VPN	Azure Point-to-Site VPN	OpenVPN Access Server
Software VPN (EC2)	Azure VM + StrongSwan/OpenVPN	Any VPN daemon on Linux VM

> **WireGuard** — modern alternative to OpenVPN/IPSec. Faster, simpler, smaller codebase. Built into Linux kernel 5.6+. Not natively supported by AWS Managed VPN but works on EC2. Good for dev/test or cost-sensitive scenarios.

🌍 Chapter 7: AWS Resource Scope — Global, Regional, Zonal and Edge

📖 Overview

Every AWS resource has a **scope** — the boundary within which it exists and can be used. Understanding scope is critical for:

- **High Availability** design (multi-AZ, multi-region)
- **Disaster Recovery** planning (what needs cross-region replication?)
- **Data Residency / Compliance** (where does data physically live?)
- **IAM and access control** (which regions/accounts can use a resource?)
- **Cost optimization** (cross-AZ and cross-region transfers cost money)

> 🔑 **Exam key:** SAP-C02 frequently tests scope awareness. Wrong scope assumption = wrong answer. Know what's global vs regional vs zonal before answering architecture questions.

Scope hierarchy:

```
Global → Regional → AZ (Zonal) → Local Zone / Wavelength Zone
                                ↑ Edge Location (CDN/DNS PoPs — not deployable)
```

🗺️ Scope Types Explained

🌐 Global Resources

- Exist **once per AWS account** — not tied to any region
- Accessible and usable from ALL regions simultaneously
- AWS manages availability — no regional failover needed
- **Examples:**

Service/Resource	Notes
IAM (users, roles, policies, groups)	Created anywhere, usable everywhere
Route 53	Hosted zones, health checks, DNS global
CloudFront	Distribution is global; serves from nearest edge
AWS Organizations	SCPs, OUs, accounts — account-level
AWS WAF (Classic)	v2 WAF is regional (except on CloudFront)
Direct Connect Gateway	Global hub for multi-region DX
S3 bucket namespace	Name is globally unique; DATA is regional
AWS Artifact	Compliance reports, global
AWS Support plan	Per account
Global Accelerator	Resource global; traffic served regionally
AWS Shield Advanced	Global DDoS protection at edge

> 🔑 **Exam key:** IAM role created in "us-east-1 console" is still global — usable in eu-west-1, ap-southeast-1, everywhere. The console just shows your current region.

🗺️ Regional Resources

- Exist within **one AWS region**
- NOT automatically replicated across regions
- Must be explicitly recreated, replicated, or deployed per region
- ARN includes region: `arn:aws:lambda:**us-east-1**:123456789012:function:my-fn`

Service/Resource	Scope	Notes
VPC	Regional	Spans all AZs in the region
Security Group	Regional	Scoped to VPC (regional)
NACL	Regional	Scoped to VPC
Internet Gateway	Regional	Attached to VPC
Route Table	Regional	Scoped to VPC
Transit Gateway	Regional	Connect VPCs in same region
VPN Connection	Regional	Terminates at VGW or TGW
VGW (Virtual Private GW)	Regional	Attached to one VPC
NAT Gateway	Regional*	*Deployed per-AZ; see Common Confusions
ALB / NLB / CLB	Regional	Serves across AZs
Auto Scaling Group	Regional	Spans AZs
Lambda	Regional	Code deployed per region
ECS / EKS cluster	Regional	Runs tasks/pods across AZs
ECR	Regional	Container registry per region
RDS (service)	Regional	Instances are AZ; Multi-AZ = 2 AZs
DynamoDB (standard)	Regional	Global Tables = multi-region
S3 (data)	Regional	Data stays in bucket's region
SNS / SQS	Regional	Topics/queues per region
EventBridge	Regional	Bus per region
Step Functions	Regional	
CodePipeline / CodeBuild	Regional	
CloudWatch	Regional	Metrics, alarms, logs per region
X-Ray	Regional	
Secrets Manager	Regional	Cross-region replication available
KMS CMK	Regional	MRK = multi-region replica
Parameter Store	Regional	
CloudTrail	Regional + Global option	Can enable global service events
Config	Regional	Aggregator = cross-region/account
GuardDuty	Regional	Must enable per region
Security Hub	Regional	Must enable per region
Inspector	Regional	
Macie	Regional	
Systems Manager	Regional	
EBS Snapshots	Regional	Can copy to other regions
AMIs	Regional	Can copy to other regions
VPC Endpoints	Regional	
PrivateLink	Regional	
DX Connection	Regional	Physical port at PoP

Service/Resource	Scope	Notes
ACM Certificate	Regional*	*us-east-1 required for CloudFront
Direct Connect Connection	Regional	

🏙️ AZ (Zonal) Resources

- Exist within **one specific Availability Zone**
- Cannot be moved to another AZ — must snapshot/recreate
- Failure of that AZ = resource unavailable

Resource	Notes
EC2 instance	Launched in one AZ; cannot move
EBS volume	Attached to EC2 in same AZ only
Subnet	Each subnet = one AZ
ENI (Elastic Network Interface)	Bound to subnet → bound to AZ
NAT Gateway (instance)	One per AZ for HA
RDS instance (Single-AZ)	One AZ; Multi-AZ = sync standby in second AZ
ElastiCache node	Per AZ
Redshift single-node	Per AZ
EFS mount target	One per AZ (EFS itself is regional)
FSx file system	Per AZ (FSx Multi-AZ = 2 AZs)

> ⚠️ **Critical:** EBS volume in `us-east-1a` CANNOT attach to EC2 in `us-east-1b`. Must: snapshot → create new volume in `us-east-1b` → attach.

📍 Local Zone Resources

- AWS infrastructure in **metro areas** (LA, Chicago, Dallas, Miami, etc.)
- Extension of parent region — same API, same console, same VPC
- **Must opt-in** per account in EC2 console settings
- Subset of services: EC2, EBS, VPC subnets, ELB, RDS, ElastiCache
- Use case: single-digit ms latency for users in that city
- Subnet in Local Zone = zonal resource in that Local Zone

```
Parent Region: us-west-2
  Local Zone: us-west-2-lax-1  (Los Angeles)
  Local Zone: us-west-2-den-1  (Denver)
```

> 🔑 **Exam key:** Local Zones are for **latency to end users** — studios, media production, gaming. Not for regulatory data residency (not a separate region). Data still counts as us-west-2.

📡 Wavelength Zone Resources

- AWS compute **embedded at 5G carrier edge** (Verizon, Vodafone, KDDI, SKT)
- Ultra-low latency: **<10ms** to 5G mobile devices
- Traffic stays on carrier network — does NOT traverse public internet
- Services: EC2, EBS, VPC subnets
- Use cases: real-time gaming, AR/VR, autonomous vehicles, live video streaming
- Parent region required — Wavelength Zone is an extension of a region

	Local Zone	Wavelength Zone
Use case	Low latency to city users	Ultra-low latency to 5G devices
Network	Internet	5G carrier network
Latency	Single-digit ms	<10ms (carrier edge)
Services	More (RDS, ELB, etc.)	EC2, EBS, VPC only
Provider	AWS	AWS + carrier partnership

🌍 **Edge Locations**

- **400+ PoPs worldwide** — NOT deployable compute
- AWS-managed infrastructure for CDN, DNS, DDoS
- You do NOT launch instances here — you configure distributions/endpoints

Service	What happens at edge
CloudFront	Caches content; TLS termination; WAF v2
Route 53	Anycast DNS resolution
Global Accelerator	Traffic ingestion; routed to regional endpoint
Shield Advanced	DDoS mitigation at edge
Lambda@Edge / CloudFront Functions	Code execution at edge PoP

📋 Master Reference Table — Quick Scope Lookup

Service	Scope	Key Exam Note
IAM	🌐 Global	Created anywhere, works everywhere
Route 53	🌐 Global	DNS, anycast from edge
CloudFront	🌐 Global	Cert must be in us-east-1
AWS Organizations	🌐 Global	SCPs apply account-wide
Direct Connect Gateway	🌐 Global	One DXGW → multi-region VPCs
Global Accelerator	🌐 Global	Anycast IPs, regional endpoints
S3 bucket name	🌐 Global namespace	Data is regional
ACM (for CloudFront)	🌐 Must be us-east-1	CloudFront only reads us-east-1 ACM
VPC	🗺️ Regional	Spans all AZs in region
Transit Gateway	🗺️ Regional	Inter-region via TGW peering
Security Group	🗺️ Regional	Scoped to VPC
ALB / NLB	🗺️ Regional	Multi-AZ by design
Lambda	🗺️ Regional	Deploy per region
DynamoDB (standard)	🗺️ Regional	Global Tables = multi-region
DynamoDB Global Tables	🌐 Multi-region	Active-active
S3 bucket data	🗺️ Regional	Stays in bucket's region
KMS CMK	🗺️ Regional	MRK = cross-region replica
CloudWatch	🗺️ Regional	Metrics don't cross regions
ACM (for ALB/NLB)	🗺️ Regional	Must match resource region
GuardDuty	🗺️ Regional	Enable per region
EC2 instance	🏢 AZ	Cannot move AZ
EBS volume	🏢 AZ	Attach only in same AZ
Subnet	🏢 AZ	One subnet = one AZ
ENI	🏢 AZ	Bound to subnet
NAT Gateway	🏢 AZ	One per AZ for HA
RDS instance	🏢 AZ	Multi-AZ = 2nd AZ standby
EFS (data)	🗺️ Regional	Mount targets are per-AZ
Local Zone subnet	📍 Local Zone	Metro extension of region
Wavelength subnet	📡 Wavelength	5G carrier edge
CloudFront PoP	🌏 Edge	400+ locations

🔑 Scope Implications for Architecture

High Availability

Scope	HA Pattern
AZ resources	Deploy in **3 AZs minimum**; ALB distributes traffic
Regional resources	Single region = regional SPOF; active-passive or active-active cross-region
Global resources	Inherently HA — AWS manages

Disaster Recovery

Resource	Cross-Region Strategy
S3	Cross-Region Replication (CRR)
RDS	Cross-region read replica → promote on failover
DynamoDB	Global Tables (active-active)
Lambda	Redeploy via IaC (CloudFormation StackSets)
EC2 AMI	Copy AMI to target region
EBS Snapshot	Copy to target region
KMS key	Create MRK or new CMK in target region
Route 53	Health checks + failover routing policy
Secrets Manager	Cross-region replication (built-in)

Data Residency / Compliance (GDPR, Data Sovereignty)

- **Regional resources** stay in their region — EU data in `eu-west-1` stays in EU ✅
- **Global resources** (IAM metadata, Route 53): stored on AWS backbone — not customer-selectable ⚠️
- **S3**: bucket data stays in region; name is global but data is not
- **CloudFront**: caches data at edge PoPs globally — use **Origin Shield** to limit caching regions; for GDPR use **geo-restriction** and separate distribution per region
- **SCP example**: deny any action where `aws:RequestedRegion` is outside EU

```
{
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {
    "StringNotEquals": {
      "aws:RequestedRegion": ["eu-west-1", "eu-central-1"]
    }
  }
}
```

> ⚠️ This SCP does NOT restrict global services (IAM, Route 53, CloudFront) — they are exempt from region conditions. Use separate controls for those.

Cost: Cross-AZ and Cross-Region Transfer

Transfer type	Cost
Same AZ, same VPC	**Free**
Cross-AZ (within region)	~$0.01/GB each direction
Cross-region	$0.02–0.09/GB (varies by region pair)
Internet egress	$0.09/GB (first 10TB/month)
CloudFront → origin	Cheaper than direct internet
DX data out	~50–70% cheaper than internet

> 🔑 **Exam cost trap:** NAT Gateway in one AZ serving private subnets in other AZs → cross-AZ charges. Best practice: one NAT GW **per AZ**.

🆚 Common Scope Confusions

NAT Gateway — Regional Service, AZ Deployment

- Service is **regional** (you manage via regional API)
- Physical instance is **per-AZ**
- **Best practice:** one NAT GW per AZ
- Single NAT GW = cross-AZ charges + AZ SPOF for all outbound internet

```
graph TD
    subgraph "Best Practice — 3 NAT GWs"
        AZ_A["AZ-A Private Subnet"] --> NATGW_A["NAT GW AZ-A"]
        AZ_B["AZ-B Private Subnet"] --> NATGW_B["NAT GW AZ-B"]
        AZ_C["AZ-C Private Subnet"] --> NATGW_C["NAT GW AZ-C"]
    end
    subgraph "Anti-Pattern — 1 NAT GW"
        AZ_A2["AZ-A Private Subnet"] -->|cross-AZ $| NATGW_S["NAT GW AZ-A only"]
        AZ_B2["AZ-B Private Subnet"] -->|cross-AZ $| NATGW_S
        AZ_C2["AZ-C Private Subnet"] -->|cross-AZ $| NATGW_S
    end
```

ACM — Regional AND Requires us-east-1 for CloudFront

- ALB/NLB/API Gateway: cert must be in **same region as the resource**
- CloudFront: cert must be in **us-east-1** regardless of where your origin is
- Common mistake: create ACM cert in eu-west-1 → try to attach to CloudFront → fails

S3 — Global Namespace, Regional Data

- Bucket name: globally unique (like a domain name)
- Bucket data: stored only in the region you chose at creation
- Replication: S3 Cross-Region Replication (CRR) is explicit — not automatic

EFS vs EBS — Regional vs AZ

	EFS	EBS
Scope	Regional (data)	AZ (volume)
Mount targets	Per-AZ (mount from any AZ)	N/A — attached directly
Multi-AZ access	✅ Yes (via mount targets)	❌ No
Protocol	NFS	Block device

KMS MRK (Multi-Region Key)

- Standard CMK: **regional** — cannot encrypt/decrypt across regions
- MRK: same key material replicated across chosen regions
- Same key ID prefix (`mrk-`) — interoperable across regions
- Use case: DynamoDB Global Tables, S3 CRR with SSE-KMS, cross-region Lambda

---

🗂️ Scope and ARN Format

ARN reveals scope:

```
arn:partition:service:region:account-id:resource-type/resource-id
```

Scope	Region in ARN	Example
Global	**Empty**	`arn:aws:iam::123456789012:role/MyRole`
Regional	Region code	`arn:aws:lambda:us-east-1:123456789012:function:MyFn`
AZ	Region code (AZ in resource)	`arn:aws:ec2:us-east-1:123456789012:subnet/subnet-abc`

> 🔑 **Exam key:** Empty region field in ARN = global resource. If you see `arn:aws:iam::...` — no region = IAM is global.

🔄 Scope Decision Flowchart

```
flowchart TD
    Q1{"Does the resource exist\nonce per AWS account\nacross ALL regions?"}
    Q1 -->|Yes| GLOBAL["🌐 GLOBAL\nIAM, Route 53, CloudFront\nOrganizations, DXGW"]
    Q1 -->|No| Q2{"Does it exist per\nAWS region, shared\nacross AZs?"}
    Q2 -->|Yes| REGIONAL["🗺️ REGIONAL\nVPC, TGW, Lambda, ALB\nDynamoDB, S3 data, KMS"]
    Q2 -->|No| Q3{"Does it exist in\none specific AZ only?"}
    Q3 -->|Yes| ZONAL["🏢 AZ / ZONAL\nEC2, EBS, Subnet, ENI\nNAT GW instance, RDS instance"]
    Q3 -->|No| Q4{"Is it at 5G\ncarrier edge?"}
    Q4 -->|Yes| WAVELENGTH["📡 WAVELENGTH\n5G carrier embedded"]
    Q4 -->|No| Q5{"Is it in a\nmetro extension?"}
    Q5 -->|Yes| LOCALZONE["📍 LOCAL ZONE\nMetro area extension"]
    Q5 -->|No| EDGE["🌍 EDGE LOCATION\nCloudFront PoP\nRoute 53, Shield"]
```

🌐 Azure Equivalent Scope Mapping

AWS Scope	Azure Equivalent	Notes
🌐 Global (IAM)	Azure AD / Entra ID (tenant-level)	Both are identity planes
🌐 Global (Route 53)	Azure DNS (global)	
🌐 Global (CloudFront)	Azure CDN / Front Door	Front Door is global
🗺️ Regional	Azure Region	Same concept
🏢 AZ (Zonal)	Azure Availability Zone	Same concept
📍 Local Zone	Azure Edge Zone	Metro extension
📡 Wavelength	Azure Edge Zone for Operators	5G carrier edge
🌍 Edge Location	Azure CDN PoP / Front Door PoP	CDN/DNS edge

🏛️ Well-Architected Framework — Resource Scope

Pillar	Scope Consideration
Operational Excellence	Tag resources with scope metadata (region, AZ). Use CloudFormation StackSets for consistent multi-region deployments. Document which resources are AZ-bound for runbook accuracy.
Security	Use SCPs to restrict which regions resources can be created in (data sovereignty). Understand IAM is global — compromised IAM credentials work in all regions. Enable GuardDuty/Security Hub in ALL regions (regional service — off by default per region).
Reliability	AZ resources: deploy in ≥3 AZs. Regional resources: plan cross-region DR. Global resources: inherently resilient. NAT GW: one per AZ to eliminate cross-AZ SPOF.
Performance Efficiency	Local Zones for sub-10ms user latency. Wavelength for <10ms 5G workloads. CloudFront edge for caching. Cross-AZ traffic adds latency — keep tightly-coupled services in same AZ.
Cost Optimization	Cross-AZ transfer ~$0.01/GB — minimize by placing dependent resources in same AZ. Cross-region transfer expensive — use regional services for local workloads. One NAT GW per AZ to avoid cross-AZ fees.
Sustainability	Minimize cross-region/cross-AZ data transfer — fewer network hops = less energy. Consolidate workloads in fewer regions if business requirements allow. Use CloudFront to serve cached content closer to users (less origin round-trips).

📋 SAP-C02 Domain Coverage — Resource Scope

Domain 1: Design Solutions for Organizational Complexity

- Multi-account scope: IAM global vs regional service enablement (GuardDuty, Security Hub per region)
- SCP region restrictions for data sovereignty across AWS Organizations
- Resource Access Manager (RAM): share regional resources (TGW, subnets, Route 53 Resolver rules) across accounts

Domain 2: Design for New Solutions

- Choosing between regional deployment vs multi-region active-active
- ACM cert placement (regional vs us-east-1 for CloudFront)
- KMS MRK design for cross-region encrypted workloads
- Global Accelerator vs CloudFront — both global, different use cases

Domain 3: Continuous Improvement for Existing Solutions

- Adding Global Tables to existing DynamoDB (regional → multi-region)
- Moving from single-AZ NAT GW to per-AZ for HA and cost
- Enabling GuardDuty/Config in missing regions via Organizations automation

Domain 4: Accelerate Workload Migration and Modernization

- AMI and snapshot copy strategy for cross-region migration
- Using StackSets to deploy infrastructure in target region
- Data migration scope: S3 CRR, DMS for database replication across regions

---

🔀 Chapter 16: VGW vs Direct Connect Gateway vs Transit Gateway — On-Premises Connectivity

📖 Overview

When connecting on-premises networks to AWS VPCs, three gateway types are central to every architecture: **Virtual Private Gateway (VGW)**, **Direct Connect Gateway (DXGW / DCG)**, and **Transit Gateway (TGW)**. These are not mutually exclusive — they are often used together in layered designs. Understanding when each is the right tool (and when it breaks down) is critical for the SAP-C02 exam.

> 🔑 **Exam key:** VGW = one VPC, one attachment point. DXGW = global bridge between DX and VPCs (no VPC-to-VPC). TGW = hub-and-spoke for many VPCs, VPNs, and DX connections in a region.

Analogy:

- **VGW** = a single door in one room — works great for one room, useless if you have 20 rooms.
- **DXGW** = a shared hallway key — lets you reach many rooms from a single entry, but rooms still can't talk to each other through it.
- **TGW** = a central lobby — everyone connects here, and routing policies control who can talk to whom.

🔧 Virtual Private Gateway (VGW)

What it is

VGW is a **VPC-side endpoint** that terminates VPN tunnels (Site-to-Site VPN) or Direct Connect Private VIFs. It is **attached to exactly one VPC** and acts as the VPC's gateway for hybrid connectivity.

Scope: Regional. Tied to one VPC. One VGW per VPC.

```
graph LR
    OnPrem["🏢 On-Premises\nRouter"] -->|"IPSec VPN or\nDX Private VIF"| VGW["🔒 VGW\n(attached to VPC)"]
    VGW --> VPC["🔷 Single VPC\n10.0.0.0/16"]
```

Attachment Types

Attachment	Protocol	Encryption	Bandwidth
Site-to-Site VPN	IPSec IKEv1/v2	AES-256	Up to 1.25 Gbps per tunnel
DX Private VIF	BGP/802.1Q	None (private link)	Up to 100 Gbps (DX port speed)

Routing

- Supports **static routes** and **dynamic BGP** (AS 64512 default, configurable)
- With BGP: VGW advertises VPC CIDR to on-premises; on-premises advertises prefixes to VPC
- Route propagation: enable **route propagation** on VPC route tables so BGP-learned routes from VGW automatically populate the route table (without manual static entries)

```
# Enable route propagation for a VGW on a route table
aws ec2 enable-vgw-route-propagation \
  --route-table-id rtb-0abc123 \
  --gateway-id vgw-0def456
```

Limitations

Limit	Value
VPCs per VGW	**1** (one VGW per VPC)
VPN tunnels per VGW	Up to 10 VPN connections
Max bandwidth (VPN)	1.25 Gbps per tunnel (no ECMP on VGW)
Transitive routing	❌ Not supported
BGP prefix limit	100 advertised routes to on-premises
ECMP	❌ Not supported on VGW VPN

> ⚠️ **Critical exam trap:** VGW does NOT support transitive routing. Traffic from on-premises through VGW into VPC **cannot** then route to another VPC (even if the VPCs are peered). For multi-VPC access from on-premises, use TGW.

When VGW breaks down

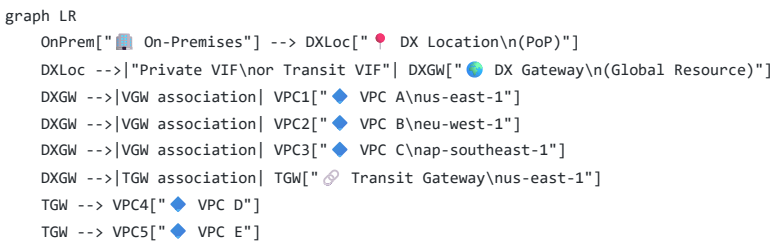
- You have **more than 1 VPC** that on-premises needs to reach → use DXGW or TGW
- You need **VPC-to-VPC routing** through a central point → use TGW
- You need **ECMP** over multiple VPN tunnels for higher bandwidth → use TGW
- You need to share connectivity across **multiple AWS accounts** → use TGW + RAM

🌍 Direct Connect Gateway (DXGW / DCG)

What it is

DXGW is a **globally scoped** AWS resource that acts as a bridge between a DX connection (via a VIF) and one or more VPCs across **any AWS Region** (except China). It decouples the physical DX connection from specific VPC or regional boundaries.

Scope: Global. Not tied to any region. One DXGW can serve VPCs in multiple regions.



Relationship to DX and VIFs

VIF Type	Connects To	DXGW Needed?
Private VIF	VGW (single VPC) or DXGW	Optional (needed for multi-VPC/multi-region)
Transit VIF	DXGW → TGW	**Required** — Transit VIF must go through DXGW
Public VIF	AWS public endpoints (S3, etc.)	No — Public VIF bypasses DXGW

> 🔑 **Exam key:** Transit VIF **always** requires a DXGW. You cannot attach a Transit VIF directly to a TGW — it must go DXGW → TGW.

VGW Association vs TGW Association

Feature	DXGW + VGW (Private VIF)	DXGW + TGW (Transit VIF)
VPCs reachable	Up to 10 VGWs (10 VPCs)	Hundreds via TGW
VPC-to-VPC routing	❌ No	✅ Yes (via TGW route tables)
BGP prefix limit	100 prefixes to on-premises	20,000 routes in TGW
Cost	Lower (no TGW per-attachment fee)	Higher (TGW attachment + processing fee)
Complexity	Lower	Higher but more scalable

Critical DXGW Limitation: No VPC-to-VPC Routing

DXGW is a **hub-and-spoke to on-premises only**. Traffic flows: on-premises ↔ DXGW ↔ VPC. Traffic does NOT flow: VPC A → DXGW → VPC B.

> ⚠️ If you need VPC-to-VPC routing via on-premises (hairpinning), DXGW alone does not do it. Add TGW to the DXGW and use TGW route tables.

Limitations

Limit	Value
VGWs per DXGW	**10**
TGWs per DXGW	**3**
DX connections per DXGW	20
Regions supported	All except China
VIF type required for TGW	Transit VIF only
BGP prefixes from on-prem	20,000 (per Transit VIF)
Overlapping CIDRs	❌ VPCs associated with same DXGW must have non-overlapping CIDRs

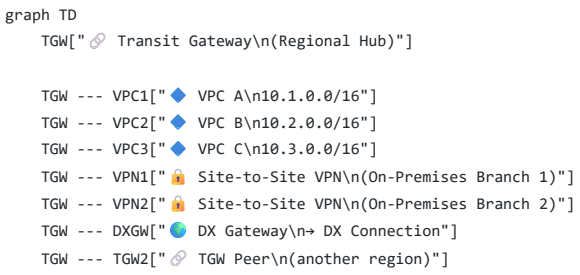
---

🔗 Transit Gateway (TGW)

What it is

TGW is a **regional** network transit hub that interconnects VPCs, VPN connections, Direct Connect (via DXGW Transit VIF), and other TGWs (via peering). It replaces complex full-mesh VPC peering with a scalable hub-and-spoke model.

Scope: Regional. TGW is created in one region. Cross-region: use TGW peering.



Attachment Types

Attachment Type	Description	Notes
VPC	Attach VPC subnets to TGW	One attachment per VPC, choose AZ subnets
VPN	Site-to-Site VPN terminates on TGW	Supports ECMP across tunnels
Direct Connect (Transit VIF)	Via DXGW → Transit VIF	Required path for DX to TGW
TGW Peering	Connect TGWs across regions	Static routes only (no BGP)
TGW Connect	GRE over VPC for SD-WAN	Higher throughput than VPN

Route Tables

TGW uses **route tables** (not subnet route tables — TGW-internal route tables):

```
TGW Route Table (Default):
  10.1.0.0/16  →  VPC A attachment
  10.2.0.0/16  →  VPC B attachment
  0.0.0.0/0    →  VPN attachment (default route to on-premises)
```

- Each attachment can be **associated** with one route table (receives routes from it)
- Each attachment can **propagate** routes into one or more route tables
- **Isolated routing:** create separate route tables to segment VPCs (e.g., prod VPCs cannot reach dev VPCs)

```
graph LR
    subgraph "Prod Route Table"
        ProdVPC1["VPC-Prod-A"]
        ProdVPC2["VPC-Prod-B"]
        DXConn["DX Connection"]
    end
    subgraph "Dev Route Table"
        DevVPC1["VPC-Dev-A"]
        DevVPC2["VPC-Dev-B"]
    end
    subgraph "Shared Services RT"
        SharedVPC["VPC-Shared\n(DNS, AD)"]
        ProdVPC1
        ProdVPC2
        DevVPC1
        DevVPC2
    end
```

ECMP on TGW

TGW supports **Equal-Cost Multi-Path (ECMP)** routing over multiple VPN tunnels:

- 2 VPN connections × 2 tunnels each = 4 active tunnels = up to **5 Gbps** aggregate
- On-premises router must support ECMP and BGP multi-path

> 🔑 **Exam key:** VGW does NOT support ECMP. TGW supports ECMP. For > 1.25 Gbps VPN bandwidth → use TGW with ECMP.

TGW Connect

TGW Connect is a **GRE (Generic Routing Encapsulation) tunnel** attachment type over an existing VPC or DX attachment:

- Used for SD-WAN appliances (Cisco, VMware, Palo Alto)
- Higher throughput than IPSec VPN (up to 5 Gbps per Connect attachment)
- BGP is required (no static routing for Connect)
- Runs over existing VPC attachment (Transport attachment)

TGW Network Manager

- Centralized management console for global network topology
- Visualize TGW peering, VPN connections, DX connections across regions
- Integrates with SD-WAN (via Connect peers)
- CloudWatch metrics: bytes in/out per attachment, packet drops

TGW Sharing via RAM

TGW can be shared across AWS accounts using **AWS Resource Access Manager (RAM)**:

- One TGW in one account (network hub account) shared to all other accounts
- Each account attaches its VPCs to the shared TGW
- Centralized network team controls routing; app teams just attach VPCs

> 🔑 **Exam key:** TGW + RAM = organization-wide hub-and-spoke networking. Essential for AWS Organizations multi-account architectures.

Limitations

Limit	Value
VPC attachments per TGW	5,000
VPN connections per TGW	5,000
TGW peering connections	10 per TGW
Route tables per TGW	20
Routes per route table	10,000 (static) / 20,000 (dynamic)
Bandwidth per VPC attachment	Up to 50 Gbps burst
Max VPN bandwidth (ECMP)	~5 Gbps aggregate
TGW is regional	✅ Yes — cross-region requires peering
BGP on peered TGW connections	❌ No — peering uses static routes only

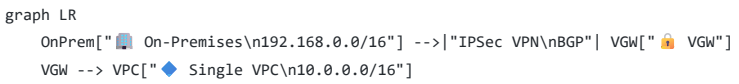
---

📊 Side-by-Side Comparison Table

Feature	VGW	DXGW	TGW
Scope	Regional (per VPC)	Global	Regional
VPCs supported	1	Up to 10 (VGW) or hundreds (via TGW)	Thousands
Multi-region	❌ No	✅ Yes (native)	❌ No (requires peering)
VPC-to-VPC routing	❌ No	❌ No	✅ Yes
VPN termination	✅ Yes	❌ No	✅ Yes
DX termination	Via Private VIF	Via Private/Transit VIF	Via Transit VIF + DXGW
ECMP (VPN)	❌ No	❌ No	✅ Yes
Transitive routing	❌ No	❌ No	✅ Yes
Shareable via RAM	❌ No	❌ No	✅ Yes
Route tables	VPC subnet route tables	N/A (pass-through)	Own TGW route tables
BGP support	✅ Yes	✅ Yes (via VIFs)	✅ Yes
Static routes	✅ Yes	❌ No (BGP only for VIFs)	✅ Yes
Cost	Low (no attachment fee)	Low (DX port cost)	Higher (per-attachment + per-GB)
Complexity	Low	Medium	High
SD-WAN/GRE	❌ No	❌ No	✅ TGW Connect
Max VPN bandwidth	1.25 Gbps	N/A	~5 Gbps (ECMP)
Azure equivalent	VPN Gateway / ExpressRoute GW	N/A (similar concept to ExpressRoute circuit peering)	Azure Virtual WAN

📐 Architecture Patterns

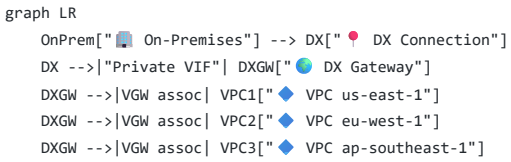
Pattern 1: Single VPC + VGW (Simple / Small Scale)



Use when: One VPC, simple connectivity, low cost, small team. Startup or dev environment. No multi-VPC routing needed.

Breaks when: More VPCs needed, need VPC-to-VPC from on-premises, need ECMP bandwidth.

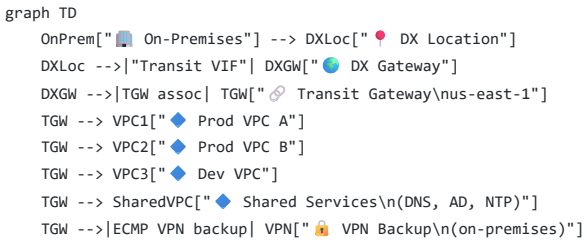
Pattern 2: Multi-VPC + DXGW (Medium Scale, Multi-Region)



Use when: Multi-region VPC access from on-premises, no need for VPC-to-VPC routing, up to 10 VPCs, single DX connection.

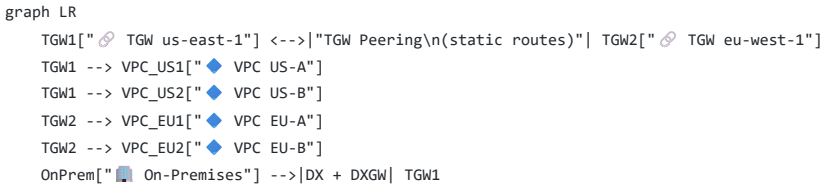
Breaks when: Need VPC-to-VPC routing, more than 10 VPCs, need shared services VPC accessible by all.

Pattern 3: Multi-VPC + TGW + DXGW (Enterprise)



Use when: Many VPCs (10+), VPC-to-VPC routing needed, shared services, centralized inspection, multi-account via RAM, DX + VPN failover.

Pattern 4: TGW Peering Across Regions



Use when: Global multi-region architecture, need VPCs in different regions to communicate, on-premises connected to one region's TGW and needs access to another region's VPCs.

Key constraint: TGW peering uses **static routes only** — no BGP between peered TGWs.

💰 Cost Comparison

Cost Component	VGW	DXGW	TGW
Gateway hourly	~$0.05/hr	Free	~$0.05/hr per TGW
VPN connection	~$0.05/hr per connection	N/A	~$0.05/hr per VPN attachment
VPC attachment	Free (VGW attaches to VPC free)	Free	**~$0.05/hr per VPC attachment**
Data processing	None	None	**$0.02/GB** (processed by TGW)
DX port	Not included	Not included	Not included
Data transfer out	Standard AWS DTO	Standard AWS DTO	Standard AWS DTO

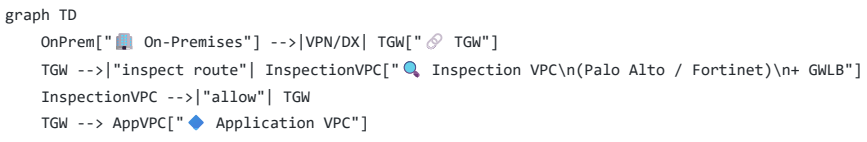
> ⚠️ **Cost trap:** TGW charges **per attachment per hour** AND **per GB processed**. A large number of VPC attachments + high traffic volume = significant cost. For low-traffic, few-VPC scenarios, DXGW + VGW may be cheaper.

> 🔑 **Exam tip:** "Cost-effective multi-VPC on-premises connectivity with < 10 VPCs, no VPC-to-VPC needed" → DXGW + Private VIFs (no TGW attachment fees). "Need VPC-to-VPC + on-premises + scalable" → TGW (accept higher cost for flexibility).

🔐 Security Considerations

Aspect	VGW	DXGW	TGW
Encryption in transit	✅ IPSec (VPN) / ❌ None (DX Private VIF)	❌ DX is private but not encrypted	✅ IPSec (VPN) / ❌ None (DX)
MACsec	❌ No	✅ Supported on DX Dedicated (link-layer)	❌ No (but upstream DX can have MACsec)
Route filtering	BGP route filters	BGP route filters per VIF	TGW route tables (fine-grained)
VPC isolation	Implicit (1 VPC per VGW)	No inter-VPC traffic	Route table isolation
Centralized inspection	❌ No	❌ No	✅ Via GWLB attachment
IAM control	IAM policies	IAM policies	IAM policies + RAM sharing
CloudTrail	✅ API calls logged	✅	✅

Centralized Inspection with TGW + GWLB:



🛡️ Reliability & Resiliency

Aspect	VGW	DXGW	TGW
Built-in HA	✅ Two tunnels per VPN connection (active/passive)	✅ Managed by AWS	✅ Multi-AZ redundant
Active/Active failover	❌ (passive failover only on VPN)	N/A	✅ ECMP enables active/active
DX resiliency	Depends on DX setup	Depends on DX setup	Same
VPN as DX backup	✅ Both attach to same VGW	✅ DX primary, VPN to VGW backup	✅ DX (via DXGW) + VPN both to TGW
BGP failover	BGP detects DX failure, falls back to VPN	BGP-driven	BGP + route priorities

DX + VPN Failover Pattern (TGW):

```
graph LR
    OnPrem["🏢 On-Premises"]
    OnPrem -->|"DX (primary)\nhigh bandwidth, low latency"| DXGW["🌍 DXGW"]
    OnPrem -->|"VPN (backup)\nlower bandwidth, over internet"| VPN["🔒 VPN Attachment"]
    DXGW -->|Transit VIF| TGW["🔗 TGW"]
    VPN --> TGW
    TGW --> VPCs["🔷 VPCs"]
```

Use **BGP MED (Multi-Exit Discriminator)** or **AS path prepending** to prefer DX over VPN. When DX fails → BGP withdraws DX routes → VPN routes take over automatically.

🌍 Scope Summary

Service	Scope	Cross-region?	Cross-account?
VGW	Regional (per VPC)	❌ No	❌ No
DXGW	**Global**	✅ Yes (native)	✅ Yes (via DX partner account)
TGW	Regional	❌ No (peering required)	✅ Yes (via RAM)

☁️ Azure / Open-Source Counterparts

AWS	Azure	Notes
VGW (VPN)	Azure VPN Gateway	Both terminate IPSec/IKEv2 VPN; Azure supports active/active by default; AWS VGW is active/passive (both tunnels up but only one active)
VGW (DX)	Azure ExpressRoute Gateway	Both provide private dedicated connectivity endpoint to VNet/VPC
DXGW	ExpressRoute Global Reach	Both allow single circuit to reach resources in multiple regions; Azure Global Reach connects circuits between locations, DXGW connects one circuit to many VPCs
TGW	Azure Virtual WAN (vWAN)	Both are regional hub-and-spoke. vWAN is fully managed with auto-BGP; TGW gives more manual control over route tables. vWAN supports Azure Firewall natively in hub; TGW requires separate inspection VPC
TGW Connect (GRE)	Azure vWAN NVA Integration	Both support SD-WAN NVA integration

Linux/Open-Source:

- **StrongSwan** — open-source IPSec daemon that plays the role of VGW on the on-premises side
- **FRRouting (FRR)** / **Quagga** — open-source BGP/OSPF routing, equivalent to on-premises BGP router connecting to VGW/TGW
- **Open vSwitch (OVS)** — conceptually similar to TGW routing (virtual switch with flow tables), used in OpenStack/KVM environments

---

📋 Well-Architected Framework Table

Pillar	VGW	DXGW	TGW
Operational Excellence	Simple to manage, single VPC focus. Route propagation automates routing.	Simplifies multi-region connectivity management. Use CloudWatch for VIF metrics.	Higher operational complexity. TGW Network Manager centralizes visibility. Infrastructure as Code (CloudFormation/Terraform) essential at scale.
Security	IPSec encryption for VPN. DX Private VIF: private but unencrypted (add MACsec or application-layer TLS). IAM for config changes.	DX: unencrypted at link layer (MACsec on dedicated connections). Limit VIF creation via IAM. VPC CIDR isolation enforced (no overlapping CIDRs).	Centralized inspection via GWLB attachment. Route table isolation between VPC groups. RAM sharing: central network team controls TGW; app teams only attach VPCs (least privilege).
Reliability	Two VPN tunnels per connection (active/passive). DX failover to VPN via BGP. Single AZ VPN endpoint risk mitigated by AWS redundancy.	Managed by AWS (HA built in). Resiliency depends on DX physical redundancy (High or Maximum Resiliency models). Multiple DX connections to same DXGW for HA.	Built-in multi-AZ. ECMP enables active/active VPN. DX primary + VPN secondary for full hybrid HA. TGW peering for cross-region DR.
Performance Efficiency	VPN: max 1.25 Gbps. DX: up to 100 Gbps (port speed). No ECMP = bandwidth ceiling.	DX: up to 100 Gbps. No traffic processing overhead. Dedicated link = predictable latency.	ECMP unlocks multi-Gbps VPN. TGW Connect (GRE): 5 Gbps per Connect attachment. VPC attachments: up to 50 Gbps burst. Right-size based on traffic patterns.
Cost Optimization	Lowest cost for single-VPC scenarios. No attachment fees.	No TGW attachment fees. Cost-effective for < 10 VPCs, multi-region access. Main cost: DX port.	Per-attachment + per-GB fees add up. Optimize: consolidate VPCs, minimize cross-attachment traffic. Right-size to avoid unused attachments. TGW Connect cheaper than many VPN connections.
Sustainability	Low overhead for small scale. Avoids unnecessary resource sprawl.	Reuse single DX connection across many VPCs (vs. separate connections per VPC).	Consolidates many peer connections into one hub (replaces N^2 peering with N spokes). Reduces operational waste from complex mesh topologies.

🎓 4 Exam Domains

Domain 1: Design Solutions for Organizational Complexity

TGW + RAM for multi-account networking:

- Central networking account owns TGW
- Share TGW via RAM to all accounts in AWS Organization
- Each account's network team attaches VPCs to shared TGW

- Centralized routing policy (route tables) controlled by network team
- No need to manage full-mesh VPC peering across accounts

DXGW for multi-region on-premises access:

- Single DX connection → DXGW → VPCs in multiple regions
- No need for per-region DX connections
- Reduces cost: one DX port serves global VPC access

Example scenario: "Company has 50 AWS accounts across 3 regions, all needing on-premises connectivity and VPC-to-VPC routing." **Answer:** TGW in each region (shared via RAM) + TGW peering between regions + DXGW with Transit VIF connecting to each TGW.

Domain 2: Design for New Solutions

VGW for greenfield simple workloads:

- New startup: one VPC, one VPN to on-premises → VGW (simplest, cheapest)
- Add DXGW when expanding to multiple regions without VPC-to-VPC needs

TGW for enterprise greenfield:

- New enterprise AWS deployment: plan for many VPCs from day 1
- TGW from the start avoids painful VGW → TGW migration later
- Design isolated route tables for prod/dev/shared upfront

DXGW + TGW for hybrid greenfield:

- New large enterprise: DX ordered, multiple VPCs planned
- Architecture: DX → DXGW → Transit VIF → TGW → VPCs
- VPN as backup on same TGW

Domain 3: Continuous Improvement for Existing Solutions

VGW → TGW migration:

1. Create TGW in the region
2. Create new VPN attachment on TGW (keep existing VGW VPN running)
3. Verify routing on TGW works
4. Switch on-premises routing to prefer TGW VPN routes (BGP MED/AS prepend)
5. Detach VGW from VPC after traffic validated
6. Enable route propagation on VPC route tables from TGW

> ⚠️ Migration window: both VGW and TGW VPNs run simultaneously during transition. Brief BGP reconvergence when switching.

Adding centralized inspection to existing TGW:

- Add Inspection VPC with GWLB
- Create new TGW route table for inspection
- Reroute spoke VPC traffic through inspection route table
- No downtime on existing VPCs (just route table changes)

Domain 4: Accelerate Workload Migration and Modernization

Using TGW during migration:

- On-premises workloads migrating to AWS VPCs
- TGW provides consistent connectivity throughout migration
- Phased approach: migrate VPC by VPC, each attaches to TGW

- On-premises still reachable via same VPN/DX during migration

SD-WAN integration via TGW Connect:

- Modernizing on-premises WAN with SD-WAN (Cisco Meraki, VMware SASE)
- TGW Connect (GRE) allows SD-WAN appliance in VPC to connect to TGW
- Higher throughput than IPSec VPN for branch office connectivity
- Enables cloud-managed WAN with AWS as hub

⚠️ Key Exam Traps

Trap	Correct Understanding
"Use DXGW for VPC-to-VPC routing"	❌ DXGW does NOT route VPC-to-VPC. Add TGW.
"VGW supports multiple VPCs"	❌ VGW = 1 VPC only.
"TGW peering uses BGP"	❌ TGW peering = static routes only.
"Transit VIF connects directly to TGW"	❌ Transit VIF → DXGW → TGW (DXGW is required middle).
"VGW supports ECMP"	❌ VGW VPN = no ECMP. TGW VPN = yes ECMP.
"DXGW is regional"	❌ DXGW is **global** — reaches VPCs in any region.
"VPN on VGW and TGW are the same cost"	❌ TGW VPN has additional per-attachment fee AND per-GB processing fee.
"TGW supports overlapping CIDRs"	⚠️ Partially — overlapping VPC CIDRs on same TGW are allowed but must use specific static routes (dynamic propagation won't work cleanly). Avoid overlapping where possible.
"DXGW requires one DX connection per region"	❌ DXGW is global — one DX connection can reach VPCs in all regions via DXGW.

🔒 Chapter 17: Traffic Segmentation in AWS

📖 Overview

Traffic segmentation = dividing network traffic into isolated groups so different workloads, tenants, or security zones cannot reach each other unless explicitly permitted. It is the foundational principle of **defense in depth** in AWS networking.

Without segmentation, a single compromised workload can reach everything on the same network. Segmentation limits **blast radius** — the damage an attacker or misconfigured application can cause.

> 🔑 **Exam key:** Segmentation ≠ encryption. Segmentation = isolation (can't reach). Encryption = confidentiality (can't read). You need both. SAP-C02 frequently tests choosing the *right* segmentation tool for the right scope.

Scope: Spans multiple levels — VPC boundary, subnet, instance, TGW route table, firewall inline.

🔧 Segmentation Mechanisms — Layer by Layer

Mechanism	OSI Level	Scope	Stateful?	Cost
VPC	L3	Account/Region boundary	N/A	Free
Subnet	L3	Within VPC, per AZ	N/A	Free
Security Group (SG)	L4	Per ENI (instance)	✅ Yes	Free
NACL	L3-L4	Per Subnet	❌ No	Free
VPC Peering	L3	Cross-VPC (no transitive)	N/A	Data transfer fee
TGW Route Tables	L3	Multi-VPC hub	N/A	Per attachment + data
AWS Network Firewall	L3-L7	VPC inline	✅ Yes	Per endpoint + data
Gateway Load Balancer (GWLB)	L3	Inline transparent	✅ Yes	Per endpoint + data
Resource Access Manager (RAM)	Control plane	Cross-account	N/A	Free

---

🔷 VPC as Segmentation Boundary

VPC is the **hardest isolation boundary** in AWS. Two VPCs have:

- Separate IP address spaces (CIDRs)
- No routing between them by default
- Separate security group namespaces
- Separate DNS namespaces

```
graph TD
    subgraph "VPC A — Production (10.0.0.0/16)"
        A1["EC2 Web Tier"]
        A2["RDS Database"]
    end
    subgraph "VPC B — Development (10.1.0.0/16)"
        B1["EC2 Dev"]
        B2["RDS Dev"]
    end
    subgraph "VPC C — Shared Services (10.2.0.0/16)"
        C1["Active Directory"]
        C2["Monitoring"]
    end
    A1 -. "NO route by default" .- B1
    C1 --> A1
    C1 --> B1
```

When to use VPC-level segmentation:

- Different environments (prod/dev/staging)
- Different teams with different security postures
- Regulatory isolation requirements (PCI, HIPAA)
- Different blast-radius domains

🔲 Subnet Segmentation

Subnets segment within a VPC — typically by **tier** (web/app/data) and **AZ**.

Subnet Type	Route Table Has	Internet Reachable?	Typical Use
Public	IGW route	✅ Yes (with EIP/Public IP)	ALB, NAT GW, Bastion
Private	NAT GW route	❌ Inbound no, outbound yes	App servers, EKS nodes
Isolated	No internet route	❌ No	Databases, internal services

> 🔑 **Exam key:** Placing an RDS instance in an *isolated* subnet (no NAT GW) + Security Group restricting access to app subnet = correct layered segmentation. "Private" subnet still has outbound internet via NAT.

🛡️ Security Groups — Instance-Level Segmentation

Security Groups are **stateful allow-lists** attached to ENIs. Key properties:

- No explicit deny (only allow rules)
- Stateful: return traffic automatically allowed
- Can reference other SGs (not just CIDRs) — **cross-account SG referencing supported via VPC peering/TGW**
- Default SG: allows all inbound from same SG, all outbound

Tiered SG pattern:

```
ALB-SG  → allows 443 from 0.0.0.0/0
App-SG  → allows 8080 from ALB-SG only
DB-SG   → allows 5432 from App-SG only
```

This creates a **chain of trust** — DB is only reachable from app layer, app only from ALB.

> ⚠️ **Limitation:** SG rules evaluated as "any match = allow." No ordering. Cannot create deny rules. Use NACL for explicit denies.

🚫 NACLs — Subnet-Level Stateless Segmentation

NACLs operate at the **subnet boundary**, evaluate rules in order (lowest number first), and are **stateless** (must allow both inbound AND outbound explicitly).

Use NACLs for:

- Explicit IP block lists (deny specific CIDRs)
- Emergency lockdown (block a compromised IP range)
- Compliance requirements for subnet-level controls

Limitation: NACLs cannot reference Security Groups — CIDR-only. Not suitable for dynamic workloads with changing IPs.

🔀 TGW Route Tables — Multi-VPC Segmentation at Scale

Transit Gateway route tables are the **enterprise-scale segmentation mechanism**. Each TGW attachment (VPC, VPN, DX, peering) is associated with one route table and can propagate into others.

```
graph LR
    subgraph "TGW"
        RT_PROD["Route Table: PROD\nPropagates: Prod VPCs + On-Prem\nNo Dev routes"]
        RT_DEV["Route Table: DEV\nPropagates: Dev VPCs + Internet\nNo Prod routes"]
        RT_SHARED["Route Table: SHARED\nPropagates: All VPCs\nRead-only shared services"]
        RT_ONPREM["Route Table: ON-PREM\nPropagates: All VPCs\nFor DX/VPN attachment"]
    end
    PROD_VPC["Prod VPC"] --> RT_PROD
    DEV_VPC["Dev VPC"] --> RT_DEV
    SHARED_VPC["Shared Services VPC"] --> RT_SHARED
    DX["DX / VPN"] --> RT_ONPREM
```

Key patterns:

- **Flat (default):** all VPCs in one route table — full mesh
- **Segmented:** separate route tables per zone — no lateral movement
- **Centralized egress:** only Egress VPC has internet route; all others send default route to TGW
- **Centralized inspection:** all inter-VPC traffic routed through Inspection VPC (Network Firewall / 3rd-party NVA)

🔍 Centralized Inspection Pattern

Most advanced segmentation: force **all east-west and north-south traffic** through an inspection VPC.

```
flowchart TD
    SPOKE_A["Spoke VPC A\n(App)"] -->|"via TGW"| INSP["Inspection VPC\n(AWS Network Firewall\nor GWLB + 3rd-party)"]
    SPOKE_B["Spoke VPC B\n(Data)"] -->|"via TGW"| INSP
    INSP -->|"allowed traffic"| SPOKE_B
    INSP -->|"north-south"| IGW["Internet Gateway"]
    INSP -->|"blocked"| DROP["🚫 Drop"]
```

Components:

- **AWS Network Firewall** — stateful L3-L7 inspection, Suricata rules, domain filtering
- **Gateway Load Balancer (GWLB)** — transparent bump-in-the-wire for 3rd-party NVAs (Palo Alto, Fortinet, Check Point)
- **TGW with appliance mode** — keeps flows symmetric (required for stateful inspection)

> 🔑 **Exam key:** TGW **appliance mode** must be enabled on the inspection VPC attachment. Without it, return traffic may take a different AZ path, breaking stateful inspection.

📊 Segmentation Mechanisms — Full Comparison

Requirement	Best Tool	Why
Isolate prod from dev entirely	Separate VPCs	Hardest boundary
Isolate DB tier from web tier	Subnets + SG	Within-VPC tiering
Block specific malicious IP	NACL	Only tool with explicit deny
Multi-account VPC isolation with selective connectivity	TGW + Route Tables	Hub-and-spoke segmentation
Inspect all traffic L7	Network Firewall + TGW	Deep packet inspection
Transparent 3rd-party NVA	GWLB	Bump-in-the-wire without changing routes
Regulatory zone isolation (PCI, HIPAA)	Separate VPCs + separate AWS accounts	Account boundary = strongest isolation
Micro-segmentation per workload	Security Groups (SG-to-SG rules)	ENI-level stateful

---

🌍 Scope

Resource	Scope
VPC	Regional
Subnet	AZ
Security Group	VPC-regional
NACL	Subnet (AZ)
TGW	Regional (peering for cross-region)
AWS Network Firewall	AZ (endpoint per AZ)
GWLB	Regional (endpoint per AZ)

☁️ Azure / On-Premises Counterparts

AWS	Azure	On-Premises
VPC	VNet	VLAN / VRF
Security Group	NSG (Network Security Group)	Firewall ACL
NACL	NSG subnet-level	ACL on L3 switch
TGW Route Tables	Azure Virtual WAN + route tables	MPLS VRF segmentation
Network Firewall	Azure Firewall	Palo Alto / Fortinet NGFW
GWLB	Azure Gateway Load Balancer	Transparent firewall cluster

---

🔐 Security Considerations

- **Least-privilege by default:** new VPCs have no routes to each other. Must explicitly enable connectivity.
- **SG over NACL for workload security:** SGs are instance-aware, stateful, and support SG references. NACLs are a blunt subnet tool.
- **Log everything:** VPC Flow Logs capture accepted and rejected traffic. Essential for forensics and compliance.
- **Avoid 0.0.0.0/0 inbound on SG:** except on public-facing load balancers. App/DB layers must restrict source.
- **TGW route tables over VPC peering mesh:** at >5 VPCs, full-mesh peering becomes unmanageable (N×(N-1)/2 connections). TGW scales linearly.
- **Appliance mode for stateful NVAs:** without it, asymmetric routing breaks session state.

🛡️ Reliability & Resiliency

- **Multi-AZ Network Firewall endpoints:** deploy one endpoint per AZ, route AZ traffic to same-AZ endpoint (avoid cross-AZ traffic for stateful inspection)
- **GWLB endpoint per AZ:** same principle — keep traffic local to AZ for HA and latency
- **TGW is regional HA:** spans all AZs in region automatically — no per-AZ configuration needed
- **Security Group changes are instant:** no downtime. NACL changes are instant. Both are non-disruptive.
- **Avoid dependency on single inspection VPC AZ:** GWLB + multi-AZ NVA fleet for true HA

📋 Well-Architected Framework

Pillar	Traffic Segmentation Consideration
Operational Excellence	Use AWS Firewall Manager to centrally manage SGs, NACLs, Network Firewall policies across accounts. Automate policy deployment via CloudFormation / Terraform.
Security	Apply defense-in-depth: VPC boundary → subnet → SG → NACL → inline inspection. Never rely on a single control. Log with VPC Flow Logs + Firewall logs → Security Lake.
Reliability	Deploy Network Firewall / GWLB endpoints in every AZ. Enable TGW appliance mode for stateful inspection. Multi-AZ NVA fleet behind GWLB eliminates single AZ dependency.
Performance Efficiency	GWLB scales automatically to NVA fleet (no manual scaling). Network Firewall is fully managed — no capacity planning. Avoid cross-AZ inspection traffic (latency + cost).
Cost Optimization	Centralized inspection VPC amortizes NVA cost across all spoke VPCs. GWLB endpoint fee only in AZs you use. TGW attachment fee per VPC — evaluate if VPC peering is cheaper for small topologies.
Sustainability	Centralized NVAs use fewer instances than per-VPC appliances. Right-size NVA fleet with GWLB auto scaling — no over-provisioning.

🎓 4 SAP-C02 Exam Domains

1. Design Solutions for Organizational Complexity

- Multi-account segmentation: use **AWS Organizations + SCPs** to prevent cross-account VPC peering outside approved paths
- **AWS Firewall Manager** centrally enforces SG and Network Firewall policies across all accounts in an Org
- **TGW shared via RAM** across accounts — segmentation maintained via route tables per account/OU

2. Design for New Solutions

- New app with PCI scope: place in **separate VPC + separate AWS account** for hard isolation
- Micro-segmentation: SG-per-workload referencing pattern
- New multi-tier app: public subnet (ALB) → private subnet (app) → isolated subnet (DB) with chained SGs

3. Continuous Improvement for Existing Solutions

- Migrating from flat VPC (all SGs allow VPC CIDR) → tiered SG model
- Migrating from VPC peering mesh → TGW with segmented route tables
- Adding centralized inspection to existing hub-and-spoke TGW (insert Inspection VPC, update route tables)

4. Accelerate Workload Migration and Modernization

- Lift-and-shift: map on-premises VLANs/VRFs to AWS subnets/VPCs
- Modernization: replace hardware firewalls with Network Firewall + GWLB
- SD-WAN integration: TGW Connect (GRE) for SD-WAN vendors → segmentation maintained in TGW route tables

---

⚠️ Key Exam Traps

Common Wrong Answer	Correct Understanding
"Use NACL to block traffic from another SG"	NACLs work on CIDRs only, not SG references
"Security Groups are stateless"	SGs are **stateful**. NACLs are stateless.
"VPC peering enables transitive routing"	VPC peering is **non-transitive**. A↔B, B↔C does NOT allow A↔C
"TGW route tables are optional — TGW is plug-and-play"	Default route table exists but flat — segmentation requires custom route tables
"Appliance mode is needed for all TGW attachments"	Only needed on the **inspection/appliance VPC** attachment
"GWLB changes application routing"	GWLB is **transparent** — no IP change, uses GENEVE encapsulation
"Network Firewall is global"	Network Firewall is **regional**, endpoint is **AZ-level**

🔌 Chapter 21: VPC Endpoints

📖 Overview

A **VPC Endpoint** allows resources inside a VPC to communicate with AWS services **privately**, without traversing the public internet, a NAT Gateway, or an Internet Gateway. Traffic stays on the AWS backbone.

> **Linux/open-source analogy:** Like a private socket directly to a service daemon — bypasses the public network stack entirely. **Azure equivalent:** Azure Private Endpoint (for Interface Endpoints) and Azure Service Endpoint (closer to Gateway Endpoints).

🔑 Two Endpoint Types

	Gateway Endpoint	Interface Endpoint (PrivateLink)
Mechanism	Route table injection (prefix list)	ENI with private IP in subnet
Services supported	S3, DynamoDB **only**	100+ AWS services + custom
Route table change	**Required**	Not needed (DNS-based)
DNS change	Optional	Required (private DNS)
Cost	**Free**	$0.01/hr per AZ + 0.01/GB data
On-premises access	❌ No	✅ Yes (via DX/VPN)
Cross-region	❌ No	❌ No (regional)
Scope	Regional	Regional (AZ-level ENI)
Security Groups	❌ Not supported	✅ Supported
Endpoint Policy	✅ Resource-based policy	✅ Resource-based policy

---

🗺️ Gateway Endpoints — How Route Tables Work

Gateway Endpoints are **purely route-based**. Without a route table entry, the endpoint exists but is **never used**.

```
# What AWS injects when you associate a route table with a Gateway Endpoint:
Destination: pl-6ca54005 (com.amazonaws.eu-west-1.s3)  →  Next hop: vpce-0abc123
```

Traffic flow WITH endpoint route:

```
EC2 (10.0.1.10) → route table lookup → prefix list matches S3 IPs
               → next hop: Gateway Endpoint
               → AWS backbone → S3 (no NAT, no IGW, no internet)
```

Traffic flow WITHOUT endpoint route:

```
EC2 (10.0.1.10) → route table lookup → 0.0.0.0/0 → NAT Gateway → Internet → S3 public IP
                                                    (costs money, crosses internet)
```

⚠️ Multi-Route-Table Trap

A VPC with 3 private subnets, each in separate route tables → **must add the endpoint route to all 3 route tables**. Subnets whose route table lacks the prefix list route still use NAT Gateway for S3/DynamoDB.

```
# Check which route tables have the endpoint route
aws ec2 describe-route-tables --filters "Name=route.destination-prefix-list-id,Values=pl-6ca54005"
```

🔌 Interface Endpoints (PrivateLink) — DNS-Based Routing

Interface Endpoints create an **ENI** with a private IP in your subnet. No route table change needed — traffic is directed by DNS resolution.

```
# Private DNS enabled (default):
nslookup sqs.eu-west-1.amazonaws.com
→ returns 10.0.1.45  (private ENI IP — stays in VPC)

# Private DNS disabled:
nslookup sqs.eu-west-1.amazonaws.com
→ returns 52.95.x.x  (public IP — goes via internet or NAT)
```

For private DNS to work:

- VPC must have `enableDnsHostnames = true`
- VPC must have `enableDnsSupport = true`
- Interface Endpoint must have **Enable private DNS name** checked

AZ Resiliency

Deploy Interface Endpoint ENIs in **every AZ** your workload uses. Each ENI gets its own IP. Traffic from an AZ hits the local ENI (no cross-AZ charge). If one AZ's ENI fails, DNS failover to another AZ's ENI.

```
# Create interface endpoint in 2 subnets (2 AZs)
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-xxx \
  --service-name com.amazonaws.eu-west-1.sqs \
  --vpc-endpoint-type Interface \
  --subnet-ids subnet-az1 subnet-az2 \
  --security-group-ids sg-xxx \
  --private-dns-enabled
```

🔒 Endpoint Policies

Both endpoint types support **resource-based policies** to restrict which resources can be accessed through the endpoint.

```
// Gateway Endpoint policy — restrict to specific S3 bucket
{
  "Statement": [{
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::my-bucket/*"
  }]
}
```

This is a **second layer** of control — even if an IAM policy allows S3 access, the endpoint policy can restrict which bucket is reachable via this endpoint. Used for data exfiltration prevention (prevent EC2 from accessing arbitrary external S3 buckets).

🌐 On-Premises Access

Endpoint Type	On-Premises Access	How
Gateway Endpoint	❌ No	Routes only exist in VPC route tables, not visible on-premises
Interface Endpoint	✅ Yes	Private IP accessible via DX Private VIF or Site-to-Site VPN

On-premises → Interface Endpoint pattern:

```
On-premises → DX Private VIF → VGW/TGW → VPC → Interface Endpoint ENI (private IP) → AWS Service
```

Requires: DNS resolution on-premises to return the ENI private IP (use Route 53 Resolver inbound/outbound, or on-premises DNS forwarder).

💰 Cost Comparison

Scenario	Without Endpoint	With Gateway Endpoint	With Interface Endpoint
EC2 → S3 (same region)	NAT GW: $0.045/GB + 0.045/hr	**Free**	$0.01/GB + 0.01/hr per AZ
EC2 → SQS	NAT GW: $0.045/GB + 0.045/hr	N/A	$0.01/GB + 0.01/hr per AZ
On-premises → S3	Public VIF or internet	N/A	DX + Interface Endpoint

> **Rule:** For S3/DynamoDB in private subnets, always use Gateway Endpoint — it's free and removes NAT dependency. For other services: Interface Endpoint saves NAT costs at scale and is required for compliance (no internet).

🏗️ VPC Endpoint vs Other Options

	VPC Endpoint	NAT Gateway	Internet Gateway	Transit Gateway
Private traffic to AWS service	✅ Best	✅ Works (costs more)	✅ Works (public IP needed)	✅ Via shared endpoints
On-premises access	Interface only	❌	❌	✅
Cost	Free (GW) / $0.01/GB (IF)	$0.045/GB + 0.045/hr	Free + DTO	$0.02/GB
Latency	Lowest (direct backbone)	Medium	Medium	Medium

🌍 Scope and Availability

- **Regional service** — endpoint belongs to one region

- **Gateway Endpoint** — regional, associated with route tables across all AZs
- **Interface Endpoint** — AZ-level ENI, but you create multiple ENIs across AZs
- Not available cross-region — create per-region endpoints for multi-region architectures

---

☁️ Azure Equivalent

AWS	Azure
Interface Endpoint (PrivateLink)	**Azure Private Endpoint** — same concept: ENI with private IP, DNS-based
Gateway Endpoint	**Azure Service Endpoint** — similar: route-based, restricts service to VNet, but no private IP
VPC Endpoint Policy	**Azure Private Link resource policy**

Key difference: Azure Private Endpoint creates a private IP (like AWS Interface Endpoint). Azure Service Endpoint does not create a private IP — it just restricts the service to your VNet CIDR (less private than AWS Gateway Endpoint in some ways, more restrictive in others).

🔗 Related Services

Service	Relationship
AWS PrivateLink	The underlying technology for Interface Endpoints. Also enables exposing your own services to other VPCs/accounts.
Route 53 Resolver	Required for on-premises DNS resolution of Interface Endpoint private DNS names
NAT Gateway	Replaced by endpoints for AWS service access from private subnets
VPC Peering / TGW	For VPC-to-VPC; endpoints are for VPC-to-AWS-service
S3 Bucket Policy	Can restrict S3 access to only requests coming via VPC Endpoint (using `aws:sourceVpce` condition)

🏛️ Well-Architected Framework

Pillar	VPC Endpoint Consideration
Operational Excellence	Use Gateway Endpoints by default for S3/DynamoDB — eliminates NAT dependency, simpler traffic path, reduces routes to manage
Security	Interface Endpoints eliminate internet exposure. Endpoint policies add resource-level access control. Use `aws:sourceVpce` in bucket/resource policies to restrict access to endpoint only
Reliability	Interface Endpoints: deploy ENIs in all AZs to avoid AZ-level single point of failure. Gateway Endpoints: add route to all relevant route tables to avoid partial subnet failure
Performance Efficiency	Direct AWS backbone path is lower latency than NAT → internet → AWS service. Gateway Endpoints add zero latency (route table lookup). Interface Endpoints add minimal ENI hop
Cost Optimization	Gateway Endpoints are free — always use for S3/DynamoDB. Interface Endpoints: calculate whether saved NAT cost exceeds endpoint hourly + data cost
Sustainability	Shorter network path (no NAT, no internet hop) = lower energy use per packet

---

📋 4 Domains

Domain	VPC Endpoint Relevance
Design Solutions for Organizational Complexity	Centralized endpoints in shared services VPC accessible via TGW (PrivateLink to internal services). Endpoint policies enforced per account
Design for New Solutions	Default to private endpoints for all new services — S3/DynamoDB via Gateway, everything else via Interface
Continuous Improvement	Audit NAT Gateway usage — any traffic to AWS services should be redirected to VPC Endpoints. Use VPC Flow Logs to identify S3/DynamoDB traffic leaving via NAT
Workload Migration and Modernization	Migrating on-premises apps to AWS: Interface Endpoints enable private access to AWS services without changing security posture

---

⚠️ Limitations

- Gateway Endpoints: S3 and DynamoDB **only**
- Gateway Endpoints: **not accessible from on-premises** (route only in VPC route table)
- Interface Endpoints: **regional only** — no cross-region access
- Interface Endpoint **private DNS** conflicts with custom DNS servers that forward `.amazonaws.com` differently
- Max 50 Interface Endpoints per VPC (soft limit, can be increased)
- Gateway Endpoint: one endpoint per service per region per VPC
- Interface Endpoints do **not** support IPv6 for all services (check service-specific docs)

🗄️ Chapter 8: Amazon Aurora

📖 Overview

Amazon Aurora is AWS's cloud-native relational database engine — a fully managed, MySQL and PostgreSQL-compatible RDBMS designed from the ground up to exploit distributed cloud storage. It delivers up to 5× the throughput of standard MySQL and 3× of PostgreSQL on the same hardware, while being fully compatible with those engines at the wire-protocol level.

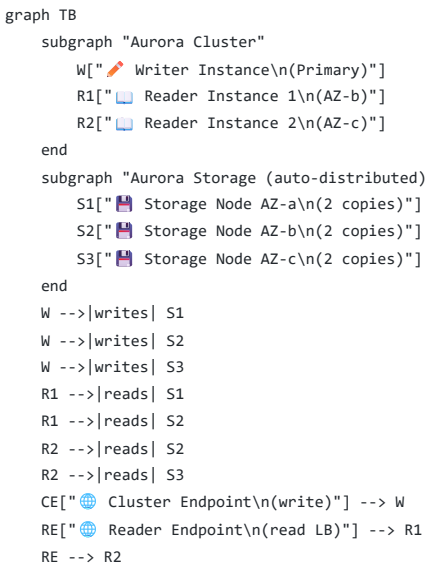
> 🔑 **Exam key:** Aurora is NOT a separate engine — it is MySQL or PostgreSQL with a reengineered storage layer. Applications connecting to Aurora use the same drivers, same SQL, same tools as vanilla MySQL/PostgreSQL.

Real-life analogy: Think of Aurora as MySQL/PostgreSQL that had its storage layer swapped from a single-server disk to a distributed SAN spread across 3 AZs — the query engine stayed the same, but the storage became massively parallel and self-healing.

Scope: Regional service. Aurora Global Database extends to up to 5 secondary regions.

🏗️ Architecture

Aurora's core innovation is the **separation of compute from storage**. The storage layer is a **shared distributed volume** replicated 6 ways across 3 AZs — the DB instances are just compute heads reading/writing to this shared volume.



Key storage facts:

- **6 copies** of data across 3 AZs (2 per AZ)
- Writes acknowledged after **4/6 quorum**
- Reads use **3/6 quorum**
- Can tolerate loss of 2 copies for writes, 3 copies for reads
- Storage auto-grows in **10 GB increments** up to **128 TiB** (no pre-provisioning needed)
- **No I/O operations visible to the compute layer** — storage handles its own replication

---

🔧 Variants & Editions

Variant	Engine Compatibility	Compute Model	Use Case
Aurora MySQL	MySQL 5.7 / 8.0	Provisioned instances	MySQL workloads
Aurora PostgreSQL	PostgreSQL 13–16	Provisioned instances	PostgreSQL workloads
Aurora Serverless v1	MySQL 5.6/5.7, PG 10/11	ACU auto-scales	Intermittent/unpredictable load
Aurora Serverless v2	MySQL 8.0, PG 13–16	Fine-grained ACU scaling	Variable load, dev/test, prod
Aurora Global Database	MySQL or PostgreSQL	Multi-region	DR, global reads
Aurora Multi-Master	MySQL 5.6 only	Multiple writers	Write HA (deprecated, limited)
Babelfish for Aurora PostgreSQL	T-SQL wire protocol	Provisioned	SQL Server migration

> 🔑 **Exam key:** Serverless v2 scales in fine-grained increments (0.5 ACU steps) while v1 had coarse doubling steps and a cold-start gap. Serverless v2 is the current recommendation.

🔗 Endpoints

Endpoint Type	Points To	Use Case
Cluster Endpoint	Current writer instance	All writes
Reader Endpoint	Load-balanced across all readers	Read scaling
Instance Endpoint	Specific instance	Diagnostics, pinned connections
Custom Endpoint	User-defined subset of instances	Tiered reader pools (large vs small)

🔒 Connectivity & Security

- Deployed inside a **VPC** — you choose subnets via a **DB Subnet Group** (must span ≥2 AZs)
- Access controlled by **Security Groups** (not NACLs — SGs are stateful, sufficient)
- **No public internet access by default** — set `Publicly Accessible = true` only for dev/test
- **SSL/TLS in transit:** enforced via `require_secure_transport` parameter (MySQL) or `ssl_mode` (PG); Aurora issues its own TLS cert; download the AWS RDS CA bundle
- **IAM Database Authentication:** passwords replaced by short-lived IAM tokens (15-min expiry); uses `AmazonRDSIAMAuth` policy; no password management needed

---

🔐 Encryption Deep-Dive

Layer	Mechanism	Key Management
At-rest (storage)	AES-256, AWS KMS	AWS-managed key (`aws/rds`) or CMK
At-rest (snapshots)	Same KMS key as source cluster	Snapshots inherit cluster key
At-rest (automated backups)	Same KMS key	Cannot change key after creation
In-transit	SSL/TLS 1.2+	AWS-managed cert (rotate with `rds-ca-2019`)
Cross-account snapshot share	Re-encrypt with target account CMK	Must share CMK with target account

> ⚠️ **Encryption cannot be enabled on an existing unencrypted Aurora cluster.** Workaround: take unencrypted snapshot → copy snapshot with encryption enabled → restore from encrypted snapshot. This is a one-time migration.

> 🔑 **Exam key:** Encryption must be enabled at cluster creation. Once encrypted, all replicas, snapshots, and restored clusters are also encrypted with the same key.

📈 Scalability & HA

Compute scaling:

- **Vertical:** Change instance class (requires brief failover for writer change)
- **Horizontal read scaling:** Add up to **15 Aurora Replicas** (vs 5 for RDS)
- **Aurora Serverless v2:** Scales from 0.5 to 128 ACUs in <1 second; writer AND readers scale independently

Storage scaling:

- Automatic — grows in 10 GB steps up to 128 TiB; no manual intervention; storage never shrinks (you pay for high-water-mark)

Connection scaling:

- **RDS Proxy** in front of Aurora: pools connections, reduces connection overhead for Lambda-heavy workloads; maintains a persistent pool to Aurora; failover-transparent to applications

---

🏢 Multi-AZ

Aurora Multi-AZ is **inherent and automatic** — the storage layer always maintains 6 copies across 3 AZs regardless of whether you have reader instances. This is architecturally different from RDS Multi-AZ.

Aspect	Aurora	RDS Multi-AZ
Storage replication	Always 6 copies / 3 AZs (built-in)	Synchronous to standby (1 copy)
Standby serves reads?	Yes (Aurora Replicas = read traffic)	No (standby is passive)
Failover time	~30 seconds (promotes existing replica)	~60-120 seconds (standby promotion)
Failover mechanism	DNS CNAME flip to existing reader	DNS flip to standby
Cost of HA	Zero storage overhead (built-in)	~2× storage cost

> 🔑 **Exam key:** Aurora always has 3-AZ storage even with a single instance. To also get compute HA, add at least one Aurora Replica in a different AZ — failover promotes it in ~30 seconds.

📖 Read Replicas

Feature	Aurora Replica	RDS Read Replica
Max count	15	5 (5 per source, up to 5 chains)
Replication lag	Milliseconds (shared storage)	Seconds–minutes (log shipping)
Promotion to primary	~30 sec (no data copy needed)	Minutes (must catch up)
Cross-region	Via Aurora Global Database	Native cross-region replicas
Read traffic routing	Automatic via Reader Endpoint	Manual per-endpoint
Cost of replica	Instance cost only (shared storage)	Instance + storage cost

> 🔑 **Exam key:** Aurora Replicas share the same storage volume as the writer — replication lag is near-zero because there's no log-shipping, just shared reads. This enables sub-30-second failovers.

🌍 Aurora Global Database

A **Global Database** spans up to **5 AWS Regions** (1 primary + 4 secondary):

```
graph LR
    P["🌎 Primary Region\nus-east-1\n(Writer + Readers)"]
    S1["🌎 Secondary Region\neu-west-1\n(Read-only Replicas)"]
    S2["🌎 Secondary Region\nap-southeast-1\n(Read-only Replicas)"]
    P -->|"< 1 sec replication\n(storage-level)"| S1
    P -->|"< 1 sec replication"| S2
```

Feature	Value
Replication lag	Typically < 1 second
Secondary regions	Up to 4
Reader instances per secondary	Up to 16
Managed Planned Failover (RPO)	~0 (synchronous before switch)
Unmanaged Failover (RTO)	< 1 minute
Use case	Global low-latency reads, cross-region DR

> 🔑 **Exam key:** Global Database replication is **storage-level**, not log-based — it does NOT go through the database engine. This is why lag is <1 second and failover is fast. DynamoDB Global Tables is active-active; Aurora Global Database is active-passive (writes only in primary region).

⏱️ RPO / RTO Table

Deployment	RPO	RTO	Notes
Single instance, no replica	Hours (last backup)	Hours (restore from backup)	Not HA — dev/test only
Single instance + automated backup	5 min (PITR)	20-40 min (restore)	Backup-only recovery
Multi-AZ (replica in another AZ)	~0 (shared storage)	~30 sec (replica promotion)	Standard HA
Aurora Global Database (planned failover)	~0	~1 min	Best RTO for regional failure
Aurora Global Database (unmanaged failover)	< 1 sec	< 1 min	Regional disaster
Serverless v2	~0 (if replicas)	~30 sec	Same as provisioned with replicas

💾 Backup & Restore

- **Automated backups:** Enabled by default; retention 1–35 days; stored in S3 (not visible to you); continuous backup to S3 (not snapshot-based)
- **Point-In-Time Recovery (PITR):** Restore to any second within retention window; creates a **new cluster** (not in-place restore)
- **Manual snapshots:** Retained until explicitly deleted; survive cluster deletion; can share cross-account; can copy cross-region
- **Backtrack:** Aurora MySQL only — rewind the cluster to a previous point **without restoring** (in-place, near-instant); backtrack window up to 72 hours; does NOT replace PITR; causes brief cluster pause
- **Database cloning:** Create a full copy of Aurora cluster using copy-on-write — near-instant, cost-efficient (only changed data stored separately); great for test environments

> 🔑 **Exam key:** Backtrack ≠ PITR. Backtrack is in-place rewind (Aurora MySQL only, brief outage). PITR creates a new cluster. Cloning uses copy-on-write — no data duplication cost.

🔄 Migration

From MySQL/PostgreSQL on-premises or RDS → Aurora

Method	When to Use	Downtime
mysqldump / pg_dump + restore	Small DBs, simple schema	Yes — full downtime
AWS DMS (homogeneous)	Any size, minimal downtime	Minimal (CDC replication)
RDS Snapshot → Aurora	RDS MySQL/PG → Aurora (same engine)	Brief (restore time)
Aurora native import	From MySQL bin log	Minimal
AWS SCT	Schema conversion (heterogeneous)	Pre-migration step only

AWS DMS for Aurora

- **Full Load + CDC:** DMS migrates existing data, then streams ongoing changes until cutover
- **Aurora as source:** Supported; DMS reads binary log (MySQL) or logical replication (PG)
- **Aurora as target:** Supported; DMS writes via standard MySQL/PG wire protocol
- **Homogeneous migration:** MySQL→Aurora MySQL — DMS handles without SCT (same engine family)
- **Heterogeneous migration:** Oracle/SQL Server→Aurora — requires **AWS Schema Conversion Tool (SCT)** first to convert schema and stored procedures, then DMS for data

> 🔑 **Exam key:** SCT converts the **schema and application code** (stored procs, functions, triggers). DMS moves the **data**. For Oracle→Aurora PostgreSQL: SCT first, DMS second.

🔗 Integration with Other AWS Services

Service	Integration	Purpose
RDS Proxy	Sits in front of Aurora	Connection pooling, failover transparency
Lambda	Data API (HTTP) or direct VPC connection	Serverless → Aurora access
S3	`LOAD DATA FROM S3` / `SELECT INTO OUTFILE S3`	Bulk load/export
IAM	IAM DB authentication	Password-free access
Secrets Manager	Auto-rotation of master password	Credential management
CloudWatch	Metrics, Enhanced Monitoring, Performance Insights	Observability
EventBridge	RDS Events (failover, backup, etc.)	Automated responses
CloudTrail	API call audit	Governance
AWS Backup	Policy-driven backup	Cross-account, cross-region copies
Kinesis Data Streams	Aurora activity streams	Near-real-time DB audit

---

⚖️ Comparison Table

Feature	Aurora	RDS MySQL/PG	Oracle RDS	Self-managed MySQL	Azure SQL
Storage	Distributed, auto-grow	EBS, manual sizing	EBS, manual	Local disk	Managed
Max storage	128 TiB	64 TiB	64 TiB	OS limit	4 TiB (General Purpose)
Read replicas	15	5	5 (Active Data Guard)	Manual	~4 (Business Critical)
Replication lag	ms	Seconds	Seconds	Variable	ms (Business Critical)
Serverless	Yes (v2)	No	No	No	Yes (Serverless v2)
Multi-region	Global Database	Cross-region replica	Cross-region replica	Manual	Auto-failover groups
Backtrack	MySQL only	No	No	No	No
Wire-protocol compat	MySQL/PG	MySQL/PG	Oracle	MySQL	T-SQL

⚠️ Limitations

Limit	Value
Max storage	128 TiB
Max Aurora Replicas per cluster	15
Max Global Database secondary regions	4
Backtrack window	72 hours (Aurora MySQL only)
Serverless v2 min ACU	0.5 ACU
Serverless v2 max ACU	128 ACU
Max DB instance classes	r6g, r7g, r8g, x2g families
Parameter group changes	Dynamic (no reboot) or static (requires reboot)
Encryption	Cannot enable on existing unencrypted cluster
Multi-Master	MySQL 5.6 only, being retired

🏛️ Well-Architected Framework

Pillar	Aurora Considerations
Operational Excellence	Use Parameter Groups for config-as-code; enable Enhanced Monitoring + Performance Insights; use Aurora Events + EventBridge for automated ops; tag clusters for cost allocation
Security	Always encrypt at rest with CMK; enforce SSL/TLS in transit; use IAM DB Authentication; deploy in private subnets; use Secrets Manager for credential rotation; enable Activity Streams for audit
Reliability	Deploy at least one Aurora Replica in a different AZ for compute HA; use Aurora Global Database for regional DR; use RDS Proxy for connection resilience; test failover with `failover-db-cluster` API
Performance Efficiency	Use Aurora Serverless v2 for variable workloads; use Reader Endpoint for read scaling; use custom endpoints to separate OLTP from analytics replicas; use Performance Insights to identify bottlenecks
Cost Optimization	Aurora Serverless v2 for dev/test (scale to zero); use Aurora I/O-Optimized for high I/O workloads (predictable pricing); stop/start dev clusters; use reserved instances for predictable prod workloads
Sustainability	Serverless v2 scales to minimum when idle; shared storage avoids per-replica storage duplication; Graviton (r6g, r7g) instances offer better performance per watt

---

📐 4 Exam Domains

Domain 1 — Design Solutions for Organizational Complexity Aurora supports multi-account patterns via RAM (share Aurora clusters), cross-account snapshot sharing, and Aurora Global Database for multi-region org deployments. Use SCPs to enforce encryption requirements across org.

Domain 2 — Design for New Solutions For new cloud-native applications: Aurora Serverless v2 for variable/unknown load. Aurora PostgreSQL with IAM auth + Secrets Manager for zero-credential-management. Aurora Global Database for global applications requiring <1 second cross-region read lag.

Domain 3 — Continuous Improvement for Existing Solutions Migrate from RDS to Aurora for better read scaling (15 replicas vs 5), faster failover (30s vs 120s), and automatic storage growth. Enable Performance Insights to identify slow queries. Use Backtrack to recover from accidental deletes without full cluster restore.

Domain 4 — Accelerate Workload Migration and Modernization Oracle/SQL Server → Aurora via SCT + DMS. Use Babelfish for Aurora PostgreSQL to run T-SQL workloads with minimal application changes. MySQL/PG RDS → Aurora via snapshot restore (minimal downtime).

🐘 Chapter 9: Amazon RDS

📖 Overview

Amazon Relational Database Service (RDS) is AWS's managed relational database service that handles provisioning, patching, backups, and failover for traditional RDBMS engines. Unlike Aurora (which re-engineered the storage layer), RDS runs the **standard, unmodified community or vendor engines** on managed EC2 + EBS infrastructure.

> 🔑 **Exam key:** RDS = managed lift-and-shift for existing relational engines. Aurora = cloud-native reengineered MySQL/PG. Choose RDS when you need a specific engine version, Oracle licensing, SQL Server features, or a non-Aurora engine.

Scope: Regional — DB instances are AZ-specific; Multi-AZ standby is in a different AZ of the same region.

Supported engines:

- Amazon Aurora (MySQL / PostgreSQL) — separate chapter
- MySQL (5.7, 8.0)
- PostgreSQL (12–16)
- MariaDB (10.4–10.11)
- Oracle (19c, 21c — BYOL or License Included)
- Microsoft SQL Server (2016–2022 — License Included or BYOL)

📐 Architecture

```
graph TB
    subgraph "RDS Multi-AZ Deployment"
        P["🖥️ Primary Instance\nAZ-a (EC2 + EBS)"]
        S["🖥️ Standby Instance\nAZ-b (EC2 + EBS)"]
        P -->|"Synchronous replication\n(block-level)"| S
    end
    subgraph "Read Replicas (optional)"
        R1["📖 Read Replica 1\nAZ-b or AZ-c"]
        R2["📖 Read Replica 2\nCross-region"]
    end
    P -->|"Async replication\n(engine log-based)"| R1
    P -->|"Async replication"| R2
    App["🌐 Application"] -->|"Write"| PE["Primary Endpoint"]
    App -->|"Read"| RE["Replica Endpoint"]
    PE --> P
    RE --> R1
```

Key components:

- **DB Instance:** EC2 compute running the DB engine
- **EBS Storage:** gp2, gp3, io1, io2 — you choose and manage size
- **DB Subnet Group:** spans ≥2 AZs; RDS places primary in one AZ, standby in another
- **Multi-AZ Standby:** synchronous replication; not readable; automatic failover
- **Read Replicas:** async replication; readable; manual promotion

---

🗂️ Storage Types

Storage Type	IOPS	Use Case	Notes
gp2	3 IOPS/GB baseline, burst to 3000	General	Legacy; prefer gp3
gp3	3000 IOPS baseline, up to 16000	General	Decouple IOPS from size
io1	Up to 40,000 IOPS	High-perf OLTP	More expensive
io2	Up to 64,000 IOPS, higher durability	Mission-critical	Best durability (99.999%)
Magnetic	Low	Legacy only	Do not use

> 🔑 **Exam key:** gp3 allows you to provision IOPS independently of storage size — more cost-efficient than gp2 for large DBs that don't need proportional IOPS. For >40,000 IOPS needs, use Aurora or switch to io2.

🔒 Connectivity & Security

- Must be in a **VPC** with a **DB Subnet Group** (≥2 AZs)
- **Security Groups** control access — no public access by default
- **SSL/TLS enforced** via parameter `rds.force_ssl=1` (PG) or `require_secure_transport=ON` (MySQL)
- **IAM DB Authentication:** supported for MySQL and PostgreSQL; token valid 15 minutes
- **Kerberos authentication:** supported for Oracle and SQL Server (via AWS Managed Microsoft AD)
- **VPC Peering / PrivateLink:** access from other VPCs; RDS doesn't support PrivateLink directly — use RDS Proxy or an NLB in front

---

🔐 Encryption Deep-Dive

Layer	Mechanism	Notes
At-rest	AES-256 via KMS	Must enable at creation; cannot add later
Snapshots	Inherit source DB key	Can re-encrypt on copy
Automated backups	Same key as DB	
Read replicas	Must also be encrypted if source is	Cross-region replica: needs CMK in target region
In-transit	SSL/TLS	Download RDS CA cert from AWS
Oracle TDE	Native Oracle TDE supported	Double-encryption possible (TDE + RDS encryption)
SQL Server TDE	Native SQL Server TDE supported	

> ⚠️ **Cannot encrypt an existing unencrypted RDS instance directly.** Workaround: snapshot → copy with encryption → restore.

📈 Scalability & HA

Vertical scaling: Change DB instance class — requires brief downtime (1-2 min for Multi-AZ: failover to standby first, upgrade standby, failover back).

Storage scaling: Auto-scaling enabled via `Max Allocated Storage` parameter — RDS expands automatically when free space < 10% AND free space < 5 GiB AND low storage lasts >5 minutes.

Read scaling: Add Read Replicas (up to 5 per source for MySQL/PG/MariaDB).

Connection scaling: RDS Proxy — pools and manages connections; reduces connection churn; IAM auth; automatic failover; supported for MySQL, PG, SQL Server.

🏢 Multi-AZ

Multi-AZ DB Instance (classic): One standby in a different AZ — synchronous block-level replication. Standby is **not readable** — purely for HA.

Multi-AZ DB Cluster (new, 2022): Two readable standby instances across 3 AZs — synchronous replication. Standbys CAN serve reads. Faster failover (~35 sec). Available for MySQL 8.0 and PostgreSQL 13+.

Feature	Multi-AZ Instance	Multi-AZ Cluster
Standby count	1	2
Standby readable	No	Yes
Failover time	60-120 sec	~35 sec
Engines	All	MySQL 8.0, PG 13+
Storage	EBS per instance	Local NVMe + EBS
Cost	~2×	~3×

> 🔑 **Exam key:** Multi-AZ standby CANNOT serve reads for the classic deployment. If you want readable standbys on RDS (non-Aurora), use Multi-AZ DB Cluster or add Read Replicas.

Failover triggers:

- Primary AZ or DB instance failure
- DB instance class change
- OS patching
- Manual `reboot with failover` API call

Failover mechanism: DNS CNAME for the DB endpoint flips to the standby's IP. Applications must use the endpoint DNS name (not IP) to benefit from automatic failover.

📖 Read Replicas

Feature	RDS Read Replica
Max replicas	5 per source (MySQL/PG/MariaDB), chaining supported
Replication	Async, engine log-based (binlog / WAL streaming)
Replication lag	Seconds to minutes
Cross-region	Yes (MySQL, PG, MariaDB)
Promotion	Manual; creates independent DB instance
Use after promotion	No longer a replica; standalone DB
Cross-region replica encryption	Needs CMK in target region
Oracle/SQL Server	No Read Replicas (use Active Data Guard for Oracle separately)

> 🔑 **Exam key:** RDS Read Replicas use async replication — there is always some lag. For zero-lag read scaling, use Aurora Replicas. Oracle RDS does NOT support read replicas natively on RDS.

⏱️ RPO / RTO Table

Deployment	RPO	RTO
Single-AZ, no replica	Up to 1 day (last backup)	20-40 min (restore)
Single-AZ + PITR	5 min	20-40 min
Multi-AZ (Instance)	~0 (sync replication)	60-120 sec
Multi-AZ (Cluster)	~0	~35 sec
Cross-region Read Replica (promoted)	Seconds-minutes (async lag)	Minutes (manual promotion)
Cross-region Read Replica + automation	Seconds	5-10 min (automated promotion via Lambda)

💾 Backup & Restore

- **Automated backups:** Daily snapshot + transaction logs; retention 0–35 days; stored in S3 (managed by AWS)
- **PITR:** Restore to any second within retention window → creates **new DB instance**
- **Manual snapshots:** Survive instance deletion; shareable cross-account; copyable cross-region; no expiry
- **Snapshot restore:** Always creates a new DB instance (cannot restore in-place)
- **AWS Backup integration:** Policy-driven backup; cross-account and cross-region copies

> 🔑 **Exam key:** Setting backup retention to 0 disables automated backups entirely (and disables PITR). Manual snapshots are always retained regardless of retention setting.

🔄 Migration

Tools

Tool	Purpose	When to Use
AWS DMS	Data migration with CDC	Minimal-downtime migrations, heterogeneous
AWS SCT	Schema + code conversion	Heterogeneous migrations (Oracle→PG, SQL Server→MySQL)
mysqldump / pg_dump	Native export/import	Small DBs, offline migration
Oracle Data Pump	Oracle export/import	Oracle → Oracle RDS
SQL Server BACPAC	SQL Server backup format	SQL Server → RDS SQL Server
RDS Snapshot Restore	Same-engine restore	RDS → RDS (same engine)
AWS MGN	Server migration (not DB-specific)	Lift-and-shift entire server

DMS Key Concepts

- **Replication instance:** EC2 instance running DMS engine — choose size based on data volume
- **Source endpoint:** Connection to source DB (on-premises, EC2, RDS, etc.)
- **Target endpoint:** Connection to target DB (RDS, Aurora, S3, Redshift, etc.)
- **Task types:** Full Load, CDC only, Full Load + CDC
- **Supported sources:** Oracle, SQL Server, MySQL, PostgreSQL, MongoDB, SAP, IBM Db2, and more
- **Ongoing replication:** DMS can run continuously post-migration for live sync

SCT Key Concepts

- **Schema conversion:** DDL (tables, views, indexes) + stored procedures, functions, triggers

- **Conversion report:** Shows automated conversion rate + manual effort items
- **Extension pack:** DMS-provided replacement functions for unsupported source engine features
- **Typical flow:** SCT converts schema → deploy to target → DMS migrates data → validate → cutover

---

🔗 Integration with Other AWS Services

Service	Purpose
RDS Proxy	Connection pooling, IAM auth, failover transparency
Secrets Manager	Auto-rotate DB credentials
IAM	DB authentication tokens
CloudWatch	Metrics, alarms, Enhanced Monitoring (per-process OS metrics)
Performance Insights	Query-level DB performance analysis
EventBridge	Respond to RDS events (failover, backup, maintenance)
Lambda	Trigger Lambda on RDS events
S3	Import/export data (MySQL: `LOAD DATA FROM S3`)
CloudTrail	API audit
AWS Backup	Centralized backup policy
VPC	Network isolation

⚖️ Comparison: RDS vs Aurora vs Self-Managed

Feature	RDS	Aurora	Self-Managed on EC2
Engine	Standard community/vendor	MySQL/PG compat, reengineered	Any
Management	Fully managed	Fully managed	You manage everything
Read replicas	Up to 5	Up to 15	Manual
Replication lag	Seconds	Milliseconds	Variable
Storage	EBS (manual size)	Auto-grow to 128 TiB	You manage
Failover	60-120 sec	~30 sec	Manual
Oracle support	Yes (BYOL or LI)	No	Yes
SQL Server	Yes	No (Babelfish only)	Yes
Serverless	No	Yes (v2)	No
Backtrack	No	MySQL only	No

⚠️ Limitations

Limit	Value
Max storage (gp3/io1)	64 TiB
Max IOPS (io2)	64,000
Read replicas per source	5 (MySQL/PG/MariaDB)
Backup retention	0-35 days
DB parameter group changes	Dynamic or static (reboot required)
Multi-AZ failover time	60-120 sec (instance), ~35 sec (cluster)
Oracle on RDS	No RAC, no Data Guard (use native DG on EC2 instead)
SQL Server	No cross-region read replicas
Encryption	Cannot add to existing unencrypted instance

🏛️ Well-Architected Framework

Pillar	RDS Considerations
Operational Excellence	Use Parameter Groups as code; automate backups via AWS Backup; use Performance Insights for query analysis; use RDS Events + EventBridge for automated remediation
Security	Enable encryption at creation (CMK preferred); enforce SSL; use IAM DB auth; keep instances in private subnets; use Secrets Manager for rotation; enable Enhanced Monitoring
Reliability	Always use Multi-AZ for production; use RDS Proxy for connection resilience; test failover regularly; set appropriate backup retention; use cross-region read replicas for DR
Performance Efficiency	Right-size instances with CloudWatch metrics; use gp3 for cost-effective IOPS; add read replicas for read-heavy workloads; use RDS Proxy to reduce connection overhead for Lambda
Cost Optimization	Use Reserved Instances (1- or 3-year) for predictable workloads; stop dev/test instances nights/weekends; use gp3 (cheaper than io1 for same IOPS); consider Aurora for MySQL/PG (better performance/cost at scale)
Sustainability	Graviton-based instances available (r6g, m6g); right-size to avoid idle over-provisioning; stop non-production instances when not in use

---

📐 4 Exam Domains

Domain 1 — Design Solutions for Organizational Complexity RDS supports multi-account patterns via cross-account snapshot sharing, AWS Backup cross-account policies, and RDS Proxy for centralizing connection management. Use SCPs to enforce encryption across org.

Domain 2 — Design for New Solutions Choose RDS for Oracle (BYOL cost control) or SQL Server workloads requiring specific Windows features. Use Multi-AZ DB Cluster for near-Aurora HA on standard MySQL/PG. Use RDS Proxy + IAM auth for serverless architectures.

Domain 3 — Continuous Improvement for Existing Solutions Migrate from single-AZ to Multi-AZ with zero downtime (AWS modifies in-place). Enable Performance Insights to identify top SQL queries. Enable storage auto-scaling to prevent capacity alerts. Enable PITR by setting backup retention > 0.

Domain 4 — Accelerate Workload Migration and Modernization Oracle/SQL Server on-premises → RDS: SCT converts schema, DMS migrates data with CDC for minimal downtime. SQL Server → Aurora PostgreSQL via Babelfish for T-SQL compatibility. Consider migrating MySQL/PG RDS → Aurora for better scaling.

⚡ Chapter 10: Amazon DynamoDB

📖 Overview

Amazon DynamoDB is AWS's fully managed, serverless, NoSQL key-value and document database. It delivers single-digit millisecond performance at any scale — from a few requests per second to millions per second — with no servers to manage, no cluster to size, and no schema migrations.

> 🔑 **Exam key:** DynamoDB is the answer when you need: massive scale, single-digit ms latency, flexible/schemaless data, serverless, or key-value/document access patterns. It is NOT a relational DB — no joins, no complex SQL, no ACID across multiple tables (only single-item/transaction-level ACID).

Real-life analogy: A hashmap in the cloud — extremely fast lookups by key, but no support for "scan all rows and compute a sum" without designing for it.

Scope: Regional service. Global Tables extend to multiple regions (active-active).

Open-source equivalent: Apache Cassandra (wide-column, similar distribution model), Redis (key-value, in-memory), MongoDB (document).

📐 Architecture & Data Model

```
graph TB
    subgraph "DynamoDB Table"
        PK["🔑 Partition Key (Hash)\nRequired — determines partition"]
        SK["🗂️ Sort Key (Range)\nOptional — enables range queries"]
        ATTR["📋 Attributes\nArbitrary, schemaless per item"]
    end
    subgraph "Storage Layer"
        P1["Partition 1\n(AZ-a, AZ-b, AZ-c replicas)"]
        P2["Partition 2\n(AZ-a, AZ-b, AZ-c replicas)"]
        P3["Partition 3\n(AZ-a, AZ-b, AZ-c replicas)"]
    end
    PK --> P1
    PK --> P2
    PK --> P3
```

Core concepts:

- **Table:** Collection of items (rows)
- **Item:** A record; max 400 KB
- **Primary Key:** Partition key alone OR partition key + sort key (composite)
- **Partition Key:** Determines which physical partition stores the item (hash function)
- **Sort Key:** Allows range queries within a partition
- **Attributes:** Schemaless — each item can have different attributes (only PK required)
- **DynamoDB is inherently multi-AZ** — every partition has 3 replicas across 3 AZs; there is no "Multi-AZ option" to enable, it is always on

---

🔑 Multi-AZ (Built-in)

DynamoDB does **not have a Multi-AZ toggle** — it is **always multi-AZ**. Every write is synchronously replicated to at least 2 of 3 AZs before acknowledged (with strong consistency), or to a majority partition for eventual consistency.

Consistency Model	Behavior	Cost
Eventually Consistent Read	Returns data from any replica (may be slightly stale)	0.5 RCU per 4 KB
Strongly Consistent Read	Returns most recent write (reads from leader)	1 RCU per 4 KB
Transactional Read	ACID across multiple items	2 RCU per 4 KB
Transactional Write	ACID across multiple items	2 WCU per 1 KB

> 🔑 **Exam key:** DynamoDB Global Tables is NOT the same as Multi-AZ. Multi-AZ is same-region 3-AZ replication (always on). Global Tables is active-active cross-region replication (optional, separate feature).

📊 Capacity Modes

Mode	How It Works	Best For	Limits
Provisioned	Set RCU + WCU in advance; auto-scaling adjusts within set range	Predictable, steady workloads	Manual capacity planning
On-Demand	Pay per request; no capacity planning; scales instantly	Spiky, unpredictable workloads	2× previous peak instantly
Reserved Capacity	1- or 3-year commitment on provisioned RCU/WCU	Cost reduction for steady state	Provisioned mode only

> 🔑 **Exam key:** On-Demand is more expensive per request than Provisioned but eliminates throttling for truly unpredictable workloads. Auto-scaling on Provisioned mode can handle gradual spikes but not instant traffic surges (takes minutes to scale). For flash sales or unknown load: On-Demand.

🗂️ Indexes

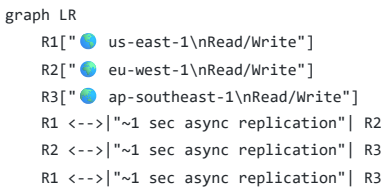
Index Type	Key	Location	Use Case
Local Secondary Index (LSI)	Same PK, different SK	Same partition	Query on alternate sort key within a partition
Global Secondary Index (GSI)	Different PK (and optional SK)	Separate partition space	Query on any attribute as if it were PK

- **LSI:** Must be defined at table creation; up to 5 per table; uses table's provisioned capacity; strong consistency supported
- **GSI:** Can be added/deleted after creation; up to 20 per table; has its own RCU/WCU; only eventually consistent; GSI over-provisioning is common exam trap

> 🔑 **Exam key:** GSI has its own capacity — if GSI is throttled, writes to the base table that update the GSI will also be throttled. Always provision GSI capacity separately.

🌍 Global Tables (Cross-Region Active-Active)

DynamoDB Global Tables provide **active-active multi-region replication** — you can write to any region and reads/writes are replicated to all other regions within ~1 second.



Feature	Value
Replication	Async, ~<1 sec typical
Consistency across regions	Eventual (last-writer-wins conflict resolution)
Conflict resolution	Last-writer-wins (based on timestamp)
Max regions	No hard limit
Requirement	Table must be empty at Global Table creation OR use Version 2019

> 🔑 **Exam key:** DynamoDB Global Tables = active-active (write to any region). Aurora Global Database = active-passive (write to primary only). This is a critical exam distinction.

DynamoDB Accelerator (DAX)

DAX is an in-memory cache for DynamoDB — fully managed, cluster-based, microsecond read latency.

Feature	Value
Latency	Microseconds (vs ms for DynamoDB)
API	DynamoDB-compatible (same API, no app code change)
Cache type	Write-through for writes; item cache + query cache for reads
HA	Multi-AZ cluster (primary + replicas)
Not suitable for	Write-heavy workloads; strongly consistent reads; Scan-heavy workloads
VPC	Deployed inside your VPC

> 🔑 **Exam key:** DAX is NOT suitable if you need strongly consistent reads (DAX always returns cached/eventually consistent data). Use DAX for read-heavy, latency-sensitive workloads (leaderboards, session stores, product catalogs).

RPO / RTO Table

Scenario	RPO	RTO
AZ failure (standard)	~0 (sync replicas)	Automatic, seconds
Region failure (Global Tables)	~1 sec (async replication lag)	Minutes (redirect DNS/app to other region)
Accidental delete (PITR)	Up to 35 days back	Minutes (restore to new table)
Table restore from backup	Last on-demand backup	1-10 min (depends on table size)

Backup & Restore

Type	Description
On-Demand Backup	Full backup at a point in time; stored in DynamoDB; no impact on performance; retained until deleted
PITR (Point-In-Time Recovery)	Enable per table; 35-day rolling window; restore to any second; restores to **new table**
AWS Backup integration	Policy-driven; cross-account; cross-region copy of DynamoDB backups
Export to S3	Export table data to S3 in JSON or ION format without using read capacity

> 🔑 **Exam key:** PITR does not pause table operations — it uses DynamoDB's continuous backup logs. Restore creates a **new table** (not in-place). Export to S3 uses DynamoDB Streams internally, not read capacity.

DynamoDB Streams

DynamoDB Streams captures a time-ordered log of item-level changes (insert, update, delete) — retained for 24 hours.

Use Case	Pattern
Trigger Lambda on changes	Lambda event source mapping to Streams
Cross-region replication (custom)	Streams → Lambda → write to other region
Audit trail	Streams → Kinesis Data Firehose → S3
Search index	Streams → Lambda → OpenSearch
Aggregation/analytics	Streams → Kinesis Data Streams → analytics

> 🔑 **Exam key:** Global Tables uses DynamoDB Streams internally for replication — enabling Streams is a prerequisite for Global Tables.

🔐 Encryption Deep-Dive

Layer	Mechanism	Notes
At-rest	AES-256 via KMS	Always encrypted; choose AWS-owned key, AWS-managed key, or CMK
In-transit	TLS 1.2+	All API calls use HTTPS
VPC Endpoint	Gateway VPC Endpoint for DynamoDB	Traffic stays in AWS network, no internet
Fine-grained access	IAM condition keys (`dynamodb:LeadingKeys`)	Restrict users to their own items

> 🔑 **Exam key:** DynamoDB is ALWAYS encrypted at rest — you cannot create an unencrypted table. The choice is which KMS key to use. AWS-owned key = no cost, no visibility. CMK = full control + cost.

🔄 Migration

Scenario	Tool
On-premises MongoDB → DynamoDB	AWS DMS (source: MongoDB; target: DynamoDB)
Cassandra → DynamoDB	AWS DMS + data modeling redesign
Relational DB → DynamoDB	Redesign data model first; DMS for data transfer
DynamoDB → DynamoDB (new table)	Data Pipeline, Export to S3 + import, DMS
Large migration	AWS DMS with Full Load + CDC

> ⚠️ **DynamoDB migration requires data model redesign.** Relational → NoSQL is not a simple lift-and-shift — you must think about access patterns, denormalization, and partition key design first.

⚠️ Limitations

Limit	Value
Max item size	400 KB
Max partition key size	2 KB
Max sort key size	1 KB
Max attribute name length	64 KB
Max table name length	255 chars
LSI per table	5
GSI per table	20
Max tables per region	2,500 (soft limit)
On-Demand max throughput	40,000 RCU / 40,000 WCU (default, can increase)
DynamoDB Streams retention	24 hours
PITR retention	35 days
Max DynamoDB transaction items	100 items

⚖️ Comparison

Feature	DynamoDB	MongoDB Atlas	Cassandra	Redis
Model	Key-value / document	Document	Wide-column	Key-value (in-memory)
Scale	Unlimited (serverless)	Manual cluster	Manual cluster	Cluster-based
Latency	Single-digit ms	ms	ms	Microseconds
Query language	DynamoDB API, PartiQL	MongoDB query language	CQL	Redis commands
ACID transactions	Single-table, up to 100 items	Yes	Lightweight transactions	No
Global	Global Tables (active-active)	Atlas Global Clusters	Manual	Manual
Fully managed on AWS	Yes	No (Atlas is separate SaaS)	No (Keyspaces is managed)	ElastiCache

---

🏛️ Well-Architected Framework

Pillar	DynamoDB Considerations
Operational Excellence	Use IaC (CloudFormation/CDK) for table definitions; use PartiQL for SQL-like queries; monitor consumed capacity vs provisioned to right-size
Security	Always use CMK for sensitive data; use VPC Gateway Endpoint (no internet traversal); use fine-grained IAM with `dynamodb:LeadingKeys` to restrict item access; enable CloudTrail for API audit
Reliability	Multi-AZ is built-in; enable PITR for accidental delete protection; use Global Tables for multi-region resilience; test Global Tables failover
Performance Efficiency	Design partition key for uniform distribution (avoid hot partitions); use DAX for microsecond reads; use GSIs for alternative access patterns; use On-Demand for spiky workloads
Cost Optimization	Use On-Demand for dev/test; use Provisioned + auto-scaling + Reserved Capacity for production; use TTL to auto-expire old items (free); export to S3 + Athena for analytics instead of Scan
Sustainability	Serverless — pay only for what you use; TTL removes unneeded data; On-Demand scales to zero (near-zero cost) when idle

---

📐 4 Exam Domains

Domain 1 — Design Solutions for Organizational Complexity DynamoDB tables are per-account/per-region. Use AWS Backup policies across org for consistent PITR coverage. IAM resource policies can restrict table access per OU. Global Tables must be set up per region explicitly.

Domain 2 — Design for New Solutions DynamoDB for session stores, shopping carts, real-time leaderboards, IoT telemetry, user profiles — any access-pattern-driven, high-scale, low-latency workload. Use DAX layer for sub-ms reads. Use Streams + Lambda for event-driven architectures.

Domain 3 — Continuous Improvement for Existing Solutions Enable PITR if not already. Convert Scan operations to Query by adding GSI on queried attributes. Move from Provisioned to On-Demand if workload is spiky. Use DAX to offload read capacity. Enable TTL to reduce storage costs.

Domain 4 — Accelerate Workload Migration and Modernization MongoDB/Cassandra on-premises → DynamoDB via DMS + data model redesign. Move session state from ElastiCache (requires manual expiry management) to DynamoDB + TTL (simpler operations). Move relational DB hot-path reads to DynamoDB (NoSQL for speed, RDS for reporting).

📊 Chapter 11: Amazon Redshift

📖 Overview

Amazon Redshift is AWS's fully managed, petabyte-scale cloud data warehouse. It uses a **massively parallel processing (MPP)** architecture with **columnar storage** to execute complex analytical queries (OLAP) across billions of rows in seconds. It is NOT designed for OLTP — it's optimized for read-heavy analytical workloads.

> 🔑 **Exam key:** Redshift = data warehouse (OLAP). RDS/Aurora = transactional database (OLTP). Use Redshift for analytics, reporting, BI tools, aggregations over large datasets. Use RDS for transactional applications.

Real-life analogy: Redshift is like a column-oriented spreadsheet in the cloud — instead of reading entire rows, it reads only the columns needed for your query, making aggregations (SUM, AVG, GROUP BY) massively faster.

Scope: Regional service. Redshift clusters are in a specific region and AZ (single-AZ by default; Multi-AZ option available for RA3 nodes).

Open-source equivalent: Apache Hive, Apache Spark SQL, Greenplum, PostgreSQL (partial wire-protocol compatibility).

🏗️ Architecture

```
graph TB
    subgraph "Redshift Cluster"
        L["🖥️ Leader Node\n(Query Planning & Coordination)"]
        C1["⚙️ Compute Node 1\n(Slice 1 | Slice 2)"]
        C2["⚙️ Compute Node 2\n(Slice 3 | Slice 4)"]
        C3["⚙️ Compute Node 3\n(Slice 5 | Slice 6)"]
    end
    L -->|"Distributes query fragments"| C1
    L -->|"Distributes query fragments"| C2
    L -->|"Distributes query fragments"| C3
    C1 -->|"Returns partial results"| L
    C2 -->|"Returns partial results"| L
    C3 -->|"Returns partial results"| L
    Client["🌐 BI Tool / JDBC/ODBC"] -->|"SQL queries"| L
    S3["🪣 S3 Data Lake"] <-->|"COPY / UNLOAD\nSpectrum"| C1
    S3 <-->|"Spectrum"| C2
```

Key components:

- **Leader Node:** Receives queries, parses, creates execution plan, coordinates compute nodes, aggregates results
- **Compute Nodes:** Execute query slices in parallel; each node has local disk (SSD/HDD)
- **Node Slices:** Each compute node is divided into slices; each slice processes a portion of data
- **Columnar Storage:** Data stored column-by-column (not row-by-row); enables compression and column-skipping
- **Data Distribution Styles:** EVEN, KEY, ALL, AUTO — determines how rows are distributed across slices

---

🔧 Node Types & Cluster Variants

Type	Nodes	Storage	Use Case
RA3 (recommended)	2–128 nodes	Managed storage (S3-backed, local SSD cache)	Decouple compute from storage, scale independently
DC2	2–128 nodes	Local SSD	Fast local I/O, fixed storage
DS2 (legacy)	HDD	Legacy	Migrate to RA3
Redshift Serverless	N/A	Managed	On-demand, no cluster management

Variant	Description
Provisioned Cluster	Fixed cluster size; manual scaling; reserved instances available
Redshift Serverless	Auto-scales RPU; pay per use; no cluster management; cold-start latency
Redshift Spectrum	Query S3 data directly without loading; uses external tables
Redshift ML	Train and run ML models inside Redshift using SQL via SageMaker
Data Sharing	Share live data across clusters/accounts without copying

---

🔒 Connectivity & Security

- Deployed in a **VPC** (Enhanced VPC routing recommended — forces COPY/UNLOAD through VPC)
- **Cluster Subnet Group:** Spans AZs; cluster placed in one AZ (Multi-AZ optional for RA3)
- **Security Groups** control access to port 5439 (default Redshift port)
- **Public accessibility** can be disabled; use **PrivateLink** for private access from other VPCs
- **SSL/TLS** enforced via cluster parameter `require_ssl`
- **IAM authentication** supported; Redshift also has native DB users
- **VPC Endpoints** for S3 access during COPY/UNLOAD (Enhanced VPC routing)

🔐 Encryption Deep-Dive

Layer	Mechanism	Notes
At-rest	AES-256 via KMS or HSM	Enable at cluster creation; cannot change after
Key management	AWS KMS CMK or CloudHSM	HSM for dedicated hardware key storage
In-transit	SSL/TLS	Parameter `require_ssl = true`
Column-level encryption	Client-side via encryption functions	`encrypt()`, `decrypt()` built-in functions
Data masking	Dynamic Data Masking (Serverless/RA3)	Mask sensitive columns per role

> ⚠️ **Encryption cannot be changed after cluster creation.** To encrypt an unencrypted cluster: create new encrypted cluster → UNLOAD data to S3 → COPY to new cluster. Or use snapshot + restore with encryption.

📈 Scalability & HA

Scaling compute:

- **Elastic Resize:** Add/remove nodes in ~15 minutes; cluster remains online (queries paused briefly)
- **Classic Resize:** Change node type or large size changes; cluster goes read-only; takes hours
- **Concurrency Scaling:** Auto-adds transient cluster capacity for read queries during bursts; first 1 hour/day free
- **Redshift Serverless:** Auto-scales RPU (Redshift Processing Units) based on workload

Scaling storage (RA3):

- RA3 decouples compute from storage — storage automatically expands in S3; local SSD is a cache
- Add compute nodes without moving data (data stays in managed S3-backed storage)

🏢 Multi-AZ

By default, a Redshift provisioned cluster is **single-AZ**. Options for HA:

Option	Description	RTO
Multi-AZ (RA3 only)	Active-standby across 2 AZs; automatic failover	~5-10 min
Snapshot + restore	Manual; restore to new cluster in different AZ	30+ min
Redshift Serverless	Managed HA; AWS handles AZ resilience	Minimal

> 🔑 **Exam key:** Redshift is primarily a data warehouse — short-term AZ failures are tolerated because data is already in S3 (for RA3). For OLTP-grade HA, use RDS/Aurora. Redshift's strength is query performance, not sub-minute failover.

📖 Read Replicas / Concurrency Scaling

Redshift does NOT have traditional Read Replicas. Instead:

Feature	How It Works
Concurrency Scaling	Auto-provisions additional read clusters during spikes; routes SELECT queries automatically
Data Sharing (cross-cluster)	Share live data to a separate "consumer" cluster for isolation
Workload Management (WLM)	Queue-based query prioritization; prevents small queries from being stuck behind large ones
Short Query Acceleration (SQA)	Auto-detects short queries and runs them in a dedicated fast queue

---

⏱️ RPO / RTO Table

Scenario	RPO	RTO
Single-AZ cluster failure	0 (no data loss, last snapshot)	30+ min (restore)
RA3 Multi-AZ failover	~0	~5-10 min
Snapshot restore (same region)	Hours (last snapshot)	20-60 min
Cross-region restore	Hours (last cross-region snapshot)	30-90 min
Redshift Serverless AZ failure	Minimal	Seconds–minutes (AWS managed)

💾 Backup & Restore

- **Automated snapshots:** Every 8 hours or every 5 GB of change; default retention 1 day; up to 35 days
- **Manual snapshots:** Retained indefinitely; copy cross-region for DR
- **Cross-region snapshot copy:** Enable on cluster; specify target region; KMS key in target region required
- **Restore:** Creates a new cluster from snapshot (RA3 restores quickly — metadata only, data in S3)
- **AWS Backup:** Supports Redshift automated + manual snapshots

---

🔄 Migration

Source	Method
On-premises data warehouse (Teradata, Netezza)	AWS SCT + DMS or native UNLOAD/COPY via S3
Oracle/SQL Server DW	AWS SCT converts DDL + SQL; DMS or S3 staging for data
S3 data lake	`COPY` command directly from S3
RDS/Aurora	DMS (Redshift as target); or export to S3 → COPY
On-premises flat files	Upload to S3 → `COPY` command

AWS SCT for Redshift:

- Converts source schema to Redshift DDL
- Converts SQL queries, stored procedures (limited)
- Assessment report shows conversion complexity

COPY command (primary bulk load method):

```
COPY tablename FROM 's3://bucket/prefix'
IAM_ROLE 'arn:aws:iam::account:role/RedshiftRole'
FORMAT AS PARQUET;
```

🔗 Integration with Other AWS Services

Service	Integration
S3	COPY (load), UNLOAD (export), Spectrum (query in place)
Glue	ETL pipeline → Redshift; Glue Data Catalog as external schema
Kinesis Data Firehose	Streaming data → Redshift (via S3 staging)
QuickSight	BI dashboard directly on Redshift
SageMaker	Redshift ML trains models via SageMaker Autopilot
Lake Formation	Fine-grained access control on Spectrum tables
DMS	Replicate transactional DB data to Redshift for analytics
EventBridge	Respond to Redshift events (resize, restore, etc.)
CloudWatch	Cluster metrics, query performance monitoring
IAM	Role-based access; IAM DB authentication

⚠️ Limitations

Limit	Value
Max nodes per cluster	128 (RA3)
Max storage per RA3.16xlarge node	64 TB managed storage
Max columns per table	1,600
Max tables per database	100,000
Max concurrent connections	500
Snapshot retention	1-35 days (automated)
Redshift Serverless max RPU	512
COPY file size recommendation	1 MB – 1 GB per file
Spectrum max S3 files per query	100,000

⚖️ Comparison

Feature	Redshift	BigQuery (GCP)	Snowflake	Athena
Model	Columnar MPP	Columnar serverless	Columnar multi-cluster	Query-in-place (S3)
Storage	Managed (RA3) or local	Google-managed	Snowflake-managed	S3
Scaling	Elastic resize	Serverless	Auto-scale	Serverless
Cost model	Instance + storage OR serverless	Per TB scanned	Storage + compute credits	Per TB scanned
SQL dialect	PostgreSQL-based	Standard SQL	ANSI SQL	Presto/Trino
Integration	Deep AWS	Deep GCP	Cloud-agnostic	Deep AWS (S3/Glue)
Best for	AWS-native DW workloads	GCP workloads	Multi-cloud DW	Ad-hoc S3 queries

---

🏛️ Well-Architected Framework

Pillar	Redshift Considerations
Operational Excellence	Use Redshift Advisor for optimization recommendations; monitor WLM queue depths; use automatic table optimization (ATO); automate snapshots with lifecycle policies
Security	Enable encryption at creation (CMK); use Enhanced VPC routing; enforce SSL; use IAM roles for COPY/UNLOAD (not access keys); enable CloudTrail; use column-level security / dynamic data masking
Reliability	Use RA3 Multi-AZ for critical workloads; enable cross-region snapshot copy for DR; use Redshift Serverless for managed HA; test restore procedures
Performance Efficiency	Choose correct distribution key (avoid skew); use sort keys for common filter columns; use compression encodings; use Concurrency Scaling for bursts; use Redshift Spectrum for cold data; use SQA for mixed workloads
Cost Optimization	Use RA3 (decouple compute/storage — scale compute independently); use Concurrency Scaling (first 1 hr/day free); use Redshift Serverless for dev/test; use Reserved Instances for production; use Spectrum for infrequently queried data
Sustainability	RA3 managed storage uses S3 (durable, efficient); Redshift Serverless scales to zero; right-size compute nodes; use compression to reduce storage

---

📐 4 Exam Domains

Domain 1 — Organizational Complexity Use Data Sharing to share live data across accounts/clusters in an org without copying. Use Lake Formation for unified data governance across S3 and Redshift Spectrum. Cross-account snapshot sharing for DR.

Domain 2 — Design for New Solutions New analytics platform: RA3 cluster + Spectrum for S3 data lake + QuickSight for BI. Use Kinesis Firehose for streaming ingestion. Use Redshift Serverless for development/occasional analytics. Use Redshift ML for in-database ML.

Domain 3 — Continuous Improvement Enable Redshift Advisor recommendations. Switch DC2 to RA3 for storage flexibility. Enable Automatic Table Optimization. Add Concurrency Scaling for query bursts. Add Redshift Spectrum to query historical cold data in S3 without loading.

Domain 4 — Migration Migrate Teradata/Netezza on-premises DW to Redshift via SCT + UNLOAD to S3 + COPY. Use DMS for ongoing replication from OLTP sources. Use Glue ETL for transformation during migration.

🔗 Chapter 12: Amazon Neptune

📖 Overview

Amazon Neptune is AWS's fully managed graph database service. It supports two popular graph query languages: **Gremlin** (property graph / Apache TinkerPop) and **SPARQL** (RDF graph / W3C standard). Neptune is purpose-built for applications with highly connected data where relationships between entities are as important as the entities themselves.

> 🔑 **Exam key:** Neptune = graph database. Use when relationships are first-class data (social networks, fraud detection, knowledge graphs, recommendation engines). Do NOT use Neptune for relational workloads or key-value patterns.

Real-life analogy: A graph database is like a map of roads and cities — not just storing cities (nodes) and roads (edges), but making it fast to answer "what's the shortest path?" or "which cities are reachable in 2 hops?"

Scope: Regional service. Multi-AZ supported.

Open-source equivalents: Neo4j (property graph), Apache Jena (RDF), JanusGraph (TinkerPop).

🏗️ Architecture

Neptune shares the same **distributed storage architecture as Aurora** — 6 copies of data across 3 AZs, storage auto-grows, quorum-based reads/writes.

```
graph TB
    subgraph "Neptune Cluster"
        W["🖍️ Primary Instance\n(Reads + Writes)"]
        R1["📖 Read Replica 1\n(AZ-b)"]
        R2["📖 Read Replica 2\n(AZ-c)"]
    end
    subgraph "Distributed Storage (Aurora-like)"
        S1["💾 Storage AZ-a\n(2 copies)"]
        S2["💾 Storage AZ-b\n(2 copies)"]
        S3["💾 Storage AZ-c\n(2 copies)"]
    end
    W --> S1
    W --> S2
    W --> S3
    R1 --> S2
    R2 --> S3
    GQL["🌐 Gremlin / SPARQL\nClient"] --> W
```

Graph models supported:

- **Property Graph (Gremlin / openCypher):** Vertices (nodes) and edges with properties; TinkerPop API; Jupyter notebooks support
- **RDF (SPARQL):** Resource Description Framework; triples (subject-predicate-object); SPARQL 1.1

🔒 Connectivity & Security

- Deployed in a **VPC** with a **DB Subnet Group** (≥2 AZs)
- Accessible via HTTPS endpoints (Gremlin WebSocket, SPARQL REST)
- **No public access by default** — must use VPC or bastion
- **IAM authentication** supported (HTTP-based auth for Gremlin/SPARQL requests)
- **VPC Security Groups** control access
- **Port:** 8182 (default)
- **Neptune Streams:** Change log (like DynamoDB Streams) for graph changes

---

🔐 Encryption Deep-Dive

Layer	Mechanism	Notes
At-rest	AES-256 via KMS	Enable at cluster creation; cannot change after
In-transit	TLS (enforced)	HTTPS-only; certificate validation required
Snapshots	Same KMS key as cluster	
Cross-account	Re-encrypt on snapshot copy	

📈 Scalability & HA

- **Vertical:** Change instance class (requires brief failover)
- **Read scaling:** Up to **15 Neptune Replicas** (same as Aurora)
- **Storage:** Auto-grows in 10 GB increments up to 64 TiB
- **Multi-AZ:** Always 3-AZ storage; add replicas in different AZs for compute HA
- **Neptune Serverless:** Auto-scales Neptune Capacity Units (NCUs); available (preview); good for variable graph workloads

---

🏢 Multi-AZ

Same model as Aurora:

- Storage: always 6 copies / 3 AZs (built-in, not optional)
- Compute HA: add at least 1 replica in a different AZ
- Failover: ~30 seconds (replica promotion); DNS CNAME flip
- Standby replicas serve reads (unlike RDS Multi-AZ standby)

📖 Read Replicas

Feature	Neptune Replica
Max replicas	15
Replication lag	Milliseconds (shared storage)
Cross-region	No native cross-region replicas (unlike Aurora Global DB)
Promotion	~30 sec
Reads	Yes — serves Gremlin/SPARQL read queries

> 🔑 **Exam key:** Neptune does NOT have cross-region replication (no equivalent to Aurora Global Database). For cross-region Neptune DR: snapshot + restore in other region.

⏱️ RPO / RTO Table

Scenario	RPO	RTO
AZ failure (with replica)	~0	~30 sec
Single instance (no replica)	Up to last backup	20-40 min
Region failure	Hours (last snapshot)	30-60 min (restore)
PITR	Up to 35 days	20-40 min

💾 Backup & Restore

- **Automated backups:** Daily, 1–35 day retention; continuous backup to S3
- **PITR:** Restore to any second within retention window → new cluster
- **Manual snapshots:** Indefinite retention; cross-account sharing; cross-region copy
- **Neptune Streams:** Capture graph changes; retain 7 days; use for change-data-capture pipelines

---

🔄 Migration

Source	Method
Neo4j	Export to CSV → Neptune Bulk Loader via S3
Relational (RDS)	Transform rows to vertices/edges → CSV → Bulk Loader
Amazon DMS	Limited support; Neptune not a primary DMS target
Neptune Bulk Loader	Primary tool; loads CSV format from S3

Neptune Bulk Loader: Fast parallel S3-to-Neptune import. Format: `application/vnd.amazon.neptune-csv` (vertices file + edges file).

🔗 Integration

Service	Purpose
S3	Bulk load data via Neptune Bulk Loader
Lambda	Query Neptune from serverless functions
SageMaker	Neptune ML — use graph structure features in ML models
Kinesis	Stream graph changes via Neptune Streams
CloudWatch	Cluster metrics
IAM	HTTP-based authentication
Jupyter/SageMaker Notebooks	Neptune Workbench for interactive graph exploration

---

⚖️ Comparison

Feature	Neptune	Neo4j	DynamoDB	RDS
Data model	Graph (PG + RDF)	Property Graph	Key-value/document	Relational
Query language	Gremlin, SPARQL, openCypher	Cypher	DynamoDB API	SQL
Managed on AWS	Fully	Self-managed on EC2	Fully	Fully
Relationship traversal	Native	Native	Limited (manual)	Joins (slow at scale)
Scale	Up to 64 TiB	Cluster-based	Unlimited	64 TiB

⚠️ Limitations

Limit	Value
Max storage	64 TiB
Max replicas	15
Max nodes/edges	Billions (practical limit: storage)
Cross-region replication	Not natively supported
Neptune Streams retention	7 days
Encryption change	Cannot change after cluster creation
PITR retention	Up to 35 days

🏛️ Well-Architected Framework

Pillar	Neptune Considerations
Operational Excellence	Use Neptune Workbench for query development; monitor query latency via CloudWatch; use Neptune Streams for change tracking
Security	Encrypt at creation with CMK; use VPC private subnets; enforce TLS; use IAM HTTP auth; enable CloudTrail
Reliability	Add replicas in different AZs; enable PITR; use automated snapshots; test failover
Performance Efficiency	Design graph schema for access patterns; use read replicas for read-heavy traversals; use Neptune ML for graph-enhanced ML
Cost Optimization	Right-size instances; use Neptune Serverless for variable graph workloads; use snapshots instead of running idle clusters for dev/test
Sustainability	Neptune Serverless scales to minimum when idle; shared storage avoids per-replica duplication

---

📐 4 Exam Domains

Domain 1 — Organizational Complexity: Neptune in VPC; IAM auth for programmatic access; CloudTrail for audit; cross-account snapshot sharing for DR.

Domain 2 — Design for New Solutions: Social network (fraud detection, recommendations) → Neptune with property graph. Knowledge graph (semantics, ontologies) → Neptune with RDF/SPARQL. Use Neptune ML for graph neural network features.

Domain 3 — Continuous Improvement: Add replicas for read scaling; enable Neptune Streams for CDC pipelines; enable PITR.

Domain 4 — Migration: Neo4j/relational → Neptune via Bulk Loader. Transform relational data to graph model (vertices + edges CSVs) → load to Neptune.

🚀 Chapter 13: Amazon ElastiCache

📖 Overview

Amazon ElastiCache is AWS's fully managed in-memory caching service. It supports two engines: **Redis** (now also offered as **Valkey**) and **Memcached**. ElastiCache sits in front of a database to absorb read traffic, dramatically reducing latency (microseconds to low milliseconds) and database load.

> 🔑 **Exam key:** ElastiCache = in-memory cache. Use to offload repetitive reads from RDS/Aurora/DynamoDB. NOT a database — data can be lost if cluster fails (unless Redis persistence enabled). Use when latency must be sub-millisecond and the data fits in memory.

Real-life analogy: ElastiCache is like a whiteboard next to a librarian — frequently requested books (data) are written on the whiteboard for instant access instead of fetching from the shelves (database) every time.

Scope: Regional service. Individual nodes are AZ-scoped. No native cross-region replication.

Open-source equivalents: Redis (upstream), Memcached (upstream).

🏗️ Architecture

```
graph LR
    App["🌐 Application\n(Cache-aside pattern)"]
    EC["🚀 ElastiCache\n(Redis Cluster)"]
    DB["🗄️ Aurora/RDS\n(Source of truth)"]
    App -->|"1. Check cache"| EC
    EC -->|"2a. Cache HIT — return data"| App
    EC -->|"2b. Cache MISS"| App
    App -->|"3. Fetch from DB"| DB
    DB -->|"4. Return data"| App
    App -->|"5. Write to cache"| EC
```

🔧 Redis vs Memcached

Feature	Redis / Valkey	Memcached
Data structures	Strings, hashes, lists, sets, sorted sets, bitmaps, HyperLogLog, streams	Strings only
Persistence	RDB snapshots + AOF logs	None
Replication	Yes (async)	No
Multi-AZ failover	Yes (automatic)	No
Pub/Sub	Yes	No
Lua scripting	Yes	No
Cluster mode	Yes (sharding)	Yes (multi-node)
Transactions	Yes (MULTI/EXEC)	No
Geo-spatial	Yes	No
Use case	Sessions, leaderboards, queues, pub/sub, complex data	Simple cache, large datasets, multi-threaded
Valkey	Drop-in Redis 7.2 fork (AWS-managed)	N/A

> 🔑 **Exam key:** Use Redis when you need persistence, replication, failover, or complex data structures. Use Memcached only for simple, pure caching with multi-threaded horizontal scaling and no HA requirements.

🔧 Redis Deployment Modes

Mode	Description	Use Case
Single-node	One Redis node, no HA	Dev/test
Cluster Mode Disabled (Replication Group)	1 primary + up to 5 replicas; all data on each node	Simple HA, read scaling
Cluster Mode Enabled (Redis Cluster)	Data sharded across up to 500 shards; each shard: 1 primary + up to 5 replicas	Massive datasets, horizontal scaling

> 🔑 **Exam key:** Cluster Mode Disabled = entire dataset on each node (limited by node memory). Cluster Mode Enabled = dataset sharded across nodes (scale beyond single-node memory limit).

🔒 Connectivity & Security

- Deployed in a **VPC** with a **Cache Subnet Group** (spans AZs)
- **Security Groups** control access; no public access by default

- **Encryption in-transit:** TLS enabled per cluster; clients must use TLS endpoint
- **Encryption at-rest:** AES-256 via KMS (Redis only, not Memcached)
- **AUTH token:** Redis password authentication; `AUTH <password>` command
- **RBAC (Redis 6+):** Role-based access control; user accounts with specific command permissions
- **Port:** 6379 (Redis default), 11211 (Memcached)

🔐 Encryption Deep-Dive

Layer	Mechanism	Notes
At-rest	AES-256 via KMS	Redis only; enable at creation; CMK supported
In-transit	TLS 1.2+	Enable at creation; client must use TLS
AUTH	Password token	Basic Redis AUTH
RBAC	IAM-like user/group model	Redis 6+; fine-grained command control
Memcached	No at-rest encryption	TLS in-transit only

📈 Scalability & HA

Redis Cluster Mode Disabled:

- Add/remove replicas (read scaling)
- Vertical: change node type (requires cluster recreation for major changes)
- Failover: primary failure → replica promoted automatically (~30-60 sec)

Redis Cluster Mode Enabled:

- **Online resharding:** Add shards without downtime
- **Shard scaling:** Scale out (add shards) or scale in (remove shards)
- Up to 500 shards × 6 nodes = 3000 nodes per cluster

Memcached:

- Add/remove nodes (horizontal scaling)
- No replication, no failover
- Client-side consistent hashing for key distribution

🏢 Multi-AZ

Redis Multi-AZ:

- Primary in one AZ; replicas in other AZs
- Automatic failover: replica in another AZ promoted on primary failure
- DNS endpoint for primary updates automatically (~30-60 sec downtime)
- **Cluster Mode Enabled:** Each shard can have replicas in different AZs

Memcached:

- No Multi-AZ support; no replication; nodes are independent
- For HA: add nodes across AZs; application handles cache miss on node failure

---

📖 Read Replicas

Feature	Redis Replica
Max replicas per shard	5 (Cluster Mode Disabled) or 5 per shard (Enabled)
Replication	Async
Cross-region	Not natively supported
Purpose	Read scaling + automatic failover source
Promotion	Automatic on primary failure

⏱️ RPO / RTO Table

Scenario	RPO	RTO
Redis primary failure (with replica)	Data in cache at time of failure (no persistence) OR last RDB/AOF	~30-60 sec (replica promotion)
Redis with AOF persistence	RPO = last AOF write (~1 sec)	1-5 min (replay AOF)
Memcached node failure	All data on that node lost	Immediate (cache miss, rebuild from DB)
Full cluster failure	Depends on persistence config	Minutes (restore from RDB snapshot)

> 🔑 **Exam key:** ElastiCache is a cache, not a database. Even with persistence, RPO/RTO are not as critical — the database is the source of truth. Cache loss = increased DB load, not data loss.

💾 Backup & Restore (Redis only)

- **RDB snapshots:** Point-in-time dump to S3; manual or automatic (daily)
- **AOF (Append-Only File):** Logs every write command; better durability; replay on restart
- **Backup retention:** 0–35 days
- **Restore:** Create new cluster from backup
- **Export to S3:** Export RDB file to S3 for analysis or cross-region restore

---

🔄 Migration

Source	Method
Self-managed Redis	Redis `DUMP` / `RESTORE`, or Online Migration tool, or `SLAVEOF` replication
ElastiCache Redis → ElastiCache Redis	Backup + restore to new cluster
Memcached → Redis	Cold migration (data not persistent; restart with new cluster)
AWS DMS	Not a typical DMS use case; DMS is for relational migrations

Online Migration to ElastiCache: AWS provides an **Online Migration** feature for migrating self-managed Redis to ElastiCache — uses Redis replication to sync with minimal downtime.

🔗 Integration

Service	Purpose
RDS/Aurora	Cache frequently read DB results (cache-aside pattern)
DynamoDB + DAX	DAX is DynamoDB-specific; ElastiCache Redis is general-purpose
Lambda	Session storage for serverless apps
ElastiCache for Redis	Session store, rate limiter, pub/sub, leaderboard
CloudWatch	Cache hit/miss ratio, evictions, memory usage metrics
Secrets Manager	Store Redis AUTH token
VPC	Private network isolation

⚖️ Comparison

Feature	ElastiCache Redis	ElastiCache Memcached	DynamoDB DAX	In-memory on EC2
Latency	Microseconds	Microseconds	Microseconds	Microseconds
Persistence	Yes (RDB/AOF)	No	No	No
Multi-AZ HA	Yes	No	Yes	Manual
Data structures	Rich	Strings only	DynamoDB types	Any
Max data	Limited by RAM	Limited by RAM	Configurable	Limited by RAM
Managed	Yes	Yes	Yes	No
Use with any DB	Yes	Yes	DynamoDB only	Yes

⚠️ Limitations

Limit	Value
Max nodes per Memcached cluster	40
Max shards (Redis Cluster Mode Enabled)	500
Max replicas per shard	5
Max node memory	Depends on node type (up to ~340 GB for cache.r6g.16xlarge)
Cross-region replication	Not natively supported
Redis AUTH token length	16-128 chars
Backup retention	0-35 days

🏛️ Well-Architected Framework

Pillar	ElastiCache Considerations
Operational Excellence	Monitor cache hit ratio (target >80%); set appropriate TTL; use CloudWatch alarms for evictions; use RBAC for operational access control
Security	Enable TLS + at-rest encryption; use RBAC (Redis 6+); deploy in private subnets; store AUTH token in Secrets Manager
Reliability	Use Redis Multi-AZ with automatic failover; design cache-aside pattern so DB is always the source of truth; test failover; set `maxmemory-policy` appropriately
Performance Efficiency	Choose right node size (memory-bound); use Cluster Mode Enabled for large datasets; monitor eviction rate; use connection pooling
Cost Optimization	Right-size nodes; use Reserved Nodes for production; Cluster Mode Enabled scales memory more efficiently than large single nodes
Sustainability	Right-size to avoid idle memory; use TTL to expire unused data (reduces memory waste); Graviton-based nodes available

---

📐 4 Exam Domains

Domain 1 — Organizational Complexity: Deploy ElastiCache in private subnets; use RAM sharing is NOT supported for ElastiCache — deploy per account; use CloudWatch cross-account for monitoring.

Domain 2 — Design for New Solutions: Session store for stateless web apps → Redis; real-time leaderboard → Redis sorted sets; rate limiting → Redis atomic operations; pub/sub messaging → Redis Pub/Sub.

Domain 3 — Continuous Improvement: Enable Multi-AZ if single-node; increase replica count for read scaling; enable AOF for better cache durability; monitor hit ratio and increase cache size if low.

Domain 4 — Migration: Migrate self-managed Redis to ElastiCache with Online Migration (minimal downtime). Memcached → ElastiCache Memcached (simple endpoint swap). Consider replacing Memcached with Redis for HA requirements.

📄 Chapter 14: Amazon DocumentDB

📖 Overview

Amazon DocumentDB (with MongoDB compatibility) is AWS's fully managed document database service. It is designed for JSON document workloads with MongoDB-compatible APIs — applications using MongoDB drivers, tools, and queries can largely work with DocumentDB without code changes.

> 🔑 **Exam key:** DocumentDB is NOT MongoDB — it is a MongoDB-compatible API layer over a purpose-built AWS storage engine (Aurora-like distributed storage). Not all MongoDB features are supported. Check the compatibility matrix for advanced features.

Real-life analogy: DocumentDB is to MongoDB what Aurora is to MySQL/PostgreSQL — same API/wire protocol, different underlying storage engine optimized for AWS.

Scope: Regional service. Multi-AZ supported via cluster replicas.

Open-source equivalent: MongoDB.

🏗️ Architecture

DocumentDB uses the **same distributed storage architecture as Aurora** — 6 copies across 3 AZs, storage auto-grows, quorum-based.

```
graph TB
    subgraph "DocumentDB Cluster"
        P["🖍️ Primary Instance\n(Reads + Writes)"]
        R1["📖 Replica 1\n(AZ-b)"]
        R2["📖 Replica 2\n(AZ-c)"]
    end
    subgraph "Distributed Storage"
        S1["💾 AZ-a (2 copies)"]
        S2["💾 AZ-b (2 copies)"]
        S3["💾 AZ-c (2 copies)"]
    end
    P --> S1 & S2 & S3
    R1 --> S2 & S3
    R2 --> S1 & S3
    App["🌐 MongoDB Driver"] -->|"Port 27017"| P
```

🔒 Connectivity & Security

- Deployed in a **VPC**; private subnets recommended; port **27017**
- **Security Groups** for access control
- **TLS in-transit:** Always enabled; cannot disable; use MongoDB TLS connection strings
- **At-rest encryption:** AES-256 via KMS; enable at creation
- **IAM Authentication:** Supported — IAM roles generate short-lived passwords
- **MongoDB connection string:** `mongodb://user:pass@cluster-endpoint:27017/?tls=true`

🔐 Encryption

Layer	Details
At-rest	AES-256 KMS (CMK supported); enable at creation only
In-transit	TLS always enforced (cannot disable)
Snapshots	Inherit cluster key

📈 Scalability & HA

- Up to **15 read replicas** per cluster (same as Aurora/Neptune)
- Storage auto-grows in 10 GB increments up to **64 TiB**
- Vertical scaling: change instance class
- No serverless option (unlike Aurora Serverless v2)

🏢 Multi-AZ

Same model as Aurora:

- Storage: always 6 copies / 3 AZs
- Compute HA: add replicas in different AZs
- Failover: ~30 sec (DNS flip to promoted replica)
- Replicas serve reads (unlike RDS standby)

📖 Read Replicas

- Up to 15 replicas; millisecond lag (shared storage); cross-region NOT natively supported
- Promotion: ~30 sec on primary failure

⏱️ RPO / RTO

Scenario	RPO	RTO
AZ failure (with replica)	~0	~30 sec
Region failure	Hours (last snapshot)	30-60 min
PITR	Up to 35 days	20-40 min

💾 Backup & Restore

- Automated backups: 1–35 day retention; PITR supported (restores to new cluster)
- Manual snapshots: indefinite retention; cross-account sharing; cross-region copy
- No export to S3 (unlike DynamoDB)

🔄 Migration

Source	Method
MongoDB (self-managed)	`mongodump` / `mongorestore` ; AWS DMS
MongoDB Atlas	AWS DMS with MongoDB source
AWS DMS	Supports MongoDB as source, DocumentDB as target; Full Load + CDC

> 🔑 **Exam key:** DMS supports MongoDB → DocumentDB migration with CDC for minimal downtime. Use `mongodump` / `mongorestore` for offline migration of small datasets.

⚖️ Comparison

Feature	DocumentDB	MongoDB Atlas	DynamoDB
API	MongoDB 4.0/5.0 compat	Native MongoDB	DynamoDB API
Data model	Document (BSON/JSON)	Document	Key-value/document
Storage engine	Aurora-like distributed	WiredTiger	AWS proprietary
Managed	Fully (AWS)	Fully (Atlas SaaS)	Fully (AWS)
Serverless	No	Yes (Atlas Serverless)	Yes (On-Demand)
Cross-region	Snapshot copy only	Atlas Global Clusters	Global Tables
MongoDB feature compat	~80-90%	100%	N/A

⚠️ Limitations

Limit	Value
Max storage	64 TiB
Max replicas	15
Max document size	16 MB
Max collections per database	No hard limit
Transactions	MongoDB 4.0-style multi-document ACID
MongoDB version compatibility	3.6, 4.0, 5.0 APIs
Not supported	Change Streams (limited), Atlas Search, MongoDB 6.0+ features

🏛️ Well-Architected Framework

Pillar	DocumentDB Considerations
Operational Excellence	Monitor via CloudWatch; use Performance Insights (if available); use profiler for slow query analysis
Security	CMK encryption at creation; TLS always on; IAM auth; private subnets; CloudTrail
Reliability	Replicas in different AZs; PITR enabled; automated snapshots; test failover
Performance Efficiency	Index key query fields; use read replicas for read scaling; choose right instance class
Cost Optimization	Right-size instances; reserved instances for production; stop dev clusters when not in use
Sustainability	Shared storage avoids per-replica duplication; right-size compute

---

📐 4 Exam Domains

Domain 1: Cross-account snapshot sharing for DR; VPC + SG controls; CloudTrail audit.

Domain 2: JSON document workloads with MongoDB compatibility → DocumentDB. Content management, catalogs, user profiles. MongoDB-compatible API means minimal code changes.

Domain 3: Migrate from single-instance MongoDB to DocumentDB cluster for HA; add replicas for read scaling; enable PITR.

Domain 4: Self-managed MongoDB → DocumentDB via DMS (Full Load + CDC). Minimal downtime migration for MongoDB-based applications moving to AWS.

📒 Chapter 15: Amazon QLDB

📖 Overview

Amazon Quantum Ledger Database (QLDB) is a fully managed ledger database that provides a **cryptographically verifiable, immutable transaction log**. Every change to data is recorded permanently with a cryptographic hash chain — you can prove that data has not been altered since it was written.

> 🔑 **Exam key:** QLDB = immutable ledger with cryptographic verification. Use when you need an audit trail that CANNOT be falsified — financial transactions, supply chain records, medical records. NOT a general-purpose database. NOT blockchain (no decentralized consensus).

Real-life analogy: QLDB is like a bank's accounting ledger — every transaction is written in permanent ink, each entry references the previous one, and the entire history can be mathematically verified. You can see any balance at any point in history.

Scope: Regional service. No Multi-AZ option (AWS manages HA internally). No cross-region replication natively.

Comparison: Unlike blockchain (Managed Blockchain), QLDB is **centralized** — owned by you, trusted by you. No distributed consensus. Much simpler and faster.

🏗️ Architecture

```
graph TB
    subgraph "QLDB Ledger"
        J["📋 Journal\n(Immutable append-only\ncryptographic hash chain)"]
        S["🗂️ Current State\n(Queryable via PartiQL)"]
        J -->|"Derived view"| S
    end
    App["🌐 Application\n(PartiQL queries)"] -->|"Reads + Writes"| S
    App -->|"Verify history"| J
    Export["📤 Export to S3\n(for analysis)"] --> J
```

Key concepts:

- **Journal:** Append-only, immutable sequence of all committed transactions; organized into blocks; each block has a hash of the previous block (blockchain-like chain)
- **Ledger:** The top-level QLDB resource; contains tables
- **Tables:** Store document data (JSON-like ION format)
- **Current state:** Queryable current snapshot of all tables; derived from journal
- **Digest:** Cryptographic proof (SHA-256 hash) of the journal at a point in time; used to verify integrity
- **Verification:** Prove any document revision has not been altered using the digest + hash chain

---

🔑 PartiQL Query Language

QLDB uses **PartiQL** — an SQL-compatible query language that works with semi-structured (JSON/ION) data.

```
-- Query current state
SELECT * FROM Vehicles WHERE VIN = 'KM8J3CA46JU622498'

-- Query history (all revisions)
SELECT * FROM history(Vehicles) AS h
WHERE h.metadata.id = 'doc-id-here'

-- Insert (creates new revision in journal)
INSERT INTO Vehicles VALUE {
    'VIN': 'KM8J3CA46JU622498',
    'Type': 'Sedan',
    'Year': 2024
}
```

🔒 Connectivity & Security

- QLDB is **not deployed in a VPC** — it is a public AWS endpoint service
- Access via **HTTPS endpoint** + IAM authentication (no VPC endpoint natively for most operations)
- **IAM policies** control all access (no database-level users)
- **Encryption at-rest:** AES-256 via KMS; AWS-managed key default; CMK supported
- **Encryption in-transit:** TLS enforced

> ⚠️ **QLDB is NOT inside your VPC** — this is a key exam gotcha. Access is via HTTPS public endpoint controlled by IAM. VPC Endpoints for QLDB are available in some regions (interface endpoint).

🔐 Encryption Deep-Dive

Layer	Details
At-rest	AES-256 KMS; AWS-managed or CMK; set at ledger creation
In-transit	TLS enforced
Journal integrity	SHA-256 hash chain; independent of KMS
Verification	Digest + proof — mathematically verifiable without decrypting

> 🔑 **Exam key:** QLDB's cryptographic integrity is separate from KMS encryption. Even if KMS key is compromised, the hash chain integrity verification tells you if data was tampered with. These are two different security properties.

📈 Scalability & HA

- **Serverless:** No capacity planning; scales automatically
- **HA:** AWS manages — QLDB stores journal across multiple AZs internally (not user-configurable)
- **No Multi-AZ configuration option** — AWS handles it transparently
- **No read replicas** — single endpoint; AWS scales internally

---

🏢 Multi-AZ

QLDB is **serverless and fully managed** — AWS handles AZ redundancy internally. There is no Multi-AZ toggle, no standby to configure, no replica to add. AWS SLA covers AZ-level resilience.

📖 Read Replicas

QLDB has **no user-configurable read replicas**. Scaling is handled automatically by AWS. For read-heavy workloads, export journal to S3 and query with Athena.

⏱️ RPO / RTO

Scenario	RPO	RTO
AZ failure	~0 (AWS-managed)	Seconds–minutes (AWS managed)
Region failure	Last export to S3	Manual rebuild from export
Accidental wrong write	N/A (immutable — cannot delete from journal)	N/A

> 🔑 **Exam key:** You CANNOT delete data from the QLDB journal. You can drop a table (removes current state view) but all journal entries remain. Immutability is the core design guarantee.

💾 Backup & Restore

- **No traditional backup/restore** — the journal IS the backup (immutable, complete history)
- **Journal export to S3:** Export journal blocks to S3 in ION or JSON format; use Athena or Spark to analyze; use for cross-region DR
- **Ledger deletion:** Destroying the ledger destroys everything (ask for confirmation)

🔄 Migration

Source	Method
Relational DB (audit tables)	Export historical data → import to QLDB tables; application layer migration
Custom audit log	Write new records to QLDB via application; backfill via bulk import
Another ledger system	Custom ETL via S3
AWS DMS	Not typically used for QLDB; no native DMS support

🔗 Integration

Service	Purpose
S3	Journal export for analysis/DR
Athena	Query exported journal data
Lambda	Trigger Lambda on QLDB Streams (document changes)
Kinesis Data Streams	QLDB Streams → Kinesis → Lambda/Firehose
IAM	All authentication/authorization
CloudTrail	API audit
CloudWatch	Ledger metrics

⚖️ Comparison

Feature	QLDB	Managed Blockchain	RDS (audit tables)	DynamoDB + Streams
Immutability	Cryptographic guarantee	Distributed consensus	No (mutable)	No (events only)
Verification	Cryptographic digest	On-chain	No	No
Decentralized	No (centralized)	Yes	No	No
Query language	PartiQL (SQL-like)	Smart contracts	SQL	DynamoDB API
Performance	High (single-owner)	Low (consensus overhead)	High	High
Use case	Financial ledger, audit	Multi-party trustless	Compliance logging	Change events
Managed	Fully	Fully	Manually managed	Fully

> 🔑 **Exam key:** QLDB vs Managed Blockchain: QLDB = centralized, trusted single-owner ledger (company owns and trusts their own data). Managed Blockchain = decentralized, multiple parties who don't trust each other. If you trust yourself = QLDB. If multiple organizations need to agree = Managed Blockchain.

⚠️ Limitations

Limit	Value
Max ledger size	No documented limit
Max document size	128 KB
Max tables per ledger	20
Transactions per second	Up to 8,000 (auto-scaled)
Cross-region replication	Not natively supported
Data deletion	Cannot delete journal entries
VPC	No native VPC deployment (public endpoint)
DMS support	Not a DMS source/target

🏛️ Well-Architected Framework

Pillar	QLDB Considerations
Operational Excellence	Use QLDB Streams to integrate with downstream systems; export journal to S3 for long-term retention; monitor via CloudWatch
Security	IAM-only authentication (no DB users); CMK for at-rest encryption; TLS enforced; CloudTrail for API audit; note public endpoint (no VPC)
Reliability	AWS manages AZ resilience; export journal to S3 for cross-region DR; test verification procedures
Performance Efficiency	Serverless — no capacity planning; use PartiQL indexes for query performance; export to S3+Athena for analytics (don't scan QLDB for analytics)
Cost Optimization	Pay per I/O + storage; export old journal to S3 (cheaper storage); use Athena for analytics on exports
Sustainability	Serverless scales to zero; no idle compute cost

---

📐 4 Exam Domains

Domain 1 — Organizational Complexity: IAM controls all QLDB access — integrate with AWS Organizations SCPs to restrict ledger access. CloudTrail captures all API operations for cross-account governance.

Domain 2 — Design for New Solutions: Financial transactions (banking credits/debits), supply chain provenance, medical record audit trails, regulatory compliance ledgers → QLDB. Need cryptographic proof of data integrity → QLDB. Multiple untrusted parties → Managed Blockchain instead.

Domain 3 — Continuous Improvement: Enable QLDB Streams → Kinesis for real-time downstream processing. Export journal to S3 + Athena for analytics. Add CMK if using AWS-managed key.

Domain 4 — Migration: Legacy audit-log tables in RDS → QLDB. Write new audit events to QLDB; backfill historical data via bulk import. No DMS support — custom ETL required.

🪣 Chapter 22: Amazon S3

📖 Overview

Amazon Simple Storage Service (S3) is AWS's **object storage** service — infinitely scalable, highly durable (11 nines = 99.999999999%), and the backbone of nearly every AWS architecture. It stores objects (files + metadata) in buckets (containers). S3 is not a filesystem — objects have no directory hierarchy (though prefixes simulate one), no partial updates, and no locking.

> **Object vs Block vs File storage:**
>
> - **Object (S3):** flat namespace, accessed via HTTP API, metadata-rich, no mount, infinite scale
> - **Block (EBS):** raw disk volumes, mounted to EC2, low latency, random I/O
> - **File (EFS/FSx):** POSIX filesystem, mountable, shared access, hierarchical

> **Linux analogy:** S3 is like a web-accessible key-value store where the key is the full object path and the value is the binary blob. **Azure equivalent:** Azure Blob Storage (Block Blobs ≈ S3 objects; Blob containers ≈ S3 buckets).

S3 is a **regional** service — data stored in a region stays in that region unless explicitly replicated. However, bucket names are **globally unique** across all AWS accounts and regions.

🗂️ Storage Classes

Storage Class	Min Duration	Min Billable Size	Retrieval Time	Retrieval Fee	AZs	Use Case
S3 Standard	None	None	Milliseconds	None	≥3	Frequently accessed data
S3 Standard-IA	30 days	128 KB	Milliseconds	Per GB	≥3	Infrequently accessed, rapid retrieval
S3 One Zone-IA	30 days	128 KB	Milliseconds	Per GB	1	Reproducible infrequent data, cost-sensitive
S3 Glacier Instant Retrieval	90 days	128 KB	Milliseconds	Per GB	≥3	Archive, quarterly access, instant retrieval
S3 Glacier Flexible Retrieval	90 days	40 KB	Minutes–hours	Per GB	≥3	Archive, occasional access, flexible timing
S3 Glacier Deep Archive	180 days	40 KB	12–48 hours	Per GB	≥3	Long-term archive, regulatory retention
S3 Intelligent-Tiering	None	None	Milliseconds–hours	None (monitoring fee)	≥3	Unknown/changing access patterns
S3 Express One Zone	None	None	Single-digit ms	None	1 (single AZ)	Latency-sensitive, ML training, HPC

Intelligent-Tiering tiers:

- Frequent Access (default, no min days)
- Infrequent Access (auto-move after 30 days, no retrieval fee)
- Archive Instant Access (auto-move after 90 days, no retrieval fee)
- Archive Access (optional, 90–180 days, retrieval fee)
- Deep Archive Access (optional, 180+ days, retrieval fee)
- Monitoring fee: $0.0025 per 1,000 objects/month

S3 Express One Zone:

- Stored in a **Directory Bucket** (separate bucket type, AZ-specific)
- 10× faster than S3 Standard
- Different API (CreateSession), does not support all S3 features (no versioning, no replication)
- Ideal for ML training data, analytics scratch space, latency-critical apps

Glacier Flexible Retrieval tiers:

- Expedited: 1–5 minutes (highest cost)

- Standard: 3–5 hours
- Bulk: 5–12 hours (lowest cost)

🪣 Bucket Fundamentals

Naming rules:

- 3–63 characters
- Lowercase letters, numbers, hyphens only
- Must start/end with letter or number
- No IP address format (192.168.0.1 not allowed)
- Globally unique across ALL AWS accounts

URLs:

- Virtual-hosted style (default): `https://bucket-name.s3.region.amazonaws.com/key`
- Path style (legacy, deprecated): `https://s3.region.amazonaws.com/bucket-name/key`

Object limits:

- Max single object size: **5 TB**
- Max single PUT: **5 GB** (use multipart for >5 GB; recommended >100 MB)
- Max object key length: 1,024 bytes (UTF-8)
- No limit on number of objects in a bucket

Metadata:

- **System metadata:** Content-Type, Content-Length, Last-Modified, ETag, x-amz-version-id (set by AWS)
- **User metadata:** custom key-value pairs, prefix `x-amz-meta-`, max 2 KB total, immutable after object creation (must copy to change)

Tags: up to 10 per object, key up to 128 chars, value up to 256 chars. Used for lifecycle, replication, cost allocation, access control.

Default account limits:

- 100 buckets per account (default soft limit)
- 1,000 buckets per account (hard limit, cannot be increased)

🔢 Versioning

Versioning can be in one of three states:

State	Description
Unversioned	Default. No version IDs. Object overwrite replaces previous.
Versioning-Enabled	Every PUT/DELETE creates a new version. Protects against accidental delete/overwrite.
Versioning-Suspended	New objects get `null` version ID. Existing versions preserved. Cannot return to unversioned.

Delete behavior with versioning enabled:

- `DELETE` without version ID → creates a **delete marker** (hides object, does not delete versions)
- `DELETE` with version ID → permanently deletes that specific version
- To restore: delete the delete marker

MFA Delete:

- Requires MFA token for: permanently deleting a version, changing versioning state
- Only bucket owner (root account) can enable MFA Delete
- Adds strong protection against accidental/malicious permanent deletion

Version IDs: unique strings per version; `null` for objects uploaded before versioning enabled or after suspension.

🔐 Access Control

S3 uses **multiple overlapping access control mechanisms** — all must allow for access to succeed.

Object Ownership & ACLs

Object Ownership settings (bucket-level):

- `BucketOwnerEnforced` (recommended): ACLs disabled, bucket owner owns all objects. Simplest model.
- `BucketOwnerPreferred`: bucket owner owns objects if uploaded with `bucket-owner-full-control` canned ACL.
- `ObjectWriter` (legacy): uploader owns the object.

ACLs (legacy): predefined grants (private, public-read, public-read-write, authenticated-read, bucket-owner-read, bucket-owner-full-control). AWS recommends disabling ACLs (BucketOwnerEnforced) for all new buckets.

Block Public Access

Four independent settings (can be set at account level or bucket level):

Setting	What It Blocks
`BlockPublicAcls`	Rejects PUT requests that grant public access via ACLs
`IgnorePublicAcls`	Ignores all public ACLs on bucket and objects
`BlockPublicPolicy`	Rejects bucket policies that grant public access
`RestrictPublicBuckets`	Restricts access to bucket with public policy to only AWS services and authorized users

> **Exam trap:** Account-level Block Public Access overrides bucket-level settings. Even if a bucket policy grants public access, account-level `BlockPublicPolicy` prevents it.

Bucket Policies

Resource-based JSON policy attached to bucket. Controls access from any principal (IAM users, roles, other accounts, services). Evaluated together with IAM policies.

```
// Force HTTPS
{
  "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:*",
  "Resource": ["arn:aws:s3:::my-bucket", "arn:aws:s3:::my-bucket/*"],
  "Condition": {"Bool": {"aws:SecureTransport": "false"}}
}
```

S3 Access Points

Named network endpoints attached to a bucket. Each Access Point has:

- Its own IAM-like policy
- Network origin control (VPC only or internet)
- Simplifies managing access at scale (one policy per team/application instead of one giant bucket policy)

Access Point ARN: `arn:aws:s3:region:account-id:accesspoint/name`

Multi-Region Access Points (MRAP): single global endpoint, routes to closest bucket replica, failover between regions. Uses AWS Global Accelerator. Supports replication and failover.

Presigned URLs

Temporary access to private objects without AWS credentials:

- Generated using SDK/CLI with caller's credentials

- Default expiry: 1 hour (max 7 days with STS token, 12 hours with IAM role)
- Valid for GET (download) or PUT (upload)
- Includes signature in URL — anyone with URL can access until expiry

```
aws s3 presign s3://bucket/key --expires-in 3600
```

🔒 Encryption

Encryption types comparison

Type	Key managed by	Key stored in AWS	Cost	Audit trail	Use case
SSE-S3	AWS	Yes (AWS-managed)	Free	No individual key audit	Default, no compliance requirement
SSE-KMS	Customer (CMK) or AWS	Yes (KMS)	KMS API cost	Yes (CloudTrail)	Compliance, audit requirement, key rotation
SSE-C	Customer	No (sent per request)	Free	No	Customer manages keys entirely
DSSE-KMS	Customer (CMK)	Yes (KMS)	Higher KMS cost	Yes	Dual-layer encryption, highest compliance
Client-side	Customer	No	Free	No	Encrypt before upload, full control

SSE-S3: AES-256, AWS manages key hierarchy, no extra cost, no per-request key access audit.

SSE-KMS:

- Uses AWS KMS CMK (Customer Managed Key or AWS-managed key)
- Each object encrypted with a unique data key; data key encrypted with CMK
- CloudTrail logs every KMS API call (GenerateDataKey, Decrypt) — full audit
- KMS has request rate limits: 5,500–30,000 API calls/sec depending on region. High-throughput S3 apps may hit KMS throttling — use S3 Bucket Keys to reduce KMS calls by 99%
- **S3 Bucket Keys:** generates a short-lived bucket-level data key from CMK, reduces KMS API calls dramatically, enabled per bucket or per object

SSE-C:

- Customer provides key in request header (must use HTTPS)
- AWS uses key to encrypt, then discards it — AWS never stores the key
- Customer must track and provide the correct key for every GET request
- AWS CLI does not support SSE-C directly (must use SDK)

DSSE-KMS: two independent layers of AES-256 encryption. Required for some government/financial compliance frameworks.

Enforce encryption via bucket policy:

```
{
  "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:PutObject",
  "Resource": "arn:aws:s3:::my-bucket/*",
  "Condition": {
    "StringNotEquals": {
      "s3:x-amz-server-side-encryption": "aws:kms"
    }
  }
}
```

Default encryption: set at bucket level — all new objects encrypted with specified method unless overridden at upload time.

🔁 Replication

	CRR (Cross-Region)	**SRR** (Same-Region)
Use case	DR, latency reduction, compliance	Log aggregation, live replication between accounts, dev/test
Versioning required	Yes — source AND destination	Yes — source AND destination
Cost	Storage + inter-region transfer	Storage only (no transfer cost)
Existing objects	NOT replicated by default (use S3 Batch Replication)	NOT replicated by default
Delete markers	Optional (configurable)	Optional (configurable)
IAM role	Required (S3 assumes role to replicate)	Required
Encryption support	SSE-S3, SSE-KMS (with KMS permissions), SSE-C not supported	Same

Replication requirements:

1. Versioning enabled on both source and destination
2. IAM role with `s3:GetObject`, `s3:ReplicateObject`, `s3:ReplicateDelete` permissions
3. Bucket policy on destination allowing source account to replicate (cross-account)

Replication Time Control (RTC):

- 99.99% of objects replicated within 15 minutes
- Replication metrics visible in CloudWatch
- Additional cost on top of standard replication
- Required for strict RPO compliance

Replica Modification Sync: replicates metadata changes (not just new objects) back from replica to source — bidirectional sync.

S3 Batch Replication: replicate existing objects that were present before replication was configured. Creates a Batch Operations job targeting existing objects.

What is NOT replicated:

- Objects already replicated from another bucket (no chaining by default)
- Delete markers (unless explicitly enabled)
- Objects in Glacier (unless restored)
- Objects encrypted with SSE-C
- Objects without replication permissions on the IAM role

🗓️ Lifecycle Policies

Two action types:

- **Transition:** move objects to a different storage class
- **Expiration:** delete objects (or versions/delete markers)

Minimum transition days (from upload/last transition):

Transition	Minimum days
Standard → Standard-IA or One Zone-IA	30 days
Standard → Glacier Instant Retrieval	90 days
Standard → Glacier Flexible Retrieval	90 days
Standard → Glacier Deep Archive	90 days
Standard-IA → Glacier	0 additional days (but 30-day minimum in IA means at least 30 days total)

> **Note:** You cannot transition directly from Standard to One Zone-IA if there is a minimum days conflict. Transition ordering must be chronological.

Expiration actions:

- Delete current version (versioning disabled or suspended)
- Expire current version (create delete marker, versioning enabled)
- Permanently delete non-current versions (specify `NoncurrentDays`)
- Delete expired delete markers
- Abort incomplete multipart uploads (specify `DaysAfterInitiation`)

Lifecycle with versioning — best practices:

- Set `NoncurrentVersionExpiration` to delete old versions after N days
- Set `AbortIncompleteMultipartUpload` (e.g., 7 days) to prevent accumulating partial uploads
- Set `ExpiredObjectDeleteMarker` to clean up orphaned delete markers

📣 Event Notifications

Trigger automated workflows on S3 object events.

Supported events: `s3:ObjectCreated:*`, `s3:ObjectRemoved:*`, `s3:ObjectRestore:*`, `s3:ReducedRedundancyLostObject`, `s3:Replication:*`, `s3:LifecycleExpiration:*`, `s3:IntelligentTiering`, `s3:ObjectTagging:*`

Destinations:

Destination	Notes
SNS	Fan-out to multiple subscribers
SQS	Decouple processing, buffering
Lambda	Immediate processing
EventBridge	ALL event types, rich filtering, fan-out to 20+ targets, event replay, archive

> **EventBridge is the recommended destination** for new architectures: catches all events (S3 only sends subset to SNS/SQS/Lambda directly), supports advanced filtering by prefix/suffix/metadata, fan-out to multiple targets, cross-account event delivery.

Enabling EventBridge: must explicitly enable on the bucket (`PutBucketNotificationConfiguration` with `EventBridgeConfiguration: {}`).

Direct notification limitation: S3 → SNS/SQS/Lambda only fires for *specific* event types. EventBridge receives *all* S3 events including metadata-level events not available via direct notifications.

⚡ Performance

Default request rate limits per prefix:

- 3,500 PUT/COPY/POST/DELETE requests/sec
- 5,500 GET/HEAD requests/sec

Prefix strategy: S3 scales independently per prefix. Use multiple prefixes (e.g., hashed key prefixes) to multiply throughput. Old advice (pre-2018) was to use random prefixes — now S3 auto-scales, but explicit prefix distribution still helps for very high throughput.

Multipart Upload:

- Recommended for objects > 100 MB
- Required for objects > 5 GB
- Up to 10,000 parts
- Each part (except last) minimum 5 MB
- Parts uploaded in parallel → higher throughput
- Failed uploads accumulate incomplete parts — use lifecycle `AbortIncompleteMultipartUpload`

S3 Transfer Acceleration:

- Uses CloudFront edge locations as entry points
- Traffic travels AWS backbone from edge to S3 region
- ~60% faster for geographically distant uploads
- Additional cost: $0.04–0.08/GB on top of standard transfer
- Test with: `bucket.s3-accelerate.amazonaws.com`
- Must be enabled per bucket; not all regions support it

Byte-Range Fetches:

- Download specific byte range of large object in parallel (like multipart but for downloads)
- Parallelize with multiple concurrent GET requests with `Range` header
- Useful for large files: CSV headers, first N bytes, parallel download reconstruction

---

🔍 S3 Select & Glacier Select

Run SQL queries directly on S3/Glacier objects — retrieve only matching data without downloading full object.

Supported formats: CSV, JSON, Apache Parquet **Supported compression:** GZIP, BZIP2 (CSV and JSON only; Parquet has built-in compression)

```
SELECT s.name, s.age FROM S3Object s WHERE s.age > 30
```

Benefits:

- Up to 400% faster, 80% less data transfer
- Billed per GB scanned and per GB returned
- Server-side filtering — reduce cost and latency

Limitations: no JOINs, no GROUP BY, no aggregate functions on Parquet, one object per query.

Glacier Select: same concept but on archived Glacier objects. Object must be in Glacier Flexible Retrieval (not Deep Archive). Useful for compliance queries on archived logs.

🔒 Object Lock

WORM (Write Once, Read Many) protection — objects cannot be modified or deleted for a retention period.

Must be enabled at **bucket creation** (cannot enable on existing bucket). Automatically enables versioning.

Retention Modes:

Mode	Who can override/shorten?	Use case
Compliance	Nobody — not even root, not even AWS Support	SEC 17a-4, FINRA, regulatory compliance
Governance	Users with `s3:BypassGovernanceRetention` permission	Internal data protection, testable WORM

Retention Period: set per-object or as bucket default. Defines until when the object version is WORM-protected.

Legal Hold:

- Separate from retention period — no expiry date
- Toggle on/off with `s3:PutObjectLegalHold` permission
- Remains until explicitly removed
- Used for litigation holds, investigation holds

Vault Lock (Glacier): similar WORM concept for Glacier Vaults. Uses a Vault Lock Policy that once locked (24-hour confirmation window) cannot be changed. Maps to Compliance mode concept.

> **Exam trap:** Object Lock Compliance mode cannot be shortened/overridden by ANY user including root. Even AWS cannot delete the object before the retention period. This is the key differentiator from Governance mode.

📊 S3 Inventory

Generates a **flat-file report** of all objects and metadata in a bucket on a scheduled basis.

Schedule: daily or weekly **Output formats:** CSV, ORC, Parquet **Destination:** another S3 bucket (can be different account) **Fields available:** key, size, last modified, ETag, storage class, encryption status, replication status, Object Lock status, intelligent-tiering tier, checksum

Use cases:

- Audit encryption status across all objects
- Find objects not yet replicated
- Feed S3 Batch Operations manifest
- Compliance reporting
- Cost analysis by storage class

Query with Athena: use Glue crawler on inventory output → query with Athena SQL for large-scale object analytics.

⚙️ S3 Batch Operations

Run operations on **billions of objects** at scale via a managed job.

Job types:

Operation	Description
`Copy`	Copy objects (between buckets, accounts, regions)
`Invoke Lambda`	Custom processing via Lambda function
`Restore`	Restore from Glacier
`Replace tags`	Bulk tag update
`Replace ACL`	Bulk ACL update
`Put Object Lock retention`	Apply retention to existing objects
`Put Object Legal Hold`	Apply legal hold to existing objects

Manifest: list of objects to process — either S3 Inventory output (CSV/ORC/Parquet) or custom CSV manifest.

Job flow:

1. Create manifest (or use Inventory)
2. Create Batch Operations job (specify operation, manifest, IAM role, completion report destination)
3. Review and confirm
4. Job runs across objects with managed retries
5. Completion report (success/failure per object) written to S3

IAM role: must have permissions for the operation on source and destination. For Lambda: must have `lambda:InvokeFunction`.

📋 Access Logs & CloudTrail

	Server Access Logging	CloudTrail Data Events
What it captures	HTTP request to bucket	S3 API calls (object-level)
Granularity	Request details (requester IP, user agent, response code)	API call, caller identity (IAM ARN), request time
Delivery	Best-effort, not guaranteed, may be hours delayed	Near real-time
Cost	Free (only pay for destination storage)	$0.10 per 100,000 events
Use for	Debugging, access pattern analysis	Security audit, who-accessed-what, compliance
Requester identity	IP address only (no IAM identity)	Full IAM identity (user, role, account)

CloudTrail S3 data events: must be explicitly enabled (not on by default — too high volume/cost). Enable for specific buckets or all S3. Captures `GetObject`, `PutObject`, `DeleteObject`, `ListObjects`.

📈 S3 Storage Lens

Organization-wide S3 storage analytics across all accounts in AWS Organizations.

Dashboard types:

- **Free (default):** 28 metrics, 14-day retention, account and bucket level
- **Advanced (paid):** 35+ metrics, 15-month retention, prefix-level metrics, CloudWatch publishing

Key metrics: total storage, object count, request count, data transfer, cost savings recommendations, replication metrics, encryption coverage, access pattern analysis.

Recommendations: Intelligent-Tiering candidates, lifecycle optimization opportunities, unencrypted objects, incomplete multipart uploads.

Scope: Organization-level (all accounts) or account-level or specific buckets.

🌐 Static Website Hosting

Enable on a bucket to serve HTML/CSS/JS as a website.

Requirements:

- Bucket name must match domain name (for Route 53 alias)
- Block Public Access must be disabled (for truly public sites)
- Bucket policy must allow `s3:GetObject` for `"Principal": "*"`
- Set index document (e.g., `index.html`) and error document (e.g., `error.html`)

URL format: `http://bucket-name.s3-website-region.amazonaws.com` (HTTP only — no HTTPS directly)

HTTPS: put CloudFront in front with ACM certificate. CloudFront → S3 origin (use OAC — Origin Access Control).

Custom domain: Route 53 Alias record to S3 website endpoint (must match bucket name exactly).

Redirect: entire bucket can redirect to another domain (e.g., `www.example.com` → `example.com`).

🌍 CORS (Cross-Origin Resource Sharing)

Needed when **browser JavaScript** on domain A makes XHR/fetch requests to S3 bucket on domain B.

CORS configuration (JSON or XML on bucket):

```
[{
  "AllowedOrigins": ["https://www.example.com"],
  "AllowedMethods": ["GET", "PUT"],
  "AllowedHeaders": ["*"],
  "MaxAgeSeconds": 3000
}]
```

Preflight request: browser sends OPTIONS request first — S3 must respond with correct CORS headers. Misconfigured CORS → browser blocks request even if server returns data.

Not needed for: server-to-server S3 access (only browsers enforce CORS), same-origin requests.

🔁 S3 Object Lambda

Transform objects on retrieval without storing modified copies.

How it works:

1. Create S3 Object Lambda Access Point
2. Client GETs object via Object Lambda Access Point
3. S3 invokes Lambda function with object URL
4. Lambda retrieves, transforms, returns modified data to client
5. Client receives transformed object — original S3 object unchanged

Use cases:

- PII redaction (mask SSNs before returning to specific users)
- Format conversion (CSV → JSON on the fly)
- Image watermarking or resizing
- Row-level filtering for different consumers
- Dynamic content enrichment

Components required:

- Standard S3 Access Point (Lambda reads original from this)
- Lambda function (transform logic)
- Object Lambda Access Point (client calls this)

💳 Requester Pays

When **Requester Pays** is enabled on a bucket:

- Data transfer costs charged to the **requester** (not bucket owner)
- Request costs charged to the **requester**
- Requester must be authenticated (no anonymous access when Requester Pays is enabled)

Use case: data distribution — you host public datasets and don't want to pay download costs. Requester (researcher, partner) pays for their own download.

How to request: requester must include `x-amz-request-payer: requester` header.

🗄️ MountPoint for S3

FUSE-based Linux filesystem client for S3. Mounts S3 bucket as a local directory.

Supports:

- Sequential read/write
- Parallel reads
- Large file workloads (ML training data, genomics, media)

Does NOT support:

- Random write (append/overwrite mid-file)
- File locking
- Renaming directories
- POSIX full compatibility

Use case: EC2 or ECS workloads that read large datasets from S3 using filesystem APIs without changing application code. Not a replacement for EFS for general filesystem workloads.

🔄 Consistency Model

Since **December 2020**, S3 provides **strong read-after-write consistency** for all operations:

Operation	Consistency
PUT new object → GET	Strong (immediate)
PUT overwrite → GET	Strong (immediate)
DELETE → GET	Strong (immediately returns 404)
LIST after PUT	Strong (object appears in listing immediately)

> **No more eventual consistency.** Pre-2020 guidance (randomize prefixes for LIST eventual consistency) is obsolete for consistency reasons (still valid for throughput scaling).

📏 Limits Table

Limit	Value
Max buckets per account	100 (default soft), 1,000 (hard limit)
Max object size	5 TB
Max single PUT size	5 GB
Multipart upload parts	1–10,000
Min part size (multipart)	5 MB (except last part)
Max object key length	1,024 bytes
Max user metadata	2 KB
Max tags per object	10
Max lifecycle rules per bucket	1,000
Max event notification configurations	100 per bucket
Presigned URL max expiry (STS)	7 days
Presigned URL max expiry (IAM role)	12 hours
S3 Inventory destination	Must be same or different bucket, must have bucket policy

---

☁️ Azure Blob Storage Comparison

Feature	AWS S3	Azure Blob Storage
Storage tiers	Standard, IA, Glacier classes	Hot, Cool, Cold, Archive
Minimum storage duration	Varies by class	Cool: 30 days, Cold: 90 days, Archive: 180 days
Retrieval fees	Glacier classes only	Cool, Cold, Archive have retrieval fees
Object versioning	Built-in, explicit enable	Blob versioning (enable per storage account)
Replication	CRR/SRR with lifecycle	GRS, GZRS (geo-redundant, auto), custom replication policies
WORM	Object Lock (Compliance/Governance)	Immutability policies (locked/unlocked)
Static website	Built-in feature	Static website feature in storage account
Lifecycle	Lifecycle rules (transition + expiration)	Lifecycle management (tiering + delete)
Analytics	S3 Storage Lens	Azure Monitor + Storage Analytics
Query in-place	S3 Select	Azure Blob Query (similar, CSV/JSON)
Consistency	Strong read-after-write	Strong consistency (since 2020 like S3)
Namespace	Global unique bucket names	Storage account (globally unique) + container name
Max object size	5 TB	190.7 TB (block blob)
Access tiers per object	Yes (IA, Glacier etc.)	Yes (Hot/Cool/Archive per blob)

Key difference: Azure Blob uses a **Storage Account** as the top-level container (globally unique name), and containers within. AWS S3 bucket name itself is globally unique. Azure allows changing access tier on existing objects without copying (Archive → Hot rehydrates in place). AWS Glacier requires restore operation.

🗺️ S3 in Data Lake Architectures

S3 is the **foundation of the AWS data lake**:

```
Ingestion → S3 Raw Zone → S3 Processed Zone → S3 Curated Zone
              ↑               ↑                   ↑
           Kinesis         Glue ETL            Athena/Redshift Spectrum
           DMS             Lambda              QuickSight
```

AWS Lake Formation: governance layer on top of S3 + Glue. Column-level, row-level security on data in S3. Manages permissions centrally instead of S3 bucket policies per consumer.

Athena: serverless SQL on S3. Reads CSV/JSON/Parquet/ORC from S3. Uses Glue Data Catalog for table metadata. Pay per TB scanned — use Parquet/ORC + partitioning to minimize scan cost.

Glue: ETL service for transforming raw S3 data. Glue Crawler discovers schema → Glue Data Catalog → Athena/Redshift Spectrum can query.

S3 + Redshift Spectrum: query S3 data directly from Redshift without loading into cluster. Uses external tables defined in Glue Data Catalog.

🏛️ Well-Architected Framework

Pillar	S3 Considerations
Operational Excellence	Use S3 Inventory for auditing, Storage Lens for analytics, EventBridge for automation triggers, lifecycle policies for automated storage management
Security	Enforce encryption (SSE-KMS + bucket policy deny non-encrypted), Block Public Access at account level, use Access Points for fine-grained access, enable CloudTrail data events for sensitive buckets, use VPC Endpoints for private access
Reliability	S3 Standard provides 11 nines durability across ≥3 AZs. Enable CRR for DR. Enable versioning to protect against accidental deletion. Use MFA Delete for critical buckets. One Zone-IA is only single-AZ — use only for reproducible data.
Performance Efficiency	Use multipart upload for large objects, byte-range fetches for parallel downloads, S3 Transfer Acceleration for global uploads, S3 Select to reduce data transfer, prefix distribution for high-throughput workloads
Cost Optimization	Use S3 Intelligent-Tiering for unknown access patterns, Lifecycle policies to auto-transition to cheaper classes, S3 Select to reduce data scanned, Gateway Endpoints to eliminate NAT costs, abort incomplete multipart uploads, analyze with Storage Lens
Sustainability	Lifecycle policies move data to energy-efficient storage (Glacier has lower operational overhead per byte), S3 Select reduces data transfer network energy, Intelligent-Tiering right-sizes storage footprint automatically

---

📋 4 SAP-C02 Domains

Domain	S3 Relevance
Design Solutions for Organizational Complexity	Multi-account S3 access (cross-account bucket policies, Access Points, S3 Access Grants), S3 Storage Lens across Organizations, Lake Formation for organization-wide data governance, MRAP for multi-region resilience
Design for New Solutions	Choosing correct storage class for new workloads, designing data lake architecture on S3, Object Lock for compliance-by-design, S3 + EventBridge for event-driven architectures, presigned URLs for secure client-side upload
Continuous Improvement	Storage Lens recommendations for cost optimization, Intelligent-Tiering for automated class optimization, Batch Operations for retroactive policy application, lifecycle rule optimization, CRR for existing buckets
Workload Migration and Modernization	Migrating on-premises object stores to S3 (DataSync, Snow family), replacing Hadoop HDFS with S3 (EMR on S3), S3 as backend for containerized apps replacing local storage

---

⚠️ Exam Traps & Common Misconceptions

Misconception	Reality
"S3 is a global service"	S3 is **regional** — data stored in specified region. Bucket names are globally unique but each bucket belongs to one region.
"Versioning can be turned off once enabled"	Cannot return to **unversioned** — only can suspend versioning
"Object Lock can be enabled on existing buckets"	Must be enabled at **bucket creation** only
"Compliance mode Object Lock can be shortened by root"	**Nobody** can shorten/remove Compliance mode retention — not root, not AWS Support
"CRR replicates existing objects"	Only new objects after replication is configured. Use **S3 Batch Replication** for existing objects.
"Delete markers are replicated by default"	Delete marker replication must be **explicitly enabled** in replication config
"SSE-C works with the AWS CLI"	AWS CLI **does not support** SSE-C — must use SDK
"Gateway Endpoint routes to on-premises"	Gateway Endpoint **only works within the VPC** — cannot be accessed from on-premises
"S3 is eventually consistent"	S3 has **strong read-after-write consistency** since December 2020
"S3 static website supports HTTPS natively"	Static website hosting is **HTTP only** — use CloudFront + ACM for HTTPS
"All storage classes provide 11 nines durability"	One Zone-IA and Express One Zone store data in **single AZ** — if AZ fails, data may be lost
"S3 Transfer Acceleration uses Direct Connect"	Transfer Acceleration uses **CloudFront edge** locations, not DX
"MFA Delete can be enabled by any admin"	MFA Delete requires **root account** credentials to enable
"Presigned URL bypasses bucket policy"	Presigned URL uses the **signer's permissions** — if signer's access is revoked, URL becomes invalid. Bucket policy deny still applies.
"S3 Intelligent-Tiering has retrieval fees for all tiers"	Frequent Access and Infrequent Access tiers have **no retrieval fee**. Archive tiers do.
"Lifecycle can transition objects to One Zone-IA from Glacier"	Cannot transition **backward** — only to same-cost or cheaper (deeper archive) classes

🔐 Chapter 23: AWS IAM — Identity and Access Management

> **Scope:** Global (IAM is a global service — users, roles, groups, and policies are not region-specific) **Counterpart:** Azure Active Directory (Entra ID) / Azure RBAC | Linux: PAM, sudoers, /etc/passwd **Exam weight:** Very high — IAM appears in ALL four SAP-C02 domains

---

📖 23.1 Service Overview

AWS Identity and Access Management (IAM) is the **foundational security service** for controlling who can do what in AWS. It provides authentication (who are you?) and authorization (what can you do?). Every API call to every AWS service is evaluated against IAM policies — no exceptions.

IAM is **free** — no additional charge for creating users, roles, groups, or policies. You pay only for the underlying AWS resources accessed.

Key capabilities:

- Fine-grained access control to any AWS service and resource
- Identity federation (SAML 2.0, OIDC, web identity)
- Temporary credentials via AWS STS (Security Token Service)
- Cross-account access without sharing credentials
- Permission boundaries and SCPs for layered governance
- Attribute-based access control (ABAC) via tags and conditions

---

🧱 23.2 Core Components

23.2.1 IAM Users

A **long-lived identity** representing a human or application. Has permanent credentials:

- **Password** — for AWS Management Console access
- **Access key ID + Secret access key** — for programmatic API access
- **MFA device** — optional but strongly recommended

```
# Create a user
aws iam create-user --user-name alice

# Create access keys (programmatic access)
aws iam create-access-key --user-name alice

# Enable MFA (virtual device)
aws iam enable-mfa-device \
  --user-name alice \
  --serial-number arn:aws:iam::123456789:mfa/alice \
  --authentication-code1 123456 \
  --authentication-code2 654321
```

> ⚠️ **Best practice:** Avoid long-lived IAM user access keys. Use roles with temporary credentials wherever possible. Root account: never use except for billing/account setup.

23.2.2 IAM Groups

A **collection of IAM users**. Attach policies to the group — all members inherit. Groups **cannot be nested** (no groups within groups). A user can belong to **up to 10 groups**.

```
# Create group, attach policy, add user
aws iam create-group --group-name Developers
aws iam attach-group-policy \
  --group-name Developers \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
aws iam add-user-to-group --group-name Developers --user-name alice
```

23.2.3 IAM Roles

A **temporary identity** with no long-lived credentials. When assumed, AWS STS issues short-lived credentials (Access Key ID, Secret Access Key, Session Token) valid for 15 minutes to 12 hours.

Roles have two policies:

1. **Trust policy** — who/what can assume this role (resource-based policy on the role)
2. **Permission policy** — what the assumed role can do (identity-based policy)

```
# Create role with EC2 trust
aws iam create-role \
  --role-name MyEC2Role \
  --assume-role-policy-document '{
    "Version": "2012-10-17",
    "Statement": [{
      "Effect": "Allow",
      "Principal": {"Service": "ec2.amazonaws.com"},
      "Action": "sts:AssumeRole"
    }]
  }'

# Attach permission policy
aws iam attach-role-policy \
  --role-name MyEC2Role \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess

# Assume role (from CLI/SDK)
aws sts assume-role \
  --role-arn arn:aws:iam::123456789:role/MyEC2Role \
  --role-session-name MySession
```

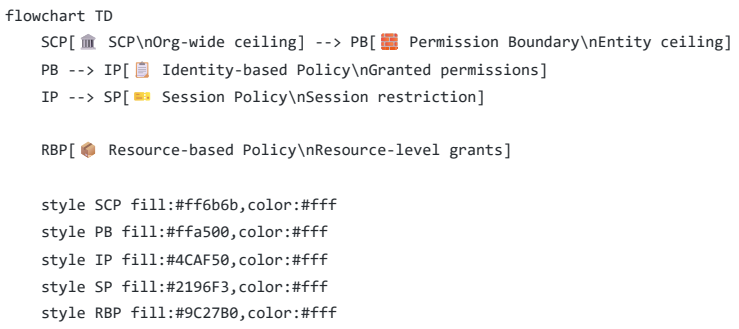
Common role use cases:

Use Case	Principal in Trust Policy	Notes
EC2 instance profile	`ec2.amazonaws.com`	No hardcoded keys in EC2
Lambda execution	`lambda.amazonaws.com`	Permissions for Lambda function
Cross-account access	`arn:aws:iam::OTHER_ACCOUNT:root`	Account B trusts account A
EKS service account (IRSA)	OIDC provider URL	Pod-level IAM, not node-level
GitHub Actions	OIDC token from GitHub	CI/CD without static keys
AWS service (e.g. CloudFormation)	`cloudformation.amazonaws.com`	Service creates resources on your behalf

23.2.4 Service-Linked Roles

Special roles **pre-created by AWS services** with fixed trust and permissions. You cannot modify the permission policy. Example: `AWSServiceRoleForElasticLoadBalancing`. AWS creates these automatically when you first use a service. Cannot be deleted while the service is using them.

📜 23.3 Policy Types



23.3.1 Identity-based Policies

Attached **to an IAM identity** (user, group, or role). Define what the identity can do.

Two types:

- **AWS Managed** — created and managed by AWS (e.g., `AmazonS3ReadOnlyAccess`). Updated by AWS automatically. Cannot be modified.
- **Customer Managed** — created by you. Full control. Recommended over inline.
- **Inline** — embedded directly in a user/group/role. Deleted when identity is deleted. Avoid — hard to audit and reuse.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::my-bucket/*",
      "Condition": {
        "StringEquals": {
          "s3:prefix": "home/${aws:username}/"
        }
      }
    }
  ]
}
```

23.3.2 Resource-based Policies

Attached **to a resource** (S3 bucket, SQS queue, KMS key, Lambda, SNS, Secrets Manager, etc.). Require a `Principal` field. Enable **cross-account access without role assumption**.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "AWS": "arn:aws:iam::222222222222:role/CrossAccountRole"
    },
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::company-shared-bucket/*"
  }]
}
```

Services that support resource-based policies:

Service	Policy Name
S3	Bucket Policy
SQS	Queue Policy
SNS	Topic Policy
KMS	Key Policy (required — no policy = no access)
Lambda	Function Policy
Secrets Manager	Resource Policy
ECR	Repository Policy
API Gateway	Resource Policy
CloudWatch Logs	Resource Policy

> ⚠️ **KMS is special:** KMS key policy is **mandatory**. Without an explicit key policy, even the root account cannot use the key. Key policies are always evaluated; identity-based policies alone are never sufficient for KMS.

23.3.3 AWS Organizations SCPs (Service Control Policies)

Ceiling on permissions for an entire AWS account or OU. SCPs do NOT grant permissions — they only restrict the maximum allowed. Even `AdministratorAccess` is limited by SCPs.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "Action": "*",
    "Resource": "*",
    "Condition": {
      "StringNotEquals": {
        "aws:RequestedRegion": ["eu-central-1", "eu-west-1"]
      }
    }
  }]
}
```

This SCP denies ALL actions outside the specified regions — enforces data residency at the org level.

SCP rules:

- SCPs affect all principals in the account **except the management account**
- Root user of member accounts IS affected by SCPs
- SCPs don't affect service-linked roles
- `FullAWSAccess` SCP must be attached for any access (default when OU created)

23.3.4 Permission Boundaries

A **maximum permissions guardrail** for a specific IAM user or role. Even if identity-based policies allow more, the boundary limits the effective permissions. Used to **safely delegate permission management** — an admin can allow developers to create roles, but only roles with a specific boundary (preventing privilege escalation).

```
# Create role with permission boundary
aws iam create-role \
  --role-name DeveloperRole \
  --assume-role-policy-document file://trust.json \
  --permissions-boundary arn:aws:iam::123456789:policy/DeveloperBoundary
```

Effective permissions = Intersection of:

- Identity-based policy (what's allowed)
- Permission boundary (maximum ceiling)

23.3.5 Session Policies

Passed **at AssumeRole time** to further restrict permissions. Cannot grant MORE than the role's attached policies — only restrict. Useful for least-privilege temporary sessions.

```
aws sts assume-role \
  --role-arn arn:aws:iam::123456789:role/DataRole \
  --role-session-name ReadOnlySession \
  --policy '{
    "Version": "2012-10-17",
    "Statement": [{
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::data-bucket/public/*"
    }]
  }'
```

🔑 23.4 Policy Evaluation Logic

This is one of the most important SAP-C02 concepts:

```
flowchart TD
    A[API Request] --> B{Explicit DENY\nin any policy?}
    B -->|Yes| DENY1[❌ DENY]
    B -->|No| C{Within\nSCP ceiling?}
    C -->|No| DENY2[❌ DENY]
    C -->|Yes| D{Within\nPermission Boundary?}
    D -->|No| DENY3[❌ DENY]
    D -->|Yes| E{Within\nSession Policy?}
    E -->|No| DENY4[❌ DENY]
    E -->|Yes| F{Explicit ALLOW\nin identity-based\nOR resource-based?}
    F -->|No| DENY5[❌ DENY - implicit]
    F -->|Yes| G{Cross-account?}
    G -->|Yes| H{BOTH identity-based\nAND resource-based\nallow?}
    H -->|No| DENY6[❌ DENY]
    H -->|Yes| ALLOW[✅ ALLOW]
    G -->|No, same account| ALLOW2[✅ ALLOW]
```

Critical rules:

1. **Default deny** — everything denied unless explicitly allowed
2. **Explicit deny always wins** — even over explicit allow
3. **Same account** — identity-based OR resource-based policy is sufficient
4. **Cross-account** — BOTH identity-based AND resource-based must allow
5. **SCPs never grant** — only restrict (no SCP = no access regardless of identity policy)

🏷️ 23.5 ABAC — Attribute-Based Access Control

Instead of creating a policy per resource, use **tags** + **policy conditions** to scale access control.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": [
      "ec2:StartInstances",
      "ec2:StopInstances",
      "ec2:TerminateInstances"
    ],
    "Resource": "*",
    "Condition": {
      "StringEquals": {
        "ec2:ResourceTag/team": "${aws:PrincipalTag/team}"
      }
    }
  }]
}
```

This policy allows users to manage EC2 instances **only when the instance's `team` tag matches the user's `team` tag**. One policy handles all teams — no per-team policy updates needed.

ABAC vs RBAC comparison:

	RBAC (Role-Based)	ABAC (Attribute-Based)
Policy count	Grows with resource count	Fixed, scales automatically
Setup	Simple, explicit	Complex initial setup
Maintenance	Update per new resource	Update tags, not policies
Granularity	Coarse (role level)	Fine (attribute level)
Best for	Stable, well-defined access patterns	Dynamic resources, many teams

---

🌐 23.6 Federation and External Identity

23.6.1 SAML 2.0 Federation

Integrates corporate identity providers (Active Directory, Okta, Ping) with AWS. Users log into the IdP → IdP returns SAML assertion → AWS STS `AssumeRoleWithSAML` → temporary credentials.

```
sequenceDiagram
    participant User
    participant IdP as Corporate IdP (AD/Okta)
    participant STS as AWS STS
    participant AWS as AWS Console/API

    User->>IdP: Authenticate (username/password + MFA)
    IdP-->>User: SAML Assertion (XML)
    User->>STS: AssumeRoleWithSAML (SAML assertion)
    STS-->>User: Temporary credentials (15min-12hr)
    User->>AWS: API calls with temp credentials
```

Required setup:

1. Register AWS as a service provider in the IdP
2. Create IAM SAML Identity Provider (upload IdP metadata XML)
3. Create IAM roles with trust to the SAML provider
4. Map SAML attributes to AWS roles (attribute: `https://aws.amazon.com/SAML/Attributes/Role`)

23.6.2 OIDC Federation (Web Identity)

For workloads: GitHub Actions, EKS (IRSA), Cognito, any OIDC-compliant IdP.

```
# Create OIDC provider for GitHub Actions
aws iam create-open-id-connect-provider \
  --url https://token.actions.githubusercontent.com \
  --client-id-list sts.amazonaws.com \
  --thumbprint-list 6938fd4d98bab03faadb97b34396831e3780aea1
```

Trust policy for GitHub Actions:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "Federated": "arn:aws:iam::123456789:oidc-provider/token.actions.githubusercontent.com"
    },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
        "token.actions.githubusercontent.com:sub": "repo:my-org/my-repo:ref:refs/heads/main"
      }
    }
  }]
}
```

23.6.3 IAM Identity Center (formerly SSO)

The **recommended** enterprise federation solution. Single sign-on for multiple AWS accounts and cloud applications.

	Direct IAM Federation	IAM Identity Center
Management	Per-account setup	Centralized (org-wide)
Account onboarding	Manual per account	Automatic via permission sets
Application SSO	No	Yes (SAML apps)
MFA	Per IdP	Centralized
SCIM provisioning	No	Yes (auto user sync)
Audit	CloudTrail per account	Centralized CloudTrail
Best for	Single account, simple setup	Multi-account enterprise

Permission Sets = reusable "role templates" applied across accounts. Define once in Identity Center → assign to account → IAM role auto-created in that account.

🔒 23.7 Conditions — Fine-Grained Control

IAM condition keys allow context-aware policies:

```
{
  "Condition": {
    "Bool": {
      "aws:MultiFactorAuthPresent": "true"        // MFA required
    },
    "StringEquals": {
      "aws:RequestedRegion": "eu-central-1",       // Region lock
      "aws:PrincipalOrgID": "o-exampleorgid111"   // Org membership required
    },
    "IpAddress": {
      "aws:SourceIp": ["10.0.0.0/8", "172.16.0.0/12"]  // IP restriction
    },
    "ArnLike": {
      "aws:SourceArn": "arn:aws:sns:*:123456789:*"      // Source ARN
    },
    "DateLessThan": {
      "aws:CurrentTime": "2026-12-31T23:59:59Z"         // Time-bound access
    },
    "StringNotEquals": {
      "aws:PrincipalAccount": "123456789012"             // Deny if specific account
    }
  }
}
```

Important condition key groups:

Prefix	Scope	Examples
`aws:`	Global, all services	`aws:SourceIp` , `aws:PrincipalTag` , `aws:RequestedRegion`
`s3:`	S3-specific	`s3:prefix` , `s3:delimiter` , `s3:ResourceAccount`
`iam:`	IAM-specific	`iam:PermissionsBoundary` , `iam:PassedToService`
`ec2:`	EC2-specific	`ec2:ResourceTag` , `ec2:Region` , `ec2:InstanceType`
`sts:`	STS-specific	`sts:ExternalId` , `sts:RoleSessionName`

23.8 Cross-Account Access Patterns

Pattern 1: Role Assumption (most common)

```
flowchart LR
    subgraph AccountA["Account A (trusting)"]
        User["IAM User/Role"]
    end
    subgraph AccountB["Account B (trusted)"]
        Role["IAM Role\n(trust policy:\nAccount A)"]
        S3["S3 Bucket"]
    end
    User -->|"sts:AssumeRole"| Role
    Role -->|"s3:GetObject"| S3
```

Account B role trust policy:

```
{
  "Effect": "Allow",
  "Principal": {"AWS": "arn:aws:iam::ACCOUNT_A:role/DevRole"},
  "Action": "sts:AssumeRole",
  "Condition": {
    "StringEquals": {"sts:ExternalId": "secret-token-123"}
  }
}
```

The `ExternalId` condition prevents the **confused deputy problem** — a third-party service can't be tricked into assuming your role on behalf of an attacker.

Pattern 2: Resource-based Policy (no role assumption)

For S3, SQS, SNS, Lambda: no need to assume a role — the resource policy directly grants cross-account access. The calling identity retains its identity (CloudTrail shows the actual caller). Simpler than role assumption for AWS-to-AWS service access.

Pattern 3: AWS Organizations `aws:PrincipalOrgID`

```
{
  "Effect": "Allow",
  "Principal": "*",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::shared-bucket/*",
  "Condition": {
    "StringEquals": {
      "aws:PrincipalOrgID": "o-exampleorgid111"
    }
  }
}
```

Anyone in the org can access — no need to enumerate individual account IDs. As accounts join/leave the org, access automatically applies/revokes.

🔐 23.9 Credential Types and Security

Credential Type	Duration	Use Case	How to Get
Root account password	Permanent	Account setup only	Existing
IAM user password	Permanent	Console login	IAM create-login-profile
IAM access keys	Permanent (until rotated)	Programmatic (avoid)	iam:create-access-key
STS temporary credentials	15 min – 12 hr	Everything recommended	sts:AssumeRole
EC2 instance metadata (IMDS)	~6 hours, auto-rotated	EC2 workloads	http://169.254.169.254
ECS task credentials	~6 hours, auto-rotated	Container workloads	ECS credential endpoint
Lambda env credentials	~1 hour, auto-rotated	Lambda functions	Environment variables

Credentials priority order (AWS SDK default chain):

1. Explicit code credentials (avoid)
2. Environment variables (`AWS_ACCESS_KEY_ID`)
3. AWS credentials file (`~/.aws/credentials`)
4. IAM role for EC2 instance (IMDS)
5. ECS task role
6. EKS IRSA (pod-level)

Access Key Best Practices

```
# List all users with access keys (audit)
aws iam list-users --query 'Users[*].UserName' --output text | \
  xargs -I {} aws iam list-access-keys --user-name {}

# Check last used date
aws iam get-access-key-last-used --access-key-id AKIAIOSFODNN7EXAMPLE

# Rotate key: create new, update application, delete old
aws iam create-access-key --user-name alice
# [update application config with new key]
aws iam update-access-key --access-key-id OLD_KEY_ID --status Inactive
aws iam delete-access-key --access-key-id OLD_KEY_ID
```

📊 23.10 IAM Access Analyzer

Analyzes resource-based policies to identify **external access** — resources shared with external principals (other accounts, public, or the internet).

```
# Create analyzer for the organization
aws accessanalyzer create-analyzer \
  --analyzer-name OrgAnalyzer \
  --type ORGANIZATION

# List findings (externally accessible resources)
aws accessanalyzer list-findings --analyzer-name OrgAnalyzer
```

Analyzer types:

Type	Zone of Trust	Flags
Account	Single account	Access from outside the account
Organization	Entire org	Access from outside the organization

Additional capabilities:

- **Policy validation** — checks policies against IAM best practices and grammar
- **Policy generation** — generates least-privilege policies from CloudTrail activity
- **Unused access findings** — identifies unused roles, access keys, passwords (90+ days unused)

```
# Generate policy from CloudTrail activity (last 90 days)
aws accessanalyzer start-policy-generation \
  --policy-generation-details '{"principalArn": "arn:aws:iam::123:role/MyRole"}' \
  --cloud-trail-details '{"trails": [{"cloudTrailArn": "arn:aws:cloudtrail:...", "allRegions": true}], "startTime": "2026-08-01T0
```

🔑 23.11 iam:PassRole — The Hidden Permission

When a service creates resources on your behalf (EC2 launches with an instance profile, Lambda creates with an execution role), the **caller must have** `iam:PassRole` permission for the role being passed to the service.

```
{
  "Effect": "Allow",
  "Action": "iam:PassRole",
  "Resource": "arn:aws:iam::123456789:role/MyEC2Role",
  "Condition": {
    "StringEquals": {
      "iam:PassedToService": "ec2.amazonaws.com"
    }
  }
}
```

Without `iam:PassRole`: user cannot launch EC2 with an instance profile even if they have `ec2:RunInstances`. With unscoped `iam:PassRole`: user can assign **any role** to a service — potential privilege escalation vector.

Best practice: always scope `iam:PassRole` to specific role ARNs and services.

📈 23.12 IAM Limits

Resource	Default Limit	Can Increase?
IAM users per account	5,000	Yes (support ticket)
Groups per account	300	Yes
Roles per account	1,000	Yes
Managed policies per account	1,500	Yes
Managed policies attached to one identity	10	Yes (up to 20)
Inline policy size	2,048 characters	No
Managed policy size	6,144 characters	No
Policy versions	5 per managed policy	No
Groups a user can belong to	10	No
Role session duration	1 hr – 12 hr	No (hard limit)
STS temporary credential duration	15 min – 12 hr	No

🌎 23.13 IAM vs Azure AD / Entra ID

Feature	AWS IAM	Azure Entra ID (AAD)
Identity types	Users, Groups, Roles	Users, Groups, Service Principals, Managed Identities
Service identity	IAM Role / Instance Profile	Managed Identity (system/user assigned)
Federation	SAML, OIDC	SAML, OIDC, WS-Federation
RBAC scope	Resource-level (ARN)	Subscription → RG → Resource
Policy format	JSON statements	Azure RBAC JSON / ARM policies
MFA	Per-user, per-role (session)	Conditional Access policies
Multi-account	AWS Organizations + SCP	Azure Management Groups
SSO	IAM Identity Center	Microsoft Entra ID (AAD)
Audit	CloudTrail	Azure Monitor / Activity Log
Cost	Free	Free tier + P1/P2 paid tiers

🏗️ 23.14 IAM in the Well-Architected Framework

Pillar	IAM Considerations
Security	Principle of least privilege. MFA for all humans. No root account usage. Rotate credentials. IAM Access Analyzer. No public access (bucket policy + IAM). Separate accounts per environment (dev/prod). SCPs to enforce security guardrails.
Reliability	IAM is globally redundant — no single region failure affects IAM. STS endpoints are regional — use regional STS endpoints for resilience. Avoid role assumption chains > 5 hops. Test permissions with Policy Simulator before deployment.
Performance Efficiency	IAM policy evaluation is sub-millisecond (in-path to every API call). Keep policies concise — very large policies have higher evaluation overhead. Use condition keys to reduce policy count (ABAC).
Cost Optimization	IAM itself is free. Least privilege reduces blast radius of misconfigurations. Permission boundaries enable self-service with guardrails (reduces ops overhead). IAM Access Analyzer unused access findings: identify over-provisioned access to clean up.
Operational Excellence	Use permission sets (Identity Center) not per-account IAM roles. Use managed policies (not inline) for visibility and reuse. Tag IAM resources for cost attribution. Automate access key rotation (Lambda + IAM). Use AWS Config for compliance monitoring.
Sustainability	No direct sustainability impact from IAM. Correct IAM configuration prevents accidental large resource provisioning (cost + carbon). Efficient ABAC reduces policy proliferation.

🗂️ 23.15 IAM and the 4 SAP-C02 Domains

Domain 1: Design Solutions for Organizational Complexity

- **Multi-account IAM**: Use AWS Organizations + IAM Identity Center. Permission sets deployed across accounts. One place to manage access for all accounts.
- **SCP strategy**: Preventive controls at org level. Deny dangerous actions (deleting CloudTrail, disabling GuardDuty, leaving org) via SCPs.
- **Cross-account roles**: Hub-and-spoke model — central security account assumes roles in member accounts for auditing, remediation.
- **Federated identity at scale**: Active Directory + AWS IAM Identity Center + SCIM provisioning. Employees onboard/offboard automatically synced.

Domain 2: Design for New Solutions

- **Zero hardcoded credentials**: All workloads use IAM roles. EC2: instance profile. Lambda: execution role. EKS: IRSA. GitHub Actions: OIDC.
- **ABAC for dynamic resources**: New resources automatically accessible to the right teams via tag-based policies.
- **Service-to-service**: Use resource-based policies for same-account access. Role assumption for cross-account.
- **Permission boundaries for developer self-service**: Developers create roles for their services, but bounded to their namespace.

Domain 3: Continuous Improvement

- **IAM Access Analyzer**: Continuously scan for external access + unused credentials.
- **AWS Config IAM rules**: `iam-user-mfa-enabled`, `iam-user-no-policies-check`, `iam-password-policy`, `access-keys-rotated`.
- **CloudTrail → Athena**: Analyze IAM usage patterns. Find unused roles. Detect privilege escalation attempts.
- **Least privilege tightening**: Generate minimum-privilege policies from actual CloudTrail usage (Access Analyzer policy generation).

Domain 4: Workload Migration and Modernization

- **On-premises AD migration**: Phase 1: Direct federation via SAML. Phase 2: migrate to IAM Identity Center with AD Connector or Managed AD.
- **Static keys → roles**: Replace application access keys with EC2 instance profiles or EKS IRSA during lift-and-shift.

- **Permission right-sizing**: Legacy apps often over-provisioned. Use Access Analyzer + policy generation to create minimum-privilege policies during modernization.
- **Container IAM**: Move from node-level EC2 roles to task-level ECS roles or IRSA (pod-level) during containerization.

🔍 23.16 Common Exam Scenarios

Scenario: Developer can't access a resource despite having the right policy → Check: SCP, permission boundary, explicit deny in resource policy, cross-account evaluation (need both sides), condition keys evaluating to false.

Scenario: Application needs to access resources in another account → Answer: Role assumption (cross-account role) OR resource-based policy on the target resource.

Scenario: Third-party vendor needs access to your S3 bucket → Answer: Cross-account role with `ExternalId` condition (prevents confused deputy). NOT a separate IAM user with access keys.

Scenario: 500 microservices each need unique least-privilege policies → Answer: ABAC — tag-based policies + service tags. One policy per permission pattern, not per microservice.

Scenario: Prevent privilege escalation by developers → Answer: Permission boundaries on all developer-created roles. IAM policy cannot exceed the boundary.

Scenario: Audit which principals can access a specific resource → Answer: IAM Access Analyzer — analyzes resource policy + identity policies to show all effective access paths.

🔗 23.17 Related Services Quick Reference

Service	Relationship to IAM
AWS STS	Issues temporary credentials when roles are assumed. Regional endpoints.
AWS Organizations	Org structure that SCPs apply to. Enables `aws:PrincipalOrgID` condition.
IAM Identity Center	Built on top of IAM roles. Manages permission sets at scale.
AWS CloudTrail	Logs all IAM API calls. Required for IAM auditing and forensics.
AWS Config	Monitors IAM configuration compliance. Managed rules for IAM best practices.
IAM Access Analyzer	Analyzes effective permissions. External access finder. Unused access finder.
Amazon Cognito	User identity for applications (not AWS console). Federates to IAM for backend access.
AWS Secrets Manager	Store/rotate credentials instead of hardcoding. IAM controls who reads secrets.
AWS KMS	Key policies (resource-based). IAM identity policies work alongside key policies.
AWS RAM	Share resources (subnets, Transit Gateway) across accounts using IAM-governed permissions.

📋 Chapter 24: AWS CloudTrail

> **Scope:** Regional service (trails are regional) with optional multi-region and organization trail options
> **Counterpart:** Azure Monitor Activity Log / Azure Audit Logs | Linux: auditd, syslog **Exam weight:** High — appears in security, compliance, and operational excellence domains

---

📖 24.1 Service Overview

AWS CloudTrail is the **audit logging service** for AWS. It records every API call made to AWS services — who called what, from where, when, and with what parameters. CloudTrail is your primary source of truth for answering: "*What happened in my AWS account?*"

CloudTrail is **not optional for production** — it is the foundational detective control. Without CloudTrail, incident response, compliance audits, and forensic investigations are impossible.

Three types of events:

- **Management events** — control plane (create/delete/modify resources: `ec2:RunInstances`, `s3:CreateBucket`, `iam:CreateRole`)
- **Data events** — data plane (read/write to data: `s3:GetObject`, `lambda:Invoke`, `dynamodb:PutItem`)
- **Insights events** — anomaly detection (unusual API call volumes, error rates)

🪜 24.2 Trail Types

24.2.1 Single-Region Trail

Records events in the region where created. Default when creating a trail. CloudTrail console shows one trail per region.

```
aws cloudtrail create-trail \
  --name MyTrail \
  --s3-bucket-name my-cloudtrail-bucket \
  --region eu-central-1
```

24.2.2 Multi-Region Trail

One trail captures events from ALL regions. Recommended over per-region trails. Catches global service events (IAM, STS, CloudFront, Route 53 — these always log to `us-east-1` regardless of trail region).

```
aws cloudtrail create-trail \
  --name MyMultiRegionTrail \
  --s3-bucket-name my-cloudtrail-bucket \
  --is-multi-region-trail \
  --include-global-service-events
```

24.2.3 Organization Trail

Created in the management account — automatically applies to **all current and future member accounts**. Logs aggregated to one S3 bucket. Member accounts cannot modify or delete the org trail. The most important trail type for enterprise environments.

```
# Run from management account
aws cloudtrail create-trail \
  --name OrgTrail \
  --s3-bucket-name org-cloudtrail-central \
  --is-multi-region-trail \
  --is-organization-trail \
  --include-global-service-events
```

Comparison:

	Single-Region	Multi-Region	Organization
Coverage	One region	All regions	All regions + all accounts
Management	Per-account	Per-account	Management account only
Member accounts	N/A	N/A	Auto-enrolled
Best for	Legacy, specific region	Single account	Enterprise, multi-account

---

📦 24.3 Event Types Deep Dive

Management Events

Enabled by default (free for first copy per region). Captures AWS API operations that create, modify, or delete resources.

```
CreateBucket, DeleteBucket, PutBucketPolicy     ← S3 management
RunInstances, TerminateInstances, CreateVpc      ← EC2 management
CreateUser, AttachRolePolicy, AssumeRole         ← IAM management
CreateFunction, DeleteFunction                   ← Lambda management
```

Read vs Write management events: can split logging by type to reduce cost.

- **Write** events (create/delete/modify) — most security-relevant
- **Read** events (describe/list/get) — high volume, less security value

Data Events

NOT enabled by default — additional cost. High volume (every S3 object access, every Lambda invocation).

Service	Data Event Examples	Use Case
S3	`GetObject`, `PutObject`, `DeleteObject`	Data exfiltration detection, compliance
Lambda	`Invoke`	Function call audit
DynamoDB	`GetItem`, `PutItem`, `DeleteItem`	Data access audit
CloudTrail Lake	`Query`	Audit query activity
SNS	`Publish`	Message audit
SQS	`SendMessage`, `ReceiveMessage`	Queue activity

```
# Enable S3 data events on specific buckets
aws cloudtrail put-event-selectors \
  --trail-name MyTrail \
  --event-selectors '[{
    "ReadWriteType": "All",
    "IncludeManagementEvents": true,
    "DataResources": [{
      "Type": "AWS::S3::Object",
      "Values": ["arn:aws:s3:::sensitive-bucket/"]
    }]
  }]'
```

Insights Events

CloudTrail Insights analyzes write management event patterns. Detects:

- Unusual API call rates (burst of `CreateUser` calls)
- Unusual error rates (spike in `AccessDenied`)

Insights events generated when anomaly detected. Delivered to same S3 bucket + CloudWatch Logs. Additional cost.

🗄️ 24.4 CloudTrail Log Storage and Management

Log Delivery

- **S3 bucket** — primary storage. Logs delivered within ~15 minutes of API call. Compressed gzip JSON files.
- **CloudWatch Logs** — optional, enables real-time alerting via Metric Filters
- **CloudTrail Lake** — managed event data store (SQL queryable, no S3 needed)

Log file format: `s3://BUCKET/AWSLogs/ACCOUNT_ID/CloudTrail/REGION/YEAR/MONTH/DAY/`

```
s3://audit-logs/AWSLogs/123456789/CloudTrail/eu-central-1/2026/11/01/
  123456789_CloudTrail_eu-central-1_20261101T1000Z_AbCdEfGh.json.gz
```

CloudTrail Log File Integrity

CloudTrail creates a **digest file** every hour. Digest contains SHA-256 hash of each log file + SHA-256 of the previous digest (hash chain). Validates no tampering.

```
aws cloudtrail validate-logs \
  --trail-arn arn:aws:cloudtrail:eu-central-1:123:trail/MyTrail \
  --start-time 2026-10-01 \
  --end-time 2026-11-01
```

If logs were deleted or modified: validation fails. Proof of tamper or proof of integrity for compliance.

S3 Bucket Protection

```
# Bucket policy: only CloudTrail service can write
{
  "Effect": "Allow",
  "Principal": {"Service": "cloudtrail.amazonaws.com"},
  "Action": "s3:PutObject",
  "Resource": "arn:aws:s3:::audit-logs/AWSLogs/*",
  "Condition": {"StringEquals": {"s3:x-amz-acl": "bucket-owner-full-control"}}
}
# Deny ALL delete/modify from any non-CloudTrail principal
{
  "Effect": "Deny",
  "NotPrincipal": {"Service": "cloudtrail.amazonaws.com"},
  "Action": ["s3:DeleteObject", "s3:DeleteBucket", "s3:PutLifecycleConfiguration"],
  "Resource": "arn:aws:s3:::audit-logs*"
}
```

Add **S3 Object Lock** (Governance mode, 1-year retention) for tamper-proof storage.

🔍 24.5 Querying CloudTrail Logs

Method 1: CloudTrail Console / Lookup Events

Quick lookup for recent events (last 90 days). Filterable by username, event name, resource ARN, time range. No Athena needed.

```
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue=AssumeRole \
  --start-time 2026-10-01 \
  --end-time 2026-11-01
```

Method 2: Athena on S3

For historical analysis beyond 90 days, complex queries, large-scale investigation.

```
-- Find all AssumeRole calls from external accounts
SELECT eventTime, userIdentity.arn, requestParameters.roleArn
FROM cloudtrail_logs
WHERE eventName = 'AssumeRole'
  AND year = '2026'
  AND month = '11'
  AND userIdentity.accountId != '123456789012'
ORDER BY eventTime DESC;
```

Partition projection eliminates Glue crawler need:

```
CREATE EXTERNAL TABLE cloudtrail_logs (...)
PARTITIONED BY (region STRING, year STRING, month STRING, day STRING)
LOCATION 's3://audit-logs/AWSLogs/123456789/CloudTrail/'
TBLPROPERTIES (
  "projection.enabled" = "true",
  "projection.region.type" = "enum",
  "projection.region.values" = "eu-central-1,eu-west-1",
  "projection.year.type" = "integer",
  "projection.year.range" = "2024,2030"
);
```

Method 3: CloudTrail Lake

Managed event data store — no S3 management, no Athena table setup. SQL query directly. Retention 7 years. OCSF (Open Cybersecurity Schema Framework) normalization available.

```
aws cloudtrail start-query \
  --query-statement "SELECT eventName, userIdentity.arn, eventTime
    FROM $EDS_ID
    WHERE eventName = 'DeleteBucket'
    AND eventTime > '2026-10-01 00:00:00'
    ORDER BY eventTime DESC"
```

Comparison: CloudTrail S3+Athena vs CloudTrail Lake:

	S3 + Athena	CloudTrail Lake
Setup	Bucket + table + crawler	Just enable
Query	Athena SQL	CloudTrail Lake SQL (subset)
Retention	S3 lifecycle (unlimited)	7 years max
Cost	S3 storage + Athena query	Lake storage + query fee
Integration	Athena → QuickSight, Glue	Lake → direct
Cross-account	Manual consolidation	Org-wide automatic
Best for	Existing Athena/data lake teams	Simplicity, managed approach

---

🔔 24.6 Real-Time Alerting with CloudWatch

CloudTrail → CloudWatch Logs → Metric Filters → Alarms → SNS

```
# Step 1: Enable CloudWatch Logs delivery
aws cloudtrail update-trail \
  --name MyTrail \
  --cloud-watch-logs-log-group-arn arn:aws:logs:eu-central-1:123:log-group:CloudTrail \
  --cloud-watch-logs-role-arn arn:aws:iam::123:role/CloudTrailCWRole

# Step 2: Create metric filter for root account usage
aws logs put-metric-filter \
  --log-group-name CloudTrail \
  --filter-name RootAccountUsage \
  --filter-pattern '{ $.userIdentity.type = "Root" && $.userIdentity.invokedBy NOT EXISTS && $.eventType != "AwsServiceEvent" }'
  --metric-transformations metricName=RootAccountUsageCount,metricNamespace=CloudTrailMetrics,metricValue=1

# Step 3: Create alarm
aws cloudwatch put-metric-alarm \
  --alarm-name RootAccountUsage \
  --metric-name RootAccountUsageCount \
  --namespace CloudTrailMetrics \
  --statistic Sum \
  --period 300 \
  --threshold 1 \
  --comparison-operator GreaterThanOrEqualToThreshold \
  --alarm-actions arn:aws:sns:eu-central-1:123:SecurityAlerts
```

AWS CIS Benchmark required alerts (exam-relevant):

Alert	Filter Pattern
Root account usage	`$.userIdentity.type = "Root"`
Unauthorized API calls	`$.errorCode = "AccessDenied"`
Console login without MFA	`$.additionalEventData.MFAUsed != "Yes"`
IAM policy changes	`$.eventName = "DeleteGroupPolicy" OR "DeleteRolePolicy" OR "PutGroupPolicy"...`
CloudTrail config changes	`$.eventName = "StopLogging" OR "DeleteTrail" OR "UpdateTrail"`
S3 bucket policy changes	`$.eventName = "PutBucketPolicy" OR "DeleteBucketPolicy"`
Security group changes	`$.eventName = "AuthorizeSecurityGroupIngress"...`
Network ACL changes	`$.eventName = "CreateNetworkAcl"...`
VPC changes	`$.eventName = "CreateVpc" OR "DeleteVpc"...`
KMS key deletion	`$.eventName = "DisableKey" OR "ScheduleKeyDeletion"`

🌊 24.7 CloudTrail Event Structure

Every CloudTrail event is JSON. Key fields for exam:

```
{
  "eventVersion": "1.08",
  "userIdentity": {
    "type": "AssumedRole",              ← Root | IAMUser | AssumedRole | AWSService | FederatedUser
    "principalId": "AROA....:SessionName",
    "arn": "arn:aws:sts::123:assumed-role/MyRole/SessionName",
    "accountId": "123456789012",
    "sessionContext": {
      "sessionIssuer": {
        "type": "Role",
        "arn": "arn:aws:iam::123:role/MyRole"
      },
      "attributes": {
        "mfaAuthenticated": "true",
        "creationDate": "2026-11-01T10:00:00Z"
      },
      "sourceIdentity": "john.doe@company.com"   ← set with sts:SetSourceIdentity
    }
  },
  "eventTime": "2026-11-01T10:15:30Z",
  "eventSource": "s3.amazonaws.com",
  "eventName": "PutObject",             ← The API call
  "awsRegion": "eu-central-1",
  "sourceIPAddress": "10.0.1.50",       ← or "AWS Internal" for service calls
  "userAgent": "aws-sdk-java/1.12...",
  "requestParameters": {                ← What was requested
    "bucketName": "my-bucket",
    "key": "important/file.txt"
  },
  "responseElements": null,             ← null for read-only; response for writes
  "errorCode": "AccessDenied",          ← present on failed calls
  "errorMessage": "...",
  "requestID": "ABC123",
  "eventID": "UUID",
  "readOnly": false,
  "resources": [{                       ← Resources affected
    "ARN": "arn:aws:s3:::my-bucket/important/file.txt",
    "accountId": "123456789012",
    "type": "AWS::S3::Object"
  }],
  "managementEvent": false,
  "eventCategory": "Data"
}
```

`userIdentity.type` values — critical for exam:

Type	Meaning
`Root`	Root account user
`IAMUser`	IAM user with long-term credentials
`AssumedRole`	Temporary role session (most common)
`FederatedUser`	SAML/web identity federation
`AWSService`	AWS service acting autonomously
`AWSAccount`	Cross-account access

🔒 24.8 CloudTrail Security Best Practices

```
flowchart TD
    CT[CloudTrail Org Trail] --> S3[Protected S3 Bucket]
    S3 --> OL[Object Lock\nGovernance Mode]
    S3 --> INT[Log File Integrity\nValidation]
    CT --> CWL[CloudWatch Logs]
    CWL --> MF[Metric Filters]
    MF --> AL[CloudWatch Alarms]
    AL --> SNS[SNS → PagerDuty/Slack]
    CT --> LAKE[CloudTrail Lake]
    LAKE --> QUERY[SQL Investigation]
    S3 --> ATHENA[Athena Queries]
```

Control	Implementation
Enable org trail	All accounts, all regions, global services
Protect logs	S3 Object Lock + bucket policy deny deletes
Validate integrity	`validate-logs` in Lambda on schedule
Encrypt logs	SSE-KMS with customer-managed key
Real-time alerts	CW Logs + Metric Filters + Alarms
Prevent disabling	SCP: deny `cloudtrail:StopLogging` , `cloudtrail:DeleteTrail`
Separate account	CloudTrail bucket in dedicated security account
Enable data events	For sensitive S3 buckets, Lambda, DynamoDB

🌎 24.9 CloudTrail in the Well-Architected Framework

Pillar	CloudTrail Considerations
Security	Enables detective controls. Audit every API call. Detect unauthorized access, privilege escalation, data exfiltration. Foundation for incident response. Encrypt logs with KMS.
Reliability	CloudTrail itself is highly available (no single-point). Multi-region trail ensures no gaps if a region is degraded. Log delivery: best-effort (minor gaps possible) — CloudTrail Lake provides more reliable storage.
Performance Efficiency	No impact on AWS API performance — CloudTrail is asynchronous, out-of-band. Athena queries: partition pruning critical for performance on large datasets. CloudTrail Lake: managed, no query optimization needed.
Cost Optimization	Management events: first copy free per region. Data events: additional cost — enable only for sensitive buckets/functions. Insights: additional cost — enable only if anomaly detection value justifies. S3 Glacier for older logs (3+ months) dramatically reduces cost.
Operational Excellence	CloudTrail Lake simplifies operations (no S3/Athena setup). Org trail reduces configuration overhead vs per-account trails. Automate incident response via EventBridge rules on CloudTrail events.
Sustainability	No direct impact. Efficient querying (Athena partition pruning) reduces compute. Archive old logs to Glacier (lower storage footprint).

🗂️ 24.10 CloudTrail in the 4 SAP-C02 Domains

Domain 1 — Organizational Complexity: Org trail aggregates all accounts into one central bucket. Delegated admin can manage org trail from security account (not management account). Config + CloudTrail together = compliance posture across the org.

Domain 2 — New Solutions: Every new solution must have CloudTrail considered for audit. Data events for sensitive workloads from day one. EventBridge + Lambda automated response patterns.

Domain 3 — Continuous Improvement: Regular analysis of CloudTrail data identifies unused permissions (feed into IAM Access Analyzer). Athena dashboards for security KPIs. CloudTrail Lake cross-account investigation.

Domain 4 — Migration and Modernization: During migration: CloudTrail captures all API calls — before/after comparison identifies permission gaps. Policy generation from CloudTrail (Access Analyzer) helps create least-privilege policies for migrated workloads.

🔗 24.11 Related Services

Service	Relationship
AWS Config	Config records RESOURCE STATE changes. CloudTrail records API CALLS. Complementary: Config = what changed; CloudTrail = who changed it.
Amazon GuardDuty	Consumes CloudTrail logs as one input source. Detects threats in CloudTrail events using ML.
AWS Security Hub	Aggregates findings from GuardDuty + Config + Inspector. Uses CloudTrail as underlying audit source.
Amazon Detective	Deeper investigation using CloudTrail, VPC Flow Logs, GuardDuty. Visualizes behavior patterns for incident investigation.
IAM Access Analyzer	Uses CloudTrail for policy generation feature.
Amazon Macie	CloudTrail logs Macie's own API calls.
EventBridge	CloudTrail events trigger EventBridge rules for automated response.
Athena	Primary query engine for CloudTrail logs stored in S3.

End of Chapter 24 — AWS CloudTrail

🏛️ Chapter 25: AWS Organizations & Service Control Policies (SCPs)

> **Scope:** Global service — Organizations and SCPs are global, not regional **Counterpart:** Azure Management Groups + Azure Policy | No direct Linux equivalent (closest: sudoers with organizational hierarchy) **Exam weight:** Very high — core to multi-account architecture (Domain 1: Organizational Complexity)

📖 25.1 AWS Organizations Overview

AWS Organizations is the **multi-account management service**. It groups AWS accounts into a hierarchy for consolidated billing, centralized governance, and policy enforcement.

Why multiple accounts?

- Security boundary: blast radius of compromise limited to one account
- Billing separation: separate cost tracking per team/project
- Service quotas: each account has independent quotas
- Environment isolation: dev/test/prod cannot accidentally interact
- Regulatory: data residency, compliance per account

Organizations is free — no charge for the service itself.

🌳 25.2 Organization Structure

```
flowchart TD
    ROOT[🌳 Root\nTop-level container]
    ROOT --> MOU[Management OU]
    ROOT --> PROD[Production OU]
    ROOT --> DEV[Development OU]
    ROOT --> SANDBOX[Sandbox OU]
    PROD --> EU[EU OU]
    PROD --> US[US OU]
    EU --> ACC1[Account: prod-eu-payments]
    EU --> ACC2[Account: prod-eu-orders]
    US --> ACC3[Account: prod-us-payments]
    DEV --> ACC4[Account: dev-team-a]
    DEV --> ACC5[Account: dev-team-b]
    SANDBOX --> ACC6[Account: experiment-1]

    style ROOT fill:#ff6b6b,color:#fff
    style MOU fill:#ffa500,color:#fff
    style PROD fill:#4CAF50,color:#fff
    style DEV fill:#2196F3,color:#fff
```

Components:

Component	Description
Root	Top-level container. SCPs attached here apply to ALL accounts.
OU (Organizational Unit)	Logical grouping of accounts. Nestable up to 5 levels deep.
Account	AWS account — the actual resource and billing boundary.
Management Account	Account that created the org. NOT subject to SCPs.
Member Account	Any account in the org (except management). Subject to SCPs.

Limits:

- Max accounts per org: default 10 (request increase to thousands)
- Max OU nesting depth: 5 levels
- Max SCPs per org: 1,000
- Max SCPs per target (account/OU): 5
- Max SCP size: 5,120 bytes (characters)

📜 25.3 Service Control Policies (SCPs) — Deep Dive

What SCPs Are

SCPs are **IAM policy documents** attached to an account or OU that define the **maximum permissions** available to all principals in that account.

Critical: SCPs do NOT grant permissions. They only restrict. An SCP allowing `s3:GetObject` doesn't mean anyone can get S3 objects — they still need an IAM policy granting it.

Think of SCPs as a **ceiling**, identity policies as the **floor you build up from**.

What SCPs Affect

- All IAM users in member accounts
- All IAM roles in member accounts
- The root user of member accounts ✅ (yes, SCPs restrict member root users)

What SCPs Do NOT Affect

- Management account (the account that owns the org)

- Service-linked roles (AWS-managed roles for services)
- AWS service-to-service calls made on your behalf

SCP Evaluation Model

```
flowchart TD
    REQ[API Request] --> DENY{Explicit DENY\nin any SCP\nin the hierarchy?}
    DENY -->|Yes| D1[❌ DENY]
    DENY -->|No| ALLOW{Explicit ALLOW\nin ALL SCPs\nin the path\nfrom root to account?}
    ALLOW -->|No| D2[❌ DENY - implicit]
    ALLOW -->|Yes| IAM[Continue to IAM\nevaluation]
```

SCP inheritance: an account inherits ALL SCPs from Root → OU → sub-OU → Account. Every SCP in the inheritance path must allow the action (when using allow-list mode) OR none of them must deny (deny-list mode).

🔧 25.4 SCP Strategies

Strategy 1: Deny-list (Recommended)

Attach `FullAWSAccess` SCP to root (allows everything). Add specific deny SCPs at the OU or account level. Everything is allowed unless explicitly denied.

```
// FullAWSAccess (attached to root — already default)
{
  "Version": "2012-10-17",
  "Statement": [{"Effect": "Allow", "Action": "*", "Resource": "*"}]
}

// Deny leaving org (attached to all member OUs)
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "Action": ["organizations:LeaveOrganization"],
    "Resource": "*"
  }]
}
```

Strategy 2: Allow-list

Remove `FullAWSAccess` SCP. Attach only specific allows. Nothing works unless explicitly allowed. Very restrictive — good for sandbox or highly regulated environments.

```
// Allow only specific services (e.g., sandbox: EC2, S3, Lambda only)
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": [
      "ec2:*", "s3:*", "lambda:*",
      "iam:*", "cloudwatch:*", "logs:*"
    ],
    "Resource": "*"
  }]
}
```

Comparison:

	Deny-list	Allow-list
Default state	Everything allowed (with IAM)	Nothing allowed
New service launch	Automatically accessible	Must be added to SCP
Operational friction	Low	High
Security model	"Allow with exceptions"	"Deny by default"
Best for	Standard environments	Sandboxes, regulated

📐 25.5 Common SCP Patterns

Data Residency (Region Lock)

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DenyNonApprovedRegions",
    "Effect": "Deny",
    "NotAction": [
      "iam:*", "sts:*", "s3:*",
      "cloudfront:*", "route53:*",
      "waf:*", "support:*",
      "trustedadvisor:*", "account:*"
    ],
    "Resource": "*",
    "Condition": {
      "StringNotEquals": {
        "aws:RequestedRegion": ["eu-central-1", "eu-west-1"]
      }
    }
  }]
}
```

Note `NotAction` for global services (IAM, STS, S3 global APIs) — these have no region and would be incorrectly blocked.

Protect Security Services

```
{
  "Sid": "DenyDisableSecurityServices",
  "Effect": "Deny",
  "Action": [
    "cloudtrail:StopLogging", "cloudtrail:DeleteTrail",
    "cloudtrail:UpdateTrail", "cloudtrail:PutEventSelectors",
    "guardduty:DeleteDetector", "guardduty:DisassociateFromMasterAccount",
    "config:DeleteConfigurationRecorder", "config:StopConfigurationRecorder",
    "securityhub:DisableSecurityHub",
    "access-analyzer:DeleteAnalyzer",
    "macie2:DisableMacie"
  ],
  "Resource": "*"
}
```

Require Tags on Resources

```
{
  "Sid": "RequireCostCenterTag",
  "Effect": "Deny",
  "Action": ["ec2:RunInstances", "rds:CreateDBInstance", "lambda:CreateFunction"],
  "Resource": "*",
  "Condition": {
    "Null": {
      "aws:RequestTag/CostCenter": "true"
    }
  }
}
```

Prevent Root Account Usage

```
{
  "Sid": "DenyRootAccess",
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {
    "StringLike": {
      "aws:PrincipalArn": "arn:aws:iam::*:root"
    }
  }
}
```

Require Encrypted S3 Uploads

```
{
  "Sid": "DenyUnencryptedS3",
  "Effect": "Deny",
  "Action": "s3:PutObject",
  "Resource": "*",
  "Condition": {
    "StringNotEquals": {
      "s3:x-amz-server-side-encryption": ["aws:kms", "AES256"]
    }
  }
}
```

Prevent VPC Changes in Production

```
{
  "Sid": "DenyVPCChanges",
  "Effect": "Deny",
  "Action": [
    "ec2:CreateVpc", "ec2:DeleteVpc",
    "ec2:ModifyVpcAttribute",
    "ec2:CreateInternetGateway",
    "ec2:AttachInternetGateway",
    "ec2:CreateSubnet", "ec2:DeleteSubnet"
  ],
  "Resource": "*"
}
```

🏷️ 25.6 Tag Policies

Tag policies (different from SCPs) enforce **tag key/value standardization** across the org. Not a deny mechanism — they flag non-compliant resources.

```
{
  "tags": {
    "Environment": {
      "tag_key": {"@@assign": "Environment"},
      "tag_value": {
        "@@assign": ["Production", "Development", "Staging", "Sandbox"]
      },
      "enforced_for": {
        "@@assign": ["ec2:instance", "s3:bucket", "rds:db"]
      }
    }
  }
}
```

Enforcement modes:

- **Report only** — non-compliant resources flagged in AWS Config
- **Prevent** — non-compliant resources cannot be created/tagged

---

🛡️ 25.7 AWS Organizations Advanced Features

Delegated Administrator

Management account can designate a member account as **delegated administrator** for specific services (Config, Security Hub, CloudTrail, GuardDuty, etc.). Delegated admin manages the service org-wide without needing management account access.

```
aws organizations register-delegated-administrator \
  --account-id SECURITY_ACCOUNT_ID \
  --service-principal config.amazonaws.com
```

Why important: management account should be empty. Security team manages security services from the security account (governed by SCPs) — not from the ungoverned management account.

AWS Control Tower

AWS-managed service that sets up a **landing zone** — a pre-configured multi-account environment following AWS best practices. Creates:

- Log Archive account (CloudTrail, Config logs)
- Audit account (security team access)
- Pre-built guardrails (SCPs + Config rules)
- Account Factory (automated account vending)

Guardrail types:

- **Preventive** — SCPs (hard enforcement)
- **Detective** — Config rules (soft: alert on violations)
- **Proactive** — CloudFormation hooks (check before deploy)

Resource Control Policies (RCPs)

Newer (2024) — complement to SCPs. SCPs restrict **what principals can do**. RCPs restrict **what can be done to resources** regardless of who calls.

```
// RCP: deny access to S3 from outside the org
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "Principal": "*",
    "Action": "s3:*",
    "Resource": "*",
    "Condition": {
      "StringNotEquals": {
        "aws:PrincipalOrgID": "o-exampleorgid"
      }
    }
  }]
}
```

Even if a bucket policy says "allow public access" — the RCP blocks external callers. Data perimeter from the resource side.

📊 25.8 Data Perimeter Architecture

Full data perimeter = four conditions preventing data exfiltration/unauthorized access:

```
1. Only org identities access org resources     → SCP: deny s3:* if PrincipalOrgID ≠ org
2. Org identities only access org resources     → SCP: deny if ResourceAccount not in org
3. Only org resources accessed via org networks → VPC endpoint policies
4. Resources reject non-org access              → RCP: deny if caller not in org
```

```
flowchart LR
    A[External Attacker] -->|❌ SCP blocks| B[Org Resource]
    C[Org Identity] -->|❌ SCP blocks| D[External Resource]
    E[Compromised Org Identity] -->|❌ VPC endpoint policy blocks| F[External S3]
    G[External Identity] -->|❌ RCP blocks| H[Org S3 Bucket\neven if public]
```

🌍 25.9 Organizations in the Well-Architected Framework

Pillar	Organizations/SCP Considerations
Security	SCPs as preventive controls — enforce security guardrails org-wide. Separate accounts for different security zones. Delegated admin keeps management account clean. RCPs for data perimeter.
Reliability	Organizations itself is highly available. Account-level isolation: one account's blast radius doesn't affect others. Account per environment prevents dev changes from impacting prod.
Performance Efficiency	Per-account service quotas: one team's Lambda concurrency usage doesn't throttle another. Account-level partitioning of resources.
Cost Optimization	Consolidated billing: volume discounts, Reserved Instance sharing across accounts. Cost allocation tags + account structure = per-team cost tracking. Sandbox OU with auto-cleanup SCPs prevents cost runaway.
Operational Excellence	Account Factory (Control Tower) automates account vending with pre-configured security. SCPs as code (Terraform/CloudFormation) — version controlled, reviewed.
Sustainability	Account structure enables per-team sustainability reporting. Sandbox SCP: auto-terminate idle resources (prevents zombie resource energy waste).

🗂️ 25.10 SCP and Organizations in the 4 SAP-C02 Domains

Domain 1 — Organizational Complexity: Core domain for SCPs. Design OU structure, SCP inheritance, account vending, delegated admin. Questions: "A company acquires another company — how do you integrate their AWS accounts?" → AWS

Organizations, OU structure, SCP guardrails.

Domain 2 — New Solutions: Design account structure for new products. SCP guardrails from day one. Account Factory for automated provisioning.

Domain 3 — Continuous Improvement: AWS Config org aggregator + SCPs = drift detection + enforcement. Upgrade from manual per-account SCPs to Control Tower managed guardrails.

Domain 4 — Migration: Migrate on-premises workloads into org structure. Landing zone setup before migration. SCPs prevent migrated workloads from accessing wrong resources.

🔗 25.11 Related Services

Service	Relationship
AWS Control Tower	Orchestrates Organizations, SCPs, Config, IAM Identity Center into a managed landing zone
AWS Config	Org aggregator collects Config findings from all accounts. Config rules = detective guardrails (vs SCPs = preventive)
IAM Identity Center	Deployed across the org — permission sets applied to multiple accounts
AWS Budgets	Org-level budgets for consolidated billing control
AWS RAM (Resource Access Manager)	Share resources (Transit Gateway, subnets) org-wide without cross-account role complexity
Amazon GuardDuty	Org-level GuardDuty: master/member model, delegated admin
AWS Security Hub	Org-level aggregation of security findings
CloudTrail	Org trail — captures all accounts, delivered to management account's S3

End of Chapter 25 — AWS Organizations & SCPs

🔑 Chapter 26: AWS IAM Identity Center (formerly AWS SSO)

> **Scope:** Regional deployment (one region per org, but access is org-wide). Recommended region: same as management account. **Counterpart:** Azure Entra ID (AAD) + Conditional Access | Okta | Ping Identity | On-prem: Active Directory + ADFS **Exam weight:** High — enterprise identity management, multi-account access

---

📖 26.1 Service Overview

AWS IAM Identity Center is the **centralized identity and access management hub** for multi-account AWS environments. It replaces the need to configure IAM users or SAML federation independently in each account.

Core value: one place to manage who has access to which AWS accounts with which permissions. Single sign-on for both AWS accounts and SAML applications (Salesforce, Slack, etc.).

It is free — no charge for Identity Center itself.

26.2 Architecture

```
flowchart TD
    IDP[Corporate IdP\nActive Directory / Okta / Google]
    IC[IAM Identity Center\nManagement Account]

    IDP -->|SAML 2.0 / SCIM| IC

    IC -->|Permission Set → IAM Role| ACC1[Account: prod-payments]
    IC -->|Permission Set → IAM Role| ACC2[Account: prod-orders]
    IC -->|Permission Set → IAM Role| ACC3[Account: dev-team]
    IC -->|SAML App| APP1[Salesforce]
    IC -->|SAML App| APP2[Slack]

    USER[Human User] -->|SSO Portal| IC
    IC -->|Temporary credentials| USER
    USER -->|Assumes auto-created role| ACC1
```

Key components:

Component	Description
Identity Source	Where users/groups come from: built-in directory, Active Directory, external IdP
Permission Set	Template defining IAM policies. Applied across accounts.
Account Assignment	Binding: Permission Set + User/Group + Account
SSO Portal	Web interface for users to see their accounts and access them
SCIM Provisioning	Automatic user/group sync from IdP to Identity Center

---

26.3 Identity Sources

Option 1: Identity Center Directory (Built-in)

Default. Manage users and groups directly in Identity Center. No external IdP needed. Good for small teams without an existing corporate directory.

```
aws identitystore create-user \
  --identity-store-id d-1234567890 \
  --user-name john.doe \
  --name '{"GivenName":"John","FamilyName":"Doe"}' \
  --emails '[{"Value":"john@company.com","Primary":true}]'
```

Option 2: Active Directory

Connect AWS Managed Microsoft AD or AD Connector.

- **AWS Managed Microsoft AD** — fully managed AD in AWS VPC. Sync to Identity Center. Best when migrating from on-premises AD.
- **AD Connector** — proxy to on-premises AD (no data stored in AWS). Best when keeping on-premises AD as the master.

Option 3: External IdP (SAML 2.0)

Okta, Google Workspace, Ping, JumpCloud, Azure AD. SAML for authentication + SCIM for user/group provisioning.

```
IdP → SCIM → Identity Center groups auto-sync
IdP → SAML 2.0 → user authentication
```

SCIM provisioning: automatic user/group sync. When employee onboards in Okta → group membership set → Identity Center picks up via SCIM → account assignments take effect. When employee leaves → deprovisioned in Okta → SCIM removes from Identity Center → all AWS access revoked simultaneously. Zero-touch lifecycle management.

Comparison of identity sources:

	Built-in Directory	AD Connector	AWS Managed AD	External IdP
User store	Identity Center	On-premises AD	AWS-managed AD	IdP (Okta/Google)
Sync	Manual	Real-time proxy	Real-time sync	SCIM
MFA	Identity Center MFA	AD MFA	AD MFA	IdP MFA
Best for	Small teams, no existing IdP	Keep on-prem AD	Migrate AD to AWS	Okta/Google enterprise
Complexity	Low	Medium	Medium	Medium

📋 26.4 Permission Sets

A **permission set** is a reusable IAM policy template:

```
# Create permission set
aws sso-admin create-permission-set \
  --instance-arn arn:aws:sso:::instance/ssoins-xxx \
  --name "DataEngineerAccess" \
  --description "Access for data engineering team" \
  --session-duration "PT8H"   # ISO 8601: 8 hours

# Attach AWS managed policy
aws sso-admin attach-managed-policy-to-permission-set \
  --instance-arn arn:aws:sso:::instance/ssoins-xxx \
  --permission-set-arn arn:aws:sso:::permissionSet/ssoins-xxx/ps-xxx \
  --managed-policy-arn arn:aws:iam::aws:policy/AmazonAthenaFullAccess

# Add inline policy
aws sso-admin put-inline-policy-to-permission-set \
  --instance-arn arn:aws:sso:::instance/ssoins-xxx \
  --permission-set-arn arn:aws:sso:::permissionSet/ssoins-xxx/ps-xxx \
  --inline-policy file://custom-policy.json

# Add permission boundary
aws sso-admin put-permissions-boundary-to-permission-set \
  --instance-arn arn:aws:sso:::instance/ssoins-xxx \
  --permission-set-arn arn:aws:sso:::permissionSet/ssoins-xxx/ps-xxx \
  --permissions-boundary '{"ManagedPolicyArn":"arn:aws:iam::aws:policy/PowerUserAccess"}'
```

What a permission set can contain:

- AWS managed policies (up to 10)
- One customer managed policy
- One inline policy (up to 10,240 characters)
- One permission boundary

Permission set deployment: assigning a permission set to an account creates an IAM role in that account: `AWSReservedSSO_PermissionSetName_<random>` . Trust policy trusts the Identity Center SAML provider. Any changes to the permission set auto-propagate to all accounts where it's assigned.

👥 26.5 Account Assignments

```
# Assign group to account with permission set
aws sso-admin create-account-assignment \
  --instance-arn arn:aws:sso:::instance/ssoins-xxx \
  --target-id 123456789012 \              # AWS Account ID
  --target-type AWS_ACCOUNT \
  --permission-set-arn arn:aws:sso:::permissionSet/ssoins-xxx/ps-xxx \
  --principal-type GROUP \
  --principal-id GROUP_ID_FROM_IDENTITY_STORE
```

Scale example — 100 accounts, 10 teams:

```
Permission Sets (10):
  - AdminAccess
  - ReadOnlyAccess
  - DataEngineerAccess
  - DevAccess
  - SecurityAuditAccess
  - BillingAccess
  - NetworkAdminAccess
  - AppDeployAccess
  - IncidentResponseAccess
  - SandboxFullAccess

Account Assignments:
  - NetworkAdmins group → NetworkAdminAccess → [all 100 accounts]
  - SecurityTeam group → SecurityAuditAccess → [all 100 accounts]
  - TeamA group → DevAccess → [dev-team-a, staging-team-a]
  - TeamA group → ReadOnlyAccess → [prod-team-a]
  - SRETeam group → IncidentResponseAccess → [all prod accounts]
```

One definition per permission set. Assignments flexible per group/account combination.

🔐 26.6 MFA in Identity Center

Identity Center supports its own MFA layer (independent of the identity source):

MFA Type	Description
TOTP (Virtual)	Authenticator app (Google Authenticator, Authy)
FIDO2 / WebAuthn	Hardware key (YubiKey), biometrics (Touch ID, Windows Hello)
Email OTP	One-time code via email (least secure)

MFA enforcement modes:

- `NEVER_MFA` — MFA not required (not recommended)
- `CONTEXT_AWARE` — Identity Center decides based on risk (new device, unusual location)
- `ALWAYS_ON` — Every login requires MFA

```
aws sso-admin update-instance-access-control-attribute-configuration \
  --instance-arn arn:aws:sso:::instance/ssoins-xxx \
  --instance-access-control-attribute-configuration \
    'AccessControlAttributes=[{Key="department",Value={Source=[{KeyType=IDENTITY_STORE,IdentityStoreAttributeName=department}]}}]
```

Attribute-based access control (ABAC) with Identity Center: Map IdP user attributes (department, costCenter, jobTitle) to session tags. IAM policies use `aws:PrincipalTag` conditions. One permission set handles all teams via ABAC.

🔄 26.7 User Experience — Access Portal

Users access the Identity Center portal: `https://d-XXXXX.awsapps.com/start`

Portal shows:

- All AWS accounts the user has access to
- All permission sets available per account
- All SAML applications assigned to the user

Click account → choose permission set → opens AWS console (federation) or downloads temporary credentials for CLI.

CLI/SDK access:

```
# Configure SSO in AWS CLI
aws configure sso
# Prompts: SSO start URL, region, account, permission set

# Or manually
aws sso login --profile my-sso-profile

# Use profile for API calls
aws s3 ls --profile my-sso-profile

# Get credentials programmatically
aws sso get-role-credentials \
  --account-id 123456789012 \
  --role-name AWSReservedSSO_DevAccess_xxx \
  --access-token TOKEN
```

Credential duration: 1–12 hours (set in permission set `session-duration`). Auto-refresh via `aws sso login` re-authentication.

🔍 26.8 Auditing Identity Center Activity

Every Identity Center event logged in CloudTrail:

Event	CloudTrail Event Name
User login	`Authenticate`
Role assumption via SSO	`GetRoleCredentials`
Account assignment created	`CreateAccountAssignment`
Permission set updated	`UpdatePermissionSet`
Permission set provisioned	`ProvisionPermissionSet`
User created	`CreateUser`

```
# Find all logins in the last 7 days
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventSource,AttributeValue=sso.amazonaws.com \
  --start-time $(date -v-7d +%Y-%m-%dT%H:%M:%S)
```

User activity traceability: CloudTrail shows:

- `userIdentity.sessionContext.sessionIssuer.arn` = the Identity Center role
- `userIdentity.principalId` = `AROAXXXXXXXXXX:user@company.com` (email embedded!)
- `userIdentity.arn` = `arn:aws:sts::123:assumed-role/AWSReservedSSO_xxx/user@company.com`

The user's email is embedded in the CloudTrail principal — full traceability without IAM Access Analyzer.

🆚 26.9 Identity Center vs Direct IAM Federation vs IAM Users

	IAM Users	Direct IAM SAML	IAM Identity Center
Credentials	Permanent (password + keys)	Temporary (STS)	Temporary (STS)
Lifecycle	Manual create/delete	Manual per account	Auto via SCIM
Multi-account	Per-account users	Per-account setup	Centralized
Permission management	Per-user policies	Per-account roles	Permission sets (global)
MFA	Per-user setting	IdP-managed	Centralized policy
Application SSO	No	No	Yes (SAML apps)
Audit	CloudTrail (username)	CloudTrail (session)	CloudTrail (email in ARN)
Recommended for	Break-glass only	Single account, simple	All multi-account orgs
Migration effort	N/A	Medium	Low–Medium

🌍 26.10 Identity Center in the Well-Architected Framework

Pillar	Identity Center Considerations
Security	Eliminates IAM users = no static long-lived credentials. Centralized MFA enforcement. SCIM: immediate revocation on offboarding. ABAC: least-privilege via attributes. CloudTrail: user email in ARN for full audit trail.
Reliability	Identity Center is highly available (AWS-managed). If IdP is down: users cannot log in (dependency). Mitigate: enable Identity Center backup MFA + break-glass IAM user in each account.
Performance Efficiency	SSO portal: single login for all accounts. No re-authentication per account switch. CLI/SDK: `aws sso login` once per session duration (up to 12 hours).
Cost Optimization	Free service. Reduces operational cost of managing per-account IAM users (hours of admin work → zero). SCIM automation: no manual account provisioning/deprovisioning.
Operational Excellence	Permission sets as code (Terraform `aws_ssoadmin_permission_set`). Assignment automation via Terraform/CloudFormation. Account Factory integration: new account → auto-assigned permission sets.
Sustainability	No direct impact. Operational efficiency gains free up engineering time.

🗂️ 26.11 Identity Center in the 4 SAP-C02 Domains

Domain 1 — Organizational Complexity: Primary service for multi-account human identity. Federate 10,000 employees to 200 AWS accounts via one Identity Center instance. Permission sets = role templates. Org-wide MFA enforcement.

Domain 2 — New Solutions: New workload account → automatically gets standard permission sets (DevAccess, ReadOnly, SecurityAudit). Account Factory + Identity Center = fully configured account in minutes.

Domain 3 — Continuous Improvement: Migrate from per-account IAM users to Identity Center. Migrate from direct SAML to Identity Center. Right-size permission sets using CloudTrail + Access Analyzer data. Remove overly broad permission sets.

Domain 4 — Migration: On-premises AD + ADFS → AWS Managed AD + Identity Center. Step-by-step: (1) Deploy AWS Managed AD. (2) Enable Identity Center. (3) Connect Identity Center to Managed AD. (4) Create permission sets. (5) Test with pilot group. (6) Migrate all users. (7) Decommission ADFS.

🔗 26.12 Related Services

Service	Relationship
AWS Organizations	Identity Center requires Organizations. Permission sets deploy across org accounts.
AWS Directory Service	Provides the Active Directory backend for Identity Center. Managed AD or AD Connector.
AWS CloudTrail	Logs all Identity Center API calls + user authentications.
AWS IAM	Identity Center creates IAM roles in each account (AWSReservedSSO_*). These roles should not be modified manually.
AWS Config	Monitors Identity Center configuration. Custom rules for permission set compliance.
AWS STS	Identity Center uses STS to issue temporary credentials when users access accounts.
AWS Control Tower	Deploys and configures Identity Center as part of the landing zone setup.
Amazon Cognito	Different service — Cognito is for APPLICATION user identity (customers), Identity Center is for AWS ACCOUNT access (employees). Not interchangeable.

⚠️ 26.13 Common Exam Traps

1. **Identity Center vs Cognito:** Cognito = customer/app identities (mobile apps, web apps). Identity Center = employee AWS account access. Exam scenario with "customers" → Cognito. "Employees needing AWS console" → Identity Center.
2. **Identity Center is not multi-region:** only one Identity Center instance per org, deployed in one region. But access is org-wide (all accounts).
3. *Cannot modify AWSReservedSSO_ roles manually:** modifying these IAM roles directly breaks Identity Center. Changes must be made in permission sets.
4. **Session duration in permission set ≠ IdP session:** Identity Center session duration (1-12 hours) is independent of the IdP session duration. User can have a 24-hour IdP session but only 8-hour AWS credentials.
5. **Permission sets require provisioning:** creating a permission set does nothing until you create account assignments AND provision (sync) to the target accounts. Provisioning can take a few minutes.
6. **SCIM vs SAML:** SCIM = user/group sync (provisioning). SAML = authentication. Both can be configured with the same IdP for full automation. SCIM alone (no SAML) = users synced but can't log in. SAML alone (no SCIM) = users can log in but must be manually created.

End of Chapter 26 — AWS IAM Identity Center

1176. [CloudTrail] A company needs to ensure all API calls across all 50 AWS accounts are captured in a single S3 bucket in their security account. What is the minimum configuration needed? ; Create an Organization Trail in the management account: `aws cloudtrail create-trail --name OrgTrail --s3-bucket-name central-audit --is-multi-region-trail --is-organization-trail` . This automatically captures events from all current and future member accounts in all regions. S3 bucket must have a bucket policy allowing CloudTrail from all org accounts to write.

    Organization trail requirement: bucket policy must include `"Principal":{"Service":"cloudtrail.amazonaws.com"}` *and* `"Condition":{"StringEquals":{"aws:SourceArn":"arn:aws:cloudtrail:*:MANAGEMENT_ACCOUNT_ID:trail/OrgTrail"}}` .
    Without this: CloudTrail cannot deliver logs. Log path: `s3://central-audit/AWSLogs/MANAGEMENT_ACCOUNT/CloudTrail-Organization/REGION/YEAR/MONTH/DAY/` *for org trail events. Per-account path:* `s3://central-audit/AWSLogs/MEMBER_ACCOUNT_ID/CloudTrail/REGION/YEAR/MONTH/DAY/` .
    Member accounts cannot modify or delete the org trail. They see it in their CloudTrail console as a read-only trail. SCP protection: deny `cloudtrail:DeleteTrail` *and* `cloudtrail:StopLogging` *on all member OUs. Combination: org trail (collection) + SCP (protection) + S3 Object Lock (storage immutability) = enterprise-grade audit trail.*

1177. [CloudTrail] A security analyst is investigating a potential data exfiltration incident. They need to find all S3 GetObject calls from a specific IAM role in the last 30 days across all accounts. CloudTrail data events are enabled. What is the most efficient query approach? ; CloudTrail Lake: `SELECT eventTime, userIdentity.arn, requestParameters.bucketName, requestParameters.key, sourceIPAddress FROM $EDS_ID WHERE eventName='GetObject' AND userIdentity.sessionContext.sessionIssuer.arn='arn:aws:iam::123:role/SuspectRole' AND eventTime > '2026-10-01'` . Cross-account, no S3/Athena table setup needed, instant results.

    Why CloudTrail Lake over Athena here: org-wide investigation requiring cross-account data. CloudTrail Lake with organization event data store automatically aggregates all accounts. Athena requires: S3 bucket with all accounts' logs + Athena table + partition management. For emergency incident response: CloudTrail Lake is faster to query (no setup, managed indexing). Athena is better for custom analytics, integration with existing data pipelines.
    Athena equivalent: Athena table on org trail S3 bucket partitioned by account, region, year, month, day. Query scans only the relevant partitions. For 50 accounts × 30 days: large scan without partition pruning. Partition projection: `WHERE account='123456789012' AND year='2026' AND month='10'` . With data events on S3: very high volume logs — costs can be significant for full 30-day S3 GetObject scan across 50 accounts.

1178. [CloudTrail] CloudTrail log validation shows some log files have failed integrity checks. What does this mean and what are the possible causes? ; Failed integrity = log file was modified or deleted after delivery to S3. Possible causes: (1) Unauthorized access to S3 bucket — log tampered to cover tracks. (2) Lifecycle policy deleted/transitioned the log before validation. (3) S3 replication corrupted the file (rare). (4) Manual editing of the log. Response: check S3 access logs for who accessed/modified the log file. Check CloudTrail for `s3:DeleteObject` / `s3:PutObject` on the log file. Escalate as security incident.

    Log validation mechanism: each log file has a SHA-256 hash. Digest file (hourly) contains hashes of all log files in that hour + hash of previous digest. Hash chain: modifying any log = its hash changes = digest file hash chain breaks. `aws cloudtrail validate-logs` walks the hash chain, re-hashes each file, compares. Failed = file doesn't match its recorded hash.
    Prevention: S3 Object Lock (Compliance mode) on CloudTrail bucket — logs physically cannot be deleted or modified until retention period expires. Combined with: bucket policy denying `s3:DeleteObject` to non-CloudTrail principals, S3 versioning, separate security account ownership of the bucket. If logs are in the same account being audited: a compromised admin can tamper. Security account bucket: compromised member account cannot access the security account's S3 bucket.

1179. [CloudTrail] A company wants to detect when any IAM role is created with a trust policy allowing an external AWS account. How do you implement real-time detection? ; EventBridge rule matching CloudTrail `CreateRole` and `UpdateAssumeRolePolicy` events → Lambda → parse `assumeRolePolicyDocument` → check each Statement's Principal for non-org account IDs → if external: SNS alert with role ARN, creator, trust policy. Alternatively: IAM Access Analyzer auto-detects external trust relationships. EventBridge rule on Access Analyzer finding creation for IAM roles → SNS.

    EventBridge rule: `{"source":["aws.iam"],"detail-type":["AWS API Call via CloudTrail"],"detail":{"eventName":["CreateRole","UpdateAssumeRolePolicy"]}}` . Lambda parses `event['detail']['requestParameters']['assumeRolePolicyDocument']` . Iterates statements: if Principal.AWS is a list/string not matching org account IDs → alert. Must also handle `"Principal":"*"` (wildcard trust = critical severity) and `"Principal":{"Federated":"..."}` (OIDC/SAML trusts).
    Access Analyzer approach: simpler — no Lambda parsing logic. Access Analyzer continuously evaluates all role trust policies. External trust → finding with `resourceType: AWS::IAM::Role` and `principalType: AWSAccount` . Access Analyzer EventBridge event: `{"source":["aws.access-analyzer"],"detail-type":["Access Analyzer Finding"]}` . Finding created event → Lambda → SNS. AWS maintains the analysis logic. Handles edge cases (conditions, wildcards) that custom Lambda might miss.

1180. [CloudTrail] What is the difference between CloudTrail management events, data events, and Insights events? When should each be enabled, and what is the cost model? ; Management events: control plane (create/delete/modify AWS resources). Enabled by default. First copy per region free. Examples: CreateBucket, RunInstances, CreateRole. Data events: data plane (read/write to data stores). NOT enabled by default. Additional cost: ~ $0.10/100,000 events. Examples: S3 GetObject/PutObject, Lambda Invoke, DynamoDB GetItem. Insights events: anomaly detection in API call patterns. Additional cost: 0.35/100,000 write management events analyzed. Enable for high-security or compliance-driven environments.

    Data events cost management: 1 million S3 data events/day = $1/day = 365/year per account. For 50 accounts with active S3 usage: $18,250/year just for S3 data events. Selective enablement:

$only enable on sensitive buckets (PII, financial data, audit logs). Use advanced event selectors to filter by bucketname, pre$
$WriteOnly` for S3:$
$only PutObject / DeleteObject (skip GetObject — high volume, less security value for most use cases). * < br >$
$* Insights cost: analyzes write management events for unusual API rates or error rates. Cost:$
$per write management event analyzed. In a busy account: 10 million write events/month \times 0.35/100k = \$35/month.$
Low cost for the value: detects credential compromise (burst of CreateUser/AttachRolePolicy calls), misconfigured applications (high AccessDenied rate). Enable Insights on management account and production accounts.

1181. [SCP] A company applies an SCP denying `ec2:RunInstances` to a production OU. A Lambda function in a production account tries to start an EC2 instance. What happens? ; Lambda execution role gets Access Denied. SCP applies to ALL IAM principals in the member account — including Lambda execution roles. Lambda → IAM evaluation → SCP ceiling check → `ec2:RunInstances` denied by SCP → 403. The Lambda function cannot start EC2 instances regardless of what its execution role policy allows.

    SCP scope: applies to users, roles (including Lambda execution roles, EC2 instance profiles, ECS task roles), and root user of member accounts. Does NOT apply to: service-linked roles, management account. Lambda service-linked role (if exists): exempt. Lambda execution role (customer-created): subject to SCP. The production OU SCP affects all resources running in that account.
    Legitimate use case: preventing unexpected EC2 launches in production Lambda accounts (cost governance, attack surface reduction). Production Lambdas should not launch new EC2 instances — if they need compute, use Lambda itself or pre-existing instances. SCP as guardrail ensures: even if Lambda code is compromised and attacker tries to launch a C5.18xlarge to mine crypto, the SCP prevents it. Defense-in-depth: IAM least privilege + SCP guardrail.

1182. [SCP] A new SCP is attached to the root OU denying `iam:CreateUser`. A developer in a member account needs to create an IAM user for a legacy application. How do you provide an exception? ; SCPs cannot have per-account exceptions when attached to root OU — they apply to ALL member accounts. Options: (1) Move the restrictive SCP from root OU to specific OUs (not the account needing the exception). (2) Create a separate OU for exceptions, move the account there, don't attach the SCP to that OU. (3) Use a separate SCP attached to the specific account that explicitly allows (but SCPs only allow or deny — the root SCP deny still wins). Correct: move SCP to specific OUs, exclude the exception account's OU.

    SCP inheritance: account inherits SCPs from root → all ancestor OUs → account itself. If root OU has a Deny SCP → ALL accounts denied regardless of account-level SCPs (explicit deny always wins). To exclude an account: it must not be in the SCP's scope. Option: remove the SCP from root, attach to each specific OU (not the exception OU). Higher management overhead but enables per-OU exceptions.
    Better long-term solution: instead of creating IAM users for legacy apps, migrate the legacy application to use IAM roles (Roles Anywhere for on-premises, instance profile for EC2). The SCP denial for `iam:CreateUser` forces modernization — good architectural pressure. If immediate fix needed: temporarily allow the exception account via OU restructuring, document the exception, set a remediation deadline.

1183. [SCP] An SCP on the root OU contains `"Effect":"Allow","Action":"*","Resource":"*"`. Does this grant all IAM users in member accounts full permissions to all AWS services? ; No. SCPs never grant permissions — they only define the maximum ceiling. An SCP Allow removes an implicit ceiling restriction, but actual access still requires an IAM identity-based policy granting the permission. This SCP (`FullAWSAccess`) means "don't restrict anything" — equivalent to no SCP at all. An IAM user with no policies still cannot do anything.

    This is the fundamental SCP misconception. SCP Allow = "this action is within the ceiling." IAM policy = "this identity is actually granted the action." Both are needed (when using deny-list approach: SCP allows everything, IAM grants specific things). When using allow-list approach: SCP allows specific things, IAM grants from within that allowed set. Neither alone is sufficient — both must align for access.
    AWS default: `FullAWSAccess` SCP is attached to root OU by default when Organizations is set up. This is intentional: don't restrict anything by default (deny-list model). The organization is "open by default, restrict as needed." This SCP can be deleted or replaced — deleting it from root blocks ALL services for ALL member accounts (catastrophic). Never delete FullAWSAccess from root without understanding the impact.

1184. [SCP] A company wants to prevent any AWS account in their org from purchasing Reserved Instances or Savings Plans without approval. How do you implement this with SCPs? ; SCP denying: `ec2:PurchaseReservedInstancesOffering`, `ec2:ModifyReservedInstances`, `savingsplans:CreateSavingsPlan`, `rds:PurchaseReservedDBInstancesOffering`, `elasticache:PurchaseReservedCacheNodesOffering`, `es:PurchaseReservedElasticsearchInstanceOffering`. Attach to all member OU. Finance team (in management account or separate billing account): not affected by SCP → can purchase RIs/SPs.

Why this matters: Reserved Instances and Savings Plans are 1-3 year commitments. Accidental purchases = years of unused cost. SCP prevents any team from purchasing commitments without going through the finance approval process. Finance purchases from management account (SCP-exempt) or billing account. After finance approves and purchases: RI/SP applies across linked accounts via consolidated billing.
Related: deny `ce:CreateCostCategoryDefinition` , `budgets:DeleteBudget` to prevent budget tampering. SCP for cost governance: (1) Deny RI/SP purchases (commitment governance). (2) Deny large instance types (cost ceiling). (3) Require cost tags on new resources. (4) Deny disabling billing alerts. Combined: full cost governance via SCPs.

1185. [SCP] What is the difference between an SCP `"Effect":"Deny","NotAction":["needed-actions"],"Resource":"*"` approach vs `"Effect":"Deny","Action":["unwanted-actions"],"Resource":"*"` ? Which is safer and why? ; NotAction+Deny: deny everything EXCEPT the listed actions. Future AWS actions auto-denied. Action+Deny: deny only the listed actions. Future AWS actions auto-allowed. NotAction+Deny is SAFER for restrictive environments: any new AWS API not in the exception list is automatically blocked. But: if you forget to add a needed action to the exception list → users blocked. Action+Deny: more permissive by default — must remember to add new actions to deny as AWS grows.

    NotAction+Deny use case: sandbox OU — only allow a specific set of services. `"Effect":"Deny","NotAction":["s3:*","ec2:*","lambda:*","iam:*","sts:*","cloudwatch:*","logs:*"]` — sandbox can only use these services. All new AWS services (Bedrock, Q, new IoT services) are automatically denied until added to the exception list. Prevents sandbox users from accidentally using services that cost a lot.
    Action+Deny use case: production OU — allow everything except specific dangerous actions. `"Effect":"Deny","Action":["ec2:DeleteVpc","s3:DeleteBucket","iam:DeleteRole","cloudtrail:DeleteTrail"]` — everything allowed except these destructive actions. When AWS launches a new service: automatically accessible (good for innovation). When you discover a new dangerous action: add it to the deny list. This is the more common production pattern.

1186. [SCP] A company uses AWS Control Tower. They want to add a custom SCP that denies creation of public S3 buckets. Can they just attach this SCP directly to the Root OU in Organizations? ; Technically yes, but NOT recommended when using Control Tower. Control Tower manages SCPs through its own guardrail mechanism. Manually modifying SCPs outside Control Tower can cause drift and break Control Tower's tracking. Correct approach: use Control Tower's custom guardrails (via Customizations for Control Tower / CfCT or Control Tower's built-in capability). Or: apply the custom SCP to specific OUs that Control Tower doesn't manage directly.

    Control Tower guardrails: Control Tower attaches its own SCPs (called guardrails) to OUs. These are managed by Control Tower and should not be manually edited. Control Tower also detects drift: if you manually modify a Control Tower-managed SCP → drift detected → Control Tower alerts. Repair: re-apply the guardrail through Control Tower console. Customizations for Control Tower (CfCT): AWS solution for adding custom SCPs and CloudFormation StackSets to Control Tower-managed accounts. Maintains SCPs as code in a CodePipeline. New accounts added to an OU: automatically get the custom SCP applied. Proper way to extend Control Tower beyond built-in guardrails. For the public S3 bucket SCP: add it via CfCT → it applies to all current and future accounts in the OU without conflicting with Control Tower's management.

1187. [Identity Center] A company has 200 AWS accounts and 5,000 employees. They want employees to access appropriate accounts via SSO with minimal admin overhead. The company uses Okta as their IdP. What is the recommended architecture? ; IAM Identity Center + Okta: (1) Connect Identity Center to Okta via SAML + SCIM. (2) Create permission sets per job function (10-15 sets). (3) Create Okta groups matching job functions (Okta_AWS_DevAccess, Okta_AWS_AdminAccess). (4) Map Okta groups to permission sets + accounts in Identity Center. (5) HR onboards employee → Okta group assignment → SCIM syncs → AWS access appears automatically.

    SCIM benefit at 5,000 employees scale: employee joins → HR adds to Okta group → SCIM push → Identity Center group membership updated → account assignments active → employee can log in. Zero manual AWS admin action. Employee leaves → HR removes from Okta → SCIM push → Identity Center removes group membership → all 200 account accesses revoked simultaneously. Without SCIM: IT team manually updates Identity Center for every hire/departure/role change = operational nightmare at scale.
    Okta group naming convention: `AWS-{PermissionSetName}-{AccountName}` or `AWS-{PermissionSetName}-ALL` (for all-account access). Okta admin assigns groups based on job function. Identity Center account assignment: `AWS-AdminAccess-ALL` group → AdminAccess permission set → all 200 accounts. `AWS-DevAccess-TeamA` group → DevAccess permission set → [dev-team-a, staging-team-a]. Granular yet manageable via Okta group management.

1188. [Identity Center] A developer needs to access multiple AWS accounts from the CLI. How do they set up and use IAM Identity Center for CLI access? ; `aws configure sso` → prompts: SSO start URL, region, select account, select permission set → creates named profile in `~/.aws/config` . Then: `aws sso login --profile profile-name` (browser authentication). For multi-account: create one profile per account. Or: use AWS SSO credential process + shell script to switch profiles. Token cacher handles refresh.

*~/.aws/config after configure sso:

```
[profile prod-payments]
sso_start_url = https://d-xxx.awsapps.com/start
sso_region = eu-central-1
sso_account_id = 111111111111
sso_role_name = AWSReservedSSO_DevAccess_abc123
region = eu-central-1

[profile dev-team]
sso_start_url = https://d-xxx.awsapps.com/start
sso_region = eu-central-1
sso_account_id = 222222222222
sso_role_name = AWSReservedSSO_DevAccess_abc123
region = eu-central-1
```

`aws s3 ls --profile prod-payments` uses prod-payments credentials. `aws s3 ls --profile dev-team` uses dev-team credentials.*
Token refresh: `aws sso login` authenticates and caches the SSO token. Token valid for the Identity Center session duration. All profiles sharing the same SSO start URL reuse the same token. Token expiry → any AWS CLI call with any SSO profile → prompts re-login. Developers log in once for the day, work across all accounts. `AWS_PROFILE=prod-payments` environment variable sets default profile for the terminal session.

1189. [Identity Center] What is the difference between IAM Identity Center and Amazon Cognito? A company is building a customer-facing web application AND needs employees to access AWS console. Which service is used for each? ; Identity Center: AWS ACCOUNT access for EMPLOYEES/WORKFORCE. Federation to AWS Management Console and CLI. Multi-account. Not for app end-users. Cognito: APPLICATION user pool for CUSTOMERS/END-USERS. Mobile/web app authentication. Issues JWTs for app APIs. No AWS console access. Use BOTH: Cognito for customer-facing app (app user sign-up/sign-in), Identity Center for employees accessing AWS console.

    Cognito use cases: customer login for e-commerce (millions of users), mobile app authentication, social login (Google, Facebook), user pool management (sign-up, forgot password, email verification). Issues JWT (ID token, access token, refresh token) for API Gateway authorization. Not AWS console access. Scales to millions of users.
    Identity Center use cases: employee SSO to AWS accounts, per-account permission sets, multi-account CLI access, AWS console federation. Does not support customer-facing application auth. Does not issue JWTs for application APIs. Scales to tens of thousands of employees (workforce scale, not consumer scale). Common exam trap: question describes "customers need to log into a web app that calls AWS services" → Cognito User Pool + Cognito Identity Pool (federate to IAM role for AWS API calls). Never Identity Center for customers.

1190. [Identity Center] A company uses IAM Identity Center with AD Connector pointing to on-premises Active Directory. The on-premises AD goes down for maintenance. What happens to AWS console access? ; AWS console access is broken during AD downtime. AD Connector is a proxy — it forwards authentication requests to on-premises AD. No AD = no authentication. Identity Center cannot issue credentials. All users are locked out of AWS console. Break-glass: have a local Identity Center user (not AD-backed) or direct IAM user in each account as emergency access.

    AD Connector dependency: authentication-time dependency. Every login → Identity Center → AD Connector → on-premises AD. If AD or the network connection to AD is unavailable: authentication fails. This is the key risk of AD Connector. Alternative: AWS Managed Microsoft AD — AD hosted in AWS, no on-premises dependency for authentication. SCIM provisioning from on-premises AD to AWS Managed AD: user exists in both places. If on-premises AD down: AWS Managed AD still works, AWS access unaffected.
    Break-glass strategy: (1) Identity Center local user with emergency admin permissions. (2) One direct IAM user per account with hardware MFA and no Identity Center dependency. (3) AWS Managed AD as a secondary authentication source. (4) CloudWatch alert on AD Connector health → PagerDuty before planned maintenance. For compliance: document the break-glass procedure, test quarterly, review after every AD maintenance event. Dependency mapping: if AWS is business-critical during AD maintenance → migrate to AWS Managed AD (no on-premises dependency).

EOF

🔥 Chapter 27: Network Security — Defense in Depth

> **Covers:** AWS Network Firewall · Gateway Load Balancer · Centralized Traffic Inspection · NAT Gateway · Internet Gateway · Egress-Only Gateway · NACL vs Security Groups · VPC Flow Logs · Palo Alto / Fortinet Integration · IDS/IPS · Geneve Protocol · Defense in Depth

---

📋 Table of Contents (Chapter 27)

- 27.1 Defense in Depth — Network Security Model
- 27.2 Internet Gateway (IGW)
- 27.3 NAT Gateway
- 27.4 Egress-Only Internet Gateway
- 27.5 Security Groups vs NACLs
- 27.6 VPC Flow Logs
- 27.7 AWS Network Firewall
- 27.8 Gateway Load Balancer (GWLB)
- 27.9 Centralized Traffic Inspection Architectures
- 27.10 Third-Party Firewalls — Palo Alto & Fortinet
- 27.11 IDS/IPS and Signature-Based Detection
- 27.12 Geneve Protocol
- 27.13 Well-Architected Framework — Network Security
- 27.14 SAP-C02 Domains — Network Security
- 27.15 Related Services Summary

🛡️ 27.1 Defense in Depth — Network Security Model

Defense in Depth is a layered security strategy — no single control is relied upon; multiple overlapping controls are stacked so that a breach of one layer does not expose the full system.

In AWS the model maps to concrete layers:



> 🔑 **Exam key:** SAP-C02 frequently tests which layer solves which problem. Know the layer, the service, and why you can't use a lower/higher layer alone.

Real-life analogy:

- Edge = city wall + gatehouse (stops known attackers before they enter)
- Perimeter = building lobby (controls who enters the building)
- Network Firewall = security checkpoint inside the building (inspects bags)
- NACL = floor-level door policy (allows/blocks entire departments)
- Security Group = office door (only specific people enter each room)
- Application = safe inside the office (last line of defense)

🌐 27.2 Internet Gateway (IGW)

Overview

IGW is a **horizontally scaled, redundant, highly available VPC component** that enables bidirectional communication between instances in a VPC and the internet. It performs NAT for instances with public IPv4 addresses.

Scope: Regional. Attached to a VPC (one IGW per VPC). Implicitly spans all AZs in the region.

Linux/Open-Source equivalent: A router with a routable external IP and `iptables MASQUERADE` for public-facing servers.

Azure equivalent: Azure Internet Gateway does not exist as a discrete resource — Azure VNets automatically route to internet. The closest concept is Azure NAT Gateway or Public IP on a subnet.

How IGW Works

```
EC2 (10.0.1.5, Elastic IP 52.x.x.x)
    |
    |  Packet: src=10.0.1.5, dst=8.8.8.8
    ▼
Internet Gateway
    |  NAT: src=10.0.1.5 → src=52.x.x.x (1:1 NAT using Elastic IP)
    ▼
INTERNET → 8.8.8.8
```

Return traffic:

```
8.8.8.8 → 52.x.x.x (Elastic IP)
IGW: dst=52.x.x.x → dst=10.0.1.5
→ Routes to EC2
```

> ⚠️ **Key point:** IGW does NOT perform NAT for IPv6. IPv6 is globally routable — no NAT needed. For IPv6-only egress, use Egress-Only Internet Gateway.

Requirements for Public Internet Access

1. Instance in a **public subnet** (route table has `0.0.0.0/0 → igw-xxx`)
2. Instance has a **public IPv4 address** or **Elastic IP**
3. **Security group** allows the desired traffic
4. **NACL** allows the desired traffic (stateless, both directions)
5. **IGW attached** to the VPC

> 🔑 **Exam trap:** If ANY of the 5 conditions above is missing, internet access fails. Common trap: public subnet route to IGW exists but no Elastic IP assigned → no internet access.

IGW Limitations

Limitation	Detail
One per VPC	Cannot attach multiple IGWs to one VPC
No bandwidth cap	IGW scales automatically; bandwidth limited by instance type
No filtering	IGW has no firewall capability — it just routes
No monitoring	IGW itself has no logs — use VPC Flow Logs
Not a NAT device for private instances	Private subnets need a NAT Gateway/NAT Instance

IGW vs Other Egress Options

Scenario	Use
EC2 needs inbound + outbound internet	IGW + public subnet + Elastic IP
Private EC2 needs outbound only (IPv4)	NAT Gateway + IGW
Private EC2 needs outbound only (IPv6)	Egress-Only IGW
Corporate office to AWS private	VPN or Direct Connect
Filtering/inspection of internet traffic	NAT GW + Network Firewall + IGW

🔀 27.3 NAT Gateway

Overview

NAT Gateway provides **outbound internet connectivity for private subnet resources** without exposing them to inbound connections. Managed service — AWS handles availability, patching, scaling.

Scope: Zonal — each NAT Gateway is deployed in one Availability Zone. For HA: deploy one per AZ.

Linux equivalent: `iptables MASQUERADE` + an EC2 instance configured as a NAT instance (legacy).

Azure equivalent: Azure NAT Gateway (very similar concept — managed NAT for private subnets).

NAT Gateway vs NAT Instance

Feature	NAT Gateway	NAT Instance
Management	Fully managed	You manage EC2
HA	Per-AZ redundancy built in	Manual HA with scripts/ASG
Bandwidth	5 Gbps to 100 Gbps (auto-scales)	Limited by instance type
Cost	Per-hour + per-GB data	EC2 + data transfer
Security Groups	Not supported	Supported
Port forwarding	Not supported	Supported
Bastion host combo	Not possible	Possible (dual use)
Use today	Default choice	Legacy/cost optimization only

> 🔑 **Exam key:** NAT Instance is legacy. NAT Gateway is the AWS-recommended choice. Exam may present NAT Instance for cost optimization scenarios (small, infrequent traffic).

NAT Gateway Types

Type	Direction	IPv6	Use Case
Public NAT Gateway	Outbound to internet via IGW	No	Private subnet EC2 → internet
Private NAT Gateway	Routes to other VPCs/on-prem via TGW	No	Private-to-private routing with overlapping CIDRs

> ⚠️ **Private NAT Gateway:** Used when two VPCs have overlapping CIDRs and need to communicate via Transit Gateway. The NAT translates the source IP to avoid overlap. No internet involved.

NAT Gateway Architecture

```
Private Subnet (10.0.2.0/24)
    |
    EC2 → Route Table: 0.0.0.0/0 → nat-gw-id
    |
    ▼
NAT Gateway (in public subnet, has Elastic IP 52.x.x.x)
    |  src=10.0.2.5 → src=52.x.x.x (port 40123)
    |
    ▼
Internet Gateway
    |
    ▼
INTERNET
```

Multi-AZ NAT Gateway Architecture (HA)

```
AZ-A: Private EC2 → NAT GW (AZ-A) → IGW
AZ-B: Private EC2 → NAT GW (AZ-B) → IGW
AZ-C: Private EC2 → NAT GW (AZ-C) → IGW
```

> ⚠️ **Cross-AZ NAT Gateway is anti-pattern:** routing from AZ-B EC2 to NAT GW in AZ-A works but incurs cross-AZ data transfer charges (~$0.01/GB) and adds single point of failure. Always use one NAT GW per AZ.

Pricing

- **Per-hour:** ~$0.045/hour$(32/month) per NAT GW
- **Per-GB processed:** ~$0.045/GB
- **Data transfer:** standard data transfer charges apply on top

> 🔑 **Exam tip:** NAT Gateway is a significant cost for high-throughput scenarios. Alternative for S3/DynamoDB: use VPC Endpoints (free data transfer, bypass NAT GW entirely).

NAT Gateway Limitations

Limitation	Value
Max bandwidth per NAT GW	100 Gbps (auto-scales from 5 Gbps)
Max simultaneous connections	55,000 per destination
Max port allocation	64,512 simultaneous connections
Ports per destination	55,000 (then new EIP needed)
IPv6 support	No (use Egress-Only IGW)
Security Groups	Not supported

> ⚠️ **Port exhaustion:** If a single EC2 opens >55,000 simultaneous connections to the same destination IP+port combo, NAT GW drops connections. Fix: use multiple NAT Gateways with multiple Elastic IPs, or redesign connection pooling.

🔵 27.4 Egress-Only Internet Gateway

Overview

Egress-Only Internet Gateway (EIGW) allows **outbound IPv6 traffic from VPC to internet while blocking unsolicited inbound IPv6 connections**.

Scope: Regional. One per VPC (same as IGW). Free to use (no hourly charge).

Why it exists: IPv6 addresses are globally routable (no NAT). A standard IGW with IPv6 allows bidirectional traffic. EIGW restricts to **egress only** — same as NAT Gateway behavior but for IPv6.

EIGW vs IGW for IPv6

Feature	IGW (IPv6)	Egress-Only IGW
Inbound allowed	Yes	No (blocked)
Outbound allowed	Yes	Yes
Use case	Public-facing IPv6 services	Private EC2 needing IPv6 updates
Cost	Free	Free

Routing Setup

```
Private subnet route table:
::/0 (all IPv6) → eigw-id

(NOT igw-id — that would allow inbound)
```

> 🔑 **Exam trap:** If the question says "private EC2 needs to download IPv6 updates but must not be reachable from internet" → Egress-Only IGW. Not NAT Gateway (NAT GW doesn't support IPv6).

🔐 27.5 Security Groups vs NACLs

This is a **fundamental exam topic** — every scenario will have NACL/SG implications.

Core Differences

Feature	Security Group	NACL
Level	Instance/ENI level	Subnet level
Statefulness	**Stateful** — return traffic auto-allowed	**Stateless** — must explicitly allow both directions
Rules	Allow only (implicit deny all)	Allow AND Deny rules
Rule evaluation	All rules evaluated, most permissive wins	Rules evaluated **in order** by rule number (lowest first), first match wins
Default behavior	New SG: deny all inbound, allow all outbound	Default NACL: allow all. Custom NACL: deny all
Association	One or more SGs per ENI	One NACL per subnet
Scope	Multiple instances across subnets	All instances in subnet
Changes	Take effect immediately	Take effect immediately
Return traffic	Automatically allowed	Must explicitly allow ephemeral ports (1024-65535)

> 🔑 **Exam key — NACL ephemeral ports:** When a client (e.g., EC2 in private subnet) initiates a connection to port 443, the response comes back on a random **ephemeral port** (1024-65535). NACL must allow inbound on ephemeral ports or responses are blocked. Security Group: stateful — response automatically allowed.

NACL Rule Evaluation Example

```
NACL Inbound Rules:
Rule 100: ALLOW   TCP 443  0.0.0.0/0
Rule 110: DENY    TCP 443  10.0.0.5/32
Rule *:   DENY    ALL

→ Traffic from 10.0.0.5 port 443: hits Rule 100 first → ALLOWED
(Rule 110 never reached — first match wins)

Fix: move Deny rule to lower number than Allow:
Rule 90:  DENY    TCP 443  10.0.0.5/32   ← evaluated first → DENIED
Rule 100: ALLOW   TCP 443  0.0.0.0/0
```

> ⚠️ **Order matters in NACLs:** To block specific IP when a broad allow exists, the deny rule MUST have a lower rule number.

When to Use Each

Use Case	Security Group	NACL
Block specific IP/CIDR	❌ (allow-only, no block)	✅ Explicit DENY rule
Restrict traffic between instances	✅ Reference other SG by ID	❌ Only CIDR-based
Subnet-wide policy	❌ Per-instance	✅ One rule applies to whole subnet
Stateful connection tracking	✅ Built in	❌ Must manage both directions
Layer of defense for compromised SG	✅ N/A	✅ Add NACL block as second layer

Security Group Reference

Security groups can reference **other security groups** as sources/destinations — not just CIDRs. This is powerful:

```
Web-SG: allow inbound 443 from 0.0.0.0/0
App-SG: allow inbound 8080 from Web-SG   ← only web tier can access app tier
DB-SG:  allow inbound 5432 from App-SG  ← only app tier can access DB
```

> 🔑 **Exam tip:** "Only allow traffic from the web tier to the application tier" → Security Group with SG-ID reference, not CIDR. CIDR-based rules break when IPs change.

Defense in Depth: SG + NACL

```
Attacker IP: 203.0.113.5 attempts port scan

NACL: DENY 203.0.113.5/32 ALL → blocked at subnet level (never reaches EC2)

Even if NACL is misconfigured:
SG: no inbound rule for 203.0.113.5 → blocked at instance level

Two independent layers must both fail for attacker to reach EC2.
```

📊 27.6 VPC Flow Logs

Overview

VPC Flow Logs capture **metadata about IP traffic** flowing through VPC network interfaces, subnets, or the entire VPC. They do NOT capture packet payloads — only connection metadata.

Scope: Regional. Can be enabled at VPC, subnet, or ENI level.

Linux equivalent: `tcpdump` for headers only (no payload), or `netstat` statistics, or `iptables LOG` target.

Azure equivalent: NSG Flow Logs (Network Security Group Flow Logs) — same concept.

What Flow Logs Capture

Default fields (in order):

```
version account-id interface-id srcaddr dstaddr srcport dstport protocol packets bytes start end action log-status
```

Example record:

```
2 123456789012 eni-abc123 10.0.1.5 10.0.2.10 52341 443 6 20 4000 1609459200 1609459260 ACCEPT OK
```

Custom format (v3+) adds:

- `vpc-id`, `subnet-id`, `instance-id`
- `tcp-flags` (SYN, ACK, FIN, RST)
- `pkt-srcaddr`, `pkt-dstaddr` (actual source when NAT involved)
- `flow-direction` (ingress/egress)
- `traffic-path` (through IGW, NAT GW, etc.)

> 🔑 **Exam key:** Flow Logs capture ACCEPT and REJECT. Rejected traffic = security group or NACL blocking. Use to audit security rules without changing them.

Flow Log Actions

Action	Meaning
`ACCEPT`	Traffic allowed by security group AND NACL
`REJECT`	Traffic denied by security group OR NACL
`NODATA`	No traffic during aggregation interval
`SKIPDATA`	Flow logs skipped (capacity constraint)

Flow Log Destinations

Destination	Use Case	Cost
CloudWatch Logs	Real-time analysis, metric filters, alarms	CloudWatch Logs ingestion ($0.50/GB)
S3	Long-term storage, Athena querying	S3 storage only (~$0.023/GB)
Kinesis Data Firehose	Real-time streaming to SIEM, Splunk, OpenSearch	Firehose processing fee

> ⚠️ **Flow Logs do NOT capture:**
>
> - DNS queries (use Route 53 Resolver Query Logs)
> - DHCP traffic
> - Instance metadata service traffic (169.254.169.254)
> - AWS Time Sync traffic (169.254.169.123)
> - Traffic to VPC Router reserved addresses

Flow Log Use Cases

```
flowchart TD
    A[Security Incident] --> B{What happened?}
    B --> C[Which IPs connected to compromised EC2?]
    B --> D[Did attacker exfiltrate data?]
    B --> E[Which security rules were bypassed?]
    C --> F[Flow Logs: filter by dstaddr=EC2 IP, action=ACCEPT]
    D --> G[Flow Logs: filter by srcaddr=EC2 IP, bytes > threshold]
    E --> H[Flow Logs: filter action=ACCEPT where expected REJECT]
```

Athena Query Examples

```sql
-- Find all rejected traffic to EC2 instance
SELECT srcaddr, dstport, COUNT(*) as attempts
FROM vpc_flow_logs
WHERE dstaddr = '10.0.1.5'
  AND action = 'REJECT'
  AND start > 1700000000
GROUP BY srcaddr, dstport
ORDER BY attempts DESC;

-- Identify top talkers (potential data exfiltration)
SELECT srcaddr, dstaddr, SUM(bytes) as total_bytes
FROM vpc_flow_logs
WHERE srcaddr LIKE '10.0.%'
GROUP BY srcaddr, dstaddr
ORDER BY total_bytes DESC
LIMIT 20;
```

Aggregation Interval

Interval	Default	Custom
1 minute	Available (more granular, higher cost)	Set `--max-aggregation-interval 60`
10 minutes	Default for ENI/subnet	Set `--max-aggregation-interval 600`

> 🔑 **Exam tip:** Flow Logs have a delay — not real-time. For real-time threat detection, use Network Firewall alert logs or GuardDuty (which uses flow logs internally but applies ML for near-real-time alerting).

Flow Logs Limitations

Limitation	Detail
No payload capture	Metadata only — can't read HTTP body or SQL queries
Not real-time	1-15 minute delay depending on interval and destination
Cost at scale	High-traffic VPCs generate massive log volumes
No filtering at source	All traffic captured (filter after ingestion)
Cannot be modified	Once created, format cannot be changed — must delete and recreate

---

🔥 27.7 AWS Network Firewall

Overview

AWS Network Firewall is a **managed, stateful Layer 3-7 network firewall** for VPCs. It provides stateful packet inspection, intrusion detection and prevention (IDS/IPS), and web filtering — without deploying or managing firewall appliances.

Scope: Regional. Deployed per-AZ (one firewall endpoint per AZ for HA).

Linux equivalent: `nftables` + `Suricata` IDS/IPS engine (AWS Network Firewall uses Suricata-compatible rule syntax).

Azure equivalent: Azure Firewall (very similar — managed firewall PaaS for VNets).

Open-source: Suricata (the IDS/IPS engine), `snort` (older alternative).

Architecture

```
VPC
├── Firewall Subnet (dedicated /28 subnet per AZ)
│   └── Firewall Endpoint (ENI injected by AWS)
├── Public Subnet
│   └── NAT Gateway, Load Balancers
└── Private Subnet
    └── Application EC2s
```

Traffic flow (ingress inspection):

```
Internet
    |
    ▼
IGW (Ingress route table: send all to firewall endpoint)
    |
    ▼
Firewall Endpoint (in Firewall Subnet)
    |  → Stateful inspection, IDS/IPS rules applied
    ▼
NAT Gateway / Load Balancer (Public Subnet)
    |
    ▼
Application EC2 (Private Subnet)
```

> ⚠️ **Asymmetric routing breaks firewall:** both ingress and egress must pass through the SAME firewall endpoint (same AZ). Asymmetric paths = stateful tracking fails = connections dropped.

Rule Types

Rule Type	Layer	Capability
Stateless rules	L3-L4	IP, CIDR, port, protocol matching. Fast. No state tracking.
Stateful rules	L3-L7	Full stateful inspection. Suricata-compatible. Domain filtering.
Managed rule groups	L3-L7	AWS-managed threat intelligence: malware, botnets, TOR exits

Stateful Rule Groups

Domain list rules (easiest to use for web filtering):

```
# Allow only specific domains (whitelist)
DENY: *.
ALLOW: amazonaws.com
ALLOW: example.com
```

Suricata-compatible rules (full IDS/IPS power):

```
# Block HTTP to known bad IP
drop http any any -> 203.0.113.5 80 (msg:"Block C2 server"; sid:1000001; rev:1;)

# Alert on SQL injection attempt
alert http any any -> any any (msg:"SQL Injection Attempt"; content:"UNION SELECT"; nocase; sid:1000002;)

# Block TLS to untrusted certificate authority
alert tls any any -> any 443 (msg:"Untrusted TLS cert"; tls.cert_issuer; content:"Evil CA"; sid:1000003;)
```

> 🔑 **Exam key:** AWS Network Firewall uses **Suricata** rule syntax. You can import existing on-premises Suricata/Snort rules. This is the migration path from on-premises IDS/IPS.

IDS vs IPS Mode

Mode	Action	Use Case
IDS (Detection)	Alert/log only. Traffic passes.	Passive monitoring, compliance reporting
IPS (Prevention)	Drop/reject matching traffic	Active blocking of threats

In AWS Network Firewall: configure rule group action as `ALERT` (IDS) or `DROP` (IPS).

```
Rule: match SQL injection signature
  IDS mode: action=ALERT → log to CloudWatch, traffic continues
  IPS mode: action=DROP  → traffic blocked, log generated
```

Network Firewall vs WAF vs Shield

Service	Layer	Granularity	Use Case
AWS Network Firewall	L3-L7 (any protocol)	VPC-level, per-AZ	Stateful inspection, IDS/IPS, domain filtering for VPC traffic
AWS WAF	L7 (HTTP/HTTPS only)	CloudFront/ALB/API GW	SQL injection, XSS, OWASP Top 10 for web apps
AWS Shield Standard	DDoS L3/L4	All AWS services	Automatic DDoS mitigation for all accounts
AWS Shield Advanced	DDoS L3-L7	CloudFront/ALB/NLB/Route53	Advanced DDoS + 24/7 DRT + cost protection

> 🔑 **Exam key:** "Web application protection" → WAF. "Network-level threat inspection for all protocols" → Network Firewall. "DDoS protection" → Shield. These are complementary, not competing.

Network Firewall Pricing

Component	Cost
Endpoint per AZ	~$0.395/hour per endpoint (~$285/month)
Data processing	~$0.065/GB
Rule group	No charge for rules

> ⚠️ **Cost consideration:** In 3-AZ region: $3 \times \$285 = \$855$/month minimum. For small workloads, consider if NACL + SG covers requirements.

⚖️ 27.8 Gateway Load Balancer (GWLB)

Overview

Gateway Load Balancer is a **transparent network appliance insertion service** — it distributes traffic to third-party virtual network appliances (firewalls, IDS/IPS, deep packet inspection) at scale, without changing the source/destination of the packet.

Scope: Regional. Zonal endpoints (GWLBe) deployed per-AZ.

Linux equivalent: No direct equivalent — GWLB uses the Geneve protocol (explained in 27.12) to encapsulate and forward packets while preserving original IP headers.

Azure equivalent: Azure Gateway Load Balancer (released 2022 — same concept, same Geneve encapsulation).

Why GWLB Exists — The Problem It Solves

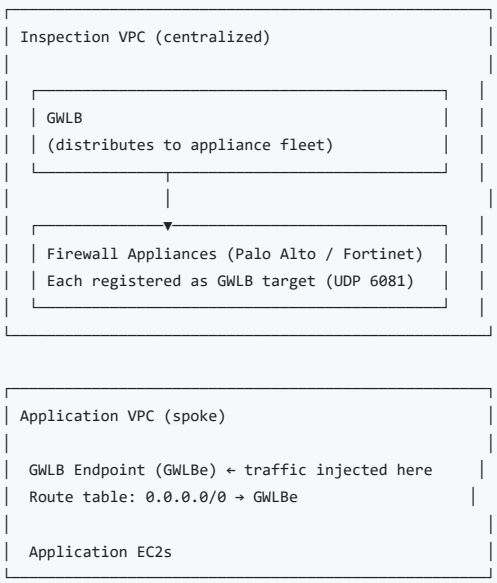
Before GWLB, inserting a third-party firewall into the traffic path required:

```
BEFORE (complex, fragile):
Client → Firewall ENI (ingress) → Internal routing → Firewall ENI (egress) → Server
         ↑ Source IP modified       ↑ complex routing rules      ↑ packet reassembly
         ↑ Original headers lost
```

With GWLB:

```
AFTER (transparent, scalable):
Client → GWLB Endpoint → GWLB → Appliance (sees original headers via Geneve) → GWLB → Server
                                  ↑ Original src/dst preserved inside Geneve tunnel
```

GWLB Architecture



GWLB Key Features

Feature	Detail
Protocol	Geneve (UDP port 6081) encapsulation
Transparency	Original IP headers preserved inside Geneve tunnel
Health checking	Removes unhealthy appliances from rotation
Stickiness	5-tuple flow stickiness — same flow always to same appliance
Scaling	Auto-scales appliance fleet behind GWLB
Cross-VPC	GWLB in inspection VPC, GWLBe in spoke VPCs
Cross-account	Supported via PrivateLink/GWLB endpoint service

> 🔑 **Exam key — Flow stickiness:** GWLB uses 5-tuple (src IP, dst IP, src port, dst port, protocol) to stick a flow to one appliance. This ensures stateful appliances see all packets of a session (not split across multiple appliances).

GWLB vs AWS Network Firewall

Aspect	AWS Network Firewall	GWLB + 3rd-party
Management	Fully managed by AWS	You manage appliance AMIs, scaling, licenses
Appliance	AWS-built (Suricata-based)	Any GWLB-compatible appliance (Palo Alto, Fortinet, Check Point)
Features	IDS/IPS, domain filtering, stateful rules	Full NGFW: SSL decryption, user-ID, advanced threat prevention
Cost	~$0.395/endpoint+0.065/GB	Appliance licensing + GWLB ($0.008/LCU-hour+0.004/GB)
Use case	AWS-native, no appliance license	Existing on-premises firewall vendor, compliance requirement for specific vendor

> 🔑 **Exam decision tree:**
>
> - "AWS-native managed firewall, no vendor dependency" → **AWS Network Firewall**
> - "Must use Palo Alto / Fortinet / Check Point (compliance or existing license)" → **GWLB + vendor appliance**
> - "Migrate on-premises Palo Alto to AWS" → **GWLB + Palo Alto VM-Series**

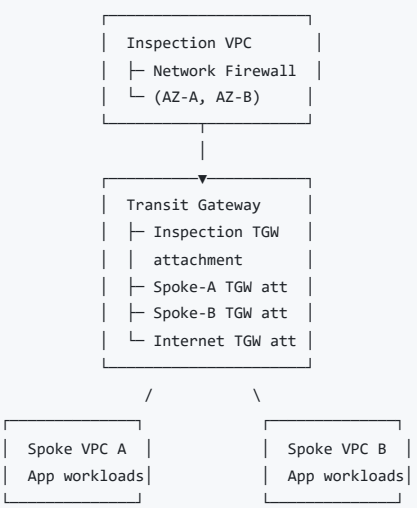
🏗️ 27.9 Centralized Traffic Inspection Architectures

Overview

For multi-VPC organizations (AWS Organizations), centralized inspection is critical — instead of deploying firewalls in every VPC, a **hub-and-spoke** model routes all traffic through a centralized inspection VPC.

Two main architectures:

Architecture 1: Transit Gateway + Network Firewall (AWS-Native)



Traffic flow (spoke-to-spoke):

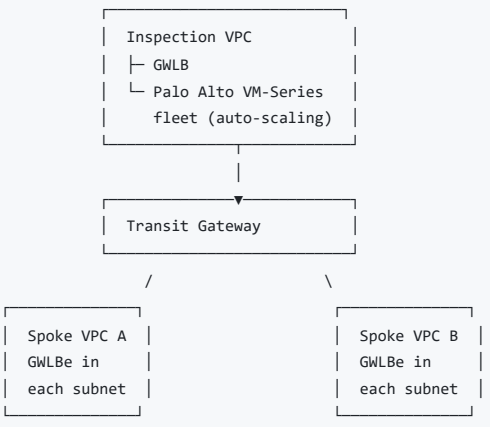
```
EC2 (Spoke A) → TGW → Inspection VPC route table → Network Firewall → back to TGW → Spoke B
```

Route tables required:

- TGW Route Table (Inspection): 10.0.0.0/8 → attachment to Inspection VPC
- TGW Route Table (Spoke): 0.0.0.0/0 → attachment to Inspection VPC
- Inspection VPC subnets: route to firewall endpoints in same AZ

> ⚠️ **AZ affinity is mandatory:** Traffic from Spoke A entering TGW in AZ-1 must route to Firewall Endpoint in AZ-1. Cross-AZ firewall routing = asymmetric routing = connection drops.

Architecture 2: Transit Gateway + GWLB + Third-Party NGFW



Architecture 3: Egress-Only Centralized Inspection (Internet Egress)

For inspecting all outbound internet traffic from all VPCs:

```
Spoke VPC → TGW → Egress VPC (NAT GW + Network Firewall) → IGW → Internet

Egress VPC contains:
  - Network Firewall (stateful IPS rules for egress filtering)
  - NAT Gateway (single egress IP, easy to allowlist externally)
  - IGW (single internet exit point)
```

> 🔑 **Exam tip — Centralized NAT:** All spoke VPCs share one set of Elastic IPs (from NAT GW in egress VPC). External partners allowlist these IPs once. Without centralization: each VPC has its own NAT GW EIP = many IPs to manage.

Architecture 4: Ingress Inspection (East-West + North-South)

```
Internet → IGW → Network Firewall (in inspection VPC) → ALB → App EC2

North-South: internet ↔ VPC (inspected by firewall)
East-West: VPC A ↔ VPC B (inspected by same firewall via TGW routing)
```

Route Table Design Summary

```
Inspection VPC - Firewall Subnet route table:
  10.0.0.0/8   → TGW attachment (send inspected traffic to destination)
  0.0.0.0/0    → IGW (for internet egress after inspection)

Inspection VPC - TGW Subnet route table:
  0.0.0.0/0    → Firewall Endpoint (AZ-A)  ← AZ-A traffic
  10.0.0.0/8   → Firewall Endpoint (AZ-A)

TGW Route Table (for spoke VPCs):
  0.0.0.0/0    → Inspection VPC attachment ← all traffic to inspection
  10.0.0.0/8   → Inspection VPC attachment ← internal traffic also inspected
```

🔴 27.10 Third-Party Firewalls — Palo Alto & Fortinet

Overview

AWS Marketplace offers **Next-Generation Firewalls (NGFW)** from leading security vendors as AMIs — Palo Alto Networks VM-Series and Fortinet FortiGate are the most common in enterprise environments.

Both integrate with GWLB for scalable, transparent traffic inspection.

Palo Alto VM-Series on AWS

Feature	Detail
Product	Palo Alto VM-Series (PA-VM-AWS)
Deployment	EC2 instance (c5.xlarge minimum, c5.4xlarge recommended)
Integration	GWLB target (UDP 6081 Geneve), PrivateLink endpoint
Licensing	BYOL or Pay-as-you-go via AWS Marketplace
Management	Panorama (centralized management), self-managed
Features	App-ID (Layer 7 app identification), User-ID, SSL decryption, WildFire (cloud sandbox), Threat Prevention
Auto-scaling	Palo Alto CloudFormation bootstrap + Auto Scaling Group

Palo Alto-specific capabilities (over AWS Network Firewall):

- **SSL/TLS decryption** — decrypt and inspect HTTPS traffic (requires cert deployment to clients)
- **App-ID** — identify applications by behavior, not just port (e.g., identify Zoom traffic on port 443)

- **User-ID** — firewall rules based on AD username, not just IP
- **WildFire** — submit unknown files to Palo Alto cloud sandbox for zero-day analysis
- **GlobalProtect** — client VPN integrated with the same NGFW policy

Fortinet FortiGate on AWS

Feature	Detail
Product	FortiGate VM (FGVM)
Deployment	EC2 instance, GWLB target
Integration	GWLB, TGW, Direct Connect
Licensing	BYOL or PAYG
Management	FortiManager (centralized), FortiAnalyzer (logging)
Features	IPS, SSL inspection, URL filtering, FortiSandbox, SD-WAN
SD-WAN	FortiGate can serve as SD-WAN hub — unique capability

Palo Alto vs Fortinet vs AWS Network Firewall

Capability	AWS Network Firewall	Palo Alto VM-Series	Fortinet FortiGate
SSL Decryption	No	✅ Yes	✅ Yes
App-ID / App Recognition	Limited (domain matching)	✅ Deep App-ID	✅ Application Control
User-ID (AD integration)	No	✅ Yes	✅ Yes
Cloud sandbox (zero-day)	No	✅ WildFire	✅ FortiSandbox
SD-WAN	No	No (separate product)	✅ Built-in
Centralized Management	AWS Firewall Manager	Panorama	FortiManager
Cost	Low (managed)	High (instance + license)	High (instance + license)
Operational burden	Low	High	High
Compliance (vendor-specific)	May not satisfy	PCI-DSS, HIPAA certified	PCI-DSS, HIPAA certified

> 🔑 **Exam decision:**
>
> - "Minimize operational overhead, AWS-native" → AWS Network Firewall
> - "Company has existing Palo Alto Panorama" → Palo Alto VM-Series + GWLB
> - "Need SSL inspection for all VPC traffic" → Palo Alto or Fortinet (Network Firewall cannot do SSL decryption)
> - "Need SD-WAN integrated with firewall" → Fortinet

Migration Path: On-Premises to AWS

```
On-premises Palo Alto PA-3200 Series
    │
    ├── Export Panorama device group configs
    ├── Deploy Palo Alto VM-Series via AWS Marketplace
    ├── Configure GWLB target group (UDP 6081)
    ├── Bootstrap VM-Series with S3 bootstrap bucket:
    │   ├── init-cfg.txt  (Panorama IP, auth keys)
    │   ├── content/      (threat signatures)
    │   └── license/      (license file)
    └── Panorama pushes security policy from on-premises → AWS VM-Series
```

🔍 27.11 IDS/IPS and Signature-Based Detection

IDS vs IPS

Aspect	IDS (Intrusion Detection System)	IPS (Intrusion Prevention System)
Action	Alert/log only	Alert + block/drop
Position	Can be out-of-band (tap/mirror)	Must be inline (traffic flows through)
Risk	False negative = missed attack	False positive = legitimate traffic blocked
AWS implementation	Network Firewall `ALERT` action	Network Firewall `DROP` action

Signature-Based Detection

Signatures are **patterns** that match known attack traffic. Each signature has:

- `sid` — unique rule ID
- `msg` — human-readable description
- `content` — byte pattern to match
- `nocase` — case-insensitive match
- `pcre` — Perl-compatible regex for complex patterns

Suricata rule anatomy:

```
action protocol src_ip src_port direction dst_ip dst_port (options)

drop tcp any any -> any 22 (msg:"SSH brute force attempt"; threshold:type both, track by_src, count 5, seconds 60; sid:2000001;)
|    |   |   |  |   | |  |
|    |   |   |  |   | |  └── destination port 22
|    |   |   |  |   | └───── destination IP (any)
|    |   |   |  |   └──────── direction (→ one-way)
|    |   |   |  └──────────── source port (any)
|    |   |   └─────────────── source IP (any)
|    |   └─────────────────── protocol (tcp)
|    └─────────────────────── action (drop/alert/pass/reject)
```

Signature Categories in AWS Network Firewall Managed Rules

Rule Group	Detects
`AWSManagedRulesCommonRuleSet`	Common attack patterns
`AWSManagedRulesThreatIntelligence`	Known malicious IPs (C2 servers, TOR exits, botnets)
`AWSManagedRulesBotControlRuleSet`	Bot traffic
`ThreatSignaturesDoS`	Denial of service patterns
`ThreatSignaturesMalware`	Known malware C2 patterns
`ThreatSignaturesPhishing`	Phishing indicators

> 🔑 **Exam key:** AWS manages signature updates for managed rule groups. You don't need to update signatures manually. For custom signatures (proprietary threat intel), use custom rule groups with Suricata syntax.

Signature Limitations

Limitation	Impact
Zero-day attacks	No signature exists → IDS/IPS misses it → need behavioral detection (GuardDuty ML)
Encrypted traffic	Signatures can't see inside TLS without decryption → SSL inspection needed
False positives	Legitimate traffic matches signature → blocks business traffic → tune rules
Performance	Deep packet inspection is CPU-intensive → limits throughput

GuardDuty vs Network Firewall (IDS/IPS)

Aspect	AWS GuardDuty	AWS Network Firewall IDS
Input	VPC Flow Logs, CloudTrail, DNS logs	Live packet inspection
Detection	ML-based anomaly + signatures	Signatures only (Suricata rules)
Real-time blocking	No (alerts only)	Yes (IPS mode drops)
Encrypted traffic	Analyzes metadata (no decryption)	Requires SSL decryption for content
Scope	Account-wide, automatic	VPC traffic explicitly routed through firewall
Zero-day	ML detects unusual behavior	Signatures miss zero-day

> 🔑 **Exam:** GuardDuty + Network Firewall are **complementary**. GuardDuty detects threats in existing traffic (no routing change needed). Network Firewall actively blocks inline. Defense in depth = both.

🔗 27.12 Geneve Protocol

Overview

Geneve (Generic Network Virtualization Encapsulation) is a network encapsulation protocol used by GWLB to transparently forward packets to appliances while preserving original source and destination IP headers.

- **RFC 8926**
- **UDP port 6081**
- Developed by VMware, Microsoft, Red Hat, Intel
- Successor to VXLAN and NVGRE

Linux equivalent: Linux kernel supports Geneve natively (`ip link add geneve0 type geneve`). Used in OpenStack, KVM, OVS (Open vSwitch).

How Geneve Works with GWLB

```
Original packet (Client → Server):
┌──────────────────────────────────────────┐
│ Outer IP Header (GWLB ↔ Appliance)       │  ← Added by GWLB
│ UDP Header (port 6081)                   │  ← Added by GWLB
│ Geneve Header (metadata: VPC, flow info) │  ← Added by GWLB
│ ┌──────────────────────────────────┐     │
│ │ INNER IP: src=Client, dst=Server │     │  ← ORIGINAL packet preserved
│ │ INNER TCP: dstport=443           │     │  ← untouched
│ │ PAYLOAD                          │     │
│ └──────────────────────────────────┘     │
└──────────────────────────────────────────┘
```

Why this matters:

- Appliance sees **original src/dst IP** inside the Geneve tunnel
- Firewall rules can use actual client IP (not GWLB IP)
- After inspection: appliance strips Geneve header, sends back to GWLB
- GWLB forwards original packet to destination

Geneve vs VXLAN vs GRE

Protocol	Port	Metadata	Use Case
Geneve	UDP 6081	Extensible TLV headers	GWLB appliance insertion, network virtualization
VXLAN	UDP 4789	VNI (24-bit tenant ID)	Overlay networking (EKS, VPC)
GRE	IP Protocol 47	Sequence number, key	VPN tunnels (AWS VPN uses modified GRE)
IPSec	ESP (50) / AH (51)	Crypto metadata	Encrypted VPN tunnels

> 🔑 **Exam tip:** Geneve/UDP 6081 = GWLB integration. If a question mentions "appliance needs to see original source IP" or "transparent inspection" → GWLB + Geneve. You don't need deep Geneve knowledge for the exam — know it's the protocol GWLB uses.

📐 27.13 Well-Architected Framework — Network Security

Pillar	Network Security Implementation
Operational Excellence	Centralized firewall management (AWS Firewall Manager). IaC for firewall rules (CloudFormation, Terraform). VPC Flow Logs → CloudWatch → automated alerts. Route table changes trigger Config rules. Network Firewall rule changes tracked in CloudTrail.
Security	Defense in depth: IGW → Network Firewall → NACL → SG → application auth. NACLs block known malicious CIDRs. SGs on principle of least privilege (no 0.0.0.0/0 inbound). Flow Logs for incident investigation. GuardDuty for threat detection. Network Firewall for IPS. GWLB + Palo Alto for SSL inspection where required.
Reliability	One NAT GW per AZ. Network Firewall endpoints per AZ. GWLB with multi-AZ appliance fleet. TGW for resilient multi-VPC connectivity. Route table health checks via Lambda + CloudWatch.
Performance Efficiency	VPC Endpoints for S3/DynamoDB (bypass NAT GW = lower latency + cost). GWLB auto-scales appliance fleet on demand. Network Firewall stateless rules for high-throughput low-risk traffic. Stateful rules only for traffic needing deep inspection.
Cost Optimization	One shared NAT GW in egress VPC (not per-VPC). VPC Endpoints eliminate NAT GW data charges for S3/DynamoDB. Network Firewall endpoints only in AZs with active workloads. GWLB: pay per LCU-hour, scale down during off-hours. Flow Logs to S3 (cheaper than CloudWatch Logs).
Sustainability	Centralized inspection VPC reduces per-VPC firewall instances. GWLB auto-scaling uses minimum instances needed. Aggregated NAT Gateway reduces total ENIs. Flow Log compression (Parquet format in S3) reduces storage.

🎯 27.14 SAP-C02 Domains — Network Security

Domain 1 — Design Solutions for Organizational Complexity: Multi-account centralized inspection architecture. AWS Firewall Manager deploys Network Firewall policies to all accounts in an OU. Centralized egress VPC with shared NAT Gateways. GWLB endpoint service shared across accounts. Transit Gateway with inspection route tables. Security Hub aggregates firewall findings from all accounts.

Domain 2 — Design for New Solutions: New VPC requiring internet access: public subnets + IGW for public resources, private subnets + NAT GW for private resources. New compliance requirement for IDS/IPS: Network Firewall with managed rule groups. New requirement for Palo Alto-specific features (SSL inspection): GWLB + Palo Alto VM-Series. New IPv6 workload: dual-stack VPC, Egress-Only IGW for private IPv6 subnets.

Domain 3 — Continuous Improvement: Migrate from NAT Instances to NAT Gateways (managed, no maintenance). Add Network Firewall to existing VPC (insert firewall subnet, update route tables). Optimize cost: add VPC Endpoints for S3 to replace NAT GW data charges. Improve visibility: enable VPC Flow Logs with 1-minute aggregation, query with Athena. Tune IPS rules: reduce false positives by moving from DROP to ALERT, analyze flow logs, re-enable DROP.

Domain 4 — Migration and Modernization: Migrate on-premises Palo Alto to GWLB + Palo Alto VM-Series. Migrate on-premises network segmentation (VLANs) to VPC subnets + NACLs + Security Groups. Migrate on-premises DMZ to AWS: Internet-facing ALB in public subnet → Network Firewall inspection → Application EC2 in private subnet. Replace hardware firewall with GWLB + Fortinet: use Panorama/FortiManager for policy migration.

🔗 27.15 Related Services Summary

Service	Relationship to Network Security
AWS WAF	Layer 7 HTTP/HTTPS protection for web apps. Complements Network Firewall (WAF for web traffic, NF for all protocols).
AWS Shield	DDoS protection. Shield Advanced integrates with WAF and CloudFront for L7 DDoS.
AWS GuardDuty	Threat detection using ML on VPC Flow Logs, CloudTrail, DNS. No inline blocking — generates findings.
AWS Security Hub	Aggregates findings from GuardDuty, Network Firewall, Macie. Single pane across accounts.
AWS Firewall Manager	Centrally deploy and manage Network Firewall, WAF, SG policies across Organizations accounts.
AWS Config	Detects non-compliant security group rules, NACL changes, route table modifications.
Amazon Inspector	EC2/container vulnerability scanning. Complements network controls with host-level visibility.
AWS CloudTrail	Logs all API calls: firewall rule changes, SG modifications, route table updates — full audit trail.
AWS Transit Gateway	Hub for centralized inspection — routes spoke VPC traffic to inspection VPC.
Amazon VPC	Foundation — all network security controls operate within or at VPC boundaries.
Route 53 Resolver DNS Firewall	Blocks DNS queries to known malicious domains. Layer before Network Firewall (DNS-level filtering).

End of Chapter 27 — Network Security

📊 Chapter 28: Additional Services

> **Covers:** Amazon Athena (Hive, Partitioning, Federated Query) · AWS Federation (SAML, OIDC, OAuth, Okta, Azure AD, Google, Ping, Auth0) · AWS KMS (CMK, AWS Managed, AWS Owned Keys)

📋 Table of Contents (Chapter 28)

- 28.1 Amazon Athena — Hive Metastore, Partitioning & Hive-Compatible Options
- 28.2 Federation in AWS — Complete Guide
- 28.3 AWS Key Management Service (KMS)
- 28.4 Aurora Serverless
- 28.5 Amazon EKS (Elastic Kubernetes Service)
- 28.6 Amazon Timestream
- 28.7 ALB Path-Based Routing
- 28.8 EC2 Purchasing Options and Reserved Instances
- 28.9 EC2 Placement Groups
- 28.10 Serverless on AWS — Lambda, App Runner, Step Functions
- 28.11 Amazon API Gateway
- 28.12 Well-Architected Framework — Chapter 28 Extended
- 28.13 SAP-C02 Domains — Chapter 28 Extended
- 28.14 Amazon ECS (Elastic Container Service)
- 28.15 AWS Fargate
- 28.18 AWS Outposts
- 28.19 AWS Local Zones

- 28.20 AWS Wavelength
- 28.21 Edge Infrastructure — Master Comparison

🦉 28.1 Amazon Athena — Hive Metastore, Partitioning & Hive-Compatible Options

📖 28.1.1 Overview

Amazon Athena is a **serverless, interactive query service** that allows you to analyze data directly in Amazon S3 using standard SQL. No infrastructure to manage, no clusters to size — you pay only for data scanned.

Engine: Based on **Presto** (open-source distributed SQL query engine), extended with Apache Iceberg, Hudi, and Delta Lake support.

Scope: Regional. Queries run in the AWS region where the data resides.

Linux/Open-source equivalent: Presto / Trino, Apache Hive, Apache SparkSQL — all query data in HDFS/S3 using similar concepts.

Azure equivalent: Azure Synapse Analytics Serverless SQL Pool — near-identical concept (serverless SQL over Azure Data Lake Storage, pay per TB scanned).

```
flowchart LR
    S3[(S3 Data Lake\ns3://bucket/table/\nyear=2024/month=01/)] --> GC[AWS Glue\nData Catalog\nHive Metastore]
    GC --> ATH[Amazon Athena\nPresto Engine]
    ATH --> RS[(S3 Results Bucket\ns3://athena-results/)]
    ATH --> CW[CloudWatch Metrics\nQuery stats]
    USER[Analyst / BI Tool\nQuickSight / JDBC] --> ATH
```

> 🔑 **Exam key:** Athena = serverless SQL on S3. No ETL needed to query raw data. Pay per TB scanned. Glue Data Catalog = schema store (Hive Metastore). Partition = cost reduction mechanism.

🗂️ 28.1.2 Hive Metastore and AWS Glue Data Catalog

What is the Hive Metastore? The Hive Metastore is a **central repository of metadata** — it stores table definitions (schema), partition information, and S3 locations for data files. Athena needs it to know: "What columns does this table have? What data type? Where is the data stored in S3?"

AWS Glue Data Catalog is AWS's fully managed, serverless implementation of the Hive Metastore. Athena uses it by default.

Component	Role
Database	Namespace grouping tables (like a schema in RDBMS)
Table	Schema definition + S3 location + SerDe + partition keys
Partition	Sub-directory of a table with a specific partition key value
Column	Name, data type, comment
SerDe	Serializer/Deserializer — how to read/write file format

External Hive Metastore (non-Glue): When you have an existing EMR cluster or on-premises Hive environment, you can configure Athena to use an external Hive Metastore via Lambda connector. Use cases:

- Existing EMR Hive workloads with large existing metadata
- On-premises Hive Metastore migration (query existing Hive tables from Athena without re-cataloging)
- Multi-environment consistency (same metastore for Hive on EMR and Athena)

> ⚠️ **External metastore limitation:** Requires Athena Hive Metastore connector Lambda function. More operational overhead than Glue Data Catalog. Use Glue unless you have specific compatibility requirements.

🗂️ 28.1.3 Partitioning

Partitioning divides a table's data into subdirectories based on column values. When you query with a `WHERE` clause on the partition key, Athena skips entire directories — scanning only relevant data.

Without partitioning:

```
-- Scans ALL data in s3://logs/ (e.g., 10 TB)
SELECT * FROM logs WHERE year = 2024 AND month = 1;
-- Cost: $50 (10TB × $5/TB)
```

With partitioning:

```
-- Scans ONLY s3://logs/year=2024/month=01/ (e.g., 50 GB)
SELECT * FROM logs WHERE year = 2024 AND month = 1;
-- Cost: $0.25 (50GB × $5/TB)
```

Hive-compatible S3 key prefix format:

```
s3://my-data-lake/
  logs/
    year=2024/
      month=01/
        day=15/
          part-00001.parquet
          part-00002.parquet
        day=16/
          part-00001.parquet
      month=02/
        day=01/
          part-00001.parquet
```

Glue Crawler automatically detects this `key=value` partition format and registers partitions in the catalog.

DDL to create partitioned table:

```
CREATE EXTERNAL TABLE logs (
  request_id STRING,
  user_id    STRING,
  action     STRING,
  bytes      BIGINT
)
PARTITIONED BY (year STRING, month STRING, day STRING)
STORED AS PARQUET
LOCATION 's3://my-data-lake/logs/';
```

Adding partitions — 3 methods:

Method	Command	Best For
Manual	`ALTER TABLE logs ADD PARTITION (year='2024', month='01', day='15') LOCATION 's3://...'`	One-time adds
MSCK REPAIR TABLE	`MSCK REPAIR TABLE logs;`	Batch discovery, scans all S3 prefixes
Partition Projection	Table properties (no ALTER needed)	High-cardinality, automated daily/hourly data

> ⚠️ **MSCK REPAIR TABLE problem:** Scans the entire S3 prefix for new partitions. For tables with years of daily data: this can take minutes/hours and consumes Athena resources. Partition Projection eliminates this.

🚀 28.1.4 Partition Projection

Partition Projection tells Athena how to **compute partition locations mathematically** without querying the metastore. Athena derives partition paths from rules instead of looking them up.

Table properties for date-range projection:

```
CREATE EXTERNAL TABLE logs (...)
PARTITIONED BY (year STRING, month STRING, day STRING)
STORED AS PARQUET
LOCATION 's3://my-data-lake/logs/'
TBLPROPERTIES (
  'projection.enabled'       = 'true',
  'projection.year.type'     = 'integer',
  'projection.year.range'    = '2020,2030',
  'projection.month.type'    = 'integer',
  'projection.month.range'   = '1,12',
  'projection.month.digits'  = '2',
  'projection.day.type'      = 'integer',
  'projection.day.range'     = '1,31',
  'projection.day.digits'    = '2',
  'storage.location.template'= 's3://my-data-lake/logs/year=${year}/month=${month}/day=${day}/'
);
```

Result: No `MSCK REPAIR TABLE` ever needed. New partitions (new days of data) are automatically queryable the moment data lands in S3.

Projection types:

Type	Description	Example
`integer`	Numeric range	year 2020-2030
`enum`	Fixed list of values	region: us-east-1, eu-west-1
`date`	Date range with format	2020-01-01 to NOW
`injected`	Value injected at query time	Custom partition keys

> 🔑 **Exam key:** Partition Projection = no MSCK REPAIR TABLE, no metastore partition records, instant new partition availability. Use for time-series data (logs, events) with predictable partition structure.

📁 28.1.5 File Formats

Format	Columnar	Splittable	Compression	Best For	Athena Cost
Parquet	✅ Yes	✅ Yes	Snappy/GZIP/ZSTD	Analytics, large scans	Lowest (skip columns)
ORC	✅ Yes	✅ Yes	Snappy/ZLIB	Hive-heavy workloads	Very low
Avro	❌ Row	✅ Yes	Snappy/Deflate	Schema evolution, streaming	Medium
JSON	❌ Row	❌ No (gzipped)	GZIP	Raw logs, debugging	High
CSV	❌ Row	✅ Yes (uncompressed)	GZIP	Simple export/import	High

> 🔑 **Exam tip:** Always convert to **Parquet** or **ORC** for Athena cost optimization. JSON/CSV = scan entire file. Parquet = scan only needed columns. Typical savings: 85-99% cost reduction vs raw JSON.

Compression Codecs:

Codec	Ratio	Speed	Splittable	Use
Snappy	Medium	Very fast	❌ No (within Parquet blocks: yes)	Default for Parquet/ORC
GZIP	High	Slow	❌ No	Archive, infrequent access
ZSTD	High	Fast	❌ No	Best balance (Parquet v2)
LZO	Low	Very fast	✅ Yes (with index)	Hadoop streaming
BZIP2	Very high	Very slow	✅ Yes	Maximum compression

> ⚠️ **Splittable matters for CSV/JSON:** If a large CSV file is GZIP compressed, Athena cannot split it across multiple workers — single-threaded query. Use uncompressed or LZO (with index) for large CSV, or better: convert to Parquet.

🪛 28.1.6 SerDe (Serializer/Deserializer)

SerDe is the component that tells Athena how to **parse file contents into rows and columns**.

SerDe	Format	Use
`LazySimpleSerDe`	CSV, TSV	Default for delimited text
`OpenCSVSerde`	CSV with quoted fields	CSV with commas in values
`JsonSerDe` (Hive)	JSON (one object per line)	Application logs, API responses
`Amazon Ion Hive SerDe`	Ion/JSON	Complex nested JSON
`RegexSerDe`	Any text with regex	Custom log formats (Apache, Nginx)
`OrcSerDe`	ORC	ORC columnar files
`ParquetHiveSerDe`	Parquet	Parquet columnar files
`AvroSerDe`	Avro	Avro with schema registry

Nginx log example with RegexSerDe:

```
CREATE EXTERNAL TABLE nginx_logs (
  ip         STRING,
  datetime   STRING,
  method     STRING,
  uri        STRING,
  status     INT,
  bytes      INT
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.RegexSerDe'
WITH SERDEPROPERTIES (
  'input.regex' = '(\\S+) \\S+ \\S+ \\[([^\\]]+)\\] "(\\S+) (\\S+) \\S+" (\\d+) (\\d+)'
)
LOCATION 's3://my-bucket/nginx-logs/';
```

🔗 28.1.7 Athena Federated Query

Athena Federated Query uses **Lambda-based data source connectors** to query data sources beyond S3 — without moving data.

Available connectors:

- Amazon RDS (MySQL, PostgreSQL, SQL Server, Oracle)
- Amazon DynamoDB
- Amazon Redshift
- Amazon DocumentDB
- Amazon OpenSearch

- On-premises JDBC (via Lambda in VPC)
- HBase on EMR
- Custom (build your own Lambda connector)

Architecture:

```
Athena SQL Query
    → Glue Data Catalog (finds connector definition)
    → Lambda connector invoked per data source
    → Lambda queries RDS/DynamoDB/etc.
    → Results returned to Athena
    → Athena joins/aggregates results
    → Results in S3
```

> 🔑 **Exam tip:** Federated Query = query RDS + S3 in a single SQL statement. JOIN operational database with data lake without ETL. Lambda connector runs in your VPC (for private RDS access).

💰 28.1.8 Cost Optimization

Technique	Impact	How
Parquet/ORC format	85-99% cost reduction	Convert with Glue ETL or CTAS
Partition pruning	90-99% cost reduction	Add WHERE clause on partition keys
Partition Projection	Operational (no MSCK cost)	Table property configuration
Column pruning	20-60%	SELECT only needed columns (not SELECT *)
Compression	30-80% storage + scan	Snappy (speed) or ZSTD (ratio)
Workgroup data limit	Cost governance	Set per-query scan limit in workgroup
CTAS	One-time conversion	`CREATE TABLE new AS SELECT ... FROM old`

CTAS example — convert JSON to partitioned Parquet:

```
CREATE TABLE logs_parquet
WITH (
  format           = 'PARQUET',
  parquet_compression = 'SNAPPY',
  partitioned_by   = ARRAY['year', 'month'],
  external_location = 's3://my-data-lake/logs_parquet/'
)
AS SELECT
  request_id, user_id, action, bytes,
  year(from_iso8601_timestamp(event_time)) AS year,
  month(from_iso8601_timestamp(event_time)) AS month
FROM logs_json;
```

⚙️ 28.1.9 Workgroups

Workgroups isolate query execution and enforce cost/security controls per team or application.

Feature	Description
Separate result locations	Each workgroup writes results to its own S3 prefix
Data scanned limit	Per-query limit (e.g., max 10GB scanned per query) → query cancelled if exceeded
Metrics per workgroup	CloudWatch metrics scoped to workgroup
Query history isolation	Users see only their workgroup queries
Encryption	SSE-S3 or SSE-KMS per workgroup result bucket
Cost allocation tags	Tag workgroup for cost attribution

> 🔑 **Exam tip:** "Limit how much data analysts can scan" → Workgroup with per-query scan limit. Prevents accidental full-table scans by junior analysts.

⚖️ 28.1.10 Athena Comparison Table

Feature	Athena	Redshift Spectrum	EMR (Hive/Spark)	Redshift
Infrastructure	Serverless	Redshift cluster required	Cluster (EC2)	Cluster
Query engine	Presto	Redshift MPP	Hive / Spark / Presto	Redshift MPP
Data location	S3	S3 (via external tables)	HDFS / S3	Redshift storage + S3
Cost model	Per TB scanned	Spectrum: per TB + cluster	Per EC2 hour	Per node hour
Startup time	Instant	Cluster must be running	5-15 min cluster start	Cluster always on
Complex transforms	Limited (SQL only)	Limited	Full (Python/Scala/R)	SQL + stored procs
ML integration	No	Redshift ML	SageMaker integration	Redshift ML
Best for	Ad-hoc S3 queries	Combined S3 + Redshift data	Complex ETL, ML pipelines	BI/DW with structured data

> 🔑 **Exam tip:** "Analyze S3 data without loading into a database" → Athena. "Query S3 data alongside Redshift tables" → Redshift Spectrum. "Complex data transformation pipelines" → EMR. "High-performance BI warehouse" → Redshift.

⚠️ 28.1.11 Athena Limitations

Limitation	Value
Concurrent DML queries	20 (default, can request increase)
Max query output size	5GB (S3 result) — use CTAS for larger
Max query execution time	30 minutes
No UPDATE/DELETE	Standard Athena SQL; Apache Iceberg tables support MERGE/DELETE
Result retention	45 days in Athena history (results in S3 forever)
Cross-region queries	Not supported directly; use S3 replication + Athena in target region
Views	Stored in Glue, cannot reference external metastore tables

---

🔐 28.2 Federation in AWS — Complete Guide

📖 28.2.1 What is Federation?

Federation allows external identities (users in Okta, Azure AD, Google) to access AWS resources **without creating IAM users**. The external Identity Provider (IdP) authenticates the user; AWS trusts the IdP assertion and issues temporary credentials via STS.

Why federation matters:

- No long-term static credentials (no IAM users with passwords/access keys)
- Central identity management in existing IdP
- Scales to thousands of users without AWS IAM user proliferation
- Temporary credentials (auto-expire) → reduced credential theft risk

Protocol landscape:

Protocol	Type	Format	Primary Use
SAML 2.0	AuthN + AuthZ	XML assertions	Enterprise SSO (workforce)
OIDC	AuthN (built on OAuth 2.0)	JWT tokens	Modern apps, CI/CD, mobile
OAuth 2.0	AuthZ only	Access tokens	API authorization

🔒 28.2.2 SAML 2.0

SAML (Security Assertion Markup Language) 2.0 is an **XML-based open standard** for exchanging authentication and authorization data between an IdP and a Service Provider (SP).

SAML flow for AWS console access:

```
sequenceDiagram
    participant U as User (Browser)
    participant IdP as Identity Provider (Okta/AD FS)
    participant AWS as AWS Sign-In Endpoint
    participant STS as AWS STS
    participant CON as AWS Console

    U->>IdP: Login with corporate credentials
    IdP->>IdP: Authenticate user, check group membership
    IdP->>U: Returns SAML Assertion (XML, signed)
    U->>AWS: POST SAML Assertion to https://signin.aws.amazon.com/saml
    AWS->>STS: AssumeRoleWithSAML (assertion + role ARN)
    STS->>AWS: Temporary credentials (15min-12hr)
    AWS->>CON: Redirect to AWS Console with session
    CON->>U: AWS Console access
```

AWS-specific SAML attributes in assertion:

```
<Attribute Name="https://aws.amazon.com/SAML/Attributes/Role">
  <AttributeValue>
    arn:aws:iam::123456789012:role/DevRole,
    arn:aws:iam::123456789012:saml-provider/OktaIdP
  </AttributeValue>
</Attribute>
<Attribute Name="https://aws.amazon.com/SAML/Attributes/RoleSessionName">
  <AttributeValue>john.doe@company.com</AttributeValue>
</Attribute>
<Attribute Name="https://aws.amazon.com/SAML/Attributes/SessionDuration">
  <AttributeValue>28800</AttributeValue>
</Attribute>
```

> 🔑 **Exam key:** SAML Role attribute = `role ARN, IdP ARN` (comma-separated pair). Both must be present. IdP ARN is the SAML provider resource in IAM. Multiple Role attributes = user can choose which role to assume.

SAML trust setup in IAM:

1. Create SAML Identity Provider in IAM (upload IdP metadata XML)
2. Create IAM role with trust policy referencing the SAML provider:

```
{
  "Principal": {"Federated": "arn:aws:iam::123456789:saml-provider/OktaIdP"},
  "Action": "sts:AssumeRoleWithSAML",
  "Condition": {
    "StringEquals": {"SAML:aud": "https://signin.aws.amazon.com/saml"}
  }
}
```

🔑 28.2.3 OIDC (OpenID Connect)

OIDC is an **identity layer on top of OAuth 2.0** — it adds authentication (who you are) to OAuth's authorization (what you can do). Uses **JWT (JSON Web Tokens)** instead of XML.

Token types:

Token	Purpose	Contains
ID Token	Authentication — who you are	User identity claims (sub, email, name)
Access Token	Authorization — what you can do	Scopes, expiry, audience
Refresh Token	Get new access tokens	Long-lived, stored securely

GitHub Actions → OIDC → AWS (no static credentials):

```
sequenceDiagram
    participant GH as GitHub Actions
    participant GHOIDC as GitHub OIDC Provider
    participant STS as AWS STS
    participant S3 as AWS S3

    GH->>GHOIDC: Request OIDC token for this workflow run
    GHOIDC->>GH: JWT token (contains repo, branch, workflow claims)
    GH->>STS: AssumeRoleWithWebIdentity(jwt, role_arn)
    STS->>STS: Verify JWT signature against GitHub OIDC endpoint
    STS->>GH: Temporary credentials (1hr)
    GH->>S3: Deploy artifacts using temp credentials
```

OIDC provider registration in IAM:

```
aws iam create-open-id-connect-provider \
  --url https://token.actions.githubusercontent.com \
  --thumbprint-list 6938fd4d98bab03faadb97b34396831e3780aea1 \
  --client-id-list sts.amazonaws.com
```

IAM role trust policy for GitHub Actions:

```
{
  "Principal": {"Federated": "arn:aws:iam::123:oidc-provider/token.actions.githubusercontent.com"},
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
      "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
    },
    "StringLike": {
      "token.actions.githubusercontent.com:sub": "repo:myorg/myrepo:*"
    }
  }
}
```

> 🔑 **Exam key:** OIDC = no static AWS credentials in CI/CD. GitHub Actions, GitLab CI, Jenkins (with OIDC plugin) all support this pattern. Much more secure than storing `AWS_ACCESS_KEY_ID` in secrets.

Kubernetes IRSA (IAM Roles for Service Accounts): EKS uses OIDC — each EKS cluster has an OIDC issuer URL. Pod service accounts can be annotated to assume an IAM role:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-app-sa
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123:role/MyAppRole
```

Pod assumes the role → gets temporary credentials via IRSA → accesses S3/DynamoDB without node-level IAM role.

🔄 28.2.4 OAuth 2.0

OAuth 2.0 is an **authorization framework** — it allows an application to access resources on behalf of a user **without the user sharing credentials**. It does NOT provide authentication (who you are) — that is OIDC's job.

OAuth grant types:

Grant Type	Use Case	Flow
Authorization Code	Web apps (server-side)	User → IdP → code → exchange for token
Authorization Code + PKCE	Mobile/SPA apps	Same + Proof Key for Code Exchange
Client Credentials	Machine-to-machine (no user)	App ID + secret → token
Device Code	CLI tools, smart TVs	Device polls for token after user authorizes on phone
Implicit	Deprecated (insecure)	Token in URL fragment — avoid

OAuth vs OIDC:

- OAuth: "App A can access Resource B on behalf of User C" — authorization only
- OIDC: OAuth + "and here is who User C is (ID token)" — authentication + authorization

AWS usage of OAuth:

- Cognito User Pools: issues OAuth tokens (JWT) for web/mobile app users
- API Gateway: validates OAuth access tokens via Cognito authorizer or Lambda authorizer
- ALB: OAuth2/OIDC authentication built into listener rules (no app code needed)

🏢 28.2.5 Federation with Specific Identity Providers

Okta

Integration Type	Method	Use Case
Okta → IAM Identity Center	SAML 2.0 + SCIM	Workforce, multi-account (recommended)
Okta → STS direct	SAML 2.0 (AssumeRoleWithSAML)	Single-account, legacy
Okta → Cognito Identity Pools	OIDC	Customer-facing apps with Okta login

Okta + IAM Identity Center setup:

1. Add "AWS IAM Identity Center" app in Okta marketplace
2. Configure SAML: Okta is IdP, Identity Center is SP (ACS URL from Identity Center)
3. Configure SCIM: generate Identity Center SCIM token, paste into Okta provisioning
4. Create Okta groups → map to Identity Center groups → assign permission sets
5. SCIM syncs users/groups automatically; SAML handles authentication

> 🔑 **Exam key:** Okta is the most common enterprise IdP for AWS. The modern pattern is always Okta → IAM Identity Center (not Okta → direct IAM SAML). Direct SAML is single-account; Identity Center handles multi-account via Organizations.

Azure AD / Microsoft Entra ID

Integration	Method	Use Case
Azure AD → Identity Center	SAML 2.0 + SCIM	Microsoft-centric enterprise, multi-account
Azure AD → IAM (direct)	SAML 2.0	Single-account
Azure AD → OIDC	OIDC provider in IAM	App workloads, CI/CD
Azure AD B2C → Cognito	OIDC	Customer apps with Microsoft login

Azure AD differences vs Okta:

Aspect	Azure AD / Entra ID	Okta
Primary strength	Microsoft ecosystem (M365, Windows, Teams)	IdP-agnostic, all platforms
On-prem AD sync	Native (Azure AD Connect)	Requires Okta AD agent
Customer identity	Azure AD B2C (separate product)	Okta Customer Identity Cloud (Auth0)
Licensing	Included with M365/Azure	Separate licensing
MFA	Azure MFA (Microsoft Authenticator)	Okta Verify, FIDO2
AWS native feel	Requires SAML app config in Azure portal	AWS app in Okta marketplace (easier)

Google Workspace

Integration	Method
Google Workspace → Identity Center	SAML 2.0 from Google Admin Console
Google as OIDC provider for IAM	Add Google OIDC provider in IAM IAM
GCP Workload Identity → AWS	OIDC (cross-cloud workload federation)

Cross-cloud pattern (GCP → AWS):

```
GCP workload → Google OIDC token (Workload Identity) → AWS STS AssumeRoleWithWebIdentity → AWS resources
```

No static AWS credentials stored in GCP. IAM role trust policy trusts Google OIDC provider with condition matching GCP service account email.

Ping Identity (PingFederate / PingOne)

- Enterprise IdP common in **financial services, healthcare, government**

- PingFederate: on-premises deployment (self-managed)
- PingOne: SaaS cloud deployment
- Both support SAML 2.0 and OIDC
- Integration with AWS Identity Center: same SAML + SCIM flow as Okta/Azure AD
- Differentiator: PingFederate has extensive on-premises integration capabilities (legacy systems, mainframes)

Auth0 (Okta Customer Identity Cloud)

- Cloud-native IdP, **OIDC-first** (vs SAML-first enterprise IdPs)
- Target: **customer identity** (millions of end users), not workforce identity
- Acquired by Okta in 2021

Integration	Use Case
Auth0 → Cognito Identity Pools	Customer app needs AWS resource access (S3 upload, DynamoDB read)
Auth0 → AssumeRoleWithWebIdentity	Direct AWS access via OIDC (no Cognito)
Auth0 → API Gateway (JWT)	Protect API Gateway with Auth0-issued JWTs

> 🔑 **Exam distinction:** Auth0 / Cognito User Pools = customer-facing app identity (millions of users, social login). Okta / Azure AD / Ping = workforce identity (employees accessing AWS accounts). Do not confuse.

🔀 28.2.6 Three Federation Patterns — Full Comparison

Pattern 1: IAM Identity Center Federation (Workforce, Multi-Account)

```
flowchart LR
    U[Employee] --> IDP[Okta / Azure AD\nSAML + SCIM]
    IDP --> IC[IAM Identity Center]
    IC --> P1[AWS Account 1\nAWSReservedSSO_Dev role]
    IC --> P2[AWS Account 2\nAWSReservedSSO_Prod role]
    IC --> P3[AWS Account N\nAWSReservedSSO_ReadOnly role]
    U --> CLI[aws sso login\nCLI credentials]
    CLI --> IC
```

When to use: Employees need access to multiple AWS accounts. Single SSO experience. Centralized permission management.

Pattern 2: IAM + STS Direct Federation (Apps, CI/CD, Single-Account)

```
flowchart LR
    APP[Application /\nCI-CD Pipeline\nGitHub Actions] --> IDP[OIDC Provider\nor SAML IdP]
    IDP --> STS[AWS STS\nAssumeRoleWithWebIdentity\nor AssumeRoleWithSAML]
    STS --> CRED[Temporary Credentials\n1hr - 12hr]
    CRED --> AWS[AWS Resources\nS3 / ECR / ECS / etc.]
```

When to use: CI/CD pipelines, applications needing AWS access, single-account workforce with existing SAML IdP.

Pattern 3: Cognito Identity Pools (Customer Identity, App Users)

```
flowchart LR
    CU[Customer\nApp User] --> CUP[Cognito User Pool\nor External IdP\nGoogle / Facebook / Auth0]
    CUP --> CIP[Cognito Identity Pool]
    CIP --> STS[AWS STS]
    STS --> ROLE[IAM Role\nper-user or\nper-group]
    ROLE --> AWS[AWS Resources\nS3 bucket upload\nDynamoDB user data]
```

When to use: Mobile/web app users need direct AWS resource access. Guest (unauthenticated) access needed. Millions of end users.

Full Comparison Table

Aspect	IAM Identity Center	IAM + STS Direct	Cognito Identity Pools
Primary use	Workforce multi-account	Apps/CI-CD/single-account	Customer-facing apps
Scale	Thousands of employees	Any	Millions of end users
Multi-account	Yes (native Organizations)	Manual role chaining	No (single account)
IdP protocols	SAML 2.0 + SCIM	SAML 2.0, OIDC	SAML, OIDC, Social (Google/Facebook/Apple)
Guest/unauth access	No	No	Yes
Credential delivery	SSO portal / `aws sso login`	Direct STS API response	Cognito SDK / AWS SDK
Management	Centralized (Org-wide)	Per-account IAM	Per Identity Pool
User directory	Built-in or external IdP	External IdP only	Cognito User Pool or external IdP
Attribute mapping	Permission set → IAM role	SAML attributes / OIDC claims	IAM role per pool or per user
Cost	Free	Free (STS API)	Cognito MAU pricing

🔑 28.2.7 AWS STS — Key APIs

API	Description	Use Case
AssumeRole	Assume a role in same or different account	Cross-account access, EC2 to Lambda role assumption
AssumeRoleWithSAML	Assume role using SAML 2.0 assertion	Enterprise SSO (Okta, AD FS, Azure AD)
AssumeRoleWithWebIdentity	Assume role using OIDC/OAuth token	GitHub Actions, mobile apps, Cognito Identity Pools
GetFederationToken	Get temporary credentials for federated user	Legacy — not recommended (user inherits caller permissions)
GetSessionToken	Get temp credentials for IAM user with MFA	MFA enforcement for sensitive operations
DecodeAuthorizationMessage	Decode encoded authorization failure message	Debugging access denied errors

Credential duration:

- AssumeRole: 15 minutes to 12 hours (default 1 hour)
- AssumeRoleWithSAML: 15 minutes to 12 hours
- AssumeRoleWithWebIdentity: 15 minutes to 12 hours
- GetFederationToken: 15 minutes to 36 hours

STS endpoint:

- Global: `sts.amazonaws.com` (always us-east-1 internally)
- Regional: `sts.eu-west-1.amazonaws.com` — recommended for latency + resiliency
- Enable regional STS endpoints: `aws iam set-security-token-service-preferences --global-endpoint-token-version v2Token`

> ⚠️ **Exam trap:** `GetFederationToken` is NOT deprecated but is discouraged. It creates credentials with the intersection of caller permissions and the specified policy. Requires the caller to have a long-term identity (IAM user). AssumeRoleWithWebIdentity is preferred for federated web identities.

🔑 28.3 AWS Key Management Service (KMS)

📖 28.3.1 Overview

AWS KMS is a **managed service for creating and controlling cryptographic keys** used to encrypt data across AWS services and custom applications. Backed by **FIPS 140-2 Level 3** validated Hardware Security Modules (HSMs).

Scope: Regional. Keys are regional resources — a key in eu-west-1 cannot be directly used in us-east-1. Use Multi-Region Keys for cross-region needs.

Linux/Open-source equivalent: HashiCorp Vault (key management + secrets), OpenSSL + PKCS#11 HSM, GnuPG for envelope encryption.

Azure equivalent: Azure Key Vault (keys, secrets, certificates). Functionally very similar — managed HSM-backed key management, envelope encryption pattern, RBAC-based access control.

> 🔑 **Exam key:** KMS does not store encrypted data — it only manages **keys**. Your application or AWS service uses KMS to encrypt/decrypt data. KMS is the control plane; your S3/EBS/RDS is the data plane.

🔑 28.3.2 Key Types — Full Comparison

AWS Owned Keys

AWS creates and manages these keys entirely within AWS infrastructure. The customer has **zero visibility and zero control**.

Characteristic	Detail
Owner	AWS (not customer)
Visibility	Not visible in customer KMS console
Control	None — customer cannot modify, disable, or audit
CloudTrail	No key usage events visible to customer
Cost	Free
Examples	S3 SSE-S3 (AES-256), DynamoDB default encryption, SQS encryption
Cross-account	Not possible
Rotation	AWS manages, customer cannot configure

> ⚠️ **Use case:** AWS Owned Keys are for general data-at-rest encryption where customer key control is not required. Suitable for non-sensitive workloads or where compliance only requires encryption (not customer key control).

AWS Managed Keys

AWS creates and manages these keys **in the customer's account**, but the customer has limited control.

Characteristic	Detail
Owner	AWS (managed on customer behalf)
Visibility	Visible in KMS console (`aws/` prefix)
Control	Cannot modify key policy, cannot disable
CloudTrail	Key usage events ARE logged to CloudTrail
Cost	Free (no per-key charge)
Name pattern	`aws/s3` , `aws/ebs` , `aws/rds` , `aws/secretsmanager`
Rotation	Automatic, every 365 days (cannot change)
Cross-account	Not supported
Scope	Only usable by the specific service that created it

Customer Managed Keys (CMK)

Customer creates and fully controls these keys.

Characteristic	Detail
Owner	Customer
Visibility	Full visibility in KMS console
Control	Full — key policy, grants, rotation, aliases, disable, delete
CloudTrail	All key operations logged
Cost	$1/month/key$+0.03/10,000 API calls
Rotation	Optional automatic (annual default) or custom period (90-2560 days)
Cross-account	Supported via key policy
Key material	AWS-generated OR imported (BYOK)
Multi-region	Supported (primary + replicas)

Three-Way Comparison Table

Feature	AWS Owned	AWS Managed	Customer Managed
Visibility in console	No	Yes (`aws/*`)	Yes
Key policy control	No	No	Yes
Disable/Delete	No	No	Yes
CloudTrail events	No	Yes (usage)	Yes (all)
Cross-account sharing	No	No	Yes
Custom rotation period	No	No (annual only)	Yes
BYOK (import material)	No	No	Yes
Multi-region	No	No	Yes
Cost	Free	Free	$1/month + API cost
Audit by customer	No	Limited	Full

```
flowchart TD
    KT[KMS Key Types] --> AO[AWS Owned Keys\nFree, no visibility\nSSE-S3, DynamoDB default]
    KT --> AM[AWS Managed Keys\nFree, read-only visibility\naws/s3, aws/ebs\nAuto-rotate yearly]
    KT --> CM[Customer Managed Keys\n$1/month, full control\nCustom policy, BYOK\nCross-account sharing]
    CM --> CMG[AWS-generated\nkey material]
    CM --> BYOK[Imported key material\nBYOK - customer HSM]
    CM --> MR[Multi-Region Keys\nSame key ID across regions]
```

External Key Material (BYOK)

Import your own key material generated outside AWS (on-premises HSM):

1. Create a CMK with no key material (origin: EXTERNAL)
2. Download AWS public key + import token from KMS
3. Encrypt your 256-bit key material with the AWS public key (RSA-OAEP)
4. Upload encrypted key material + import token to KMS
5. KMS decrypts with its private key, stores your key material in HSM

BYOK limitations:

- Rotation is manual (delete + re-import new key material)
- If you delete the key material: data encrypted with it is permanently unrecoverable
- Key material expiration: optional, set an expiry date for auto-deletion

> 🔑 **Exam trap:** BYOK does NOT mean you can take the key out of AWS. Once imported, the key material is protected by AWS HSMs. You cannot export it. BYOK = you generated it, you control the source. For true HSM custody: use CloudHSM.

CloudHSM-Backed Keys

- Customer provisions and manages CloudHSM cluster (EC2-based HSMs)
- Keys stored exclusively in customer-controlled HSM
- AWS never has access to key material
- FIPS 140-2 Level 3 with customer-exclusive HSM custody
- Use KMS custom key store pointing to CloudHSM cluster
- ~$1.45/hour per HSM instance(1,050/month per HSM)
- Minimum 2 HSMs for HA

> 🔑 **Exam key:** "Keys must never be accessible to AWS" → CloudHSM. "FIPS 140-2 Level 3 with customer-exclusive custody" → CloudHSM. Standard KMS: AWS HSMs, but AWS manages them. CloudHSM: customer-managed HSM hardware in AWS data center.

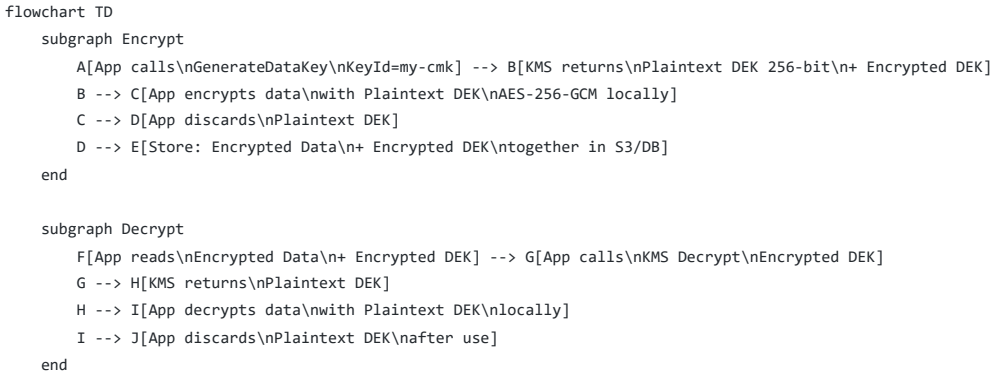
🔒 28.3.3 KMS Encryption Operations

API	Input	Output	Limit
Encrypt	Plaintext (≤4KB) + KeyId	Ciphertext blob	4KB plaintext max
Decrypt	Ciphertext blob	Plaintext	Ciphertext must have been encrypted by KMS
GenerateDataKey	KeyId + key spec	Plaintext DEK + Encrypted DEK	Use for envelope encryption
GenerateDataKeyWithoutPlaintext	KeyId + key spec	Encrypted DEK only	Deferred decrypt (generate key, encrypt data later)
ReEncrypt	Ciphertext + new KeyId	New ciphertext	Never exposes plaintext — atomic decrypt+encrypt
GenerateRandom	NumberOfBytes	Random bytes	Cryptographically secure random from HSM

> 🔑 **Exam key:** `ReEncrypt` is used when rotating to a new CMK without ever having the plaintext in your application. Source ciphertext → KMS (decrypt with old key) → KMS (encrypt with new key) → new ciphertext. Plaintext never leaves the HSM.

🧅 28.3.4 Envelope Encryption

KMS enforces envelope encryption because the Encrypt API is limited to 4KB. Your application data (MB, GB, TB) is encrypted locally with a Data Encryption Key (DEK); only the DEK is managed by KMS.



Why envelope encryption:

1. **Performance:** KMS API is remote (HTTPS). Encrypting 1GB via KMS API = 262,144 API calls. Encrypting locally with a DEK = 1 KMS call + fast local AES.
2. **Cost:** 1 KMS API call per encryption operation (not per MB)
3. **Scale:** DEK can encrypt unlimited data; KMS manages key hierarchy
4. **Re-keying:** to change CMK, just re-encrypt the DEK (not all data)

S3 SSE-KMS uses envelope encryption internally:

- You upload an object → S3 calls `GenerateDataKey` (your CMK) → gets DEK → encrypts object with DEK → stores encrypted object + encrypted DEK → discards plaintext DEK
- You download → S3 calls `Decrypt` (your CMK) with encrypted DEK → gets plaintext DEK → decrypts object → returns to you

---

📜 28.3.5 Key Policies

Every CMK has exactly one key policy (JSON document, max 32KB). **Key policy is the primary access control mechanism for KMS keys** — IAM policies alone are insufficient.

Default key policy (created via console):

```
{
  "Statement": [
    {
      "Sid": "Enable IAM User Permissions",
      "Effect": "Allow",
      "Principal": {"AWS": "arn:aws:iam::123456789012:root"},
      "Action": "kms:*",
      "Resource": "*"
    },
    {
      "Sid": "Allow key administrators",
      "Effect": "Allow",
      "Principal": {"AWS": "arn:aws:iam::123456789012:role/KeyAdminRole"},
      "Action": ["kms:Create*","kms:Describe*","kms:Enable*","kms:List*",
                 "kms:Put*","kms:Update*","kms:Revoke*","kms:Disable*",
                 "kms:Get*","kms:Delete*","kms:ScheduleKeyDeletion","kms:CancelKeyDeletion"]
    },
    {
      "Sid": "Allow key users",
      "Effect": "Allow",
      "Principal": {"AWS": "arn:aws:iam::123456789012:role/AppRole"},
      "Action": ["kms:Encrypt","kms:Decrypt","kms:ReEncrypt*",
                 "kms:GenerateDataKey*","kms:DescribeKey"]
    }
  ]
}
```

> 🔑 **Critical exam point:** The `root` principal in the key policy enables IAM policies to control key access. **Without the root principal in the key policy, NO IAM identity-based policy can grant access to the key — even the account root user.** This is the most important KMS policy concept.

Cross-account key sharing:

1. Add the other account (or role in other account) to the key policy
2. In the other account, create an IAM policy granting `kms:Decrypt` etc. on the key ARN
3. Both must be configured — key policy + IAM policy in the other account

---

🔄 28.3.6 Key Rotation

Key Type	Rotation	Configurable	Effect on Existing Data
AWS Owned	AWS manages	No	N/A
AWS Managed	Automatic, every 365 days	No	Old data still decryptable
CMK (automatic)	Optional, annual default	Yes (90-2560 days, 2024 feature)	Old data still decryptable
CMK (on-demand)	Immediate rotation	Yes (any time)	Old data still decryptable
CMK BYOK	Manual (re-import)	N/A	Old data still decryptable

> 🔑 **Exam key — rotation misconception:** Key rotation does NOT break existing encrypted data. KMS retains all previous key versions (backing keys) for decryption. New data uses the new key version. Old data still decrypts with the old version. You can always decrypt old ciphertext.

What rotation does:

- Creates a new cryptographic backing key (new AES-256 key material)
- New data encrypted with new backing key

- Old backing key retained for decrypting old ciphertext
- Key ID, ARN, alias — all unchanged (transparent to callers)

What rotation does NOT do:

- Does NOT re-encrypt existing data (you must do this manually if required)
- Does NOT change key ID or ARN
- Does NOT expire old backing keys (they're kept indefinitely for decryption)

🌎 28.3.7 Multi-Region Keys

Multi-Region Keys allow the **same key material to be used in multiple AWS regions** — enabling cross-region decrypt without re-encryption.

Feature	Detail
Key ID	Same across all regions (`mrk-` prefix)
Key material	Replicated, identical in all regions
API calls	Still regional (call us-east-1 KMS for data encrypted there)
Use case	Cross-region active-active, DR failover, globally consistent encryption
Promotion	Replica can be promoted to primary
Independent	Each replica is an independent key resource (can have different policies)

Use case — Global DynamoDB Table:

- DynamoDB Global Table replicates data across us-east-1, eu-west-1
- With single-region CMK: data in eu-west-1 cannot be decrypted (wrong region key)
- With Multi-Region Key: same key material in both regions, decryption works everywhere

---

🎫 28.3.8 KMS Grants

Grants are a **programmatic, time-limited delegation** of key access without modifying the key policy.

```
aws kms create-grant \
  --key-id arn:aws:kms:us-east-1:123:key/key-id \
  --grantee-principal arn:aws:iam::123:role/AppRole \
  --operations Decrypt GenerateDataKey \
  --name "app-grant-2024"
```

Grant use cases:

- AWS services use grants on your CMKs internally (EBS creates a grant when you encrypt a volume)
- Short-lived access for a specific operation
- Service-level delegation without key policy modification

> ⚠️ **Grant vs Key Policy:** Key policy = durable, administrative access control. Grant = programmatic, time-limited, per-operation delegation. Grants can be created and retired by applications without key admin involvement.

🔗 28.3.9 KMS with AWS Services

Service	KMS Integration	Key Type	Notes
S3	SSE-KMS	AWS Managed (`aws/s3`) or CMK	Enable S3 Bucket Key to reduce KMS API calls 99%
EBS	Volume encryption	AWS Managed (`aws/ebs`) or CMK	Set at volume creation; cannot change later
RDS	Storage encryption	AWS Managed (`aws/rds`) or CMK	Set at creation; cannot change; snapshots inherit
Secrets Manager	Secret value encryption	AWS Managed (`aws/secretsmanager`) or CMK	CMK for cross-account secret sharing
Lambda	Environment variable encryption	CMK	Encrypt sensitive env vars at rest
CloudWatch Logs	Log group encryption	CMK	Optional; adds API call cost per log delivery
DynamoDB	Table encryption	AWS Owned (default), AWS Managed, or CMK	CMK enables CloudTrail key usage visibility
SQS	Message encryption	AWS Managed (`aws/sqs`) or CMK	CMK needed for cross-account queue access
SNS	Topic encryption	CMK	Encrypts messages at rest

S3 Bucket Key: When enabled, S3 generates a bucket-level data key (from your CMK), cached for up to 5 minutes. Instead of calling KMS for every object PUT/GET, S3 uses the cached bucket key. Result: 99% fewer KMS API calls = 99% lower KMS cost for S3-heavy workloads.

⚠️ 28.3.10 KMS Limitations

Limitation	Value	Workaround
Encrypt API plaintext limit	4KB	Use envelope encryption (GenerateDataKey)
Key scope	Regional	Multi-Region Keys for cross-region
Key deletion waiting period	7-30 days minimum	Cannot be immediate — by design
Cryptographic operations quota	30,000/sec (varies by operation)	Request quota increase; use data key caching
Key policy size	32KB	Restructure with grants for many principals
Aliases per key	50	Sufficient for most use cases
Keys per region	100,000 (soft)	Request increase

Data Key Caching (AWS Encryption SDK): Instead of calling KMS for every encryption operation, cache the plaintext DEK in memory for N seconds or N uses. Reduces KMS API calls proportionally. Trade-off: cached DEK in memory is a security risk if process is compromised during cache window.

📐 28.4 Well-Architected Framework — Additional Services

Pillar	Athena	Federation	KMS
Operational Excellence	Workgroups for team isolation. Partition Projection eliminates maintenance. CTAS automates format conversion. Athena Named Queries for reusable queries.	SCIM automates user provisioning/deprovisioning. Permission sets as code (Terraform). CloudTrail logs all federation events.	Key aliases for human-readable key management. CloudTrail logs all key operations. Automated rotation. IaC key policies.
Security	Workgroup result encryption (SSE-KMS). Athena access via IAM policies + resource-based policies. Data encrypted at rest in S3 (SSE-KMS). VPC endpoints for Athena (private query).	No static long-term credentials. Temporary STS credentials. IdP MFA enforcement. Session duration limits. Least-privilege permission sets.	CMK for sensitive data. Key policy least privilege (separate admin vs user). Cross-account key sharing with explicit policy. Disable unused keys.
Reliability	Workgroup query retry. Athena is serverless (no cluster to fail). CTAS to create reliable long-term data copies.	STS regional endpoints for federation reliability. Identity Center multi-region failover. IdP HA (Okta/Azure AD are SaaS — HA by design). AD Connector dependency on on-premises AD.	Multi-Region Keys for DR. Automatic key rotation. KMS SLA: 99.999% availability. Key deletion waiting period prevents accidental data loss.
Performance Efficiency	Parquet + Snappy = 10x-100x faster queries. Partition pruning eliminates unnecessary data scan. Columnar format = skip unneeded columns. Parallel query execution across Presto workers.	STS regional endpoints reduce federation latency. Credential caching (15min-12hr) reduces STS calls. OIDC token caching in CI/CD.	S3 Bucket Key reduces KMS API calls 99%. Data Key Caching (Encryption SDK) reduces API calls. GenerateDataKeyWithoutPlaintext for bulk operations.
Cost Optimization	Parquet + partitioning = 85-99% scan reduction. Workgroup data scan limits prevent runaway queries. CTAS converts expensive JSON to cheap Parquet. S3 lifecycle policies archive old query results.	IAM Identity Center: free (no per-user cost). STS: API calls are free. Cognito Identity Pools: MAU-based pricing ($0.00015/MAU above free tier).	AWS Owned/Managed keys: free. CMK: $1/month — delete unused keys. S3 Bucket Key: 99% KMS API cost reduction for S3. Consolidate CMKs where compliance allows.
Sustainability	Parquet compression reduces S3 storage (smaller footprint).	Centralized identity management reduces per-service auth overhead. SCIM automation reduces admin operations.	Single CMK for multiple services (where compliance allows). Automated rotation reduces manual key management effort.

Pillar	Athena	Federation	KMS
	Partition pruning = less compute per query. S3 Intelligent-Tiering for Athena data.		

🎯 28.5 SAP-C02 Domains — Additional Services

Domain 1 — Organizational Complexity: Athena: centralized data lake with organization-wide Glue Data Catalog. Cross-account Athena: Athena in analytics account queries S3 data in data-producer accounts (via cross-account S3 bucket policy + Glue resource policy). Federation: Identity Center for multi-account employee access. SAML federation from corporate IdP (Okta, Azure AD) to 200 accounts via Identity Center. KMS: CMKs shared cross-account for multi-account data encryption. Firewall Manager + KMS for organization-wide encryption compliance.

Domain 2 — New Solutions: Athena: new data lake analytics capability — serverless, no infrastructure, query S3 directly. New compliance requirement for audit logs → S3 + Athena + Glue (no Redshift cluster needed). Federation: new application using GitHub Actions CI/CD → OIDC federation (no static keys). New mobile app → Cognito Identity Pools + Auth0. KMS: new regulated workload → CMK for all data at rest, S3/EBS/RDS CMK-encrypted, CloudTrail key usage logging.

Domain 3 — Continuous Improvement: Athena: existing JSON logs are expensive to query → CTAS to Parquet + partitioning → 95% cost reduction. Add Partition Projection to eliminate MSCK REPAIR overhead. Federation: migrate from IAM users to Identity Center. Migrate from on-premises ADFS to Okta + Identity Center. KMS: migrate from AWS Managed Keys to CMK for compliance. Enable S3 Bucket Key for existing buckets to reduce KMS costs.

Domain 4 — Migration: Athena: migrate on-premises Hive/Impala queries to Athena (near-compatible SQL, same Parquet/ORC files). Use External Hive Metastore connector to query existing Hive metadata during migration. Federation: migrate on-premises AD + ADFS to AWS Managed AD + Identity Center. Migrate static IAM user credentials to OIDC-based federation for CI/CD pipelines. KMS: migrate on-premises HSM-encrypted data to AWS KMS CMK. BYOK: import existing key material from on-premises HSM to KMS CMK.

🗄️ 28.4 Aurora Serverless

📖 28.4.1 Overview

Aurora Serverless is an on-demand, auto-scaling configuration for Amazon Aurora. Instead of provisioning fixed DB instances, Aurora Serverless automatically starts up, shuts down, and scales capacity based on application demand.

Two versions exist — v1 (legacy) and v2 (current recommended).

Scope: Regional. Storage is AZ-redundant (same as provisioned Aurora — 6 copies across 3 AZs).

Linux equivalent: PostgreSQL + pgBouncer connection pooler on an auto-scaling EC2 group — conceptually similar but manually managed.

Azure equivalent: Azure SQL Database Serverless — auto-pause/resume, vCore-based scaling, very similar concept.

📊 28.4.2 Aurora Serverless v1 vs v2

Feature	Serverless v1	Serverless v2
Scaling speed	Minutes (step-wise, brief connection interruption)	Sub-second, continuous, no connection drops
Pause/Resume	Yes — scales to 0 ACU when idle (~25s resume)	No — minimum 0.5 ACU always running
ACU range	1–256 ACU	0.5–128 ACU
Multi-AZ	No	Yes
Read replicas	No	Yes (mix serverless v2 + provisioned readers)
Aurora Global Database	No	Yes
Data API	Yes	Yes
Blue/Green Deployments	No	Yes
Best for	Dev/test, infrequent/unpredictable access	Production variable workloads

> 🔑 **Exam key:** v1 = pause to zero (zero cost when idle, slow to resume). v2 = production-grade instant scaling. For SAP-C02 "serverless production database" scenarios → Aurora Serverless v2.

⚡ 28.4.3 Aurora Capacity Units (ACU)

- **1 ACU ≈ 2 GB RAM** + proportional CPU and network
- **v2 minimum:** 0.5 ACU (~1 GB RAM) — never fully pauses, always warm
- **v2 maximum:** 128 ACU (~256 GB RAM)
- **Scaling:** v2 scales in fine-grained steps within seconds; old connections remain alive
- **Billing:** per ACU-second (v2) — you pay only for capacity used

---

🔗 28.4.4 Aurora Serverless Data API

- **HTTP-based API** to execute SQL against Aurora Serverless — no persistent TCP connection needed
- No VPC required from the caller (Lambda, API GW, etc.)
- Use for: **stateless Lambda functions** that cannot maintain persistent DB connections
- Authentication: IAM-based (SigV4) — no DB username/password in Lambda code
- **Limitation:** higher latency than native driver; not suitable for high-throughput OLTP
- **RDS Proxy alternative:** persistent connection pool for Lambda → Aurora (reduces connection exhaustion)

```
Lambda (stateless) ──HTTP──► Data API Endpoint ──► Aurora Serverless
Lambda (stateful)  ──TCP───► RDS Proxy (pool) ──► Aurora Serverless
```

📊 28.4.5 Aurora Serverless vs Provisioned Aurora vs RDS

Aspect	Provisioned Aurora	Aurora Serverless v2	RDS
Scaling	Manual + read replica auto-scaling	Instant ACU scaling (writer + readers)	Manual vertical
Cost model	Per instance-hour	Per ACU-second	Per instance-hour
Multi-AZ	Yes	Yes	Yes
Global Database	Yes	Yes	No
Connection pooling	RDS Proxy	RDS Proxy	RDS Proxy
Best for	Steady high-load predictable	Variable, spiky workloads	Simple managed DB

⚠️ 28.4.6 Exam Traps

1. **"Zero cost when idle"** → v1 only (pause to 0 ACU). v2 minimum 0.5 ACU always running.
2. **"Aurora Serverless for Global Database"** → v2 only, not v1.
3. **"Lambda connecting to Aurora without VPC"** → Data API (both v1 and v2).
4. **"Aurora Serverless v2 scales instantly"** → correct, sub-second, connections preserved.
5. **"Aurora Serverless v1 for production high-traffic"** → wrong; v1 scaling takes minutes and can drop connections.
6. **"Serverless = no storage costs"** → wrong; storage is billed same as provisioned Aurora.

☸️ 28.5 Amazon EKS (Elastic Kubernetes Service)

📖 28.5.1 Overview

Amazon EKS is a **managed Kubernetes control plane** service. AWS manages the etcd cluster, API server, scheduler, and controller manager. You manage worker nodes (EC2 or Fargate), networking configuration, and add-ons.

Scope: Regional. Control plane spans multiple AZs automatically. Node groups are deployed into specific subnets (AZ-specific or multi-AZ).

Linux/Open-source equivalent: Self-managed Kubernetes via `kubeadm`, k3s (lightweight K8s), kind (K8s in Docker).

Azure equivalent: Azure Kubernetes Service (AKS) — near-identical managed K8s offering.

📊 28.5.2 EKS Node Types

Node Type	Who Manages	Cost Model	Use Case
Managed Node Groups	AWS manages ASG, AMI updates, rolling upgrades	Per EC2 instance-hour	Standard production workloads
Self-Managed Nodes	You manage everything	Per EC2 instance-hour	Maximum customization
Fargate Profiles	AWS manages (serverless pods)	Per vCPU-second + GB-second	Stateless workloads, no node ops
EKS on Outposts	AWS control plane, on-prem nodes	Per node-hour	On-premises K8s

---

🔍 28.5.3 EC2 Node vs EC2 Instance

> This distinction frequently appears in SAP-C02 scenarios involving EKS.

Term	Definition
EC2 Instance	A virtual machine provisioned in AWS — the raw compute resource
EC2 Node (K8s)	An EC2 instance that has been registered as a Kubernetes worker node — runs kubelet, kube-proxy, container runtime (containerd)
Pod	One or more containers co-located on a node, sharing network namespace
Node Group	A collection of EC2 nodes with identical configuration (instance type, AMI, IAM role) managed as a unit

```
flowchart TD
    EKS[EKS Cluster] --> NG[Node Group]
    NG --> N1[EC2 Node / Worker]
    NG --> N2[EC2 Node / Worker]
    N1 --> P1[Pod: app-container]
    N1 --> P2[Pod: sidecar]
    N2 --> P3[Pod: app-container]
    N2 --> P4[Pod: monitoring]
```

ALB path-based routing to EC2 nodes (exam scenario):

- AWS Load Balancer Controller creates ALB from K8s Ingress
- Ingress rules map URL paths → K8s Services → Pod groups
- ALB target group uses IP mode (direct pod IPs) or instance mode (node port)
- Path `/api/*` → Service A → Pods on Node Group A
- Path `/web/*` → Service B → Pods on Node Group B

🌐 28.5.4 EKS Networking

- **VPC CNI (`aws-vpc-cni`):** Each pod receives a **real VPC IP address** (secondary IP from node ENI)
- **Pod-to-pod communication:** Direct VPC routing — no overlay (VXLAN) overhead
- **Security Groups for Pods:** Attach SGs directly to pods via `SecurityGroupPolicy` CRD
- **Prefix delegation:** Assign /28 CIDR prefixes to ENI — more pod IPs per node (up to 110 pods on large instances)
- **EKS endpoint modes:**
  - Public: `kubectl` reachable from internet
  - Private: `kubectl` only from within VPC
  - Public+Private: both (most common for managed clusters)

🔐 28.5.5 EKS IAM Integration

IRSA (IAM Roles for Service Accounts):

- K8s ServiceAccount → annotated with IAM role ARN → OIDC token → STS `AssumeRoleWithWebIdentity`
- Pod assumes a **specific IAM role** — not the broad node instance profile
- Least privilege: each microservice pod has its own IAM role

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: s3-reader
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/S3ReaderRole
```

EKS Access Entries (modern): Map IAM principals → K8s access policies via EKS API (replaces aws-auth ConfigMap).

🔧 28.5.6 Key EKS Add-ons

Add-on	Purpose
AWS Load Balancer Controller	Provisions ALB (Ingress) / NLB (Service) from K8s manifests
Cluster Autoscaler	Scales node groups based on pending pods
Karpenter	Fast, flexible node provisioning (replaces Cluster Autoscaler for many use cases)
EBS CSI Driver	Dynamic EBS PersistentVolume provisioning
EFS CSI Driver	Shared ReadWriteMany volumes (EFS)
CoreDNS	Cluster-internal DNS resolution
kube-proxy	Manages network rules on each node

📊 28.5.7 EKS vs ECS vs App Runner

Aspect	EKS	ECS	App Runner
Orchestration	Kubernetes	AWS-native	Fully managed
K8s ecosystem	Full	No	No
Ops complexity	High	Medium	Very low
Node management	Manual or managed NG	Fargate or EC2	None
Use when	Existing K8s workloads, complex microservices	AWS-native containers	Simple web APIs, no infra ops

---

⚠️ 28.5.8 Exam Traps

1. **"No EC2 to manage in EKS"** → Use Fargate profiles — but: no DaemonSets, no EBS PVs, no GPU.
2. **"Pod needs IAM access to S3"** → IRSA (not node instance profile — node profile is too broad, grants access to ALL pods on node).
3. **"kubectl from developer laptop to private EKS cluster"** → VPN/Direct Connect to VPC + private endpoint enabled.
4. **"Shared storage across multiple pods simultaneously"** → EFS CSI (ReadWriteMany), not EBS (ReadWriteOnce per node).
5. **"EKS node scaling"** → Cluster Autoscaler or Karpenter. EC2 Auto Scaling alone does not register new nodes with K8s.
6. **"EKS Fargate DaemonSet"** → not supported on Fargate.

⏱️ 28.6 Amazon Timestream

📖 28.6.1 Overview

Amazon Timestream is a **serverless time-series database** purpose-built for IoT telemetry, operational metrics, and DevOps monitoring data. It automatically scales ingestion and query capacity and manages data tiering between fast and cost-efficient storage.

Scope: Regional.

Linux/Open-source equivalent: InfluxDB, TimescaleDB (PostgreSQL extension), Prometheus + Thanos (long-term storage).

Azure equivalent: Azure Time Series Insights, Azure Data Explorer (ADX) — ADX is more powerful for analytics.

🏗️ 28.6.2 Architecture — Two-Tier Storage

```
flowchart LR
    INGEST[Write API
Kinesis / IoT Core / SDK] --> MEM[Memory Store
Recent data
hours to days
Fast, expensive]
    MEM -->|Auto-tiering| MAG[Magnetic Store
Historical data
months to years
Columnar, cheap]
    QUERY[Query Engine
SQL + time-series
functions] --> MEM
    QUERY --> MAG
```

- **Memory store:** In-memory, SSD-backed. High write throughput. Configurable retention (1 hour to 2 years).
- **Magnetic store:** Columnar format (similar to Parquet). Low cost. Configurable retention (unlimited).
- **Automatic tiering:** Data moves from memory to magnetic based on retention policy — transparent to queries.

📋 28.6.3 Data Model

```
Database
  └── Table
        └── Record
              ├── time (nanosecond precision, required)
              ├── dimensions (string metadata: device_id, region, sensor_type)
              └── measures (numeric/string/boolean: temperature, cpu_percent, status)
```

Multi-measure records: Store multiple metrics in one record per timestamp — more efficient than one record per metric.

🔍 28.6.4 Query Language

SQL with built-in time-series functions:

```sql
-- CPU utilization averaged every 5 minutes for the last hour
SELECT bin(time, 5m) AS t,
       AVG(measure_value::double) AS avg_cpu
FROM metrics
WHERE measure_name = 'cpu_utilization'
  AND time > ago(1h)
GROUP BY bin(time, 5m)
ORDER BY t

-- Linear interpolation for missing data points
SELECT time, interpolate_linear(CREATE_TIME_SERIES(time, measure_value::double)) AS interpolated
FROM metrics
WHERE measure_name = 'temperature'
```

🔗 28.6.5 Integrations

Source / Destination	How
Kinesis Data Streams	Lambda consumer writes to Timestream
AWS IoT Core	IoT Rule action → Timestream direct write
Amazon MSK / Kafka	Lambda/Kinesis bridge
Grafana	Native Timestream plugin (most common for dashboards)
Amazon QuickSight	JDBC connector
SageMaker	ML on time-series data

💰 28.6.6 Pricing

Component	Cost
Writes	Per million records written
Memory store	Per GB-hour stored
Magnetic store	Per GB-month (much cheaper than memory)
Queries	Per GB scanned
Compute	No separate compute charge — serverless

📊 28.6.7 Timestream vs Alternatives

Aspect	Timestream	InfluxDB (EC2)	DynamoDB	RDS/TimescaleDB
Management	Fully serverless	You manage EC2	Managed	Managed
Time-series functions	Built-in (`bin` , `interpolate` , `time_series`)	Built-in	Manual in queries	TimescaleDB extension
Auto-scaling	Yes	Manual	Yes	Manual vertical
Retention tiers	Memory + magnetic auto-tier	Manual	Manual TTL	Manual partitioning
Best when	IoT/DevOps, zero ops overhead	Existing InfluxDB, full control	Simple KV time-series	PostgreSQL ecosystem

---

⚠️ 28.6.8 Exam Traps

1. **"Store IoT sensor data, query trends over months"** → Timestream (not DynamoDB — DynamoDB works but is not optimized for time-series aggregations).
2. **"Timestream for transactional workloads"** → wrong; write-optimized time-series append only, not OLTP.
3. **"How to query both recent and historical time-series"** → Timestream query engine accesses both tiers automatically; no manual tier selection needed.
4. **"Real-time IoT dashboard"** → Timestream + Grafana (most common pattern in exam scenarios).

⚖️ 28.7 ALB Path-Based Routing

📖 28.7.1 Overview

Application Load Balancer (ALB) operates at **Layer 7 (HTTP/HTTPS/gRPC)**. It routes requests to different target groups based on request content — path, host, HTTP method, headers, query strings, or source IP.

Scope: Regional. ALB itself spans all AZs in a region.

Azure equivalent: Azure Application Gateway — same Layer 7 routing concept with path-based rules.

Linux equivalent: nginx `location` blocks; HAProxy `use_backend` with ACL rules.

📊 28.7.2 ALB Routing Rule Types

Rule Type	Pattern Example	Use Case
Path-based	`/api/*` → API TG; `/images/*` → CDN TG	Microservices on different paths
Host-based	`api.example.com` → API TG; `www.example.com` → Web TG	Virtual hosting, multi-tenant
HTTP method	`POST /orders` → write TG; `GET /orders` → read-only TG	CQRS pattern
Query string	`?version=2` → v2 TG	API versioning
HTTP header	`X-Device-Type: mobile` → mobile TG	Mobile vs desktop backend
Source IP	`10.0.0.0/8` → internal TG	Internal vs external routing
Combined (AND)	path `/api/*` AND header `X-Auth: true`	Complex routing logic

> 🔑 **Exam key:** Rules are evaluated in priority order (lowest number = highest priority). Default rule (no condition) catches everything else.

🪜 28.7.3 Path-Based Routing to EC2 Nodes

```
flowchart LR
    CLIENT[Client
HTTPS Request] --> ALB[ALB
Listener :443]
    ALB -->|/api/*| TGA[Target Group A
EC2 API nodes
port 8080]
    ALB -->|/images/*| TGB[Target Group B
EC2 Image nodes
port 9090]
    ALB -->|/admin/*| TGC[Target Group C
EC2 Admin nodes
port 7070]
    ALB -->|default| TGD[Target Group D
EC2 Web nodes
port 80]
    TGA --> EC2A1[EC2: api-01]
    TGA --> EC2A2[EC2: api-02]
    TGB --> EC2B1[EC2: img-01]
    TGC --> EC2C1[EC2: admin-01]
```

Target Group configuration per service:

- Health check: per TG (different paths, ports, thresholds per service)
- Deregistration delay: time ALB waits before stopping traffic to unhealthy/deregistering instance
- Sticky sessions: AWSALB cookie (ALB-generated) or AWSALBAPP cookie (app-generated) — sticks user to same EC2 node

---

🔗 28.7.4 ALB with EKS (Load Balancer Controller)

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
spec:
  rules:
  - http:
      paths:
      - path: /api/*
        pathType: Prefix
        backend:
          service:
            name: api-service
            port:
              number: 8080
      - path: /web/*
        pathType: Prefix
        backend:
          service:
            name: web-service
            port:
              number: 80
```

ALB Load Balancer Controller creates one ALB, with one listener, with path rules pointing to target groups backed by pod IPs (IP mode) or node ports (instance mode).

⚠️ 28.7.5 Limits and Exam Traps

- **Max listener rules per ALB:** 100 (soft limit)
- **Max target groups per ALB:** 100
- **"Route based on URL path"** → ALB (not NLB — NLB is L4, no HTTP routing)
- **"NLB for path-based routing"** → impossible; NLB routes by TCP/UDP port only
- **"SSL termination at ALB then plaintext to EC2"** → valid, common pattern (offloads TLS from EC2)
- **"ALB for WebSocket"** → yes, ALB supports WebSocket upgrade
- **"Same ALB for multiple domains with different certs"** → yes, SNI-based, up to 25 certs per listener

---

💰 28.8 EC2 Purchasing Options and Reserved Instances

📖 28.8.1 Overview

AWS offers multiple purchasing models to balance **flexibility, cost, and commitment**. Choosing the right model is a SAP-C02 core skill — exam scenarios frequently test cost optimization decisions.

📊 28.8.2 Purchasing Options Comparison

Option	Commitment	Discount vs On-Demand	Payment Options	Use Case
On-Demand	None	0%	Per-second billing	Unpredictable workloads, short-term, dev/test
Reserved (Standard RI)	1 or 3 year	Up to 72%	All-upfront / Partial / No-upfront	Steady-state 24/7 production workloads
Reserved (Convertible RI)	1 or 3 year	Up to 66%	All-upfront / Partial / No-upfront	Steady-state but uncertain future instance type
Compute Savings Plans	1 or 3 year	Up to 66%	Hourly commitment ($)	EC2 + Lambda + Fargate, any family/region/OS
EC2 Instance Savings Plans	1 or 3 year	Up to 72%	Hourly commitment ($)	Specific instance family + region, more flexible than RI
Spot Instances	None	Up to 90%	Bid on spare capacity	Fault-tolerant, stateless, batch, ML training
Dedicated Host	On-demand or 1/3 year	Varies	Per physical host	BYOL (Oracle/SQL Server), compliance, hardware affinity
Dedicated Instance	None	Small premium	Per-instance	Dedicated hardware, no host control
Capacity Reservation	None (pay On-Demand rate)	0%	Per-hour	Guaranteed capacity in specific AZ for DR/launch

---

📊 28.8.3 Reserved Instance Types

Feature	Standard RI	Convertible RI
Exchange	Cannot exchange (only modify size within family)	Can exchange for different family, OS, tenancy
Max discount	72%	66%
RI Marketplace	Can sell unused RIs	Cannot sell
Scope	Regional or Zonal	Regional or Zonal
Use when	Certain about instance type for full term	Uncertain about future requirements

Zonal RI: Reserved capacity + discount for specific AZ. **Regional RI:** Discount applied to any instance in any AZ in the region (no capacity reservation).

📊 28.8.4 Savings Plans vs Reserved Instances

Aspect	Compute Savings Plan	EC2 Instance Savings Plan	Standard RI
Scope	EC2 (any family/region/OS) + Lambda + Fargate	Specific instance family + region	Specific instance type + region + OS
Flexibility	Maximum	High	Lowest
Max discount	66%	72%	72%
Management	Applied automatically	Applied automatically	Manage RI inventory
AWS recommendation	Preferred for most	When higher discount needed	Legacy, less common now

---

🔴 28.8.5 Spot Instances

- **Up to 90% discount** on spare EC2 capacity
- **2-minute interruption notice** via instance metadata (`/latest/meta-data/spot/termination-time`) or EventBridge
- **Spot Fleet:** mix of instance types and AZs to maintain target capacity even if one pool is interrupted
- **Interruption strategies:** Terminate (default) | Stop (EBS-backed, resume later) | Hibernate (RAM to EBS)
- **Spot capacity pools:** each (instance type, AZ) is an independent pool
- **Diversification:** use multiple instance types and AZs to reduce interruption probability

```
flowchart LR
    SF[Spot Fleet] -->|capacity pool 1| C5[c5.xlarge
AZ-A]
    SF -->|capacity pool 2| M5[m5.xlarge
AZ-A]
    SF -->|capacity pool 3| C5B[c5.xlarge
AZ-B]
    SF -->|capacity pool 4| M5B[m5.xlarge
AZ-B]
```

🏢 28.8.6 Dedicated Hosts vs Dedicated Instances

Aspect	Dedicated Host	Dedicated Instance
Physical server	Yes — specific server assigned to you	Dedicated hardware, but you do not control the host
Host affinity	Yes — can pin instance to specific host	No host control
BYOL	Yes — use your own per-socket/per-core licenses	Limited — depends on license type
Visibility	See sockets, cores, host ID	No host visibility
Cost	Per physical host	Per instance (small premium)
Use case	Oracle DB BYOL, Windows Server BYOL, compliance requiring physical isolation	Dedicated hardware without BYOL needs

---

⚠️ 28.8.7 Exam Traps

1. **"Reduce cost for 24/7 production, 3 years"** → Standard RI, 3yr all-upfront = maximum discount (72%).
2. **"Flexible discount across EC2 + Lambda + Fargate"** → Compute Savings Plan (not RI — RI is EC2 only).
3. **"Physical server for Oracle BYOL"** → Dedicated Host (not Dedicated Instance — no host control).
4. **"Spot interrupted — EBS data?"** → EBS persists unless Terminate strategy; instance store data lost.
5. **"Reserve AZ capacity for DR without RI commitment"** → On-Demand Capacity Reservation.
6. **"Exchange m5.xlarge RI for c5.2xlarge"** → Convertible RI only (Standard RI cannot change family).
7. **"Sell unused Standard RI"** → RI Marketplace (Convertible RI cannot be sold).

🖥️ 28.9 EC2 Placement Groups

📖 28.9.1 Overview

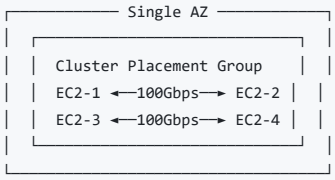
EC2 Placement Groups control the **physical placement** of EC2 instances on underlying AWS hardware to optimize for either **network performance** or **fault isolation**.

Three types: **Cluster** · **Spread** · **Partition**

📊 28.9.2 Placement Group Comparison

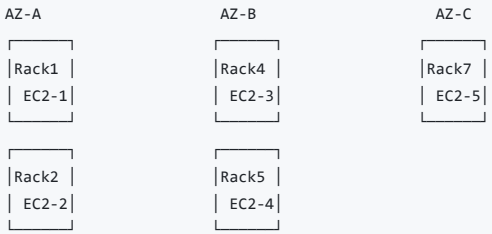
Feature	Cluster	Spread	Partition
Purpose	Max network performance	Max fault isolation per instance	Fault isolation per partition, large scale
AZ span	Single AZ only	Multi-AZ	Multi-AZ
Network	Up to 100 Gbps between instances	Standard (up to 25 Gbps)	Standard
Max instances	No limit	7 per AZ	Hundreds per AZ
Failure isolation	None (correlated — same rack)	Per instance (different racks)	Per partition (different racks)
Use case	HPC, ML training (all-reduce), tightly-coupled MPI	Small HA groups (Zookeeper, etcd, DB replicas)	Large distributed systems (Cassandra, Kafka, HBase, HDFS)

🔴 28.9.3 Cluster Placement Group



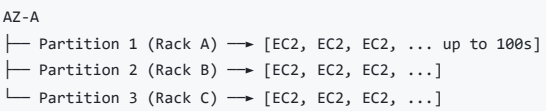
- All instances on the same low-latency, high-bandwidth network fabric
- Same physical rack or adjacent racks
- **Risk:** rack failure → all instances potentially affected
- **Enhanced Networking (ENA):** required for 100 Gbps — use instances that support ENA (C5n, P3, etc.)

🟢 28.9.4 Spread Placement Group



- Each instance on a **different rack** (different power + network)
- **Max 7 instances per AZ** — limits scale
- Multi-AZ: yes

🔵 28.9.5 Partition Placement Group



- **Up to 7 partitions per AZ**

- Each partition has its **own rack** (no shared hardware between partitions)
- Instances within a partition **may share hardware** (acceptable for replicated systems)
- **Instance metadata:** shows which partition an instance is in → application can place replicas in different partitions
- **Use:** Hadoop HDFS, Kafka (3 brokers → 3 partitions), Cassandra (RF=3, 1 replica per partition)

⚠️ 28.9.6 Exam Traps

1. **"Low-latency HPC inter-node communication"** → Cluster Placement Group.
2. **"Cassandra 3-way replication, no correlated hardware failure"** → Partition PG (3 partitions, 1 replica per partition).
3. **"7 Zookeeper nodes each on separate hardware"** → Spread Placement Group.
4. **"Cluster PG spans multiple AZs"** → FALSE — Cluster PG is single AZ only.
5. **"Spread PG for 100 instances"** → impossible; max 7 per AZ (use Partition PG instead).
6. **"Move existing instance into placement group"** → must stop, modify placement, start (cannot migrate running instance).

⚡ 28.10 Serverless on AWS — Lambda, App Runner, Step Functions

📖 28.10.1 AWS Lambda

Lambda is AWS's **event-driven, serverless compute** service. You deploy code (function) and AWS handles execution, scaling, and availability automatically.

Scope: Regional. Each invocation runs in a single AZ (not user-selectable).

Azure equivalent: Azure Functions.

Linux equivalent: No direct equivalent — conceptually similar to running code in cron jobs or daemon processes triggered by events.

📊 Lambda Key Parameters

Parameter	Default	Maximum
Memory	128 MB	10,240 MB (10 GB)
CPU	~0.07 vCPU	~6 vCPU (at 10,240 MB) — proportional to memory
Timeout	3 seconds	15 minutes
Deployment package (zip)	—	50 MB (direct upload), 250 MB (unzipped from S3)
Container image	—	10 GB
Concurrent executions	1,000/region	Adjustable (account-level soft limit)
Ephemeral storage (/tmp)	512 MB	10,240 MB
Environment variables	—	4 KB total
Layers	—	5 layers, 250 MB total unzipped
Payload (sync)	—	6 MB request, 6 MB response
Payload (async)	—	256 KB

🔁 Lambda Execution Model

```
flowchart TD
    INV[Invocation] --> CHECK{Warm env
available?}
    CHECK -->|Yes| WARM[Warm start
~1ms overhead
reuse /tmp, globals]
    CHECK -->|No| COLD[Cold start
100ms - 1s
download code
init runtime
run init code]
    WARM --> EXEC[Execute handler]
    COLD --> EXEC
```

- **Provisioned Concurrency:** Pre-warmed execution environments → eliminates cold starts
- **Reserved Concurrency:** Caps maximum concurrent invocations for a function (throttles if exceeded)

🔐 Lambda in VPC

- Lambda functions can be placed in a VPC to access private resources (RDS, ElastiCache, EFS)
- Uses **Hyperplane ENI** (shared across multiple Lambda functions in same VPC/SG/subnet config)
- **No public internet** inside VPC Lambda — add NAT Gateway for outbound internet
- **VPC endpoints** recommended for AWS service access (S3, DynamoDB, SQS) from VPC Lambda

---

🌍 Lambda@Edge and CloudFront Functions

Feature	Lambda@Edge	CloudFront Functions
Runtime	Node.js, Python	JavaScript (ES5)
Timeout	5s (viewer), 30s (origin)	2ms
Memory	Up to 10 GB	2 MB
Code size	1 MB (viewer), 50 MB (origin)	10 KB
Triggers	Viewer Request/Response, Origin Request/Response	Viewer Request/Response only
Use case	Auth, complex URL rewriting, A/B testing	Simple redirects, header manipulation, URL normalization
Cost	Per invocation + duration	Per invocation (much cheaper)

📖 28.10.2 AWS App Runner

App Runner is a **fully managed container application service** — deploy from source code or container image with zero infrastructure management.

Scope: Regional.

Azure equivalent: Azure Container Apps (very similar positioning).

Key Features

Feature	Detail
Source	GitHub (auto-build) or ECR image
Auto-scaling	Based on concurrent requests; scale-to-zero optional
Custom domain	Built-in, ACM cert auto-provisioned
VPC connector	Connect to VPC private resources (RDS, ElastiCache)
Health checks	HTTP path or TCP
Observability	CloudWatch metrics + logs, X-Ray tracing

📊 Serverless Compute Comparison

Aspect	Lambda	App Runner	ECS Fargate	EKS
Code model	Functions	Container	Container	Container
Max duration	15 min	Unlimited	Unlimited	Unlimited
Scale to zero	Yes (on-demand)	Optional	No	No
Ops complexity	Very low	Very low	Medium	High
K8s ecosystem	No	No	No	Yes
Best for	Event-driven short tasks	Simple web APIs and services	Flexible containers	Complex microservices

---

📖 28.10.3 AWS Step Functions

Step Functions provides **serverless workflow orchestration** — define state machines to coordinate AWS services, Lambda, human approvals, and external systems.

Scope: Regional.

Azure equivalent: Azure Logic Apps (low-code), Azure Durable Functions (code-based).

Open-source equivalent: Apache Airflow (data pipelines), Temporal.io (general workflows).

📊 Workflow Types

Feature	Standard Workflow	Express Workflow
Execution semantics	Exactly-once	At-least-once
Max duration	1 year	5 minutes
Throughput	2,000 executions/second	100,000 executions/second
Execution history	Full (CloudWatch Events)	CloudWatch Logs only
Pricing	$0.025/1,000 state transitions	$1/million executions + duration
Use case	Long-running, human approval, audit trail	High-volume, short-duration, idempotent

State Types

State	Purpose
Task	Invoke Lambda, ECS, DynamoDB, SNS, SQS, API GW, Glue, SageMaker, etc.
Choice	Conditional branching based on data
Wait	Pause for duration or until timestamp
Parallel	Execute multiple branches concurrently
Map	Iterate over array, run state machine per item
Pass	Pass input to output unchanged
Succeed / Fail	Terminal states

Error Handling

```
"Retry": [{
  "ErrorEquals": ["Lambda.ServiceException", "States.TaskFailed"],
  "IntervalSeconds": 2,
  "MaxAttempts": 3,
  "BackoffRate": 2.0
}],
"Catch": [{
  "ErrorEquals": ["States.ALL"],
  "Next": "ErrorHandler"
}]
```

Integration Patterns

Pattern	Behavior	Use Case
Request-Response	Call service, immediately proceed	Fire-and-forget
Synchronous (.sync:2)	Call service, wait for completion	ECS task, Glue job
Wait for Task Token	Send token, wait for callback	Human approval, external system

```
flowchart TD
    START([Start Order]) --> VALIDATE[Validate Order
Lambda]
    VALIDATE -->|valid| CHARGE[Charge Payment
Lambda]
    VALIDATE -->|invalid| FAIL_V([Fail: Invalid])
    CHARGE -->|success| INVENTORY[Reserve Inventory
DynamoDB]
    CHARGE -->|failed| REFUND[Refund Notification
SNS]
    REFUND --> FAIL_P([Fail: Payment])
    INVENTORY --> NOTIFY[Notify Customer
SNS]
    NOTIFY --> DONE([Success])
```

⚠️ 28.10.4 Exam Traps

Lambda:

1. **"Lambda max timeout"** → 15 minutes.
2. **"Eliminate cold starts"** → Provisioned Concurrency.
3. **"Lambda in VPC + internet access"** → NAT Gateway required.
4. **"Large Lambda deployment package (500 MB)"** → container image (up to 10 GB).
5. **"Lambda called too frequently, causing throttles"** → Reserved Concurrency limit reached or account limit; request increase.

App Runner: 6. **"Deploy web API from GitHub, zero infra ops"** → App Runner. 7. **"App Runner access to RDS in private VPC"** → VPC Connector.

Step Functions: 8. **"Retry on Lambda throttle, catch errors"** → Step Functions with Retry/Catch (not Lambda calling Lambda). 9. **"Human review step in automated workflow"** → Waitfor Task Token + human system calls callback. 10. **"High-volume 100k/sec workflow orchestration"** → Express Workflow. 11. **"Step Functions Standard max duration"** → 1 year.

🔌 28.11 Amazon API Gateway

📖 28.11.1 Overview

Amazon API Gateway is a **fully managed service** to create, publish, maintain, monitor, and secure APIs at any scale. It acts as the front door to backend services (Lambda, EC2, HTTP endpoints, or directly to AWS services).

Scope: Regional, Edge-Optimized, or Private.

Azure equivalent: Azure API Management (APIM).

Linux/open-source equivalent: Kong (API gateway), nginx + Lua scripting, Traefik.

📊 28.11.2 API Types

Type	Protocol	Deployment	Features	Best For
REST API	HTTP/HTTPS	Regional / Edge-Optimized / Private	Full feature set: caching, API keys, usage plans, transformation, WAF	Full-featured APIs, SaaS, internal
HTTP API	HTTP/HTTPS	Regional	Reduced (no caching, no API keys) but 70% cheaper, lower latency	Simple proxies, cost-sensitive
WebSocket API	WebSocket	Regional	Persistent connections, route by action	Chat, live dashboards, gaming
Private API	HTTP/HTTPS	VPC-only (via Interface Endpoint)	REST API features, VPC-only access	Internal microservices

---

🔗 28.11.3 Integration Types

Integration	Behavior	Use Case
Lambda Proxy	Full request → Lambda, Lambda controls response	Standard Lambda backend
Lambda Custom	Request/response transformed via mapping templates (VTL)	Data reshaping before Lambda
HTTP Proxy	Pass-through to any HTTP endpoint	Proxy to existing web service
AWS Service	Direct SDK call to AWS service (DynamoDB, SQS, SNS, Step Functions)	No Lambda layer needed
Mock	Return static response from API GW	CORS preflight, stubs for testing

---

🔐 28.11.4 Authorization Options

Method	How It Works	Use Case
IAM Auth	SigV4-signed request; caller needs `execute-api:Invoke` IAM permission	Service-to-service, internal AWS clients
Cognito Authorizer	API GW validates JWT issued by Cognito User Pool	Web/mobile apps with Cognito login
Lambda Authorizer (token)	Custom Lambda validates token (JWT, OAuth, SAML)	Any custom token format
Lambda Authorizer (request)	Full HTTP request context passed to Lambda for auth	Complex auth (multiple headers, IPs)
API Key	`x-api-key` header; not authentication — for usage tracking and throttling	Rate-limited SaaS tiers
JWT Authorizer (HTTP API)	HTTP API native — validates JWT from any OIDC provider	HTTP API, simple JWT auth

---

🚀 28.11.5 Stages, Deployments, and Canary

- **Stage:** Named deployment target (dev, staging, prod). Each stage is independently configured.
- **Deployment:** Snapshot of API config deployed to a stage.
- **Stage variables:** Key-value pairs available as `${stageVariables.name}` in integration URI, used to point to different Lambda aliases or backends per stage.
- **Canary:** Route X% of traffic to new deployment, rest to stable. Monitor then promote or roll back.

🛡️ 28.11.6 Throttling and Usage Plans

- **Account default:** 10,000 RPS, burst 5,000 requests
- **Stage/method level:** Override per stage or per method
- **Usage Plans:** Define rate + quota limits, associate with API Keys → per-customer throttling for SaaS

```
Usage Plan: Bronze
  Rate: 100 RPS | Quota: 10,000/day
  API Key: customer-A-key

Usage Plan: Gold
  Rate: 1,000 RPS | Quota: 1,000,000/day
  API Key: customer-B-key
```

💾 28.11.7 Caching (REST API Only)

Setting	Value
Available on	REST API only (not HTTP API)
TTL range	0–3,600 seconds (default 300s)
Cache capacity	0.5 GB to 237 GB
Cache key	URL path + all query params (customizable)
Invalidation	Client sends `Cache-Control: max-age=0`
Cost	Per hour per cache size

🌐 28.11.8 Edge-Optimized vs Regional vs Private

Type	How Deployed	Certificate Location	Use Case
Edge-Optimized	Routed through CloudFront PoPs	ACM in **us-east-1**	Global clients, low latency worldwide
Regional	Deployed in specific AWS region	ACM in **same region**	Clients in same region, private API, WebSocket
Private	VPC Interface Endpoint only	ACM in same region	Internal microservices, no public exposure

> 🔑 **Exam key:** Edge-Optimized API Gateway cert → must be in **us-east-1** (same rule as CloudFront).

📊 28.11.9 REST API vs HTTP API

Feature	REST API	HTTP API
Cost	Higher (~$3.50/million)	~~70% less (~~$1.00/million)
Latency	Higher	Lower
Caching	Yes	No
API Keys / Usage Plans	Yes	No
Request/Response transformation	Yes (VTL mapping templates)	No
WAF integration	Yes	No
X-Ray tracing	Yes	Yes
Resource policies	Yes	No
JWT Authorizer	No (use Lambda Auth)	Yes (native)
OIDC/OAuth	Via Lambda Authorizer	Native support
Private APIs	Yes	No
WebSocket	No (separate type)	No

⚠️ 28.11.10 Exam Traps

1. **"Caching in API Gateway"** → REST API only; HTTP API has no built-in cache.
2. **"Cheapest API Gateway option"** → HTTP API (70% less than REST API).
3. **"Per-customer rate limits for SaaS API"** → REST API + Usage Plans + API Keys.
4. **"Internal-only API, no public internet"** → Private API + resource policy with `aws:SourceVpc`.
5. **"Real-time bidirectional communication"** → WebSocket API.
6. **"Edge-Optimized API certificate region"** → us-east-1.
7. **"Call DynamoDB from API GW without Lambda"** → AWS Service Integration (REST API).
8. **"JWT validation without writing Lambda"** → HTTP API JWT Authorizer (or Cognito Authorizer on REST API).
9. **"Canary deployment for API"** → Stage canary — split traffic between current and new deployment.

🏛️ 28.12 Well-Architected Framework — Chapter 28 Extended

Pillar	Aurora Serverless	EKS	Timestream	ALB Routing	EC2 Purchasing	Placement Groups	
Operational Excellence	v2 eliminates capacity planning. RDS Proxy for connection management.	Managed node groups reduce patch ops. Karpenter for auto-provisioning.	No infrastructure to manage. Grafana dashboards for observability.	Path-based routing = service segmentation without multiple LBs.	Savings Plans auto-apply — no RI inventory management.	Documented placement strategy per workload type.	I L F (
Security	IAM + Secrets Manager for DB auth. Data API removes VPC requirement for Lambda. Encrypt with CMK.	IRSA: pod-level IAM (not node-level). SGs for Pods. Private EKS endpoint.	Encrypt at rest (CMK optional). IAM-controlled access. VPC endpoint.	HTTPS only (ACM cert). WAF on ALB. SG: ALB → EC2 only.	Dedicated Host for compliance.	Cluster PG: ensure SG restricts inter-node traffic.	L r p F e V f r a
Reliability	v2: Multi-AZ, instant scaling, no connection drops. RDS Proxy for connection pooling.	Multi-AZ node groups. Pod Disruption Budgets. ALB health checks. Karpenter re-provisions quickly.	Serverless — no single point of failure. Multi-AZ redundant.	Multi-AZ ALB. Target group health checks. Deregistration delay for graceful shutdown.	On-Demand Capacity Reservation for DR. Spot Fleet diversification for fault tolerance.	Spread PG for HA critical instances. Partition PG for distributed system replica isolation.	L r p c S F r A a
Performance	Sub-second ACU scaling. Data API for stateless Lambda. Provisioned Concurrency analog = min ACU.	EKS Fargate for variable pod density. EBS CSI for fast storage. EFA for HPC pods.	Memory store for recent queries. Adaptive query engine. Columnar magnetic store.	Connection reuse (HTTP/2). Target group stickiness for session affinity.	Cluster PG: 100 Gbps for HPC. Spot Fleet: access high-end instance types cheaper.	Cluster PG: lowest latency for inter-node traffic. ENA required.	L r (S F F f c t
Cost Optimization	v2: ACU-second billing — pay only for active capacity. v1: pause to zero for dev/test.	Spot node groups for batch EKS workloads. Fargate for spiky workloads (no idle nodes).	Magnetic store much cheaper than memory store for historical data.	One ALB for multiple services (path routing) vs multiple LBs.	Compute Savings Plans for flexible 66% discount. Standard RI 72% for predictable. Spot 90% for tolerant.	No cost for placement groups themselves. Cluster PG: fewer high-bandwidth instances needed.	L (r c S F E h A s z
Sustainability	Scale to near-zero	Karpenter right-sizes	Magnetic tier uses	Efficient routing	Spot: uses otherwise-	Cluster PG: fewer	L S

Pillar	Aurora Serverless	EKS	Timestream	ALB Routing	EC2 Purchasing	Placement Groups	
	with v2 min ACU. No idle provisioned DB resources.	nodes — no over-provisioned node groups. Fargate: per-pod billing.	columnar storage — efficient. No idle compute.	reduces unnecessary backend invocations.	idle AWS capacity (reuse of existing hardware).	instances needed for same HPC throughput.	

🎯 28.13 SAP-C02 Domains — Chapter 28 Extended

Domain 1 — Organizational Complexity: Multi-account EKS clusters with IRSA for cross-account IAM. Centralized API Gateway with AWS Organizations + Firewall Manager WAF. Step Functions cross-account execution via resource policies. Compute Savings Plans at payer account level (Consolidated Billing). ALB in shared services VPC exposed to spoke VPCs via PrivateLink.

Domain 2 — New Solutions: New IoT platform → Timestream for metrics + Grafana for dashboards + Kinesis for ingestion. New event-driven order processing → API Gateway + Lambda + Step Functions + DynamoDB. New containerized microservices from on-prem K8s → EKS with IRSA + ALB Ingress controller. New variable-load database → Aurora Serverless v2.

Domain 3 — Continuous Improvement: Migrate Lambda from zip to container image for larger dependency packages. Migrate REST API to HTTP API for cost savings (validate no REST-only features needed first). Add Provisioned Concurrency to Lambda for latency-sensitive API Gateway integrations. Add Partition Projection to Athena queries on Timestream-exported S3 data. Optimize EC2 costs: move On-Demand to Compute Savings Plans + Spot for non-critical batch.

Domain 4 — Migration: Migrate on-premises Cassandra → EKS on EC2 with Partition Placement Group (3 partitions = 3 replication zones). Migrate on-premises cron + shell scripts → Step Functions + Lambda (serverless, monitored, retried). Migrate on-premises nginx routing rules → ALB listener rules (path/host-based). Migrate Oracle BYOL from shared EC2 to Dedicated Host (license compliance). Migrate on-premises Kubernetes to EKS: use AWS Migration Hub + eksctl.

🌊 28.14 Amazon ECS (Elastic Container Service)

📖 28.14.1 Overview

Amazon ECS is AWS's **native container orchestration service**. It manages the lifecycle of Docker containers — scheduling, placement, scaling, health checking, and service discovery — without requiring Kubernetes knowledge.

Scope: Regional. Tasks can be spread across multiple AZs.

Linux/open-source equivalent: Docker Swarm (native), Nomad (HashiCorp).

Azure equivalent: Azure Container Instances (ACI) for single tasks; Azure Container Apps for services.

🏗️ 28.14.2 ECS Core Concepts

Concept	Definition
Cluster	Logical grouping of tasks/services and capacity providers (EC2 or Fargate)
Task Definition	Blueprint: container image(s), CPU/memory, networking mode, IAM roles, volumes
Task	Running instance of a Task Definition — one-off or part of a service
Service	Maintains N running tasks, integrates with ALB/NLB, handles rolling deployments
Container Instance	EC2 instance registered with ECS cluster running the ECS agent
Capacity Provider	Abstracts the compute layer (EC2 ASG or Fargate)

🌐 28.14.3 ECS Networking Modes

Mode	Description	Use Case
awsvpc	Each task gets its own ENI + private VPC IP	Recommended; task-level SGs, VPC routing, Fargate requires this mode
bridge	Docker bridge network on host; uses dynamic host port mapping	Legacy EC2 containers; compatible with older Docker networking
host	Container shares EC2 host network stack	Maximum network performance; no port conflicts allowed
none	No external networking	Isolated tasks

> 🔑 **Exam key:** Fargate **requires** `awsvpc` mode. `awsvpc` = each task has its own SG — enables fine-grained security.

🔐 28.14.4 ECS IAM Roles

Role	Purpose
Task Execution Role	ECS agent permission: pull image from ECR, write logs to CloudWatch, read Secrets Manager/SSM secrets
Task Role	Application permissions: what your container code can do (S3, DynamoDB, SQS, etc.)
EC2 Instance Profile	For EC2-backed clusters: allows ECS agent to communicate with ECS control plane

```
ECS Agent (Task Execution Role) ──pulls──► ECR
                                ──logs──► CloudWatch Logs
                                ──reads─► Secrets Manager

Container App (Task Role) ──reads──► S3
                          ──writes─► DynamoDB
```

> 🔑 **Exam key:** Two separate roles. Task Execution Role = ECS infrastructure. Task Role = application logic. Confusing them is a common exam trap.

🚀 28.14.5 ECS Launch Types

Feature	EC2 Launch Type	Fargate Launch Type
Infra management	You manage EC2 instances	AWS manages fully
Pricing	Per EC2 instance-hour	Per vCPU-second + GB-second
DaemonSets equivalent	ECS daemon scheduling (1 task per instance)	Not supported
GPU support	Yes	Limited (Fargate Arm64 only, no GPU)
EBS volumes	Yes (bind mount)	Ephemeral only (20 GB default)
EFS volumes	Yes	Yes (shared persistent storage)
Custom networking	All modes	awsvpc only
Spot instances	Via Spot Fleet capacity provider	Fargate Spot (up to 70% discount)

📦 28.14.6 ECS Service Deployments

Deployment Type	Behavior	Use Case
Rolling Update	Replace old tasks with new tasks; configurable min/max healthy %	Zero-downtime gradual rollout
Blue/Green (CodeDeploy)	Create new task set → shift ALB traffic → validate → terminate old	Instant rollback capability
External	Managed by external controller	Custom deployment orchestration

Rolling update parameters:

- `minimumHealthyPercent`: minimum % of desired tasks that must remain healthy (e.g., 50% = replace half at a time)
- `maximumPercent`: maximum % of desired tasks allowed during deployment (e.g., 200% = double count during rollout)

🔗 28.14.7 ECS Service Discovery and Load Balancing

ALB Integration:

- Service registers tasks as ALB target group targets
- `awsvpc` mode: target by IP (each task IP directly)
- `bridge` mode: target by instance + dynamic port
- Path-based routing: multiple ECS services behind one ALB (same as EC2 path routing)

Service Discovery (Cloud Map):

```
Service A → registers as api.myapp.local (A records)
Service B → resolves api.myapp.local → task IPs
```

- AWS Cloud Map creates DNS records per running task IP
- Used for service-to-service communication without ALB

📊 28.14.8 ECS Auto Scaling

Scaling Type	Mechanism
Service Auto Scaling	Target tracking or step scaling on CloudWatch metrics (CPU, memory, ALB requests/target, custom)
EC2 Capacity Provider (cluster auto scaling)	Scale underlying EC2 ASG based on cluster capacity pressure
Fargate	Scales tasks independently (no underlying instance ASG to manage)

📊 28.14.9 ECS vs EKS vs App Runner

Aspect	ECS	EKS	App Runner
Ecosystem	AWS-native	Kubernetes (portable)	AWS-native, zero-config
Learning curve	Medium	High	Very low
Control	High	Maximum	Minimal
DaemonSet equivalent	ECS daemon tasks	DaemonSets	Not applicable
Service mesh	ECS Service Connect / App Mesh	Istio, Linkerd, App Mesh	Not supported
Best for	AWS-native containerized microservices	K8s ecosystem, portability	Simple web APIs, fast onboarding

---

⚠️ 28.14.10 Exam Traps

1. **"Task Execution Role vs Task Role"** → Execution = ECS agent/infra; Task = app code. Separate roles.
2. **"ECS + Fargate + EBS"** → Fargate has no EBS support; ephemeral storage only. For persistence: EFS.
3. **"One ALB for multiple ECS services with path routing"** → Yes — each service maps to a target group; ALB rules route by path/host.
4. **"ECS Fargate DaemonSet"** → not possible; use EC2 launch type daemon scheduling.
5. **"ECS task needs to read from Secrets Manager"** → Task Execution Role must have Secrets Manager read permission.
6. **"ECS Blue/Green deployment"** → requires CodeDeploy + ALB. Rolling update does not use CodeDeploy.
7. **"ECS service needs IAM permission to write to DynamoDB"** → Task Role (not Execution Role, not instance profile).

🌥️ 28.15 AWS Fargate

📖 28.15.1 Overview

AWS Fargate is a **serverless compute engine for containers** — runs ECS tasks and EKS pods without provisioning or managing EC2 instances. You define CPU and memory per task/pod; AWS handles the rest.

Scope: Regional. Fargate tasks run in customer VPC subnets (awsvpc mode).

Azure equivalent: Azure Container Instances (ACI) — serverless container execution.

Key distinction: Fargate is not a service itself — it is a **launch type/provider** for ECS and EKS.

📊 28.15.2 Fargate for ECS vs EKS

Aspect	Fargate for ECS	Fargate for EKS
Configuration	Capacity Provider or Launch Type: FARGATE	Fargate Profile (namespace + label selector)
Networking	awsvpc (task gets ENI + VPC IP)	awsvpc (pod gets ENI + VPC IP)
Storage	Ephemeral (20 GB default, up to 200 GB) + EFS	Ephemeral (20 GB) + EFS
DaemonSets	ECS daemon scheduling (EC2 only)	Not supported
Spot	Fargate Spot (ECS)	No Spot for EKS Fargate
GPU	No	No

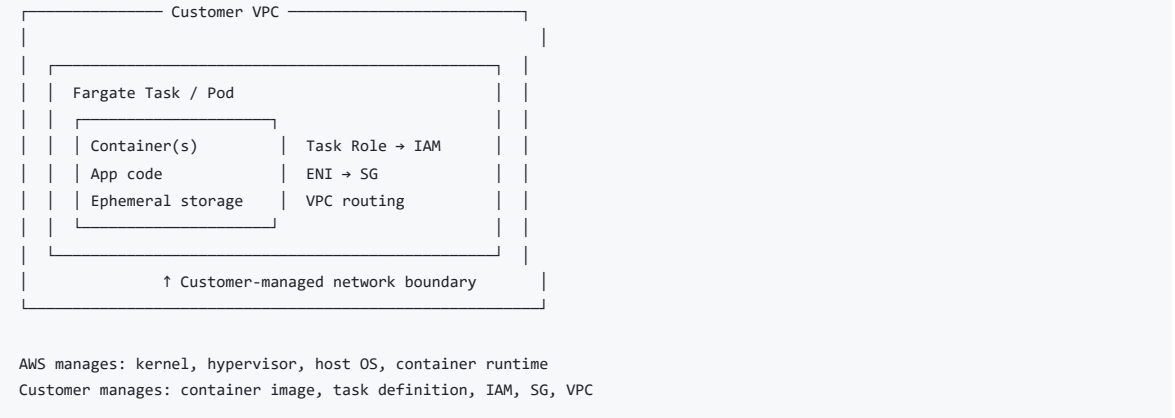
💰 28.15.3 Fargate Pricing Model

Resource	Billing Unit
vCPU	Per vCPU-second while task/pod is running
Memory	Per GB-second while task/pod is running
Ephemeral storage	First 20 GB free; charged per GB-second above 20 GB
Fargate Spot	Up to 70% discount; task can be interrupted with 2-min notice

Available CPU/Memory combinations (ECS):

vCPU	Memory Range
0.25 vCPU	0.5 GB – 2 GB
0.5 vCPU	1 GB – 4 GB
1 vCPU	2 GB – 8 GB
2 vCPU	4 GB – 16 GB
4 vCPU	8 GB – 30 GB
8 vCPU	16 GB – 60 GB
16 vCPU	32 GB – 120 GB

🔐 28.15.4 Fargate Security Model



Security isolation: Each Fargate task runs on dedicated kernel infrastructure (not shared with other customers). AWS manages patching of underlying OS — customer patches only container image.

📦 28.15.5 Fargate Storage

Storage Type	Capacity	Persistence	Use Case
Ephemeral	20 GB (default), up to 200 GB	Lost when task stops	Temp files, scratch space
EFS	Unlimited	Persistent, shared	Shared config files, media, persistent state
S3	Unlimited	Persistent, object	Large objects via SDK (not mounted filesystem)
Secrets Manager / SSM	N/A	External	Credentials, config (read at startup)

> 🔑 **Exam key:** No EBS for Fargate. EFS = only persistent filesystem mount. Ephemeral storage in /tmp is not shared between tasks.

🔗 28.15.6 Fargate Networking

- **awsvpc mandatory:** Each task/pod gets dedicated ENI in customer VPC subnet
- **Security Groups:** Apply at task level (not at instance level — no shared instance) → fine-grained micro-segmentation
- **Public IP:** Assign automatically if task in public subnet + `assignPublicIp: ENABLED`
- **Private only:** Place in private subnet + NAT Gateway for outbound internet
- **VPC Endpoints:** Recommended for ECR pull, CloudWatch Logs, S3, Secrets Manager → avoid NAT Gateway costs

```
flowchart LR
    TASK[Fargate Task
ENI + Private IP] -->|outbound| NAT[NAT Gateway
public subnet]
    NAT --> IGW[Internet Gateway] --> INTERNET[Internet / ECR]
    TASK -->|AWS services| VPCe[VPC Endpoints
ECR, S3, CloudWatch
Secrets Manager]
```

🤸 28.15.7 Fargate Spot

Feature	Detail
Discount	Up to 70% vs standard Fargate
Interruption	2-minute warning via ECS task state change event
Supported	ECS only (not EKS Fargate)
Use case	Batch jobs, stateless workers, non-critical tasks
Mixed strategy	Capacity Provider strategy: weight FARGATE 1, FARGATE_SPOT 4 → 80% Spot

```
"capacityProviderStrategy": [
  { "capacityProvider": "FARGATE",      "weight": 1, "base": 1 },
  { "capacityProvider": "FARGATE_SPOT", "weight": 4 }
]
```

📊 28.15.8 Well-Architected Framework — ECS + Fargate

Pillar	ECS	Fargate
Operational Excellence	ECS Service Connect for service-to-service observability. CodeDeploy Blue/Green for safe deployments. Container Insights for metrics.	No EC2 patch management. Fargate manages OS/kernel. Focus on app, not infra.
Security	Task Execution Role + Task Role separation. awsvpc = per-task SG. Secrets Manager injection at task start.	AWS-managed kernel isolation per task. No shared host OS. Image scanning via ECR.
Reliability	Multi-AZ task placement. Service Auto Scaling. ALB health checks auto-replace unhealthy tasks.	AWS-managed container runtime reliability. Task re-scheduling on failure.
Performance	EC2 launch type: GPU, EBS, custom AMI. Daemon tasks for sidecars.	Right-size per task (0.25–16 vCPU). No noisy-neighbor EC2 contention.
Cost Optimization	Spot capacity providers for EC2-backed clusters. Pack tasks per instance.	Fargate Spot 70% savings. Per-second billing. No idle instance cost between tasks.
Sustainability	Bin-pack tasks on EC2 instances to maximize utilization.	Per-task billing = zero idle compute. AWS optimizes physical infrastructure utilization.

🎯 28.15.9 SAP-C02 Domains — ECS + Fargate

Domain 1 — Organizational Complexity: Multi-account ECS cluster strategy: shared services cluster in network account + spoke account workloads via RAM-shared VPC subnets. Fargate tasks in spoke VPCs pull images from central ECR via VPC endpoint. Organizations SCPs prevent public ECR repositories.

Domain 2 — New Solutions: New containerized microservices: ECS Fargate + ALB (path-based) + Service Connect (service mesh) + EFS (shared config). All serverless — no EC2 to manage. Fargate Spot for batch processing layer.

Domain 3 — Continuous Improvement: Migrate ECS EC2 launch type to Fargate to eliminate instance patching overhead. Use Fargate Spot for non-critical services (70% savings). Add Container Insights for per-container metrics. Blue/Green deployments via CodeDeploy to eliminate deployment downtime.

Domain 4 — Migration: Migrate on-premises Docker Compose → ECS Task Definitions (1:1 mapping of services to containers). Migrate bare-metal batch jobs → ECS Fargate Spot batch tasks. Use AWS App2Container tool to containerize Java/.NET apps and generate ECS/EKS artifacts.

⚠️ 28.15.10 Exam Traps — Fargate

1. **"Fargate = ECS only"** → wrong; Fargate works with both ECS and EKS.
2. **"Mount EBS to Fargate task for database"** → not possible; use EFS for persistence or RDS/Aurora for DB.
3. **"Fargate task needs to pull image from private ECR without NAT Gateway"** → VPC endpoint for ECR (com.amazonaws.region.ecr.api + com.amazonaws.region.ecr.dkr + S3 gateway endpoint).
4. **"Reduce Fargate cost for batch workloads"** → Fargate Spot (ECS only, not EKS).
5. **"Fargate supports GPU for ML inference"** → no GPU support (use ECS EC2 with GPU-optimized AMI).
6. **"Fargate task SG vs EC2 task SG"** → Fargate (awsvpc): SG on task ENI. EC2 bridge mode: SG on EC2 instance (shared across tasks).
7. **"Patch container OS on Fargate"** → AWS patches underlying host OS; you patch container image (base image layers).
8. **"DaemonSet on ECS Fargate"** → not possible. ECS daemon scheduling = EC2 launch type only.

🏛️ 28.16 Well-Architected Framework — ECS + Fargate (Summary Row)

(Added as extended row to the 28.12 WAF table above — see 28.15.8 for full detail.)

🎯 28.17 SAP-C02 Domains — ECS + Fargate (Summary)

(Added as extended bullets to the 28.13 SAP-C02 section — see 28.15.9 for full detail.)

🏭 28.18 AWS Outposts

📖 28.18.1 Overview

AWS Outposts is a **fully managed service** that extends AWS infrastructure, services, APIs, and tools to virtually any on-premises or edge location. AWS delivers, installs, operates, and manages the physical rack(s) at your facility — you get native AWS services running locally, using the same APIs and Console as the cloud.

Key premise: Run AWS services where latency requirements, data residency regulations, or local data processing needs prevent using a public AWS Region.

Scope: Physical rack(s) at customer site, linked to a parent AWS Region via a reliable network connection (Direct Connect recommended). All management plane operations go through the parent region.

Azure equivalent: Azure Stack Hub / Azure Stack Edge — same concept, on-premises managed extension of Azure.

Linux/VMware equivalent: VMware Cloud on AWS, but inverted — Outposts brings AWS to VMware-like on-prem environments.

🏗️ 28.18.2 Form Factors

Form Factor	Description	Use Case
Outposts Rack	Full 42U rack, 1–96 racks per site	Large enterprises, data centers, factories
Outposts Servers	1U/2U servers	Smaller locations, retail stores, branch offices, space-constrained sites

Outposts Rack sizes: Start at 1 rack; scale to 96 racks per site. Each rack ships pre-installed with the selected EC2 instance types and storage configurations (called a **configuration**).

⚙️ 28.18.3 Supported AWS Services on Outposts

Service	On Outposts
EC2	Yes — selected instance families (M5, C5, R5, G4, I3en, etc.)
EBS	Yes — local block storage (gp2, io1)
S3 on Outposts	Yes — object storage locally (separate from S3 in region)
RDS	Yes — MySQL, PostgreSQL on Outposts
ECS	Yes
EKS	Yes
ElastiCache	Yes — Redis
EMR	Yes
ALB	Yes — local Application Load Balancer
VPC	Yes — Outpost is a subnet extension of your AWS VPC

> 🔑 **Key:** Not all AWS services are available on Outposts. Services that run locally = low latency. Services not on Outposts = accessed via parent region over the network link.

🌐 28.18.4 Networking Architecture

```
flowchart LR
    subgraph OnPrem["On-Premises Site"]
        OPR[Outposts Rack]
        OPR --> LGW[Local Gateway
LGW]
        OPR --> SLG[Service Link
VPN tunnel to region]
    end

    subgraph VPC["AWS VPC (Parent Region)"]
        RSUB[Region Subnet
AZ-A]
        OSUB[Outpost Subnet
extension of VPC]
    end

    LGW -->|on-prem LAN routing| CORP[Corporate Network]
    SLG -->|Direct Connect or internet| RSUB
    OPR -->|Outpost Subnet = VPC subnet| OSUB
```

- **Outpost Subnet:** A subnet in your VPC with `OutpostArn` — resources in this subnet run on the Outpost physically
- **Local Gateway (LGW):** Routes traffic between Outpost instances and on-premises network without going to the region
- **Service Link:** Encrypted VPN connection from Outpost to parent region for management plane, API calls, and services not available locally
- **Direct Connect recommended:** For reliable, low-latency Service Link; internet connection is backup

---

🔐 28.18.5 Data Residency and Compliance

Outposts addresses these regulatory requirements:

- **Data residency:** Data stored on Outpost EBS/S3 never leaves the physical location
- **Data sovereignty:** Compliance with GDPR, local data laws requiring data within a country/facility
- **Air-gapped or restricted connectivity:** Future: Outposts with disconnected mode (limited services without region connectivity)
- **Latency:** Sub-millisecond to local applications (on-prem to Outpost = LAN latency)

> 🔑 **Exam key:** "Data must stay in the building / country and you need EC2/RDS" → Outposts. "Data must stay in AWS region" → standard region with data residency SCP.

💰 28.18.6 Pricing

- **Hardware fee:** 3-year term commitment or On-Demand (higher cost). Priced per configuration (rack size + instance types + storage).
- **EC2 on Outposts:** Same On-Demand or Savings Plan pricing as in the region, minus the data transfer to region.
- **S3 on Outposts:** Per GB stored + per GB data retrieved.
- **No charge for the AWS infrastructure management itself** — AWS owns and maintains the hardware.

⚠️ 28.18.7 Outposts Limitations

- Requires **reliable network link to parent region** for management (Service Link). Loss of connectivity = cannot launch new instances, but existing running instances continue.
- Not available in all regions or countries (Outpost must ship to your location).
- Limited service availability vs full region.
- You are responsible for physical security, power, cooling, and network of your facility.

⚠️ 28.18.8 Exam Traps

1. **"Outposts runs independently without internet/DX"** → partial. Existing workloads continue during connectivity loss, but management plane requires connectivity. True offline = Snowball Edge.
2. **"Outposts = same pricing as Region"** → EC2/EBS pricing similar, but hardware commitment cost is separate.
3. **"S3 on Outposts = same as S3 in region"** → different service — separate endpoints, different limits, no S3 intelligent-tiering, no lifecycle to Glacier.
4. **"Outposts for DR backup site"** → wrong use case; Outposts = low-latency local processing. DR backup = S3/Glacier, DRS.
5. **"Any AWS service on Outposts"** → FALSE. Only supported services listed above. Lambda is NOT on Outposts.
6. **"Outpost subnet = isolated network"** → FALSE. Outpost subnet is an extension of your AWS VPC — same CIDR space, same SGs, same route tables.

🌆 28.19 AWS Local Zones

📖 28.19.1 Overview

AWS Local Zones are **extensions of an AWS Region** that place compute, storage, database, and other select AWS services **closer to large population centers, industry, and IT centers** where no full AWS Region exists.

Key premise: Ultra-low latency to end users in a specific metro area, while still using standard AWS APIs and VPC networking. No hardware shipped to your premises — AWS manages infrastructure in its own facility in the metro area.

Example: AWS Region us-east-1 (N. Virginia). Local Zone: Los Angeles (us-east-1-lax-1a) — brings EC2/EBS to LA without a full us-east-1 deployment.

Scope: Extension of parent region. Local Zone appears as an additional AZ. Resources in Local Zones are still part of the parent region's VPC.

Azure equivalent: Azure Edge Zones (Preview) — similar concept of metro-area extensions.

🌎 28.19.2 Available Services in Local Zones

Not all services available — subset of regional services:

Category	Services
Compute	EC2 (selected instance families), Auto Scaling
Storage	EBS, FSx for Lustre
Networking	VPC, Subnets, Direct Connect, VPN, ELB (ALB, NLB)
Containers	ECS, EKS
Database	RDS (MySQL, PostgreSQL, MariaDB)
Content Delivery	No CloudFront PoP in LZ — use standard CloudFront

> 🔑 Not available in Local Zones: Lambda, S3, DynamoDB, SQS, SNS, many managed services.

🏗️ 28.19.3 Architecture

```
flowchart LR
    subgraph Region["AWS Region (us-east-1)"]
        AZA[AZ us-east-1a]
        AZB[AZ us-east-1b]
        S3[S3 / Lambda / DynamoDB]
    end

    subgraph LZ["Local Zone (us-east-1-lax-1a — Los Angeles)"]
        LZ_EC2[EC2 Instances]
        LZ_EBS[EBS Volumes]
        LZ_ALB[ALB]
    end

    USERS[LA End Users
< 10ms latency] -->|low latency| LZ_ALB
    LZ_EC2 -->|< 10ms to users| USERS
    LZ_EC2 -->|~60ms to region| S3
```

- Local Zone subnet = subnet in your VPC with `AvailabilityZone = us-east-1-lax-1a`
- Same VPC routing, same SGs
- Data between LZ and parent region = standard AWS inter-region latency (~10-60ms depending on distance)
- Direct Connect to Local Zone = carrier-provided connectivity in the metro area

---

📊 28.19.4 Local Zones vs Availability Zones vs Outposts

Feature	Availability Zone	Local Zone	Outpost
Location	AWS data center in region	AWS-operated metro facility	Your on-premises facility
Who owns hardware	AWS	AWS	AWS (at your site)
Latency to users	Regional (10-100ms)	Metro-area (<10ms)	Sub-millisecond (LAN)
Service breadth	Full	Subset	Subset
Data residency	In region	In metro area	At your site
Cost	Standard	Slight premium	Hardware commitment
Opt-in required	No	Yes	Yes (order hardware)
Use case	Standard workloads	Low-latency metro apps	On-prem data residency

---

🎯 28.19.5 Use Cases

- **Media & Entertainment:** Video rendering, live streaming production — content creators in LA need <10ms to AWS compute
- **Real-time gaming:** Game servers co-located with player base
- **Financial trading:** Low-latency order processing near exchange co-location
- **AR/VR:** Real-time rendering requiring <20ms round-trip
- **Healthcare imaging:** PACS (Picture Archiving and Communication Systems) near hospitals in metro

---

⚠️ 28.19.6 Exam Traps

1. **"Local Zones = separate AWS Region"** → FALSE. LZ is an extension of a parent region — same VPC, same APIs, opt-in AZ.
2. **"All AWS services available in Local Zones"** → FALSE. Limited subset: EC2, EBS, ECS, EKS, RDS, ALB/NLB. No Lambda, S3, DynamoDB.
3. **"Local Zone for data residency in specific country"** → only if a Local Zone exists in that country; else Outpost.

4. **"Opt-in required"** → TRUE. Local Zones are opt-in — must enable in account settings (Account → Local Zones → Enable).
5. **"Direct Connect to Local Zone"** → yes, via carrier connections at Local Zone facility. Standard DX goes to parent region.

📡 28.20 AWS Wavelength

📖 28.20.1 Overview

AWS Wavelength **embeds AWS compute and storage services within telecommunications providers' 5G networks**. Wavelength Zones are AWS infrastructure deployments inside telco 5G data centers at the edge of the mobile network.

Key premise: Applications running in Wavelength Zones can serve mobile devices and end users with **single-digit millisecond latency** by eliminating the need for traffic to traverse the public internet to reach an AWS Region. Traffic stays within the 5G network until it reaches the Wavelength Zone.

Scope: Wavelength Zone is an extension of a parent AWS Region. Similar to Local Zones but co-located inside telco infrastructure (Verizon, Vodafone, KDDI, SK Telecom, etc.).

Azure equivalent: Azure Edge Zones with Carrier (partnered with AT&T, etc.) — same concept.

📐 28.20.2 Architecture

```
flowchart LR
    subgraph Region["AWS Region (us-east-1)"]
        RSVC[Regional Services
S3, DynamoDB, RDS]
        RVPC[VPC + Subnets]
    end

    subgraph WZ["Wavelength Zone (BOS-WL1, Verizon Boston)"]
        WL_EC2[EC2 Instances
in Wavelength Zone]
        WL_EBS[EBS Volumes]
        CGW[Carrier Gateway
routes to/from 5G network]
    end

    MOBILE[5G Mobile Device] -->|5G Radio| TOWER[5G Base Station
Verizon RAN]
    TOWER -->|stays in carrier network| WZ
    CGW -->|ultra-low latency| WL_EC2
    WL_EC2 -->|backbone| RSVC
```

- **Carrier Gateway (CGW):** Routes traffic between Wavelength Zone and the carrier's mobile network. Analogous to IGW but for carrier traffic.
- **No standard IGW in Wavelength Zone** — traffic enters/exits via Carrier Gateway only.
- **Wavelength Zone subnet:** Subnet in your VPC with AvailabilityZone set to the Wavelength Zone name (e.g., `us-east-1-wl1-bos-wlz-1`).

📊 28.20.3 Supported Services in Wavelength Zones

Service	Available
EC2	Yes (selected types: t3, c5, r5, g4dn for GPU)
EBS	Yes (gp2, io1)
ECS	Yes
EKS	Yes
EC2 Auto Scaling	Yes
ALB	Yes
S3, DynamoDB, Lambda	No — access via parent region over backbone

---

📊 28.20.4 Wavelength vs Local Zone vs Outpost vs Region

Feature	AWS Region	Local Zone	Wavelength Zone	Outpost
Location	AWS data center	Metro AWS facility	Telco 5G data center	Your facility
Who manages	AWS	AWS	AWS (in telco DC)	AWS
Network entry	Public internet / DX	Public internet / DX	5G carrier network	LAN
Target latency	10-100ms	< 10ms to metro	Single-digit ms to 5G device	Sub-ms (LAN)
Connectivity	Standard AWS	Opt-in AZ	Carrier Gateway	Service Link + LGW
Primary use case	Standard cloud	Metro low-latency	5G mobile / IoT edge	On-prem + cloud same API
Telco partnership	No	No	Yes (Verizon, Vodafone, KDDI...)	No

🎯 28.20.5 Use Cases

- **Connected vehicles:** Real-time processing of vehicle telemetry, V2X communication at highway speeds
- **AR/VR on mobile:** Game streaming to 5G devices — render in Wavelength, stream to phone
- **Smart factories / Industry 4.0:** 5G-connected robots and sensors processing data locally at the factory edge
- **Live video production:** Stadium/event live streaming with <5ms processing
- **Real-time ML inference:** Object detection on 5G-connected cameras — inference runs in Wavelength, not in the cloud region

---

🔐 28.20.6 Security in Wavelength

- **Carrier Gateway:** Only route for ingress/egress from carrier network; no IGW — cannot reach via public internet directly
- **Security Groups:** Same SG model as VPC — applied to EC2 ENIs in Wavelength Zones
- **No direct public IP:** Wavelength EC2 instances use carrier IP addresses (assigned by telco), not standard AWS EIPs
- **VPC connectivity:** Wavelength subnet is part of VPC — can communicate with region via VPC backbone (not via carrier network)

⚠️ 28.20.7 Exam Traps

1. **"Wavelength = separate AWS Region"** → FALSE. Extension of parent region — same VPC, same APIs, opt-in zone.
2. **"EIP on Wavelength EC2"** → FALSE. Wavelength uses carrier IP addresses, not standard EIPs.

3. **"IGW in Wavelength Zone"** → FALSE. No standard IGW — Carrier Gateway is the entry point from mobile network.
4. **"Wavelength for any low-latency app"** → Only when users are on a partnered 5G carrier network. Local Zone for general metro low-latency.
5. **"S3/Lambda/DynamoDB in Wavelength"** → NOT available locally. Access via parent region over backbone.
6. **"Wavelength opt-in"** → TRUE. Must enable Wavelength Zones in account settings, like Local Zones.

📊 28.21 AWS Edge Infrastructure — Master Comparison

Feature	AWS Region	Availability Zone	Local Zone	Wavelength Zone	AWS Outposts
Infrastructure owner	AWS	AWS	AWS	AWS (in telco DC)	AWS (at your site)
Location	AWS campus	Within region	Metro city	Telco 5G edge	Your data center/factory
Opt-in required	No	No	Yes	Yes	Yes (order hardware)
Latency to local users	10–100ms	10–100ms	< 10ms	1–5ms (5G)	< 1ms (LAN)
Full AWS service set	Yes	Yes	No (subset)	No (EC2/EBS only)	No (subset)
Data residency	In region	In AZ	In metro area	In carrier DC	At your site
Network entry	Internet / DX / VPN	Same as region	Internet / DX at LZ	Carrier Gateway (5G)	Local Gateway (LAN)
Connectivity to region	N/A (is region)	Within region	High-speed backbone	VPC backbone	Service Link (DX rec.)
Hardware commitment	No	No	No	No	Yes (3-year or On-Demand)
Primary use case	Standard cloud	HA within region	Low-latency metro apps	5G mobile/IoT edge	On-prem data residency / processing

---

🏛️ 28.22 Well-Architected Framework — Edge Infrastructure

Pillar	Outposts	Local Zones	Wavelength
Operational Excellence	Same AWS APIs/Console as cloud — one operational model for on-prem + cloud. AWS manages hardware.	Standard EC2/VPC tooling — no new ops model needed. Opt-in AZ.	Same EC2/VPC tooling. Carrier partnership handles 5G edge networking.
Security	Data stays on-prem (data residency). Same IAM, SGs, VPC. Outpost subnet = VPC subnet. You secure the physical facility.	Same security controls as region. Data stays in metro facility (AWS-managed).	Carrier Gateway replaces IGW — no direct internet exposure. Same SG/VPC model.
Reliability	Service Link loss = management unavailable, but running workloads continue. Multi-Outpost for HA.	Parent region AZ + Local Zone AZ = active-active HA.	Carrier network reliability separate from AWS region reliability. Design for carrier outage.
Performance	Sub-ms LAN latency for on-prem applications. Local RDS = eliminates WAN DB latency.	< 10ms to metro end users. GPU EC2 for real-time rendering near users.	Single-digit ms for 5G devices. GPU instances (g4dn) for ML inference at edge.
Cost Optimization	3-year hardware commitment = predictable cost. S3 on Outposts = local storage saves DX data transfer costs.	Pay for LZ EC2 only when used. Eliminate multi-region active-active if LZ covers the use case.	Pay only for Wavelength EC2. Reduces data transfer from device → cloud → device latency-sensitive paths.
Sustainability	Consolidates on-prem + cloud into single managed infrastructure = higher utilization vs dedicated on-prem servers.	AWS-managed facility = optimized PUE. No customer-owned hardware for metro presence.	Co-location in existing telco DC = no new facility needed.

🎯 28.23 SAP-C02 Domains — Edge Infrastructure

Domain 1 — Organizational Complexity: Multi-account Outposts management: Outpost shared via AWS RAM to spoke accounts. Centralized networking account owns LGW and on-prem routes. SCPs in all accounts enforce data residency. Organizations-level Service Link via Transit Gateway.

Domain 2 — New Solutions: New manufacturing plant monitoring: Outposts Rack running EC2+RDS locally (sub-ms to PLCs) + S3 on Outposts for local telemetry storage + batch sync to region S3 via DataSync. New mobile gaming backend: Wavelength Zones in 5 carrier markets + parent region for persistence (DynamoDB, S3) + CloudFront for static assets.

Domain 3 — Continuous Improvement: Migrate on-prem SQL Server to RDS on Outposts → same latency, fully managed. Add Local Zone in user-dense metro area to reduce ALB response time from 80ms to <10ms. Enable Wavelength for existing mobile app → add Wavelength Zone subnets to existing VPC, deploy EC2 in WZ for inference, no app code change needed.

Domain 4 — Migration: Migrate factory floor applications to Outposts: use AWS Application Migration Service to lift-and-shift VMs → EC2 on Outposts. Retain same IP ranges via LGW. Migrate on-premises Oracle to RDS on Outposts for regulatory compliance. Migrate CDN origin servers to Local Zones near user base to reduce origin latency.

End of Chapter 28 — Additional Services

🔒 Chapter 29: Security Add-Ons

> **Covers:** AWS ACM · AWS CloudHSM · Certificates in CloudFront/ALB/API GW · AWS Config · Amazon GuardDuty · AWS CloudTrail (advanced) · Amazon Macie · AWS Control Tower · AWS Security Hub · Delegated Administrators · AWSSecurityAudit Permission Set · IAM Access Analyzer · Identity Center Built-in Directory

---

📋 Table of Contents (Chapter 29)

- 29.1 AWS Certificate Manager (ACM)
- 29.2 AWS CloudHSM
- 29.3 Certificates in CloudFront, ALB, and API Gateway
- 29.4 AWS Config
- 29.5 Amazon GuardDuty
- 29.6 AWS CloudTrail — Advanced
- 29.7 Amazon Macie
- 29.8 AWS Control Tower
- 29.9 AWS Security Hub
- 29.10 Delegated Administrators in AWS Organizations
- 29.11 AWSSecurityAudit Permission Set
- 29.12 IAM Access Analyzer
- 29.13 IAM Identity Center — Built-in Directory
- 29.14 Well-Architected Framework — Security Add-Ons
- 29.15 SAP-C02 Domains — Security Add-Ons
- 29.16 Related Services Summary

🏅 29.1 AWS Certificate Manager (ACM)

📖 Overview

AWS Certificate Manager (ACM) is a managed service for **provisioning, managing, and deploying SSL/TLS certificates** for use with AWS services and internal connected resources. Public certificates are **free** and automatically renewed.

Scope: Regional for most services (ALB, NLB, API Gateway). **us-east-1 only** for CloudFront (global service requirement).

Linux/Open-source equivalent: Let's Encrypt + certbot (free DV certs, manual or automated renewal), OpenSSL (self-signed), CFSSL (Cloudflare PKI toolkit).

Azure equivalent: Azure App Service Managed Certificates (free TLS for App Service), Azure Key Vault Certificates (managed cert lifecycle), DigiCert/GlobalSign integration.

🔑 29.1.1 Certificate Types

Type	Cost	Validation	Renewal	Export	Use Case
ACM Public Certificate	Free	DNS or Email (DV only)	Automatic	❌ Cannot export	ALB, CloudFront, API GW, NLB
ACM Private Certificate (Private CA)	$400/CA/month + 0.75/cert	Internal (CA-signed)	Automatic or manual	✅ Exportable (with private key)	Internal services, mTLS, code signing
Imported Certificate	Free (import action)	External CA validates	**Manual** (you manage)	N/A (you brought it)	Third-party certs (DigiCert, Sectigo)

> 🔑 **Exam key:** Public ACM certs **cannot be exported** (private key never leaves ACM). If you need the cert on an EC2 instance directly, import a cert with its private key, or use ACM Private CA (exportable).

🔑 29.1.2 Domain Validation Methods

Method	How	Pros	Cons
DNS validation	Add CNAME record to DNS zone	Persists across renewals (no action needed)	Requires DNS zone access
Email validation	AWS emails WHOIS admin contacts	No DNS access needed	Expires — must re-validate if cert expires

> 🔑 **Exam tip:** DNS validation is strongly recommended. ACM auto-renews via the CNAME record silently. Email validation requires action on each renewal — risk of cert expiry if email is ignored.

🔑 29.1.3 Wildcard and SAN Certificates

Wildcard: `*.example.com` covers:

- `app.example.com` ✅
- `api.example.com` ✅
- `sub.app.example.com` ❌ (does NOT cover two levels deep)

SANs (Subject Alternative Names): One certificate with multiple FQDNs:

```
example.com
www.example.com
api.example.com
admin.example.com
```

ACM allows up to 10 domain names per cert by default (can be increased).

🔑 29.1.4 ACM Private CA

ACM Private CA allows you to build a **private PKI hierarchy** for internal services:

```
Root CA (ACM Private CA)
    └── Subordinate CA (ACM Private CA)
            ├── Internal TLS certs (web services, microservices)
            ├── mTLS client certs (API clients)
            └── Code signing certs
```

Cost: ~$400/month per CA + $0.75 per private certificate issued.

Key difference from KMS: Private CA manages the certificate lifecycle (issue, revoke, renew). KMS manages encryption keys. They complement each other — Private CA can use a KMS CMK to protect the CA private key.

Use cases:

- mTLS for ALB (client certificate authentication)
- mTLS for API Gateway
- Internal microservices TLS (zero-trust service mesh)
- IoT device certificates

🔑 29.1.5 ACM Limitations

Limitation	Detail
Public certs not exportable	Cannot install on EC2 directly
DV only for public certs	No OV (Organization Validated) or EV (Extended Validation)
CloudFront requires us-east-1	Even if workload is in eu-central-1
No IP address certs	DNS name only (no cert for 1.2.3.4)
2048-bit RSA minimum	1024-bit not supported
Certificate pinning incompatible	Auto-rotation changes fingerprint

---

🔐 29.2 AWS CloudHSM

📖 Overview

AWS CloudHSM provides **dedicated Hardware Security Module (HSM) appliances** within your VPC. Unlike KMS (where AWS manages the HSMs), CloudHSM gives you **exclusive control** — AWS personnel cannot access your key material.

Scope: Regional. Deployed as a **cluster** of HSM instances across multiple AZs.

Linux/Open-source equivalent: On-premises Thales Luna HSM, Utimaco HSM, or nCipher HSM connected to application servers via PKCS#11.

Azure equivalent: Azure Dedicated HSM (same concept — dedicated HSM in your network, exclusive customer control).

🔑 29.2.1 CloudHSM vs KMS Comparison

Aspect	KMS (CMK)	KMS (BYOK)	CloudHSM	ACM Private CA
Key custody	AWS manages HSMs	Customer generates, AWS stores	Customer-exclusive	ACM manages
AWS key access	AWS can theoretically access (key policy-controlled)	AWS stores encrypted key material	**AWS has zero access**	N/A
FIPS 140-2	Level 3	Level 3	**Level 3 (dedicated)**	Level 3 (via KMS/CloudHSM)
Standards interface	AWS-only API	AWS-only API	PKCS#11, JCE, CNG	ACM API
Cost	$1/key/month	$1/key/month	~$1,044/month/HSM	$400/CA/month
Operational burden	Low (managed)	Medium	High (cluster management)	Medium
Use case	General encryption	Regulatory BYOK	Exclusive HSM control	Internal PKI

> 🔑 **Exam decision:**
>
> - "Encryption keys, AWS-managed infrastructure" → **KMS CMK**
> - "Regulatory requires key material generated externally" → **KMS BYOK (CMK with EXTERNAL origin)**
> - "Regulatory requires keys NEVER accessible to AWS" → **CloudHSM**
> - "Internal PKI / mTLS certificates" → **ACM Private CA**
> - "Database TDE with customer-controlled HSM" → **CloudHSM**

🔑 29.2.2 CloudHSM Architecture

```
VPC
├── AZ-A
│   └── CloudHSM Instance 1 (ENI in your subnet)
├── AZ-B
│   └── CloudHSM Instance 2 (ENI in your subnet)
│
└── Application EC2
    └── CloudHSM Client (PKCS#11 library)
        └── communicates on port 2223/2224 to HSM ENIs
```

Cluster synchronization: Key material is automatically synchronized across all HSMs in the cluster. Quorum (M-of-N) authentication available for critical operations.

CloudHSM Client SDK: Linux (`cloudhsm-client`), Windows — provides PKCS#11, JCE, and CNG interfaces to application code.

🔑 29.2.3 CloudHSM Use Cases

1. **Database Transparent Data Encryption (TDE):** Oracle TDE or SQL Server TDE using CloudHSM as the key store — database encryption key protected by HSM.
2. **PKI Root CA key protection:** Store root CA private key in CloudHSM, issue certs via ACM Private CA backed by CloudHSM.
3. **Document signing:** Legal documents, code signing — HSM ensures signing key never exposed.
4. **FIPS 140-2 Level 3 compliance:** Some regulations (DoD, government) require dedicated, certified HSM.
5. **SSL/TLS offloading:** Web servers using CloudHSM for TLS private key — private key operations performed in HSM.

🌐 29.3 Certificates in CloudFront, ALB, and API Gateway

📖 Overview

SSL/TLS certificate deployment varies by service. The **most critical exam point:** CloudFront certificates must **always** be in **us-east-1**, regardless of where your origin or users are.



🔑 29.3.1 CloudFront Certificate Requirements

Requirement	Detail
Certificate region	**MUST be us-east-1** — CloudFront is a global service
SNI (Server Name Indication)	Free — requires clients to support SNI (all modern browsers/clients do)
Dedicated IP	$600/month — for legacy clients that don't support SNI
Minimum TLS version	Configurable: TLSv1, TLSv1.1, TLSv1.2, TLSv1.3 (recommend TLSv1.2 minimum)
Custom domain	Route 53 Alias or CNAME → CloudFront distribution domain
Wildcard	Supported (`*.example.com` in ACM us-east-1)

> ⚠️ **Exam trap:** Company has workloads in eu-west-1. They create an ACM cert in eu-west-1 for their CloudFront distribution. This will FAIL — CloudFront requires ACM certs in us-east-1.

🔑 29.3.2 ALB Certificate Requirements

Feature	Detail
Certificate region	Same region as ALB
Multiple certs per listener	Up to 25 certs per HTTPS listener (SNI-based selection)
Default cert	Selected if no SNI match from client
mTLS	ALB verifies client cert against trust store (ACM Private CA or S3 CA bundle)
TLS policy	Choose TLS security policy (ELBSecurityPolicy-TLS13-1-2-2021-06 recommended)
Certificate rotation	Automatic via ACM — ALB picks up renewed cert without restart

ALB mTLS configuration:

```
Listener → Mutual Authentication → Trust store (S3 bucket with CA cert bundle)
→ ALB validates client cert → passes client cert headers to targets:
  X-Amzn-Mtls-Clientcert: <base64 DER cert>
  X-Amzn-Mtls-Clientcert-Subject: CN=client.example.com
```

🔑 29.3.3 API Gateway Certificate Requirements

API Type	Certificate Location
REST API — Edge-Optimized	us-east-1 (routes through CloudFront)
REST API — Regional	Same region as API Gateway
HTTP API	Same region
WebSocket API	Same region
Custom domain name	ACM cert + Route 53 Alias or CNAME
mTLS	Truststore (CA bundle) uploaded to S3

> 🔑 **Exam key:** Edge-Optimized API Gateway = cert in us-east-1 (same as CloudFront). Regional API Gateway = cert in same region. This parallels the CloudFront requirement because Edge-Optimized APIs ARE fronted by CloudFront.

🔑 29.3.4 Certificate Rotation Flow

```
ACM auto-renews cert 60 days before expiry
    |
    ▼
New cert issued (same ARN, new serial number)
    |
    ├── ALB: automatically uses new cert (no restart)
    ├── CloudFront: automatically uses new cert (no restart)
    └── API Gateway: automatically uses new cert (no restart)

Result: zero-downtime cert rotation
```

> ⚠️ **Certificate pinning breaks ACM auto-rotation:** If a client pins to a specific cert fingerprint, it will reject the new cert after ACM rotation. Avoid certificate pinning with ACM-managed certs.

⚙️ 29.4 AWS Config

📖 Overview

AWS Config is a **continuous configuration recording and compliance evaluation service**. It records the state of AWS resources over time and evaluates them against desired configurations.

Scope: Regional. Multi-region/multi-account via **Config Aggregator**.

Linux/Open-source equivalent: Chef InSpec, Open Policy Agent (OPA), Puppet compliance reporting.

Azure equivalent: Azure Policy (compliance evaluation), Azure Resource Graph (resource state queries).

> 🔑 **Config vs CloudTrail distinction:**
>
> - **Config** = WHAT is the current/past state of a resource (configuration snapshot, compliance status)
> - **CloudTrail** = WHO made an API call, WHEN, from WHERE (action audit log)

🔑 29.4.1 Core Components

```
flowchart TD
    RC[AWS Resources\nEC2, SG, S3, IAM...] -->|changes| CR[Config Recorder]
    CR -->|config items| CH[Config History\nS3 Bucket]
    CR -->|change notifications| SNS[SNS Topic]
    CR --> RULES[Config Rules\nEvaluation]
    RULES -->|compliant / non-compliant| DASH[Config Dashboard]
    RULES -->|non-compliant| REM[Remediation\nSSM Automation]
    DASH --> AGG[Config Aggregator\nmulti-account/region]
```

🔑 29.4.2 Config Rules

Rule Type	How Defined	Trigger	Example
AWS Managed	AWS-provided	Config change or periodic	`restricted-ssh`, `s3-bucket-public-read-prohibited`
Custom Lambda	Your Lambda function	Config change or periodic	Custom business logic
Custom Guard	CloudFormation Guard (declarative)	Config change	Declarative compliance rules
Proactive	CloudFormation Hook	Before resource creation	Block non-compliant resource creation

Key managed rules for exam:

Rule	Checks
`restricted-ssh`	SG does not allow unrestricted SSH (port 22) from 0.0.0.0/0
`restricted-common-ports`	SG blocks ports 20, 21, 3389, 3306, 4333 from 0.0.0.0/0
`s3-bucket-public-read-prohibited`	S3 bucket does not allow public read
`encrypted-volumes`	EBS volumes are encrypted
`rds-storage-encrypted`	RDS instances have storage encryption
`cloudtrail-enabled`	CloudTrail is enabled in account
`mfa-enabled-for-iam-console-access`	IAM users with console access have MFA
`iam-password-policy`	Account password policy meets requirements
`vpc-flow-logs-enabled`	VPC Flow Logs enabled for all VPCs
`root-account-mfa-enabled`	Root account has MFA enabled

🔑 29.4.3 Conformance Packs

Pre-packaged collections of Config rules for specific compliance frameworks:

Pack	Framework
`AWS-CIS-AWS-Benchmark-Level1`	CIS AWS Foundations Benchmark Level 1
`AWS-NIST-800-53-rev5`	NIST SP 800-53 Rev 5
`AWS-PCI-DSS-3-2-1`	PCI DSS 3.2.1
`AWS-HIPAA-Security-Rule-2003`	HIPAA Security Rule
`AWS-GxP-21-CFR-Part-11`	FDA 21 CFR Part 11 (life sciences)

Deploy a conformance pack → all included rules deployed → compliance report generated per resource.

🔑 29.4.4 Config Aggregator

Multi-account, multi-region view:

- Aggregator account (usually Security/Audit account) collects Config data from all member accounts
- Requires: either Organizations-level aggregator (auto-discovers accounts) or manual account list
- Delegated admin: member account can be designated as Config aggregator without using management account

---

🔍 29.5 Amazon GuardDuty

📖 Overview

Amazon GuardDuty is a **managed intelligent threat detection service** using machine learning, anomaly detection, and threat intelligence feeds. It requires **no agents, no infrastructure changes** — enables on a per-region, per-account basis.

Scope: Regional. Multi-account via Organizations (delegated admin).

Azure equivalent: Microsoft Defender for Cloud — Threat Protection plans (Defender for Servers, Defender for Storage, etc.).

> 🔑 **GuardDuty vs Config vs Security Hub:**
>
> - **GuardDuty** = active threat detection (anomalous behavior, malicious activity)
> - **Config** = compliance and configuration drift detection
> - **Security Hub** = aggregation layer — collects findings from GuardDuty, Config, Macie, etc.

🔑 29.5.1 Data Sources and Protection Plans

Data Source / Plan	What It Analyzes	Default
VPC Flow Logs	Network traffic metadata — port scans, C2 communication	Enabled by default
CloudTrail Management Events	API calls — credential abuse, unusual activity	Enabled by default
CloudTrail S3 Data Events	S3 GetObject/PutObject — data exfiltration from S3	Enabled by default
DNS Logs	DNS queries — malware C2 via DNS, DGA domains	Enabled by default
EKS Audit Logs	Kubernetes API activity	Optional (EKS Protection plan)
RDS Login Activity	Unusual RDS authentication patterns	Optional (RDS Protection plan)
Lambda Network Activity	Suspicious Lambda outbound traffic	Optional (Lambda Protection plan)
Runtime Monitoring	Process-level activity on EC2/ECS/EKS	Optional (Runtime Monitoring plan)
Malware Protection	EBS volume scanning of flagged instances	Optional (add-on)

🔑 29.5.2 Finding Categories and Examples

Category	Example Finding	Meaning
UnauthorizedAccess	`UnauthorizedAccess:EC2/SSHBruteForce`	SSH brute force from external IP
CryptoCurrency	`CryptoCurrency:EC2/BitcoinTool.B!DNS`	EC2 resolving crypto mining pool domains
Trojan	`Trojan:EC2/BlackholeTraffic`	EC2 contacting known sinkholed/blackholed IPs
Backdoor	`Backdoor:EC2/Spambot`	EC2 sending spam on port 25
Recon	`Recon:EC2/PortProbeUnprotectedPort`	Port probe on EC2 with unprotected port
CredentialAccess	`CredentialAccess:IAMUser/AnomalousBehavior`	ML-detected anomalous IAM activity
Impact	`Impact:S3/MaliciousIPCaller`	S3 API from known malicious IP
Exfiltration	`Exfiltration:S3/AnomalousBehavior`	Unusual S3 data retrieval (ML-detected)
Persistence	`Persistence:IAMUser/NetworkPermissions`	New network permissions added to IAM user
DefenseEvasion	`DefenseEvasion:IAMUser/CloudTrailLoggingDisabled`	CloudTrail disabled by IAM user

🔑 29.5.3 GuardDuty Multi-Account Architecture

```
Management Account
    └── Enable GuardDuty delegated admin → Security Account

Security Account (GuardDuty delegated admin)
    ├── Receives ALL findings from all member accounts
    ├── Cannot be disabled by member accounts
    ├── Centralized suppression rules, trusted IP lists
    └── Findings → EventBridge → Lambda auto-remediation

Member Accounts (auto-enrolled via Organizations)
    ├── GuardDuty enabled automatically
    └── Cannot disable or modify GuardDuty settings (delegated admin controls)
```

🔑 29.5.4 GuardDuty Automated Response Pattern

```
GuardDuty Finding: UnauthorizedAccess:EC2/SSHBruteForce
    │
    ▼
EventBridge Rule: source=aws.guardduty, detail-type=GuardDuty Finding
    │
    ▼
Lambda Function:
    1. Extract EC2 instance ID from finding
    2. Extract attacker source IP
    3. Add NACL DENY rule for attacker IP
    4. Isolate EC2: modify SG to deny all inbound (except forensic team SG)
    5. Create EBS snapshot (forensic evidence)
    6. Send SNS alert to security team
    7. Create Jira/ServiceNow incident
```

📋 29.6 AWS CloudTrail — Advanced

📖 Additional Details

This section supplements Chapter 24. Focus on advanced features tested at professional level.

🔑 29.6.1 Organization Trail vs Individual Trail

Aspect	Organization Trail	Individual Trail
Scope	All accounts in Organizations	Single account
Created in	Management account	Any account
Member account visibility	Read-only (cannot modify)	Full control
Coverage	All current AND future accounts	One account
S3 destination	Usually in Log Archive account	Any account
Use case	Enterprise audit, compliance	Single-account or dev/test

---

🔑 29.6.2 CloudTrail Lake

Managed event data lake for CloudTrail:

Feature	Detail
Storage	Managed (no S3 setup), 7-year retention option
Query	SQL (SELECT, WHERE, GROUP BY, etc.) directly on events
No ETL	No need for Glue + Athena table setup
Federation	Query CloudTrail + external event sources
Cost	$2.50/GB ingested (retention \leq 7yr)$;0.005/GB queried
Latency	Events queryable within minutes

CloudTrail Lake vs Athena on S3:

- **Lake:** Zero setup, higher cost, immediate SQL access
- **Athena on S3:** Lower cost ($0.50/TB scanned), requires Glue table, partition management

---

🔑 29.6.3 CloudTrail Insights

Anomaly detection on **write management events**:

- Detects: unusual API call rates, unusual error rates
- Example: `ConsoleLogin` calls spike from 10/hour to 10,000/hour → `InsightType: ApiCallRateInsight`
- Insight events delivered to same S3 bucket and CloudWatch Logs as trail
- Cost: ~$0.35/100,000 write management events analyzed

🔎 29.7 Amazon Macie

📖 Overview

Amazon Macie uses **ML and pattern matching** to automatically discover, classify, and protect sensitive data stored in **Amazon S3**.

Scope: Regional. Multi-account via Organizations.

Azure equivalent: Microsoft Purview (data classification, sensitivity labels, governance across Azure data stores).

Linux equivalent: OpenDLP (open-source data loss prevention), `grep` /regex scripts — not comparable in scale or automation.

🔑 29.7.1 Finding Types

Category	Finding Type	Meaning
Sensitive Data	`SensitiveData:S3Object/Personal`	PII detected in S3 object
Sensitive Data	`SensitiveData:S3Object/Credentials`	AWS keys, passwords in S3 object
Sensitive Data	`SensitiveData:S3Object/Financial`	Credit card numbers, bank account data
Sensitive Data	`SensitiveData:S3Object/Medical`	Health data (HIPAA PHI)
Policy	`Policy:IAMUser/S3BlockPublicAccessDisabled`	S3 Block Public Access turned off
Policy	`Policy:IAMUser/S3BucketPubliclyAccessible`	S3 bucket is publicly accessible
Policy	`Policy:IAMUser/S3BucketSharedExternally`	Bucket accessible by external AWS account

🔑 29.7.2 Custom Data Identifiers

For organization-specific sensitive data patterns:

```
Name: EmployeeID
Regex: EMP-[0-9]{6}
Keywords: ["employee", "staff", "personnel"]
Maximum match distance: 50 characters

→ Macie flags any S3 object containing "EMP-123456" near "employee"
```

Use cases:

- Internal project codes
- Customer contract numbers
- Proprietary data formats specific to your industry

🔑 29.7.3 Macie Automated Response

```
Macie Finding: SensitiveData:S3Object/Credentials
    │
    ▼
EventBridge Rule → Lambda:
    1. Identify S3 object with exposed credentials
    2. Move object to quarantine bucket (restricted access)
    3. Revoke/rotate any found credentials (if AWS keys)
    4. Alert security team via SNS
    5. Tag S3 bucket for compliance review
```

🪜 29.8 AWS Control Tower

📖 Overview

AWS Control Tower is a **managed landing zone service** that sets up and governs a secure, well-architected multi-account AWS environment. It is **opinionated** — follows AWS best practices for multi-account architecture.

Scope: Regional (landing zone deployed in a home region), but manages accounts across regions.

Azure equivalent: Azure Landing Zone Accelerator (ARM/Bicep templates) — conceptually similar, but not a managed service (it deploys templates, not a control plane).

🔑 29.8.1 Landing Zone Architecture

```
flowchart TD
    ROOT[Root OU] --> SECOU[Security OU]
    ROOT --> SBOU[Sandbox OU]
    ROOT --> WLOU[Workloads OU]

    SECOU --> LOGA[Log Archive Account\nAll CloudTrail + Config logs]
    SECOU --> AUDIT[Audit Account\nSecurity tooling, cross-account roles]

    SBOU --> DEV1[Developer Sandbox 1]
    SBOU --> DEV2[Developer Sandbox 2]

    WLOU --> DEVOU[Dev OU]
    WLOU --> TESTOU[Test OU]
    WLOU --> PRODOU[Prod OU]

    DEVOU --> DEVACCT[Dev Account]
    PRODOU --> PRODACCT[Prod Account]
```

🔑 29.8.2 Guardrail Types

Type	Mechanism	Example	Mandatory?
Preventive	SCP	Disallow deletion of CloudTrail log archive	Mandatory / Elective
Detective	Config Rule	Detect if MFA not enabled for root	Mandatory / Strongly Rec / Elective
Proactive	CloudFormation Hook	Block S3 bucket creation without encryption	Elective

Mandatory guardrails (always enforced, cannot disable):

- Disallow changes to CloudTrail set up by Control Tower
- Disallow changes to Config recording set up by Control Tower
- Disallow deletion of Log Archive

🔑 29.8.3 Account Factory

Account Factory = **self-service account vending** via AWS Service Catalog:

```
Developer requests new account → Service Catalog → Account Factory:
    1. Creates new AWS account via Organizations
    2. Enrolls account in Control Tower
    3. Applies mandatory guardrails
    4. Sets up Identity Center access
    5. Configures Config recorder
    6. Creates CloudTrail trail

Time: ~20-30 minutes automated
```

Account Factory for Terraform (AFT):

- Git repo defines account configurations (HCL)
- Pull request to repo → CodePipeline → Account Factory vends account
- IaC-driven account vending for large organizations

🔑 29.8.4 Drift Detection

Control Tower detects **drift** when:

- Someone manually edits a Control Tower-managed SCP
- Someone manually deletes a Config rule created by Control Tower
- Account is removed from the org outside Control Tower

Drift → Alert in Control Tower console → "Re-register OU" to remediate.

> ⚠️ **Exam trap:** Never modify Control Tower-managed SCPs or Config rules directly in Organizations/Config console. Use Control Tower's own console or Customizations for Control Tower (CfCT) for additions.

📊 29.9 AWS Security Hub

📖 Overview

AWS Security Hub is a **centralized security posture management (CSPM) service** that aggregates, normalizes, and prioritizes security findings from multiple AWS security services and third-party tools.

Scope: Regional (with optional cross-region aggregation to a home region).

Azure equivalent: Microsoft Defender for Cloud — Security Posture Management + Workload Protection plans.

🔑 29.9.1 Security Hub Architecture

```
flowchart LR
    GD[GuardDuty\nFindings] --> SH[Security Hub]
    MACIE[Macie\nFindings] --> SH
    IA[IAM Access\nAnalyzer] --> SH
    INSP[Amazon\nInspector] --> SH
    FW[Firewall\nManager] --> SH
    CFG[Config\nNon-compliant] --> SH
    TP[3rd Party\nSplunk / CrowdStrike] --> SH

    SH --> DASH[Security Hub\nDashboard]
    SH --> ASFF[ASFF Normalized\nFindings]
    SH --> EB[EventBridge\nAutomated Response]
    SH --> AGG[Cross-Account\nAggregation]
```

🔑 29.9.2 Security Standards

Standard	What It Checks
AWS Foundational Security Best Practices	200+ AWS service-specific controls
CIS AWS Foundations Benchmark v1.4	CIS hardening controls for AWS accounts
PCI DSS v3.2.1	Payment Card Industry requirements
NIST SP 800-53 Rev 5	US federal security controls
AWS NIST Cybersecurity Framework	NIST CSF controls mapped to AWS

Security Hub runs these checks using Config rules — **Config must be enabled** for Security Hub standards to work.

🔑 29.9.3 ASFF (Amazon Security Finding Format)

Common JSON schema for all Security Hub findings:

```
{
  "SchemaVersion": "2018-10-08",
  "Id": "arn:aws:securityhub:...",
  "ProductArn": "arn:aws:securityhub::product/aws/guardduty",
  "GeneratorId": "arn:aws:guardduty:...",
  "AwsAccountId": "123456789012",
  "Severity": { "Label": "HIGH", "Normalized": 70 },
  "Title": "EC2 instance is communicating with a known C2 server",
  "Resources": [{ "Type": "AwsEc2Instance", "Id": "arn:aws:ec2:..." }],
  "Workflow": { "Status": "NEW" },
  "RecordState": "ACTIVE"
}
```

Workflow statuses: `NEW` → `NOTIFIED` → `SUPPRESSED` / `RESOLVED`

🏛️ 29.10 Delegated Administrators in AWS Organizations

📖 Overview

Delegated administrator allows a **member account** to manage specific AWS services on behalf of the entire organization — keeping the management account minimally used.

Why important: The management account is the most privileged account. If compromised, the attacker controls the entire org. Minimize services running in the management account.

🔑 29.10.1 Delegated Admin by Service

Service	Delegated Admin Limit	Typical Account
Security Hub	1 per region	Security/Audit account
GuardDuty	1 per region	Security account
Macie	1 per region	Security account
AWS Config	1 aggregator account	Security/Audit account
IAM Access Analyzer	1 per region	Security account
Firewall Manager	1 per org	Network/Security account
AWS Backup	1 per region	Operations account
Account Management	Multiple	Finance/Admin accounts

🔑 29.10.2 Enabling Delegated Admin

```
# Step 1: Enable trusted access for the service
aws organizations enable-aws-service-access \
  --service-principal guardduty.amazonaws.com

# Step 2: Register delegated administrator
aws organizations register-delegated-administrator \
  --account-id 111111111111 \
  --service-principal guardduty.amazonaws.com

# Step 3: In the delegated admin account, enable the service for the org
# (Done in GuardDuty/Security Hub/etc. console in that account)
```

🔐 29.11 AWSSecurityAudit Permission Set

📖 Overview

`SecurityAudit` is an **AWS-managed IAM policy** providing read-only access to security-relevant configuration across AWS services. It is commonly used as an Identity Center **permission set** for cross-account security auditing.

ARN: `arn:aws:iam::aws:policy/SecurityAudit`

🔑 29.11.1 What SecurityAudit Grants

Read access to security configurations of:

- IAM (users, roles, policies, access keys, MFA status)
- CloudTrail (trails, event selectors, log file validation)
- AWS Config (rules, recorders, delivery channels)
- VPC (security groups, NACLs, route tables, flow logs)
- S3 (bucket policies, ACLs, public access settings)
- KMS (key metadata, key policies — NOT key material)
- GuardDuty, Security Hub, Macie (findings, settings)
- CloudWatch (alarms, log groups — NOT log contents)
- EC2 (instance metadata, AMIs, snapshots — NOT data)

🔑 29.11.2 SecurityAudit vs ReadOnlyAccess

Aspect	SecurityAudit	ReadOnlyAccess
Focus	Security configuration	All AWS service data
S3 object read	❌ Cannot read object contents	✅ Can read S3 object contents
KMS decrypt	❌ Cannot decrypt	❌ Cannot decrypt (neither policy grants decrypt)
CloudWatch Logs read	Metadata only	✅ Can read log events
RDS data read	❌ Cannot query DB	❌ Cannot query DB
Risk	Low (config-only, no data)	Higher (can access data-plane)
Use case	Security auditor, compliance	Developer read-only access

> 🔑 **Exam use case:** "External auditor needs to assess security controls across 50 accounts without accessing business data" → Identity Center permission set using `SecurityAudit` policy, deployed to all accounts. Zero data access risk.

🔑 29.11.3 Cross-Account Audit Pattern

```
Security/Audit Account
    └── Identity Center permission set: SecurityAuditAccess (uses SecurityAudit policy)
            └── Assigned to: Security-Team group
            └── Accounts: ALL (all 200 accounts)

Security analyst logs in → Identity Center portal →
Selects any of 200 accounts → Assumes AWSReservedSSO_SecurityAuditAccess_xxx role →
Full read-only security config visibility in that account
```

🔬 29.12 IAM Access Analyzer

📖 Overview

IAM Access Analyzer identifies **resources shared with external principals** (outside your account or organization) using automated reasoning (formal logic, not ML). It also provides **policy validation** and **policy generation** from CloudTrail activity.

Scope: Regional analyzer per account, or Organization-level analyzer.

Azure equivalent: Azure IAM Access Review (periodic review process), Azure Resource Graph (query access policies).

🔑 29.12.1 Analyzer Types

Type	Zone of Trust	"External" means
Account analyzer	Your account	Any principal outside your account
Organization analyzer	Your organization	Any principal outside your organization

> 🔑 **Exam key:** Use **organization-level analyzer** when you want to find resources accessible outside your org. An account-level analyzer would flag cross-account access within the org as "external" — generating false positives for legitimate cross-account sharing within your organization.

🔑 29.12.2 Resources Analyzed

Resource	What Access Analyzer Checks
S3 Buckets	Bucket policy, ACL granting access to external principals
IAM Roles	Trust policy allowing external account to assume the role
KMS Keys	Key policy allowing external account access
Lambda Functions	Resource-based policy allowing external invoke
SQS Queues	Queue policy allowing external account to send/receive
Secrets Manager Secrets	Secret policy allowing external access
SNS Topics	Topic policy allowing external publish/subscribe

🔑 29.12.3 Policy Generation

Access Analyzer analyzes **CloudTrail events** to generate a **least-privilege IAM policy**:

```
1. Enable policy generation for IAM user/role
2. User performs normal work for 1-90 days
3. Access Analyzer analyzes CloudTrail: what APIs were actually called?
4. Generates IAM policy containing only those actions + resources
5. Review and apply the generated policy

Result: IAM policy based on actual usage, not guessed permissions
```

> 🔑 **Exam use case:** "Right-size overly permissive IAM roles" → IAM Access Analyzer policy generation.
> Alternative: AWS IAM Access Advisor (shows last-used dates per service) — simpler but less granular.

🔑 29.12.4 Policy Validation

Before applying an IAM policy, validate it:

```
aws accessanalyzer validate-policy \
  --policy-document file://policy.json \
  --policy-type IDENTITY_POLICY
```

Returns:

- **Errors:** Policy syntax errors (would be rejected by IAM)
- **Security warnings:** Overly permissive (wildcard actions, wildcard resources)
- **Warnings:** Best practice violations
- **Suggestions:** Recommendations for improvement

🔑 29.12.5 Unused Access Analyzer

Identifies:

- IAM roles not used in last N days
- IAM users with unused access keys
- IAM users with unused passwords
- IAM roles with unused permissions (granted but never used)

Use to right-size permissions and decommission unused access. Cost: per IAM resource analyzed per month.

📁 29.13 IAM Identity Center — Built-in Directory

📖 Overview

Identity Center includes its own **managed directory** — no external IdP required. Users and groups are managed directly in the Identity Center console.

When to use: Small organizations (< ~50 users), proof of concept, break-glass emergency accounts.

🔑 29.13.1 Identity Sources Comparison

Identity Source	Management	SCIM Sync	SSO to Non-AWS	MFA	Best For
Identity Center Directory	Manual in console	N/A (native)	Limited (SAML SP-init)	TOTP, WebAuthn, Email OTP	< 50 users, no existing IdP
AD Connector	On-premises AD	N/A (live proxy)	No	AD-based	On-premises AD, no cloud IdP
AWS Managed AD	AWS-hosted AD	N/A	Limited	AD-based	Cloud-primary, needs AD services
External IdP (Okta/Azure AD)	IdP admin console	✅ SCIM auto-sync	✅ Full SSO	IdP-managed	Enterprises with existing IdP

🔑 29.13.2 MFA Options in Identity Center

MFA Method	Device	Phishing Resistant	Setup
TOTP	Google Authenticator, Authy	No	User scans QR code
WebAuthn / FIDO2	YubiKey, Touch ID, Face ID, passkeys	✅ Yes	User registers hardware key or biometric
Email OTP	Any device with email	No	Fallback method

> 🔑 **Exam tip:** For highest security MFA → WebAuthn/FIDO2 (phishing-resistant, cannot be intercepted by MITM unlike TOTP). Required for high-security accounts (finance, security team, privileged access).

🔑 29.13.3 Built-in Directory Limitations

- No SCIM sync from external HR systems (manual user management)
- No SSO to non-AWS SAML SP applications (limited compared to full IdP)
- Password reset flow is Identity Center-managed (not integrated with corporate systems)
- Cannot sync groups from corporate directory automatically
- Suitable for break-glass accounts even when using external IdP: always maintain 1-2 Identity Center native users as emergency access (no external IdP dependency)

📐 29.14 Well-Architected Framework — Security Add-Ons

Pillar	Security Add-Ons Implementation
Operational Excellence	Config conformance packs as code (CloudFormation). Automated remediation via SSM Automation. Security Hub dashboards for daily security posture review. GuardDuty findings → EventBridge → ticketing system (Jira/ServiceNow). Control Tower Account Factory for repeatable, compliant account provisioning.
Security	ACM for zero-touch cert management (no cert expiry incidents). CloudHSM for regulatory HSM-exclusive requirements. GuardDuty always-on threat detection (no infrastructure). Macie for PII discovery and data classification. IAM Access Analyzer to find external resource exposure. Security Hub aggregates all findings in one place. Config + conformance packs for continuous compliance. SecurityAudit permission set for auditor access without data risk.
Reliability	ACM auto-renewal prevents cert-expiry outages. GuardDuty malware protection isolates infected instances automatically. Control Tower ensures all accounts have baseline guardrails (no "forgotten" accounts with weak security). Config remediation auto-fixes configuration drift before it causes failures. CloudHSM cluster (2+ HSMs in 2 AZs) for HSM HA.
Performance Efficiency	ACM cert auto-rotation transparent to ALB/CloudFront (zero-downtime). GuardDuty: no agents = no CPU/memory overhead on EC2. Config: recording is lightweight. Security Hub: aggregation, not active scanning. S3 Bucket Key for ACM/KMS (reduces KMS API calls 99%).
Cost Optimization	ACM public certs free. Config: pay per item recorded (exclude non-security resources from recording). GuardDuty: tiered pricing, turn off unused protection plans (e.g., EKS Protection if not using EKS). Security Hub: first 10,000 findings free/month. Macie: targeted scanning (specific buckets) vs all-buckets. CloudHSM: minimum 2 HSMs needed — only use if compliance truly requires it (KMS is much cheaper).
Sustainability	Managed services (GuardDuty, Config, Macie, Security Hub) eliminate need for self-managed security tooling infrastructure. Automated remediation reduces human intervention. Centralized delegated admin reduces management overhead across accounts.

🎯 29.15 SAP-C02 Domains — Security Add-Ons

Domain 1 — Design Solutions for Organizational Complexity: Multi-account security governance: Control Tower landing zone + Security Hub delegated admin + GuardDuty org-wide + Config aggregator in audit account. Delegated admins for all security services in Security account. SecurityAudit permission set via Identity Center for auditors. Organization-level IAM Access Analyzer for external resource exposure. Macie org-wide for PII discovery across all S3 buckets.

Domain 2 — Design for New Solutions: New web application requiring SSL/TLS: ACM cert (DNS validation) + ALB HTTPS listener (same-region cert). New CloudFront distribution: ACM cert in us-east-1. New compliance requirement for IDS: GuardDuty (enable all protection plans). New internal PKI for microservices mTLS: ACM Private CA + ALB mTLS truststore. New multi-account environment: Control Tower landing zone.

Domain 3 — Continuous Improvement: Improve visibility: Security Hub + Config conformance packs + GuardDuty Insights. Right-size IAM permissions: IAM Access Analyzer policy generation from CloudTrail. Detect PII in S3: enable Macie with automated sensitive data discovery. Migrate from on-premises HSM to AWS: CloudHSM (exact compliance parity) or KMS BYOK (if compliance allows). Remove manual cert management: migrate to ACM (auto-renewal).

Domain 4 — Migration and Modernization: Migrate manual cert management (DigiCert with manual renewal) to ACM (import existing cert → migrate to ACM-managed → enable auto-renewal). Migrate on-premises security monitoring (Splunk) to AWS: GuardDuty + Security Hub + EventBridge + Firehose → Splunk integration. Migrate manual account provisioning to Account Factory (Control Tower). Migrate per-account security tooling to centralized org-level services.

🔗 29.16 Related Services Summary

Service	Relationship to Security Add-Ons
AWS WAF	Works with ACM certs on CloudFront/ALB. WAF rules applied at same layer as TLS termination.
AWS Shield	DDoS protection for services using ACM certs (CloudFront, ALB).
AWS KMS	ACM Private CA uses KMS CMK to protect CA private key. CloudHSM alternative to KMS for key storage. GuardDuty findings about KMS activity.
AWS CloudTrail	GuardDuty analyzes CloudTrail events. IAM Access Analyzer uses CloudTrail for policy generation. Config records API calls (with CloudTrail as source). Security Hub ingests CloudTrail-based Config findings.
AWS EventBridge	Automation hub: GuardDuty/Macie/Security Hub findings → EventBridge → Lambda/SNS/SSM for response.
AWS Lambda	Config custom rules, GuardDuty/Macie automated response, Athena Federated Query connectors.
AWS Organizations	Foundation for multi-account: Control Tower, Security Hub, GuardDuty, Macie, Config, IAM Access Analyzer all use Organizations for multi-account deployment.
AWS IAM Identity Center	SecurityAudit permission set deployed to all accounts for auditors. Built-in directory for break-glass.
Amazon S3	Macie scans S3. Config delivers snapshots to S3. CloudTrail delivers logs to S3. ACM certificate for S3 static website (via CloudFront).
Amazon Inspector	Vulnerability assessment (EC2, Lambda, containers) — feeds findings to Security Hub. Complements GuardDuty (runtime threats) and Config (configuration compliance).

🔒 29.X Service Control Policies (SCPs) vs IAM Permission Boundaries

📖 29.X.1 Overview

Both SCPs and Permission Boundaries are **restrictive guardrails** — they set a ceiling on what permissions can be used. Neither grants permissions. Both must allow an action alongside identity policies for it to succeed.

They operate at **different levels** of the IAM permission model.

📊 29.X.2 Side-by-Side Comparison

Aspect	Service Control Policies (SCPs)	IAM Permission Boundaries
What they apply to	All IAM entities in an AWS Organizations account or OU	A single specific IAM user or role
Set by	Organization management account / delegated admin	IAM administrator in the same account
Scope	All principals in the account (users, roles)	Only the entity they're directly attached to
Purpose	Prevent member accounts from exceeding org-wide policy	Delegate IAM creation without allowing privilege escalation
Grants permissions?	No — restricts only	No — restricts only
Affects account root user?	Yes (member account root), No (management account root)	No — root user never affected
Affects resource-based policies?	Cross-account only (SCP of caller's account applies)	No — resource-based policies not affected
Where defined	AWS Organizations (management account / delegated admin)	IAM — `PermissionsBoundary` field on user or role
Who can remove it	Org admin only — member account cannot override	IAM admin with `iam:PutUserPermissionsBoundary` permission
Inheritance	Inherits down the OU hierarchy	Not inherited — per-entity attachment

---

⚙️ 29.X.3 Effective Permissions Formula

Every AWS authorization decision evaluates all applicable policies:

```
Effective Permissions = Identity Policy ∩ SCP ∩ Permission Boundary ∩ Session Policy
```

All layers must **allow** an action. Any single layer that **denies** or does not allow → request denied.

```
┌──────────────────────────────────────────────────┐
│  REQUEST: iam:CreateRole                         │
│                                                  │
│  SCP:                ✅ allows iam:CreateRole    │
│  Permission Boundary: ✅ allows iam:CreateRole   │
│  Identity Policy:     ✅ allows iam:CreateRole   │
│                                                  │
│  RESULT: ✅ ALLOWED                              │
└──────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────┐
│  REQUEST: ec2:RunInstances in us-west-2          │
│                                                  │
│  SCP:                ❌ denies all except eu-central-1 │
│  Permission Boundary: ✅ allows ec2:RunInstances │
│  Identity Policy:     ✅ allows ec2:RunInstances │
│                                                  │
│  RESULT: ❌ DENIED (SCP blocks it)               │
└──────────────────────────────────────────────────┘
```

🎯 29.X.4 Use Cases

SCPs — organization-wide guardrails:

```
// Restrict all dev OU accounts to eu-central-1 only
{
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {
    "StringNotEquals": {
      "aws:RequestedRegion": "eu-central-1"
    }
  }
}
```

Common SCP patterns:

- Prevent CloudTrail deletion across all member accounts
- Block IAM user creation (enforce SSO-only access)
- Prevent leaving AWS Organizations
- Restrict EC2 instance types to cost-approved sizes
- Deny disabling Security Hub / GuardDuty

Permission Boundaries — safe IAM delegation:

```
// Boundary: DeveloperBoundaryPolicy
// Allows developer to assume only dev-scoped roles
{
  "Effect": "Allow",
  "Action": ["s3:*", "lambda:*", "logs:*", "dynamodb:*"],
  "Resource": "*"
}
```

Then developer can create Lambda execution roles — but any role they create **cannot exceed** the boundary. Even if they attach `AdministratorAccess` to a role they create, the boundary caps it.

📐 29.X.5 Architecture Diagrams

```
flowchart TD
    MGMT[Management Account
Org Root] -->|SCP: deny non-eu-central-1| PROD_OU[Production OU]
    MGMT -->|SCP: deny ec2:RunInstances for t3.large+| DEV_OU[Dev OU]
    PROD_OU --> ACC_A[Account A
All IAM entities capped by SCP]
    PROD_OU --> ACC_B[Account B
All IAM entities capped by SCP]
    DEV_OU --> ACC_C[Account C - Dev
All IAM entities capped by SCP]

    subgraph Account C
        ADMIN[IAM Admin
sets Permission Boundary] -->|attach boundary| DEV_ROLE[Developer Role
Boundary = DeveloperPolicy]
        DEV_ROLE -->|can create| NEW_ROLE[New Lambda Role
capped at DeveloperPolicy]
    end
```

📊 29.X.6 Key Differences Summary

Question	SCP	Permission Boundary
"Block entire account from a region?"	✅ Yes	❌ No — account-wide
"Prevent one role from escalating?"	❌ Blunt — affects all	✅ Yes — per-entity
"Enforce across 100 accounts?"	✅ Apply to OU	❌ Must attach to each entity
"Safe IAM delegation to dev team?"	❌ Not granular enough	✅ Yes — canonical use case
"Prevent disabling GuardDuty?"	✅ SCP on all accounts	❌ Would need per-role boundary
"Limit a contractor role scope?"	❌ Affects all in account	✅ Attach to contractor role only

---

⚠️ 29.X.7 Exam Traps

1. **"SCPs grant permissions"** → FALSE. SCPs only restrict. You still need IAM identity policies to allow actions.
2. **"SCP denies override explicit IAM Allow"** → TRUE. SCP Deny → action blocked regardless of IAM policy.
3. **"Management account is restricted by SCPs"** → FALSE. Management account is exempt from SCPs.
4. **"Permission boundary allows access to S3, therefore the role can access S3"** → FALSE. Identity policy must also allow S3. Boundary alone is not sufficient.
5. **"SCP applied to OU — does it affect the management account?"** → No. Management account never subject to SCPs.
6. **"Developer creates a role with AdministratorAccess — but has a permission boundary"** → Role effective permissions = AdministratorAccess ∩ Boundary. If boundary only allows S3+Lambda, role can only do S3+Lambda.
7. **"Prevent all IAM users from disabling CloudTrail in all 50 accounts"** → SCP applied to root OU (one policy, all accounts). Permission boundary would require attaching to every IAM user — not scalable.
8. **"Session policies"** → STS assume-role session policies further restrict temporary credentials — same ∩ logic applies.

🏛️ 29.X.8 Well-Architected — SCPs and Permission Boundaries

Pillar	SCP	Permission Boundary
Security	Enforce least-privilege at account level. Prevent lateral movement between accounts.	Safe IAM delegation — no privilege escalation via role creation.
Operational Excellence	Apply once to OU — propagates automatically to all new accounts.	Enforce via IAM policy: require `iam:PutRolePermissionsBoundary` before `iam:CreateRole` .
Reliability	Prevent accidental disable of logging/monitoring services.	Prevent misconfigured roles with over-broad permissions causing outages.
Cost Optimization	SCPs blocking expensive instance types or regions save cost org-wide.	Boundary limiting dev roles to dev-sized resources prevents cost overruns.
Sustainability	—	—
Performance	No performance impact — evaluated at authorization, not runtime.	No performance impact.

📜 29.Y AWS Policy Types — Complete Guide

📖 29.Y.1 Overview

AWS authorization is a **multi-layer evaluation engine**. Every API call passes through up to seven distinct policy types. Understanding each type — what it controls, where it lives, who sets it, and how it interacts with others — is essential for both real-world security design and the SAP-C02 exam.

```
┌──────────────────────────────────────────────────────────────────┐
│                   AWS Authorization Decision Engine              │
│                                                                  │
│  1. SCP (org-level ceiling)                                      │
│  2. RCP (org-level resource ceiling)                             │
│  3. Permission Boundary (entity-level ceiling)                   │
│  4. Identity-based Policy (what the principal is allowed to do)  │
│  5. Resource-based Policy (what can be done TO the resource)     │
│  6. VPC Endpoint Policy (what can transit through the endpoint)  │
│  7. Session Policy (what this specific session is scoped to)     │
│  8. ACL (cross-account S3/network legacy control)                │
│                                                                  │
│  ALLOW only when ALL applicable layers allow AND no explicit Deny│
└──────────────────────────────────────────────────────────────────┘
```

🆔 29.Y.2 Entity vs. Principal

These two terms are often confused. They overlap but are not identical.

Term	Definition	Examples
IAM Entity	An IAM object that can be created and managed in IAM — something that *exists* in the identity store	IAM user, IAM role, IAM group
IAM Principal	An actor that can make an API request — something that *authenticates and acts*	IAM user, IAM role (assumed), AWS service, federated user, anonymous public

Key distinctions:

- IAM **group** = entity (exists in IAM) but **not** a principal (groups cannot make API calls directly)
- AWS **service principals** (e.g., `lambda.amazonaws.com`) = principals but not IAM entities (they don't live in your IAM)
- A **federated user** (via SAML/OIDC) = principal (can make calls) but not an IAM entity (not persisted in your IAM)
- An **assumed role session** = principal (ARN: `arn:aws:sts::123:assumed-role/MyRole/SessionName`) but the underlying entity is the role

```
IAM Entities ⊂ Principals (all entities are principals)
But Principals ⊃ Entities (not all principals are entities)

Principals include:
  ├── IAM entities: users, roles (groups are NOT principals)
  ├── AWS service principals: lambda.amazonaws.com
  ├── Federated principals: arn:aws:iam::123:federated-user/Alice
  ├── Assumed role sessions: arn:aws:sts::123:assumed-role/Role/Session
  └── Anonymous: * (in resource policies = unauthenticated public access)
```

> 🔑 **Exam key:** "Principal" in a resource-based policy can be `*` (everyone), a specific ARN, or an AWS service. "Entity" is only used for objects that exist inside your IAM account.

📋 29.Y.3 Identity-Based Policies

Identity-based policies are **attached to IAM entities** (users, roles, groups). They define what actions that entity is allowed or denied to perform on which resources.

Two Types: Managed vs Inline

Feature	AWS Managed Policy	Customer Managed Policy	Inline Policy
Created by	AWS	You	You (embedded)
Reusable	Yes — attach to many entities	Yes — attach to many entities	No — 1:1 with one entity
Version history	Yes (up to 5 versions)	Yes (up to 5 versions)	No
Visible in IAM	Yes	Yes	No (hidden inside entity)
Delete entity	Policy remains	Policy remains	Policy deleted with entity
Use case	Standard policies (ReadOnlyAccess, PowerUserAccess)	Custom org policies	Strict 1:1 permission lockdown
SAP-C02 recommendation	Use AWS Managed for standard roles	Use Customer Managed for custom permissions	Avoid except for strict coupling requirement

AWS Managed Policy examples:

- `AdministratorAccess` — full access everything
- `ReadOnlyAccess` — read everything
- `AWSSecurityAuditPolicy` — security audit read
- `AmazonS3FullAccess` — full S3

Policy structure:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowS3ReadInEU",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::my-eu-bucket",
        "arn:aws:s3:::my-eu-bucket/*"
      ],
      "Condition": {
        "StringEquals": {
          "aws:RequestedRegion": "eu-central-1"
        }
      }
    }
  ]
}
```

Evaluation for identity-based policy (same account, no resource policy):

```
Action allowed if:
  ✅ At least one Statement with Effect=Allow matches
  ❌ No Statement with Effect=Deny matches
  ❌ No SCP Deny
  ❌ No Boundary Deny
```

🏢 29.Y.4 Resource-Based Policies

Resource-based policies are **attached to the resource itself** (not to the principal). They define who can access the resource and what they can do.

Services that support resource-based policies:

Service	Resource Policy Name
S3	Bucket Policy
KMS	Key Policy (required — KMS has no other way to grant access)
Lambda	Function Policy (resource policy)
SQS	Queue Policy
SNS	Topic Policy
ECR	Repository Policy
Secrets Manager	Resource Policy
IAM Role	Trust Policy (controls who can assume the role — `sts:AssumeRole`)
API Gateway	Resource Policy
CloudWatch Logs	Resource Policy
Glacier	Vault Access Policy

Key behaviors:

```
Same account:
  Allow in EITHER identity policy OR resource policy → access granted
  (resource policy acts like an additional grant — OR logic)

Cross-account:
  BOTH identity policy (in requester account) AND resource policy (on resource) must allow
  AND SCPs in both accounts must allow
  (AND logic for cross-account)
```

Example — S3 Bucket Policy allowing cross-account access:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "AWS": "arn:aws:iam::999999999999:role/DataPipelineRole"
    },
    "Action": ["s3:GetObject"],
    "Resource": "arn:aws:s3:::my-bucket/*"
  }]
}
```

KMS Key Policy — special case:

KMS **requires** an explicit key policy. Without a key policy granting access, even AdministratorAccess cannot use the key. The default key policy grants the root user full access, enabling IAM policies to delegate.

```
{
  "Effect": "Allow",
  "Principal": { "AWS": "arn:aws:iam::123456789012:root" },
  "Action": "kms:*",
  "Resource": "*"
}
```

> 🔑 **Exam key:** If a KMS key policy does NOT include the root account, even the account root cannot recover access — you must contact AWS Support.

🕯️ 29.Y.5 VPC Endpoint Policies

VPC Endpoint Policies control **what traffic is allowed to pass through a VPC endpoint** to an AWS service. They act as a filter — even if the IAM identity policy allows an action, the endpoint policy can restrict which specific resources or actions are accessible via that endpoint.

Two types of VPC endpoints:

- **Gateway Endpoint** (S3, DynamoDB) — endpoint policy on the gateway
- **Interface Endpoint / PrivateLink** (most other services) — endpoint policy on the interface endpoint

Evaluation logic:

```
Request via VPC endpoint:
  ✅ Identity policy allows
  ✅ VPC Endpoint policy allows
  ✅ Resource policy allows (if applicable)
  → Access granted

Endpoint policy RESTRICTS — it does not grant. Cannot allow more than identity policy.
```

Example — restrict S3 gateway endpoint to only allow access to specific bucket:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": "*",
    "Action": ["s3:GetObject", "s3:PutObject"],
    "Resource": "arn:aws:s3:::company-data-bucket/*"
  }]
}
```

Any VPC traffic attempting to access other S3 buckets via this endpoint is **denied**, even if the IAM policy allows it.

Restrict S3 bucket to VPC endpoint only (bucket policy condition):

```
{
  "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:*",
  "Resource": ["arn:aws:s3:::my-bucket", "arn:aws:s3:::my-bucket/*"],
  "Condition": {
    "StringNotEquals": {
      "aws:SourceVpce": "vpce-0123456789abcdef0"
    }
  }
}
```

> 🔑 **Exam key:** `aws:SourceVpce` condition in S3 bucket policy = "only accessible from this specific VPC endpoint." Prevents data exfiltration via public internet even if credentials are compromised.

🔒 29.Y.6 IAM Permission Boundaries (Deep Dive)

(See full comparison with SCPs in section 29.X. This section covers mechanics.)

A Permission Boundary is a **managed policy attached to an IAM entity as a boundary**, not as a regular policy. It defines the **maximum permissions** that entity can ever have, regardless of what identity-based policies are attached.

What it does NOT do:

- Does not grant any permissions itself
- Does not restrict resource-based policies
- Does not affect other entities (only the one it's attached to)

- Does not affect SCPs (separate layer)

Mechanics — creating a role with boundary enforcement:

```
IAM Admin has: iam:CreateRole + condition iam:PermissionsBoundary = BoundaryArn

Developer (with boundary) calls iam:CreateRole:
  → Must specify PermissionsBoundary = BoundaryArn in request
  → Condition on admin's policy enforces this
  → New role's effective = role's identity policy ∩ BoundaryArn
  → Developer cannot create a role more powerful than the boundary
```

Delegation pattern with boundary:

```
Management creates:
  DeveloperRole (identity policy: s3:*, lambda:*, iam:CreateRole)
  DeveloperBoundary (allows: s3:*, lambda:*)

DeveloperRole policy condition:
  iam:CreateRole requires iam:PermissionsBoundary = DeveloperBoundaryArn

Result:
  Developer CAN create roles → but any role they create is capped at DeveloperBoundary
  Developer CANNOT create AdministratorAccess role because:
    - Created role gets DeveloperBoundary
    - Even if AttachRolePolicy gives AdminAccess → effective = AdminAccess ∩ DeveloperBoundary = DeveloperBoundary
```

🏛️ 29.Y.7 AWS Organizations Resource Control Policies (RCPs)

RCPs are a **newer policy type** in AWS Organizations (generally available 2024). They complement SCPs but control the **resource side** rather than the principal side.

Feature	SCP	RCP
Controls	What principals in member accounts can DO	What can be DONE TO resources in member accounts
Applied to	Principals (IAM entities)	Resources
Perspective	Principal-centric	Resource-centric
Blocks	Principals from taking actions	Actions taken on resources
Default	Full Allow (permissive)	Full Allow (permissive)
Use case	Prevent principals from taking actions in wrong region/service	Prevent external principals from accessing internal resources

RCP example — prevent cross-account access to S3 buckets from outside the org:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "Principal": "*",
    "Action": "s3:*",
    "Resource": "*",
    "Condition": {
      "StringNotEquals": {
        "aws:PrincipalOrgID": "o-exampleorgid"
      },
      "BoolIfExists": {
        "aws:PrincipalIsAWSService": "false"
      }
    }
  }]
}
```

This ensures no S3 bucket in any member account can be accessed by principals outside the organization — even if bucket policies allow it.

RCP vs SCP — complementary controls:

```
Without RCP: SCP blocks your principals, but external principals can still access your resources
             if resource policies allow it

With RCP:    BOTH your principals AND external access to your resources are governed

Combined:
  SCP → your people can't do unauthorized things externally
  RCP → outsiders can't do unauthorized things to your resources
```

> 🔑 **Exam key (2024+):** RCPs prevent data exfiltration at the org level. Even if a resource policy allows `"Principal": "*"`, the RCP can block external access entirely. Exam may test: "prevent all S3 data from being accessible outside the org" → RCP.

📋 29.Y.8 Access Control Lists (ACLs) — Detailed Explanation

ACLs are a **legacy access control mechanism** in AWS. They predate IAM and are now largely superseded by resource policies, but remain important to understand — especially S3 ACLs and network ACLs.

S3 ACLs

S3 ACLs control access at the **object or bucket level** using a grantee + permission model. They were the original S3 access control mechanism before bucket policies existed.

S3 ACL Grantees:

Grantee	Description
Canonical User ID	Unique identifier for an AWS account (used in S3 ACLs for cross-account)
AWS Account email	Grants access to another AWS account by email
Amazon S3 predefined group — AllUsers	`http://acs.amazonaws.com/groups/global/AllUsers` — public (unauthenticated + authenticated)
Amazon S3 predefined group — AuthenticatedUsers	Any authenticated AWS account (NOT just your account — essentially the whole world of AWS accounts)
Log Delivery group	Grants S3 access logging service permission to write logs

S3 ACL Permissions:

Permission	On Bucket	On Object
READ	List objects	Read object data + metadata
WRITE	Create, overwrite, delete objects	Not applicable to objects
READ_ACP	Read bucket ACL	Read object ACL
WRITE_ACP	Write bucket ACL	Write object ACL
FULL_CONTROL	All of the above	All of the above

Object Ownership and ACL interaction:

AWS introduced **Object Ownership** setting on S3 buckets:

Setting	Behavior
BucketOwnerEnforced (default, recommended)	ACLs disabled. Object ownership = bucket owner always. Use bucket policies for access control.
BucketOwnerPreferred	If uploader includes `bucket-owner-full-control` canned ACL, ownership transfers to bucket owner
ObjectWriter (legacy)	Uploader retains object ownership — original behavior

> 🔑 **Exam key: Modern AWS best practice = BucketOwnerEnforced (ACLs disabled)**. For cross-account uploads, use bucket policy instead of ACL. ACLs are legacy — exam may test "why can't the bucket owner see uploaded objects?" → Object Ownership = ObjectWriter; fix = BucketOwnerPreferred or BucketOwnerEnforced.

Canned ACLs — predefined shorthand:

Canned ACL	Who gets what
`private`	Owner only (default)
`public-read`	Owner full control + AllUsers READ
`public-read-write`	Owner full control + AllUsers READ + WRITE
`authenticated-read`	Owner full control + AuthenticatedUsers READ
`bucket-owner-read`	Object owner full control + bucket owner READ
`bucket-owner-full-control`	Object owner and bucket owner both get FULL_CONTROL

Network ACLs (NACLs)

NACLs are **stateless packet filters** at the subnet level in a VPC (covered in Chapter 27 — Network Security, but key points for policy comparison):

Feature	NACL	Security Group
Level	Subnet	ENI (instance/resource)
State	Stateless (must allow inbound + outbound explicitly)	Stateful (return traffic auto-allowed)
Allow + Deny	Both	Allow only (implicit deny)
Evaluation	Rules numbered 1–32766, lowest wins	All rules evaluated, most permissive wins
Default	Default NACL: allow all in/out	Default SG: allow all outbound, block all inbound
Applies to	All resources in subnet	Only resources with SG attached

NACL rule format:

```
Rule# | Type  | Protocol | Port Range | Source/Dest | Allow/Deny
100   | HTTP  | TCP      | 80         | 0.0.0.0/0   | ALLOW
200   | HTTPS | TCP      | 443        | 0.0.0.0/0   | ALLOW
300   | SSH   | TCP      | 22         | 10.0.0.0/8  | ALLOW
*     | ALL   | ALL      | ALL        | 0.0.0.0/0   | DENY   ← catch-all
```

> 🔑 **NACL exam key:** Ephemeral ports (1024–65535) must be allowed outbound for response traffic (stateless). Explicit deny rules can block specific IPs — use NACL not SG for deny rules.

🎭 29.Y.9 Session Policies

Session policies are IAM policies passed **at the time of role assumption** as inline JSON or ARN. They **further restrict** the permissions of the temporary session — they cannot grant more than the role's identity policy.

```
Session Effective = Role Identity Policy ∩ Role Permission Boundary ∩ Session Policy ∩ SCP
```

Session policies apply to **temporary credentials** obtained via STS. Three main STS operations differ in what session policies they accept and what they mean:

AssumeRole

```
Caller: IAM user, IAM role, AWS service
Use: internal role switching, cross-account access, privilege scoping

STS:AssumeRole(
  RoleArn = "arn:aws:iam::TARGET:role/TargetRole",
  RoleSessionName = "my-session",
  Policy = { inline session policy JSON }    ← optional, restricts further
  PolicyArns = [{ arn: "managed-policy" }]  ← optional, restrict to managed policy
)

Trust policy on TargetRole must explicitly allow the caller principal via sts:AssumeRole.

Session credentials: AccessKeyId, SecretAccessKey, SessionToken
Duration: 15 min – 12 hours (default 1 hour; max depends on role's MaxSessionDuration)
```

Cross-account AssumeRole flow:

```
Account A (requester)                Account B (resource)
  IAM Role/User                        Target Role
  Identity policy: sts:AssumeRole  →   Trust policy allows Account A role
  SCP: allows sts:AssumeRole       →   SCP Account B: allows assumed actions
  Optional Session Policy          →    Role identity policy
                                        Permission Boundary (if any)
                                   Result: intersection of all
```

AssumeRoleWithSAML

```
Caller: SAML IdP (Okta, Azure AD, ADFS) on behalf of federated user
Use: enterprise SSO — employees log in via corporate IdP, get AWS credentials

Flow:
  1. User logs into IdP (Okta/Azure AD)
  2. IdP authenticates user, generates SAML 2.0 Assertion
  3. SAML Assertion includes: NameID, AWS role ARN, session duration, attributes
  4. App/AWS calls STS:AssumeRoleWithSAML(RoleArn, SAMLAssertion, Policy?)
  5. STS validates assertion signature against IdP metadata (registered in AWS IAM)
  6. Returns temporary credentials

Trust policy condition:
{
  "Effect": "Allow",
  "Principal": { "Federated": "arn:aws:iam::123:saml-provider/OktaProvider" },
  "Action": "sts:AssumeRoleWithSAML",
  "Condition": {
    "StringEquals": {
      "SAML:aud": "https://signin.aws.amazon.com/saml"
    }
  }
}

Session policy: same as AssumeRole — further restricts session
Duration: up to 12 hours (controlled by SAML attribute + MaxSessionDuration)
```

Key difference from AssumeRole:

- No IAM user/role initiates — the **SAML Federation provider** is the principal
- Role selection can be in the SAML assertion itself (IdP sends `https://aws.amazon.com/SAML/Attributes/Role` attribute)
- User does not need an IAM entity — identity lives in the IdP

AssumeRoleWithWebIdentity

```
Caller: Web/mobile app using OIDC IdP (Cognito, Google, Apple, Facebook, any OIDC provider)
Use: mobile app users, web app users, K8s pod IRSA

Flow:
  1. User logs into OIDC IdP (Google, Cognito User Pool)
  2. IdP returns JWT (id_token or access_token)
  3. App calls STS:AssumeRoleWithWebIdentity(RoleArn, WebIdentityToken=JWT, Policy?)
  4. STS validates JWT against IdP's OIDC discovery endpoint (JWKS)
  5. Returns temporary credentials

Trust policy condition (Cognito User Pool example):
{
  "Effect": "Allow",
  "Principal": { "Federated": "cognito-identity.amazonaws.com" },
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
      "cognito-identity.amazonaws.com:aud": "us-east-1:pool-id"
    },
    "ForAnyValue:StringLike": {
      "cognito-identity.amazonaws.com:amr": "authenticated"
    }
  }
}

EKS IRSA trust policy:
{
  "Effect": "Allow",
  "Principal": {
    "Federated": "arn:aws:iam::123:oidc-provider/oidc.eks.us-east-1.amazonaws.com/id/EXAMPLED539D4633E53DE1B71EXAMPLE"
  },
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
      "oidc.eks.us-east-1.amazonaws.com/id/EXAMPLED:sub": "system:serviceaccount:default:s3-reader"
    }
  }
}

Session policy: same as others — further restricts session
Duration: up to 12 hours
```

Key difference from AssumeRoleWithSAML:

- JWT (OIDC) instead of SAML 2.0 XML assertion
- Common for mobile apps, SPA frontends, K8s pods
- Cognito Identity Pools simplify this flow (handles STS call for app)

Comparison — Three AssumeRole Variants

Feature	AssumeRole	AssumeRoleWithSAML	AssumeRoleWithWebIdentity
Caller identity	IAM entity or AWS service	SAML federation	OIDC token (JWT)
IdP type	N/A (AWS IAM)	SAML 2.0 (Okta, ADFS, Azure AD)	OIDC (Cognito, Google, Apple)
Token format	None (IAM credentials)	SAML XML Assertion	JWT (JSON Web Token)
Requires IAM user?	Yes (must have IAM identity)	No (identity in IdP)	No (identity in IdP)
Trust policy principal	IAM ARN or service	`"Federated": SAML-provider ARN`	`"Federated": OIDC-provider ARN`
Session policy allowed?	Yes	Yes	Yes
Max session duration	Up to MaxSessionDuration (12h)	Up to 12h	Up to 12h
Common use case	Cross-account, service-to-service	Enterprise SSO, corporate users	Mobile/web apps, Kubernetes IRSA

📊 29.Y.10 All Policy Types — Master Comparison Table

Policy Type	Attached To	Who Sets It	Grants Permissions?	Can Deny?	Cross-Account?	Scope
Identity-based (Managed)	IAM user/role/group	IAM admin	Yes	Yes (explicit)	N/A	Per entity
Identity-based (Inline)	IAM user/role/group	IAM admin	Yes	Yes (explicit)	N/A	Per entity
Resource-based	AWS resource (S3, KMS, SQS…)	Resource owner	Yes (same account), Required (cross-account)	Yes	Yes	Per resource
SCP	Org account / OU	Org management account	No	Yes (implicit + explicit)	N/A	All entities in account
RCP	Org account / OU	Org management account	No	Yes	Yes (restricts cross-account)	All resources in account
Permission Boundary	IAM user or role	IAM admin	No	No (caps max)	No	Per entity
VPC Endpoint Policy	VPC endpoint	VPC/network admin	No (restricts)	Yes (deny to restrict)	Partial	Per endpoint
Session Policy	STS session	Role-assumer	No (restricts)	Yes	N/A	Per session
S3 ACL	S3 object or bucket	Object/bucket owner	Yes (limited)	No	Yes	Per object/bucket
Network ACL	VPC subnet	VPC admin	Yes (allow)	Yes (deny)	N/A	Per subnet

⚠️ 29.Y.11 Exam Traps — All Policy Types

1. **"IAM group can be a Principal in a resource policy"** → FALSE. Groups cannot be principals. Use role or user ARNs.
2. **"Resource policy alone grants access cross-account"** → FALSE. Cross-account = resource policy AND identity policy of the requester must both allow.
3. **"NACL is stateful"** → FALSE. NACL is stateless. Must allow both inbound and outbound. SG is stateful.
4. **"KMS key policy grants root full access by default"** → The DEFAULT key policy does, but customer-created keys with custom policies may not. Without root in key policy, only key policy can grant access (IAM policies alone insufficient).
5. **"VPC endpoint policy grants additional permissions"** → FALSE. Endpoint policy restricts — it can only reduce what identity policy allows through the endpoint.
6. **"Session policy grants additional permissions beyond the role"** → FALSE. Session policy intersects with role — can only restrict, never expand.
7. **"AssumeRoleWithWebIdentity requires an IAM user"** → FALSE. OIDC token from Google/Cognito is sufficient — no IAM user needed.
8. **"AssumeRoleWithSAML and AssumeRoleWithWebIdentity are interchangeable"** → FALSE. SAML = XML assertion from enterprise IdP (Okta/ADFS). WebIdentity = JWT from OIDC provider (Cognito/Google). Different token formats.
9. **"AuthenticatedUsers ACL grantee = same AWS account users"** → FALSE. AuthenticatedUsers = ANY authenticated AWS account worldwide. Use bucket policies for account-scoped access.
10. **"RCP applies to the management account"** → FALSE. Like SCPs, RCPs do not apply to the management account.
11. **"Permission boundary on role affects roles assumed from it"** → FALSE. Boundary is on the specific entity; assumed-role sessions inherit the role's permissions but not the boundary (session policy is different).
12. **"S3 BucketOwnerEnforced disables ACLs"** → TRUE. With this setting, ACLs are disabled; all access control via bucket policy only.

🏛️ 29.Y.12 Well-Architected Framework — Policy Types

Pillar	Application
Security	Use SCPs for org-wide guardrails. Use RCPs to prevent data exfiltration. Use VPC endpoint policies to restrict service access to specific buckets/resources. Disable S3 ACLs (BucketOwnerEnforced). Use Permission Boundaries for safe IAM delegation. Never use `AuthenticatedUsers` ACL grantee.
Operational Excellence	Customer Managed Policies over inline (reusable, versioned, auditable). AWS Managed Policies for standard roles. Session policies scope contractor/temporary access without creating new roles.
Reliability	Resource-based policies on Lambda/SQS/SNS enable cross-service invocation without extra IAM plumbing. KMS key policy must include root to avoid lock-out.
Performance	No runtime impact — all policies evaluated at authorization time, not execution time.
Cost Optimization	VPC Endpoint policies + bucket policies preventing public access = avoid accidental data transfer costs. Managed policies reduce IAM management overhead.
Sustainability	Granular session policies reduce over-provisioned long-lived credentials.

📨 Chapter 30: Amazon SQS — Simple Queue Service

> **Covers:** Standard vs FIFO Queues · Visibility Timeout · DLQ · Long Polling · Lambda Integration · Extended Client Library · Security · Pricing · Limits

📋 Table of Contents (Chapter 30)

- 30.1 Overview
- 30.2 Queue Types — Standard vs FIFO
- 30.3 Core Concepts
- 30.4 Dead Letter Queue (DLQ)

- 30.5 Security
- 30.6 SQS + Lambda Integration
- 30.7 SQS Extended Client Library
- 30.8 SQS vs Other Messaging Services
- 30.9 Pricing
- 30.10 Limits
- 30.11 Exam Traps
- 30.12 Well-Architected Framework
- 30.13 SAP-C02 Domains
- 30.14 Related Services

📖 30.1 Overview

Amazon SQS is a **fully managed message queuing service** that decouples the components of a distributed application. Producers send messages to a queue; consumers poll the queue and process messages independently.

Key characteristics:

- **Pull-based** — consumers poll for messages (unlike SNS which pushes)
- **At-least-once delivery** (Standard) or **exactly-once** (FIFO)
- Messages stored **redundantly across multiple AZs** within a region
- No message broker to manage — fully serverless

Scope: Regional. Queue URLs are regional (`https://sqs.us-east-1.amazonaws.com/123456789/my-queue`). No cross-region queues natively — use replication patterns for DR.

Linux/Open-source equivalent: RabbitMQ (AMQP), Apache ActiveMQ, Redis Streams, Beanstalkd.

Azure equivalent: Azure Service Bus Queues (feature-rich, similar to SQS FIFO), Azure Storage Queues (simpler/cheaper, similar to SQS Standard).

```
flowchart LR
    P1[Producer\nEC2 / Lambda] -->|SendMessage| Q[(SQS Queue)]
    P2[Producer\nApp Server] -->|SendMessage| Q
    Q -->|ReceiveMessage\nlong poll| C1[Consumer\nLambda]
    Q -->|ReceiveMessage| C2[Consumer\nEC2 Worker]
    C1 -->|DeleteMessage| Q
    C2 -->|DeleteMessage| Q
    Q -->|maxReceiveCount exceeded| DLQ[(Dead Letter\nQueue)]
```

⚖️ 30.2 Queue Types — Standard vs FIFO

Feature	Standard Queue	FIFO Queue
Delivery	At-least-once (duplicates possible)	Exactly-once processing
Ordering	Best-effort (not guaranteed)	Strict FIFO within Message Group ID
Throughput	Nearly unlimited	300 TPS / 3,000 TPS (batching) / 70,000 TPS (High Throughput Mode)
Deduplication	No built-in	5-minute deduplication window
Queue name	Any name	Must end in `.fifo`
Use case	High throughput, order unimportant	Financial transactions, order processing
Lambda trigger	Yes	Yes (one invocation per Message Group ID)
DLQ type required	Standard DLQ	FIFO DLQ

> 🔑 **Exam key:** FIFO = exactly-once + ordered. Standard = throughput king. Cannot convert Standard → FIFO after creation.

FIFO — Message Group ID and Deduplication ID

- **Message Group ID:** Groups messages into ordered streams. Messages with the same Group ID are processed in order. Multiple Group IDs = parallel processing (each group independent).
- **Deduplication ID:** Prevents duplicates within a 5-minute deduplication window. If you resend a message with the same Dedup ID within 5 minutes, SQS silently drops it.
- **Content-based deduplication:** FIFO option to auto-generate Dedup ID as SHA-256 of message body — no need to generate IDs in producer code.

```
Example: payment processing
Group ID = "order-12345"  → all steps for order 12345 processed in order
Group ID = "order-99999"  → processed in parallel with order 12345
Dedup ID = "payment-attempt-1" → retry sends same ID → SQS drops duplicate
```

🔧 30.3 Core Concepts

Visibility Timeout

When a consumer receives a message, SQS makes it **invisible** to other consumers for the Visibility Timeout duration.

```
Timeline:
t=0:   Consumer A receives message → message hidden for 30s (default)
t=25:  Consumer A finishes → calls DeleteMessage → message gone permanently
t=30:  (if Consumer A had NOT deleted) → message reappears → Consumer B receives it

Visibility Timeout too short → message reprocessed before consumer finishes → duplicates
Visibility Timeout too long → if consumer crashes, long wait before retry
Best practice: set to 6x your average processing time
```

Parameter	Default	Min	Max
Visibility Timeout	30 seconds	0 seconds	12 hours

- **Change in-flight:** consumer can call `ChangeMessageVisibility` to extend timeout mid-processing
- **Per-message override:** set different visibility timeout per `ReceiveMessage` call

Message Retention Period

Parameter	Default	Min	Max
Retention	4 days	1 minute	14 days

Messages not consumed within the retention period are **automatically deleted**.

Delivery Delay

Messages can be delayed before becoming visible:

Scope	Min	Max
Queue-level default delay	0s	900s (15 min)
Per-message delay	0s	900s

> ⚠️ **FIFO queue:** per-message delay not supported. Queue-level delay only.

Long Polling vs Short Polling

	Short Polling	Long Polling
Behavior	Returns immediately (empty if no messages)	Waits up to 20s for a message
Empty responses	Frequent (costly)	Rare
Cost	Higher (more API calls)	Lower
Latency	Minimal (sub-ms)	Up to 20s wait
Enable	`WaitTimeSeconds=0`	`WaitTimeSeconds=1-20`

> 🔑 **Exam key:** Always use long polling in production. Short polling = wasted API calls = higher cost + higher CPU on consumer.

Message Attributes

Up to 10 custom metadata attributes per message. Types: String, Number, Binary. Useful for routing/filtering without parsing message body.

Batch Operations

Operation	Max per call
`SendMessageBatch`	10 messages
`ReceiveMessage`	10 messages
`DeleteMessageBatch`	10 messages
`ChangeMessageVisibilityBatch`	10 messages

10 messages = 1 API call = 1/10th the cost. Always batch for high-throughput applications.

☠️ 30.4 Dead Letter Queue (DLQ)

A DLQ is a separate SQS queue that receives messages that **repeatedly fail processing**.

```
flowchart LR
    Q[(Source Queue\nmaxReceiveCount=3)] -->|received 1x| C[Consumer\nfails]
    C -->|visibility timeout| Q
    Q -->|received 2x| C
    C -->|visibility timeout| Q
    Q -->|received 3x = maxReceiveCount| DLQ[(Dead Letter Queue)]
    DLQ --> Alert[CloudWatch Alarm\nSNS Alert]
    DLQ --> Redrive[DLQ Redrive\nafter bug fix]
    Redrive --> Q
```

Key parameters:

- `maxReceiveCount` : 1–1000. After this many receives → message moved to DLQ.
- DLQ **must be the same type** as source queue (Standard → Standard DLQ, FIFO → FIFO DLQ).
- DLQ has its **own retention period** — set longer than source queue to allow investigation.
- **DLQ redrive (2021 feature):** move messages from DLQ back to source after fixing consumer bug. No manual message extraction needed.

Operational best practice:

```
CloudWatch Metric: ApproximateNumberOfMessagesVisible on DLQ
Alarm: > 0 messages → SNS → PagerDuty alert
This means: a message has failed maxReceiveCount times → engineer must investigate
```

> 🔑 **Exam key:** DLQ depth alarm is the standard pattern for detecting processing failures. Without DLQ, poison-pill messages loop forever blocking the queue.

🔐 30.5 Security

Encryption

Type	Key	Cost	Use case
SSE-SQS	AWS-owned key (free)	No extra charge	Default encryption for compliance
SSE-KMS	AWS Managed or CMK	KMS API call per message	Audit trail, cross-account, fine-grained control
In-transit	TLS/HTTPS	Free	Always enforced

> ⚠️ Enforce HTTPS-only via queue policy condition: `"Condition": {"Bool": {"aws:SecureTransport": "false"}}` with Effect Deny.

Queue Policy (Resource-Based Policy)

Controls who can send/receive messages. Required for:

- Cross-account access
- AWS services publishing to queue (SNS, S3 event notifications, EventBridge)

```
{
  "Effect": "Allow",
  "Principal": {"Service": "sns.amazonaws.com"},
  "Action": "sqs:SendMessage",
  "Resource": "arn:aws:sqs:us-east-1:123:my-queue",
  "Condition": {"ArnEquals": {"aws:SourceArn": "arn:aws:sns:us-east-1:123:my-topic"}}
}
```

VPC Endpoint

Interface VPC Endpoint for SQS: private access from VPC without internet, NAT Gateway, or IGW.

- Endpoint policy: further restrict which queues and actions are accessible via the endpoint.

⚡ 30.6 SQS + Lambda Integration

Lambda can be an SQS **event source** — Lambda manages the polling loop automatically.

```
SQS Queue → Lambda Event Source Mapping → Lambda function invoked with batch of messages
```

Key behaviors:

- Lambda scales: one Lambda invocation per batch. Multiple batches processed concurrently.
- **Standard queue:** Lambda scales to 1,000 concurrent invocations (burst limit applies).
- **FIFO queue:** Lambda scales to one concurrent invocation **per Message Group ID** (preserves order).
- On Lambda success: messages deleted automatically.
- On Lambda failure: messages return to queue (visibility timeout expires) → retry → DLQ.

Partial Batch Failure (ReportBatchItemFailures):

```
def handler(event, context):
    failed_items = []
    for record in event['Records']:
        try:
            process(record)
        except Exception:
            failed_items.append({"itemIdentifier": record['messageId']})
    return {"batchItemFailures": failed_items}
# Only failed messages return to queue — successful ones are deleted
# Without this: one failure → entire batch returns → reprocessing all messages
```

> 🔑 **Exam key:** Enable `ReportBatchItemFailures` to avoid reprocessing successfully-handled messages when one message in a batch fails.

📦 30.7 SQS Extended Client Library

Problem: SQS message size limit is 256KB. Large payloads (images, documents, XML) exceed this.

Solution: SQS Extended Client Library (Java, Python, other SDKs):

1. Producer: if payload > threshold (default 256KB or custom) → upload to S3 → send S3 pointer in SQS message
2. Consumer: receives SQS message → detects S3 pointer → downloads from S3 → processes full payload
3. After processing: delete SQS message AND S3 object

```
Producer → S3 (stores 10MB XML) → SQS (stores "s3://bucket/key") → Consumer → S3 (downloads XML)
```

> 🔑 **Exam key:** "Payload > 256KB in SQS" → Extended Client Library + S3. Do not store large payloads in SQS natively.

⚖️ 30.8 SQS vs Other Messaging Services

Feature	SQS	SNS	EventBridge	Kinesis	Amazon MQ
Model	Queue (pull)	Pub/Sub (push)	Event bus (push)	Stream (pull)	Broker (AMQP/MQTT)
Consumers	One consumer per message	Many subscribers	Many targets via rules	Many consumers (independent)	One or many (queue/topic)
Ordering	FIFO only	FIFO topic only	No native ordering	Per-shard ordering	Yes
Retention	14 days max	No retention (push only)	Archive + replay	1–365 days	Per broker config
Fan-out	No (use SNS→SQS)	Native	Via rules	No	Topic-based
Scheduling	No	No	Yes (Scheduler)	No	No
Protocol	AWS SDK/HTTP	AWS SDK/HTTP	AWS SDK/HTTP	AWS SDK/HTTP	AMQP, STOMP, MQTT, OpenWire
Use case	Decoupling, worker queues	Notifications, fan-out	Event routing, automation	Streaming analytics	Lift-and-shift from MQ

> 🔑 **Exam decision tree:**
>
> - Decouple one producer from one consumer → **SQS**
> - One event to many consumers → **SNS + SQS fan-out**
> - Route AWS service events by content → **EventBridge**
> - Ordered stream, replay, analytics → **Kinesis**
> - Migrate ActiveMQ/RabbitMQ → **Amazon MQ**

💰 30.9 Pricing

	Standard	FIFO
First 1M requests/month	Free	Free
Per million requests	$0.40	$0.50
Data transfer in	Free	Free
Data transfer out (same region)	Free	Free

Cost optimization:

- Use batch operations: 10 messages per `SendMessageBatch` = 1 API call = $0.00000004 per message
- Long polling: fewer empty `ReceiveMessage` calls
- VPC endpoints: avoid NAT Gateway data charges for SQS traffic from private subnets

---

📏 30.10 Limits

Parameter	Limit
Message size	256 KB (use Extended Client for larger)
Message retention	1 min – 14 days
Visibility timeout	0 – 12 hours
Delivery delay	0 – 900 seconds
In-flight messages (Standard)	120,000
In-flight messages (FIFO)	20,000
Batch size	10 messages
Queue name length	80 characters
Message attributes	10 per message
FIFO throughput (standard)	3,000 TPS with batching
FIFO High Throughput Mode	70,000 TPS
Queues per account	No limit (soft: 1,000 default)

⚠️ 30.11 Exam Traps

1. **"Exactly-once delivery"** → FIFO queue only. Standard = at-least-once (duplicates possible). Consumer must be idempotent for Standard queues.
2. **"Ordered messages"** → FIFO queue with same Message Group ID. Different group IDs are ordered independently.
3. **"Message keeps reappearing"** → visibility timeout too short, OR consumer not calling `DeleteMessage` after processing.
4. **"Fan-out to multiple systems"** → SQS alone cannot fan out. Use SNS → multiple SQS queues.
5. **"Message body > 256KB"** → Extended Client Library storing payload in S3. Not a queue limit increase.
6. **"FIFO throughput limit"** → 3,000 TPS with batching (default). Enable High Throughput Mode for 70,000 TPS.
7. **"DLQ type mismatch"** → Standard source requires Standard DLQ. FIFO source requires FIFO DLQ. Mismatch = configuration error.
8. **"Long polling vs short polling cost"** → Long polling = fewer empty responses = fewer API calls = lower cost. Always use long polling.
9. **"Lambda + FIFO: ordering lost"** → Lambda respects FIFO order. One concurrent invocation per Message Group ID ensures order. Parallel groups run concurrently.
10. **"Delete SQS queue to clear all messages"** → deletes the queue itself. Use `PurgeQueue` API to clear messages while keeping the queue.

🏗️ 30.12 Well-Architected Framework — SQS

Pillar	SQS Implementation
Operational Excellence	DLQ + CloudWatch alarm for processing failures. Dead-letter queue redrive for recovery. SQS metrics (ApproximateAgeOfOldestMessage) to detect consumer lag.
Security	SSE-KMS for sensitive data. Queue policy enforcing HTTPS-only (deny `aws:SecureTransport=false`). VPC endpoint for private access. Least-privilege IAM roles (separate send vs receive permissions).
Reliability	Messages stored across multiple AZs. FIFO for exactly-once delivery. DLQ prevents message loss. Visibility timeout sized to processing time + buffer. Retry with exponential backoff.
Performance Efficiency	Long polling to reduce API calls. Batch operations (10x cost reduction). FIFO High Throughput Mode (70,000 TPS) for high-volume ordered workloads. Lambda event source mapping auto-scales consumers.
Cost Optimization	Batch operations. Long polling. VPC endpoints (eliminates NAT GW charges). Standard queue when ordering not required (cheaper at $0.40 vs 0.50$/million).
Sustainability	Message batching reduces API calls (fewer compute cycles per message processed). Auto-scaling consumers scale to zero when queue empty — no idle resources.

🎯 30.13 SAP-C02 Domains — SQS

Domain 1 — Organizational Complexity: Multi-account SQS fan-out: SNS in account A → SQS in account B (cross-account queue policy). Centralized dead-letter analysis queue aggregating DLQ events from multiple application queues. SCP restricting SQS queue creation to approved regions.

Domain 2 — New Solutions: Design decoupled order processing: API GW → Lambda → SQS → Lambda consumer → RDS. FIFO queue for financial transactions requiring exactly-once. Auto-scaling EC2 worker fleet based on `ApproximateNumberOfMessagesVisible` metric.

Domain 3 — Continuous Improvement: Replace synchronous microservice calls with SQS (async decoupling). Add DLQ to existing queue with no DLQ (message loss prevention). Switch from short polling to long polling (cost reduction). Enable SSE-KMS on existing unencrypted queue (compliance).

Domain 4 — Migration: Migrate on-premises ActiveMQ to Amazon MQ (lift-and-shift, preserves AMQP). Or re-architect to SQS (native AWS, no broker management). Decision: ActiveMQ-specific features (transactions, message selectors) → MQ. Simple point-to-point → SQS.

🔗 30.14 Related Services

Service	Relationship
Amazon SNS	Fan-out: SNS publishes to multiple SQS queues. SQS provides durable buffering for SNS subscribers.
AWS Lambda	Event source mapping: Lambda polls SQS, processes batches, scales concurrency automatically.
Amazon EventBridge	EventBridge rules can target SQS queues. EventBridge Pipes can use SQS as source.
Amazon S3	S3 event notifications can publish to SQS. Extended Client Library stores large payloads in S3.
Amazon EC2 Auto Scaling	Scale worker fleet based on SQS queue depth (step scaling policy on `ApproximateNumberOfMessagesVisible`).
AWS KMS	SSE-KMS for queue encryption with customer-managed keys.
Amazon CloudWatch	Queue metrics: depth, age of oldest message, in-flight count. DLQ alarms.
Amazon MQ	Alternative for lift-and-shift of ActiveMQ/RabbitMQ workloads requiring AMQP/MQTT/STOMP protocols.

End of Chapter 30 — Amazon SQS

📣 Chapter 31: Amazon SNS — Simple Notification Service

> **Covers:** Standard vs FIFO Topics · Subscribers · Fan-Out Pattern · Message Filtering · Mobile Push · SMS · Security · DLQ · Pricing

---

📋 Table of Contents (Chapter 31)

- 31.1 Overview
- 31.2 Topic Types — Standard vs FIFO
- 31.3 Subscriber Types
- 31.4 SNS Fan-Out Pattern
- 31.5 Message Filtering
- 31.6 Security
- 31.7 Mobile Push Notifications
- 31.8 SMS
- 31.9 DLQ for SNS
- 31.10 Message Archiving and Replay
- 31.11 Pricing
- 31.12 Exam Traps
- 31.13 Well-Architected Framework
- 31.14 SAP-C02 Domains
- 31.15 Related Services

📖 31.1 Overview

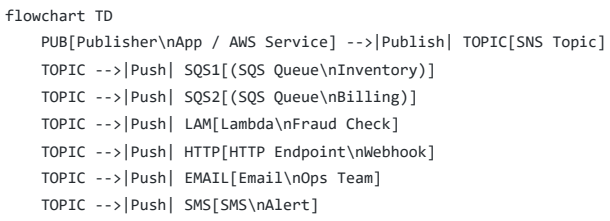
Amazon SNS is a **fully managed pub/sub messaging service** for application-to-application (A2A) and application-to-person (A2P) communication. A publisher sends one message to an SNS topic; SNS **pushes** it to all registered subscribers simultaneously.

Key characteristics:

- **Push-based** — SNS delivers to subscribers (unlike SQS where consumers poll)
- **Fan-out** — one message → many subscribers
- No message retention (Standard topics) — if delivery fails and subscriber has no DLQ, message is lost
- Scope: **Regional**. Topics are regional resources.

Linux/Open-source equivalent: RabbitMQ topic exchange (AMQP), Apache Kafka (pub/sub), Redis Pub/Sub.

Azure equivalent: Azure Service Bus Topics (pub/sub with subscriptions/filters), Azure Event Grid (event-driven, similar to SNS for event notifications).



⚖️ 31.2 Topic Types — Standard vs FIFO

Feature	Standard Topic	FIFO Topic
Delivery	At-least-once	Exactly-once
Ordering	Best-effort	Strict FIFO
Throughput	Unlimited	300 TPS / 3,000 with batching
Subscribers	SQS, Lambda, HTTP/S, Email, SMS, Mobile, Firehose	SQS FIFO only
Deduplication	No	Yes (5-min window)
Topic name	Any	Must end in `.fifo`
Use case	Notifications, fan-out, alerts	Ordered event streams to multiple consumers

> 🔑 **Exam key:** SNS FIFO → SQS FIFO only. Cannot subscribe Lambda or HTTP directly to FIFO topic. For FIFO with Lambda: SNS FIFO → SQS FIFO → Lambda.

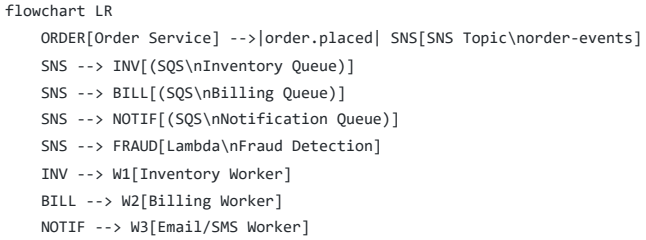
📋 31.3 Subscriber Types

Protocol	Subscriber	Notes
SQS	SQS Queue	Durable. Decouples producer speed from consumer speed.
Lambda	Lambda Function	Immediate processing. Scales automatically.
HTTP/HTTPS	External endpoint	Webhook. Must confirm subscription (GET to confirm URL).
Email	Email address	Human-readable. Must manually confirm. JSON or plain text.
SMS	Phone number	Transactional or promotional. Per-country pricing.
Mobile push	APNs (iOS), FCM (Android), ADM (Amazon), Baidu (China)	Requires Platform Application registration.
Kinesis Data Firehose	Firehose delivery stream	Stream SNS messages to S3, Splunk, OpenSearch, Redshift.

> ⚠️ **SNS cannot deliver directly to Kinesis Data Streams.** Path: SNS → Lambda → KDS, or SNS → Kinesis Data Firehose.

🔀 31.4 SNS Fan-Out Pattern

The fan-out pattern solves the problem of delivering one event to multiple independent processing pipelines.



Benefits:

- Each SQS queue has its own DLQ — one failing pipeline does not affect others
- Each consumer scales independently

- Adding a new consumer = add new subscription to SNS topic (no producer code change)
- Producers are completely decoupled from consumers

Cross-account fan-out:

- SNS in account A publishes to SQS in account B
- Requires SQS queue policy in account B allowing `sns.amazonaws.com` to `sqs:SendMessage`

🔍 31.5 Message Filtering

By default, all subscribers receive all messages. **Filter policies** let subscriptions receive only matching messages.

Filter policy targets:

- **Message attributes** (available on Standard and FIFO topics)
- **Message body** (must enable `FilterPolicyScope: MessageBody` — supported since 2022)

Filter policy examples:

```
// Subscription 1: EU orders only (attribute-based)
{
  "region": ["eu-west-1", "eu-central-1"],
  "orderType": ["standard", "express"]
}

// Subscription 2: high-value orders (numeric range)
{
  "orderValue": [{"numeric": [">=", 1000]}]
}

// Subscription 3: everything except cancelled
{
  "status": [{"anything-but": "cancelled"}]
}
```

Architecture with filtering:

```
flowchart LR
    SNS[SNS Topic\nAll Orders] -->|filter: region=eu| EU[(SQS\nEU Processing)]
    SNS -->|filter: region=us| US[(SQS\nUS Processing)]
    SNS -->|filter: value >= 1000| VIP[(SQS\nVIP Processing)]
    SNS -->|no filter| AUDIT[(SQS\nAudit All)]
```

> 🔑 **Exam key:** Message filtering is per-subscription (not per-topic). Each subscriber can have a different filter. No filter = receives all messages.

🔐 31.6 Security

Encryption

- **SSE-SNS:** AWS-managed key. Free. Encrypts at rest within SNS.
- **SSE-KMS:** Customer Managed Key. Per-API-call KMS charge.
- **In-transit:** TLS always.

Topic Policy (Resource-Based)

Controls who can publish and subscribe. Required for cross-account publishing.

```
{
  "Effect": "Allow",
  "Principal": {"Service": "cloudwatch.amazonaws.com"},
  "Action": "SNS:Publish",
  "Resource": "arn:aws:sns:us-east-1:123:ops-alerts",
  "Condition": {"StringEquals": {"AWS:SourceAccount": "123456789012"}}
}
```

Subscription Confirmation

HTTP/HTTPS subscribers must confirm their subscription — SNS sends a GET request with a token; endpoint must call `ConfirmSubscription`. Prevents unauthorized subscriptions receiving messages.

Data Protection Policy

SNS supports PII masking in messages. Detect and redact sensitive data (credit card numbers, SSNs, emails) before delivering to subscribers. Useful for compliance (PCI-DSS, HIPAA) without Lambda preprocessing.

VPC Endpoint

Interface VPC Endpoint for SNS: private access from VPC without internet.

📱 31.7 Mobile Push Notifications

SNS supports push notifications to mobile devices across platforms.

Setup flow:

1. **Platform Application** — register with Apple APNs (iOS certificate/key) or Google FCM (API key) in SNS
2. **Platform Endpoint** — register each device token (obtained from mobile OS) as an endpoint
3. **Publish** — publish to endpoint (single device) or to SNS topic (broadcast to all registered endpoints)

Platform	Service	Notes
iOS	Apple APNs	Certificate or token-based auth (token preferred)
Android/Web	Google FCM	API key or service account
Amazon devices	Amazon ADM	For Kindle/Fire devices
China Android	Baidu Cloud Push	For apps distributed in China

TTL (Time-to-Live): How long the mobile push platform holds an undelivered notification before discarding it. Set via `AWS.SNS.MOBILE.APNS.TTL` message attribute.

📱 31.8 SMS

Type	Priority	Use case
Transactional	High	OTP codes, security alerts, order confirmations
Promotional	Lower	Marketing messages (may require opt-in by law)

Key settings:

- **Spending limit:** monthly cap per account (default $1 — must increase via support)
- **Sandbox mode:** test without sending real SMS (registered destination numbers only)
- **Opt-out management:** SNS automatically handles STOP/UNSTOP replies per country requirements
- **Origination number/Sender ID:** customize how your SMS appears to recipients

---

☠️ 31.9 DLQ for SNS

SNS DLQ is **per-subscription** (not per-topic) and captures **delivery failures** (not processing failures):

	SQS DLQ	SNS Subscription DLQ
What it captures	Processing failures (consumer failed)	Delivery failures (SNS could not deliver to subscriber)
Configured on	Source queue	Individual SNS subscription
Used with	SQS consumer workflows	HTTP, Lambda, SQS subscriber failures

Configure on subscription:

```
aws sns set-subscription-attributes \
  --subscription-arn arn:aws:sns:... \
  --attribute-name RedrivePolicy \
  --attribute-value '{"deadLetterTargetArn":"arn:aws:sqs:us-east-1:123:my-dlq"}'
```

🗂️ 31.10 Message Archiving and Replay

Available on **SNS FIFO topics**:

- Enable archiving on the FIFO topic with a retention period
- Replay: replay archived messages from a specific point in time to a specific SQS FIFO subscriber
- Use case: disaster recovery, reprocessing after a consumer bug was fixed, onboarding new subscribers to catch up

> Standard topics: no archiving/replay. For replay with Standard topics, use EventBridge Archive + Replay.

💰 31.11 Pricing

Component	Cost
First 1M API requests/month	Free
API requests (publish, subscribe, etc.)	$0.50 / million
HTTP/HTTPS deliveries	$0.06 / million
SQS deliveries	No charge (SQS charges apply)
Lambda deliveries	No charge (Lambda charges apply)
Email/Email-JSON	$2.00 / 100,000
SMS (US)	~$0.00645 / message
Mobile push (first 1M/month)	Free; $0.50 / million after
Kinesis Firehose deliveries	No SNS charge (Firehose charges apply)

⚠️ 31.12 Exam Traps

1. **"Fan-out to multiple independent systems"** → SNS topic + multiple SQS queue subscriptions. SQS alone has one consumer per message.
2. **"Only certain subscribers get certain messages"** → SNS message filtering via filter policies. Not topic-level routing.
3. **"Ordered delivery to multiple subscribers"** → SNS FIFO topic + SQS FIFO subscribers. Not Standard topic.
4. **"HTTP subscriber not receiving messages"** → subscription not confirmed. SNS sends confirmation request; endpoint must respond.
5. **"SNS → Kinesis Data Streams"** → NOT directly supported. Use SNS → Lambda → KDS, or SNS → Kinesis Data Firehose.
6. **"Mobile push notification to iOS"** → SNS Platform Application (APNs). Not SQS (pull-based, not suitable for push).

7. **"Retain messages for replay"** → Standard SNS has no retention. Use SQS (14 days), Kinesis (365 days), or EventBridge Archive. SNS FIFO has archiving.
8. **"One-to-one messaging with guaranteed processing"** → SQS (queue, one consumer per message). SNS is pub/sub (one-to-many, no guaranteed processing).
9. **"SNS FIFO subscriber must be Lambda"** → False. SNS FIFO only supports SQS FIFO subscribers. Lambda requires Standard topic or SQS FIFO → Lambda.
10. **"Cross-account SNS → SQS"** → requires SQS queue policy granting `sqs:SendMessage` to `sns.amazonaws.com` with source ARN condition.

🏛️ 31.13 Well-Architected Framework — SNS

Pillar	SNS Implementation
Operational Excellence	Message filtering reduces noise (subscribers only process relevant messages). Subscription DLQ captures delivery failures for investigation. CloudWatch metrics: NumberOfMessagesPublished, NumberOfNotificationsFailed.
Security	SSE-KMS for sensitive notification data. Topic policy with SourceAccount condition prevents unauthorized cross-account publishing. HTTPS-only for HTTP subscribers. Data protection policy for PII masking.
Reliability	SNS redundantly stores messages across AZs within region. DLQ per subscription prevents silent delivery failure. Fan-out to SQS: each SQS queue provides durable buffering if subscriber is slow.
Performance Efficiency	SNS FIFO for ordered fan-out. Message filtering at SNS layer (not at subscriber) reduces downstream compute. Direct Lambda invocation for real-time processing without queue buffer.
Cost Optimization	Message filtering reduces unnecessary SQS messages and Lambda invocations. SNS → Firehose for archival (cheaper than SNS → Lambda → S3). Batch publish where supported.
Sustainability	Message filtering eliminates processing cycles for irrelevant messages across all subscribers. Serverless model: zero resource consumption when no messages published.

🎯 31.14 SAP-C02 Domains — SNS

Domain 1 — Organizational Complexity: Cross-account event notification: SNS in security account → SQS in each member account (compliance alert distribution). Centralized notification hub: one SNS topic → email + PagerDuty (HTTP) + Slack (HTTP) + SQS (audit). Firewall Manager + SNS for security policy violation alerts across org.

Domain 2 — New Solutions: E-commerce order event fan-out: SNS → SQS(inventory) + SQS(billing) + SQS(notification) + Lambda(fraud). Mobile app OTP: SNS SMS transactional. New consumer pattern: add SQS subscription to existing SNS topic (zero producer change).

Domain 3 — Continuous Improvement: Add message filtering to reduce downstream processing cost. Add subscription DLQ to capture failing deliveries (previously silent failures). Switch HTTP endpoint to SQS subscription for durability (HTTP endpoint downtime = message loss; SQS buffers messages).

Domain 4 — Migration: On-premises email alerts → SNS Email. On-premises message broker fan-out → SNS + SQS. Legacy pub/sub system → SNS Standard topic with SQS subscribers per consumer type.

🔗 31.15 Related Services

Service	Relationship
Amazon SQS	Primary SNS subscriber for durable fan-out. Each SQS subscription provides independent buffering and DLQ.
AWS Lambda	Direct invocation from SNS. No polling needed. Lambda scales with message volume.
Amazon EventBridge	Complementary: SNS for broad notifications, EventBridge for content-based routing. SNS can send to EventBridge via HTTP destination.
Amazon Kinesis Data Firehose	SNS subscriber for streaming notifications to S3, OpenSearch, Splunk without Lambda.
AWS CloudWatch	CloudWatch Alarms publish to SNS topics for operational notifications.
Amazon SES	SES for transactional email at scale (better deliverability than SNS email). SNS email suited for ops alerts, not marketing.
AWS KMS	SSE-KMS for SNS topic encryption with customer-managed keys.
AWS Firewall Manager / GuardDuty / Security Hub	These services publish findings/alerts to SNS for notification workflows.

End of Chapter 31 — Amazon SNS

⚡ Chapter 32: Amazon EventBridge

> **Covers:** Event Buses · Rules · Event Patterns · Input Transformation · Pipes · Scheduler · Schema Registry · API Destinations · Cross-Account/Region · Archive & Replay · Security

---

📋 Table of Contents (Chapter 32)

- 32.1 Overview
- 32.2 Event Buses
- 32.3 Events — Structure and Publishing
- 32.4 Rules and Event Pattern Matching
- 32.5 Input Transformation
- 32.6 EventBridge Pipes
- 32.7 EventBridge Scheduler
- 32.8 Schema Registry
- 32.9 API Destinations
- 32.10 Archive and Replay
- 32.11 Cross-Account and Cross-Region
- 32.12 EventBridge vs SNS vs SQS
- 32.13 Security
- 32.14 Pricing
- 32.15 Exam Traps
- 32.16 Well-Architected Framework
- 32.17 SAP-C02 Domains
- 32.18 Related Services

📖 32.1 Overview

Amazon EventBridge is a **serverless event bus** that makes it easy to connect applications using data from AWS services, SaaS applications, and custom applications. It routes events to targets based on rules with content-based pattern matching.

Key characteristics:

- **Event-driven:** decouple event producers from consumers
- **Content-based routing:** route by any field in the event JSON
- **Schema-aware:** built-in schema registry with code generation
- **Successor to CloudWatch Events** — same underlying service, enhanced console and features
- **Scope:** Regional. Global endpoints available for multi-region HA.

Linux/Open-source equivalent: Apache Kafka + custom routing layer, NATS JetStream, Apache Camel.

Azure equivalent: Azure Event Grid (very similar — serverless event routing with rules and subscribers).

```
flowchart LR
    AWS[AWS Services\nEC2/S3/RDS/...] -->|automatic| DEFAULT[Default Event Bus]
    APP[Custom App] -->|PutEvents API| CUSTOM[Custom Event Bus]
    SAAS[SaaS Partner\nZendesk/Datadog] -->|partner integration| PARTNER[Partner Event Bus]
    DEFAULT --> RULE1{Rule 1\nEC2 terminated}
    DEFAULT --> RULE2{Rule 2\nS3 upload}
    CUSTOM --> RULE3{Rule 3\norder.placed}
    RULE1 --> LAMBDA[Lambda]
    RULE1 --> SNS[SNS Topic]
    RULE2 --> SQS[(SQS Queue)]
    RULE3 --> STEP[Step Functions]
    RULE3 --> SQS2[(SQS Queue)]
```

🚌 32.2 Event Buses

Bus Type	Purpose	Cost	Who publishes
Default	AWS service events	Free for AWS events	AWS services automatically
Custom	Application/microservice events	$1/million custom events	Your apps via `PutEvents` API
Partner	SaaS integrations	$1/million events	SaaS partner (Zendesk, Datadog, etc.)

Default event bus: Every AWS account has one in each region. AWS services automatically publish events here — EC2 state changes, S3 lifecycle events, RDS snapshots, CodePipeline state changes, etc.

Custom event bus: Create for your own application events. Supports cross-account resource policy (other accounts can publish to your bus or you can target their bus).

Partner event bus: Activate a SaaS partner integration in EventBridge console → SaaS partner can push events directly to your account. Examples: Zendesk ticket created, Datadog alert fired, GitHub PR opened, Stripe payment processed.

📄 32.3 Events — Structure and Publishing

All EventBridge events are JSON with a standard envelope:

```
{
  "version": "0",
  "id": "abc-123",
  "source": "aws.ec2",
  "account": "123456789012",
  "time": "2024-01-15T10:30:00Z",
  "region": "us-east-1",
  "resources": ["arn:aws:ec2:us-east-1:123:instance/i-abc123"],
  "detail-type": "EC2 Instance State-change Notification",
  "detail": {
    "instance-id": "i-abc123",
    "state": "terminated"
  }
}
```

Custom event publishing (PutEvents API):

```
import boto3
client = boto3.client('events')
client.put_events(Entries=[{
    'Source': 'com.mycompany.orders',
    'DetailType': 'Order Placed',
    'Detail': '{"orderId": "ORD-1234", "amount": 99.99, "region": "eu"}',
    'EventBusName': 'my-custom-bus'
}])
```

PutEvents limits: 10 entries per API call, 256KB total payload.

🎯 32.4 Rules and Event Pattern Matching

Rules match events and route them to targets.

Rule types:

1. **Event pattern rule:** matches events by content → routes to targets
2. **Schedule rule:** triggers targets on a time schedule (cron or rate)

Up to 5 targets per rule. Each target can have its own input transformation.

Target types include: Lambda, SQS, SNS, Step Functions, ECS Task, CodePipeline, Kinesis Data Streams, Kinesis Firehose, API Gateway, EventBridge bus (cross-account/region), Batch, SSM Run Command, SSM Automation, CodeBuild, EC2, and 270+ via EventBridge Scheduler.

Event Pattern Matching — Full Reference

```
// Exact match
{"source": ["aws.ec2"], "detail-type": ["EC2 Instance State-change Notification"]}

// Prefix match on detail field
{"detail": {"bucketName": [{"prefix": "prod-"}]}}

// Anything-but (exclusion)
{"detail": {"state": [{"anything-but": ["running", "stopped"]}]}}

// Exists check (error events from CloudTrail)
{"detail": {"errorCode": [{"exists": true}]}}

// Numeric range
{"detail": {"orderValue": [{"numeric": [">=", 100, "<", 1000]}]}}

// Wildcard (suffix match)
{"detail": {"filename": [{"wildcard": "*.pdf"}]}}

// Complex: EC2 terminated in prod account, specific region
{
  "source": ["aws.ec2"],
  "account": ["111111111111"],
  "region": ["eu-central-1"],
  "detail": {"state": ["terminated"]}
}
```

> 🔑 **Exam key:** EventBridge pattern matching is against the FULL event JSON (including `source`, `account`, `region`, `detail-type`, and `detail`). SNS filtering is only on message attributes. EventBridge is more powerful for content-based routing.

🔄 32.5 Input Transformation

Transform the event JSON before sending to a target — without needing a Lambda function just to reshape data.

Three modes:

1. **Matched event:** send full original event (default)
2. **Input path + template:** extract fields, build custom JSON
3. **Constant JSON:** send a fixed JSON regardless of event content

Example — extract fields and build Slack message:

```
Input path:
  $.detail.instance-id  → <instance>
  $.region              → <region>
  $.detail.state        → <state>

Input template:
  {"text": "Instance <instance> in <region> changed state to <state>"}

Result sent to Lambda/HTTP:
  {"text": "Instance i-abc123 in us-east-1 changed state to terminated"}
```

> 🔑 **Exam key:** Use Input Transformation to avoid writing a Lambda whose only job is reformatting an event. Saves cost and latency.

🔧 32.6 EventBridge Pipes

Pipes provide **point-to-point integration** with optional filtering and enrichment between a source and a target.

```
flowchart LR
    SRC[Source\nSQS/Kinesis/DDB Streams\nMSK/MQ] -->|filter| FILT[Optional Filter\ndrop unwanted events]
    FILT -->|enrich| ENR[Optional Enrichment\nLambda/Step Fn/API GW]
    ENR -->|deliver| TGT[Target\nLambda/SQS/SNS/EventBridge\nStep Fn/API GW/270+ services]
```

Sources: SQS, Kinesis Data Streams, DynamoDB Streams, Amazon MSK (Kafka), Amazon MQ (RabbitMQ).

Why Pipes instead of Lambda polling:

- No custom polling code
- Built-in batching, retry, error handling
- Enrichment stage without chaining Lambdas
- Pay per event processed (not per Lambda invocation for empty polls)

Use case: DynamoDB Streams → Pipe (filter `INSERT` events only) → Lambda (enrich with customer data from RDS) → EventBridge bus → multiple consumers.

⏰ 32.7 EventBridge Scheduler

Standalone scheduling service — separate from EventBridge rules.

Feature	EventBridge Scheduler	CloudWatch Events Scheduled Rules
Schedule types	One-time, rate, cron	Rate, cron only
Time zones	270+ time zones	UTC only
Targets	270+ AWS APIs directly	~20 target types
Flexible window	Yes (spread invocations)	No
Scale	Millions of schedules	Limited
Cost	$1/million invocations (14M free/month)	Included in EventBridge

Flexible time window: Instead of triggering exactly at 9:00 AM, trigger within a 15-minute window around 9:00 AM. Prevents thundering herd when thousands of schedules fire simultaneously.

One-time schedule: Fire once at a specific date/time, then auto-delete. Use for: "send report email on 2025-12-31 at 23:59 UTC+1".

Examples:

```
Rate: rate(5 minutes)          → every 5 minutes
Cron: cron(0 9 ? * MON-FRI *)  → weekdays at 9 AM UTC
One-time: at(2025-12-31T23:59:00) → specific date/time
```

📐 32.8 Schema Registry

EventBridge auto-discovers event schemas from your event buses and stores them.

Features:

- **Schema discovery:** enable on a bus → EventBridge infers JSON schema from events
- **Code bindings:** download generated code (Java, Python, TypeScript) for type-safe event handling
- **Pre-built schemas:** AWS service events + SaaS partner schemas included
- **IDE integration:** AWS Toolkit for VS Code uses schema registry for autocomplete

Use case: Teams writing Lambda functions for custom events get type-safe classes auto-generated — no manual schema maintenance.

🌐 32.9 API Destinations

Send events to any **external HTTP endpoint** — Splunk, PagerDuty, Slack, Salesforce, custom webhooks.

Features:

- **Built-in retry:** exponential backoff with configurable attempts
- **Rate limiting:** max invocations per second (prevents overwhelming external API)
- **Auth types:** API Key, OAuth 2.0 Client Credentials, Basic Auth
- **Connection object:** reusable auth config (stored securely, not in rule)

Example: GuardDuty finding → EventBridge rule → API Destination → PagerDuty incident API → on-call engineer paged. No Lambda needed.

```
EventBridge Rule → API Destination → https://events.pagerduty.com/v2/enqueue
                                     Auth: API Key header
                                     Rate: 300/second max
                                     Retry: 3 attempts, exponential backoff
```

🗄️ 32.10 Archive and Replay

Archive: Store a copy of all events matching a filter on a custom event bus.

```
Custom Event Bus → Archive (retention: 30 days, filter: all events)
                          ↓
                   S3-backed storage (no direct S3 access, managed by EventBridge)
```

Replay: Re-publish archived events from a time range back to the event bus — triggers all matching rules again.

Use cases:

- Consumer bug discovered → fix Lambda → replay last 24 hours of events → reprocess
- New microservice deployed → replay 7 days of events to populate its data store
- Disaster recovery: restore event stream after regional failure

> 🔑 **Exam key:** Archive + Replay is EventBridge-specific. SNS Standard has no replay. SQS has retention (14 days) but no selective replay. EventBridge Archive is the answer for "replay events after a consumer bug."

🌏 32.11 Cross-Account and Cross-Region

Cross-account event routing:

```
Account A (producer):
  EventBridge rule → target: event bus in Account B

Account B (consumer):
  Resource policy on event bus: allow Account A to send events
  Rules on Account B event bus → Lambda/SQS/etc.
```

Cross-region routing:

```
EventBridge rule in us-east-1 → target: event bus in eu-central-1
Use case: replicate events for DR or regional processing
```

Global Endpoints (active-active multi-region):

- Create a global endpoint with two regional event buses
- Route 53 health checks on each regional endpoint
- EventBridge Pipe: events routed to healthy region

- **Event replication:** events published to primary region are replicated to secondary
- Failover: if primary region health check fails → all new events route to secondary
- Use case: mission-critical event processing with no single region dependency

---

⚖️ 32.12 EventBridge vs SNS vs SQS

Feature	SQS	SNS	EventBridge
Model	Queue (pull)	Pub/Sub (push)	Event bus (push, rules)
Routing	None (send to specific queue)	Message attribute filters	Full JSON content matching
Fan-out	No (use SNS)	Yes (multiple subscribers)	Yes (multiple targets per rule)
Ordering	FIFO only	FIFO topic only	No native ordering
Retention/Replay	14 days retention	No (Standard); FIFO archive	Archive + Replay (custom buses)
Scheduling	No	No	Yes (Scheduler)
SaaS integration	No	No	Yes (Partner event sources)
Schema registry	No	No	Yes
External HTTP target	No	HTTP/S endpoint	API Destinations (with retry/auth)
Max targets per rule	N/A	N/A	5 per rule
Cost model	Per request	Per request	Per event published
AWS service events	Manual setup	Manual setup	Automatic (default bus)
Best for	Decoupled worker queues	Broadcast notifications	Event-driven automation, SaaS

> 🔑 **Decision guide:**
>
> - Worker queue, retry, DLQ → **SQS**
> - Broadcast to many systems instantly → **SNS + SQS fan-out**
> - React to AWS service events, SaaS events, content-based routing → **EventBridge**
> - All three together: EventBridge (routing) → SNS (broadcast) → SQS (buffer) → Lambda (process)

🔐 32.13 Security

Resource policy on event bus: Controls who can publish events to the bus and create rules. Required for cross-account.

```
{
  "Effect": "Allow",
  "Principal": {"AWS": "arn:aws:iam::ACCOUNT-B:root"},
  "Action": "events:PutEvents",
  "Resource": "arn:aws:events:us-east-1:ACCOUNT-A:event-bus/my-bus"
}
```

IAM controls:

- `events:PutEvents` — publish events
- `events:PutRule` — create/update rules
- `events:PutTargets` — add targets to rules
- Target permissions: EventBridge needs IAM role to invoke Lambda, write to SQS, etc.

Encryption:

- At rest: AWS-owned key (default) or CMK

- In transit: TLS always

No VPC endpoint for EventBridge bus itself. Targets (Lambda, SQS) can be in VPC. Use API Destinations for private endpoints via Direct Connect or VPN.

💰 32.14 Pricing

Component	Cost
AWS service events (default bus)	Free
Custom/partner events	$1.00 / million events
Cross-account/cross-region delivery	$0.01 / million events
EventBridge Pipes	$0.40 / million events processed
Scheduler (first 14M/month)	Free
Scheduler (after 14M)	$1.00 / million invocations
Archive storage	S3-equivalent pricing
Schema discovery	$0.10 / million events scanned

Cost comparison vs alternatives:

- EventBridge rule → Lambda: $1/million events + Lambda cost
- SNS → Lambda: $0.50/million SNS + Lambda cost
- For simple "invoke Lambda on S3 upload": S3 event notification (free) is cheaper than EventBridge

---

⚠️ 32.15 Exam Traps

1. **"React to EC2 state change automatically"** → EventBridge rule on default bus (aws.ec2 source). Not CloudWatch Events (same thing, just old name).
2. **"Schedule Lambda at 9am New York time"** → EventBridge Scheduler with `America/New_York` timezone. CloudWatch Events only supports UTC.
3. **"SaaS (Zendesk/Datadog) events into AWS"** → EventBridge partner event source. Not Lambda webhook listener.
4. **"Send events to Splunk/PagerDuty/Slack without Lambda"** → EventBridge API Destinations with built-in retry and auth.
5. **"Route events to multiple targets based on content"** → EventBridge rules. SNS filters on attributes only; EB filters on full event JSON.
6. **"EventBridge vs CloudWatch Events"** → same service. EventBridge is the new name. Exam may use either. CloudWatch Events scheduled rules still work but Scheduler is the modern approach.
7. **"Replay events after bug fix"** → EventBridge Archive + Replay. Not SQS (SQS has retention but no selective replay). Not SNS (no retention).
8. **"Reshape event JSON before Lambda"** → EventBridge Input Transformation. Avoid Lambda just for reshaping.
9. **"One-time event at specific future time"** → EventBridge Scheduler one-time schedule. Not Step Functions Wait state (different use case).
10. **"Cross-account event routing"** → resource policy on target account event bus + rule in source account targeting that bus.
11. **"EventBridge to DynamoDB directly"** → Not a native target. Use Lambda or Pipes as intermediary.
12. **"Max targets per rule"** → 5. For more targets: fan out to SNS topic, which delivers to many subscribers.

🏗️ 32.16 Well-Architected Framework — EventBridge

Pillar	EventBridge Implementation
Operational Excellence	Schema registry for event documentation. CloudWatch metrics per rule (MatchedEvents, FailedInvocations, ThrottledRules). Dead-letter SQS for failed rule invocations. Archive + Replay for operational recovery.
Security	Resource policies for cross-account bus access. IAM least privilege for `PutEvents`, `PutRule`, `PutTargets`. API Destinations OAuth 2.0 for external services. CMK encryption for sensitive event data on custom buses.
Reliability	Events stored redundantly across AZs. Failed target invocations retried (configurable retry policy). DLQ on rules captures failed invocations. Global endpoint + event replication for multi-region HA. Archive prevents event loss.
Performance Efficiency	Content-based filtering at EventBridge (not at Lambda) reduces compute. Input Transformation avoids Lambda overhead for reshaping. Pipes replaces custom polling Lambda (managed, efficient). Scheduler flexible window distributes load.
Cost Optimization	S3 event notifications (free) for simple Lambda triggers — no EventBridge needed. API Destinations vs Lambda: no Lambda cost for external webhooks. Disable unused rules (events still match but no target invocation).
Sustainability	Serverless event routing scales to zero. Content-based filtering reduces downstream Lambda invocations. Schema-based code generation reduces developer time (fewer compute cycles wasted on trial and error).

🎯 32.17 SAP-C02 Domains — EventBridge

Domain 1 — Organizational Complexity: Cross-account event bus: security events from all member accounts → central security account EventBridge bus → SIEM via API Destination. Organizations integration: resource policy allowing org-wide `PutEvents`. Multi-account compliance automation: Config rule violation event → EventBridge → SNS → security team.

Domain 2 — New Solutions: Microservice decoupling: OrderService publishes `order.placed` to custom bus → Inventory, Billing, Notification services subscribe via rules. SaaS integration: Stripe payment events → partner event bus → Lambda → DynamoDB. Event-driven pipeline: S3 upload → EventBridge → Step Functions state machine.

Domain 3 — Continuous Improvement: Migrate CloudWatch Events scheduled rules to EventBridge Scheduler (timezone support). Add Archive + Replay for existing event bus (operational recovery capability). Replace Lambda polling pattern with EventBridge Pipes (DynamoDB Streams → Pipe → target). Add schema registry to undocumented custom event bus.

Domain 4 — Migration: Migrate on-premises event bus (IBM MQ, TIBCO) to EventBridge + SQS. Replace cron jobs on EC2 with EventBridge Scheduler (serverless, managed). Replace SNS fan-out with EventBridge where content-based routing is needed.

🔗 32.18 Related Services

Service	Relationship
Amazon SQS	EventBridge rule target. EventBridge Pipes source. Provides durable buffering for EventBridge-routed events.
Amazon SNS	EventBridge rule target for fan-out. EventBridge Pipes target.
AWS Lambda	Most common EventBridge target. Invoked synchronously by EventBridge rules.
AWS Step Functions	EventBridge rule target to start state machine execution. Pipes enrichment stage.
Amazon Kinesis	EventBridge rule target (Data Streams). EventBridge Pipes source (Data Streams, Firehose).
Amazon DynamoDB	EventBridge Pipes source (DynamoDB Streams). EventBridge rule cannot write to DynamoDB directly.
AWS CloudWatch	Metrics for EventBridge rules (MatchedEvents, FailedInvocations). EventBridge replaced CloudWatch Events.
AWS CloudTrail	CloudTrail management events appear on EventBridge default bus — pattern match on API calls for real-time response.
Route 53	Health checks for EventBridge Global Endpoints multi-region failover.
Amazon MSK / Amazon MQ	EventBridge Pipes sources for Kafka and RabbitMQ streams.

End of Chapter 32 — Amazon EventBridge

1489. [SNS] Describe the SNS topic policy structure needed to allow only HTTPS delivery to an SQS queue subscriber. ; Queue policy on the SQS queue:

```
{
  "Effect": "Allow",
  "Principal": {"Service": "sns.amazonaws.com"},
  "Action": "sqs:SendMessage",
  "Resource": "arn:aws:sqs:...",
  "Condition": {
    "ArnEquals": {"aws:SourceArn": "arn:aws:sns:..."},
    "Bool": {"aws:SecureTransport": "true"}
  }
}
```

SNS itself delivers to SQS using its own internal mechanism (not HTTPS — it's an AWS-to-AWS call). The `aws:SecureTransport` condition ensures the SQS API call from SNS uses TLS. SNS→SQS is always internal AWS HTTPS — this condition is satisfied by default.

For HTTPS-only at the SNS level (publisher to SNS): add SNS topic policy denying publishes over non-HTTPS: `"Condition":{"Bool":{"aws:SecureTransport":"false"}}` with `Effect: Deny`. This ensures producers can only publish to the topic over HTTPS. For SQS queue policy: `aws:SecureTransport: true` condition is redundant for SNS internal calls but enforces HTTPS for any other caller trying to use the queue directly.
Multi-layer HTTPS enforcement: SNS topic policy (deny non-HTTPS publish) + SQS queue policy (deny non-HTTPS SendMessage) + Lambda function policy (deny non-HTTPS invoke). Layered defense ensures data in transit is encrypted at every hop. IAM condition `aws:SecureTransport` checks if the request was made using HTTPS/TLS.

Chapter 28 Extended — Aurora Serverless, EKS, Timestream, ALB, EC2, Lambda, App Runner, Step Functions, API Gateway

1573. [Aurora Serverless] A startup needs a dev/test MySQL database that is unused 20 hours per day. They want to pay zero cost during idle hours. Which Aurora Serverless version should they use, and what feature enables zero cost? - A) Aurora Serverless v2 with min ACU = 0 - B) Aurora Serverless v1 with pause/resume enabled - C) Aurora Serverless v2 with

scale-to-zero flag - D) Provisioned Aurora with auto-stop schedule > **Answer: B**. v1 supports pause to 0 ACU = zero cost. v2 minimum 0.5 ACU → always incurs cost.

1574. [Aurora Serverless] A production application experiences unpredictable traffic spikes from 100 to 50,000 concurrent connections. The team needs instant scaling with no connection drops. Which solution is best? - A) Aurora Serverless v1 with max ACU = 256 - B) Aurora Provisioned with horizontal read replica auto-scaling - C) Aurora Serverless v2 with appropriate ACU range - D) Aurora Serverless v1 with increased cooldown period > **Answer: C**. v2 provides sub-second scaling without connection disruption. v1 takes minutes and can interrupt connections.

1575. [Aurora Serverless] A Lambda function needs to query Aurora Serverless without maintaining persistent VPC connections and without storing database credentials. What is the recommended approach? - A) RDS Proxy in the VPC with Lambda VPC configuration - B) Aurora Data API with IAM authentication - C) Direct TCP connection from Lambda to Aurora endpoint - D) Secrets Manager rotation with Lambda environment variable injection > **Answer: B**. Data API = HTTP-based, no TCP connection, IAM auth via SigV4. No VPC required for Lambda.

1576. [Aurora Serverless] Which feature is NOT available with Aurora Serverless v1? - A) Pause to zero ACU when idle - B) Aurora Global Database - C) Aurora Data API - D) MySQL and PostgreSQL compatibility > **Answer: B**. Global Database requires v2. v1 supports Data API, pause/resume, MySQL/PostgreSQL.

1577. [Aurora Serverless] A company migrates from Aurora Serverless v1 to v2. What is the key operational difference they should expect? - A) v2 requires manual ACU configuration for each request - B) v2 can be deployed as part of a Multi-AZ Aurora cluster with reader instances - C) v2 cannot be used with RDS Proxy - D) v2 charges per connection, not per ACU > **Answer: B**. v2 supports Multi-AZ, read replicas, and Global DB. v1 supports none of these.

1578. [EKS] A company runs an EKS cluster. Each microservice pod needs to access a different S3 bucket with specific IAM permissions. A developer suggests using the EC2 node instance profile. A Solutions Architect disagrees. What is the correct approach and why? - A) Node instance profile is correct — it provides least-privilege pod-level access - B) IRSA (IAM Roles for Service Accounts) — pod-level IAM role via K8s ServiceAccount annotation - C) Kube2IAM — legacy approach required for managed node groups - D) STS AssumeRole from application code using hardcoded credentials > **Answer: B**. Node instance profile grants ALL pods on the node the same broad IAM access. IRSA maps each ServiceAccount to a specific IAM role, achieving least privilege per microservice.

1579. [EKS] A pod in an EKS cluster needs to write to an EBS volume. When the developer creates a PersistentVolumeClaim with ReadWriteMany, the pod fails to bind. Why? - A) EBS volumes require IRSA configuration before mounting - B) EBS volumes support ReadWriteOnce only — one node at a time - C) EBS CSI driver does not support managed node groups - D) The pod must use Fargate to access EBS volumes > **Answer: B**. EBS is block storage = ReadWriteOnce (one node). For ReadWriteMany shared storage use EFS CSI (ReadWriteMany).

1580. [EKS] An EKS cluster needs to scale node groups automatically when pods are pending. Two options are under consideration: Cluster Autoscaler and Karpenter. What is the primary advantage of Karpenter? - A) Karpenter integrates with AWS Savings Plans automatically - B) Karpenter provisions directly from EC2 (faster, right-sized, no need to pre-define node groups) - C) Karpenter is the only option that works with managed node groups - D) Karpenter requires Fargate, Cluster Autoscaler requires EC2 > **Answer: B**. Karpenter provisions nodes directly via EC2 Fleet, right-sizes per pending pod spec, much faster than Cluster Autoscaler + managed node group ASG.

1581. [EKS] A developer cannot reach the private EKS API server endpoint using kubectl from their laptop. VPN to the VPC is configured and working. What is the likely cause? - A) EKS cluster does not support private endpoints - B) The EKS cluster has public endpoint access disabled and private endpoint enabled; kubectl needs VPC routing - C) kubectl is deprecated for EKS — use eksctl only - D) IAM permissions for the developer's role are missing the KMS:Decrypt permission > **Answer: B**. When only private endpoint is enabled, kubectl must come from within the VPC (via VPN, Direct Connect, or Cloud9 in VPC).

1582. [EKS] An ALB needs to route /api/* to one K8s service and /web/* to another, sharing a single load balancer. How is this configured? - A) Create two ALBs — one per service — with separate Ingress resources - B) Use AWS Load Balancer Controller with a single Ingress resource defining path-based rules - C) Use NLB with port-based routing (8080 → api, 80 → web) - D) Use Route 53 weighted routing to split between two ALBs > **Answer: B**. ALB Load Balancer Controller creates one ALB from a single Ingress with path rules → two target groups.

1583. [EKS] A DaemonSet needs to run a monitoring agent on every EKS node. The cluster uses Fargate profiles for all workloads. Will this work? - A) Yes — Fargate supports DaemonSets via privileged pod configuration - B) No — EKS Fargate does not support DaemonSets; use sidecar containers instead - C) Yes — use Fargate profile with DaemonSet toleration - D) No — switch to self-managed nodes, managed node groups do not support DaemonSets > **Answer: B**. Fargate = serverless, no node concept for DaemonSets. DaemonSets require EC2 nodes (managed or self-managed).

1584. [Timestream] A company collects IoT sensor data from 50,000 devices. Each device sends temperature and humidity every 10 seconds. The team needs to query 1-year trend data and real-time last-hour data. Which database is most appropriate and why? - A) DynamoDB with TTL for old records and GSI for time queries - B) RDS PostgreSQL with time-based partitioning - C) Amazon Timestream with memory store (recent) + magnetic store (historical) - D) Amazon Kinesis Data Streams with S3 sink for historical > **Answer: C**. Timestream built for time-series: memory store for recent fast queries, magnetic store for cheap historical, auto-tiering, built-in time-series SQL functions.

1585. [Timestream] A developer writes a query to calculate the average CPU utilization for all servers in 5-minute bins over the last 24 hours. Which Timestream function groups time into fixed intervals? - A) `WINDOW(time, 5m)` - B) `bin(time, 5m)` - C) `round_time(time, 300)` - D) `DATE_TRUNC('minute', time)` > **Answer: B**. `bin(time, 5m)` is the Timestream function for time-interval binning.

1586. [Timestream] An IoT platform ingests sensor data via AWS IoT Core. The team wants to store all sensor readings directly in Timestream without writing a Lambda function. What is the recommended integration? - A) IoT Core → Kinesis → Lambda → Timestream - B) IoT Core Rule with direct Timestream action - C) IoT Core → SQS → Timestream consumer - D) IoT Greengrass → Timestream SDK > **Answer: B**. IoT Core Rules have a direct Timestream write action — no Lambda needed.

1587. [Timestream] Which scenario is LEAST appropriate for Amazon Timestream? - A) Monitoring EC2 CPU utilization trends over 90 days - B) Storing 5-year historical financial transactions with complex relational joins - C) Fleet management tracking vehicle location every 30 seconds - D) DevOps alerting based on application error rate over time > **Answer: B**. Timestream = time-series append-only; complex relational OLTP with joins → RDS/Aurora. Options A, C, D are classic time-series workloads.

1588. [ALB] A company runs microservices behind one ALB. Traffic to /api/* should go to an API EC2 Auto Scaling Group. Traffic to /static/* should go to an S3-backed origin. Traffic to anything else should go to a web EC2 group. What ALB configuration achieves this? - A) Three separate ALBs, each with a different DNS record - B) One ALB listener with three rules: path /api/, *path /static/*, and default rule - C) One ALB with three separate listeners on different ports - D) Route 53 path-based routing to three separate EC2 groups > **Answer: B**. ALB path-based rules on one listener: rule 1 (/api/) → *TG-A, rule 2 (/static/*) → TG-S3, default → TG-Web.

1589. [ALB] A developer says they need an NLB for "smart routing based on URL path." Is this correct? - A) Yes — NLB supports path-based routing at Layer 7 - B) No — NLB is Layer 4 (TCP/UDP); path routing requires ALB (Layer 7) - C) Yes — NLB with WAF enabled supports path routing - D) No — path routing requires API Gateway, not ALB or NLB > **Answer: B**. NLB = Layer 4, routes by IP + port only. Path routing = ALB (Layer 7).

1590. [ALB] Users report that their shopping cart is lost when ALB routes them to a different EC2 instance. The application stores session data in instance memory. What ALB feature can resolve this without changing the application? - A) Enable ALB cross-zone load balancing - B) Enable sticky sessions on the target group (AWSALB cookie) - C) Enable slow-start mode on the target group - D) Enable connection draining with a 300-second timeout > **Answer: B**. Sticky sessions (cookie-based) route each user to the same EC2 instance for the cookie duration.

1591. [ALB] An ALB listener has reached its maximum number of routing rules. The team wants to add more microservice path routes. What is the maximum rule limit and how can this be addressed? - A) 10 rules — upgrade to NLB for more - B) 100 rules — use nested Lambda authorizers to implement additional routing - C) 100 rules — refactor to use host-based routing with multiple domain names or use API Gateway as a secondary router - D) Unlimited rules — this error indicates a misconfiguration > **Answer: C**. 100 rules per listener soft limit. Solution: host-based routing to segment traffic, or use API GW behind ALB for complex routing.

1592. [EC2 Purchasing] A company needs EC2 compute for a 3-year production database workload. The instance type is known (m5.4xlarge). They want maximum cost savings. Which option is best? - A) Compute Savings Plans, 3-year, no-upfront - B) Standard Reserved Instance, 3-year, all-upfront - C) Convertible Reserved Instance, 3-year, all-upfront - D) EC2 Instance Savings Plans, 3-year, partial-upfront > **Answer: B**. Standard RI + all-upfront + 3-year = up to 72% discount. Known instance type, 3-year commitment = Standard RI wins.

1593. [EC2 Purchasing] A data science team runs GPU training jobs that can be interrupted. Jobs checkpoint every 30 minutes to S3. Which purchase model reduces EC2 costs most? - A) Reserved Instances (GPU family, 1-year) - B) Dedicated Hosts for GPU instances - C) Spot Instances with interruption handling and checkpoint restart - D) On-Demand with Auto Scaling > **Answer: C**. Spot = up to 90% discount. ML training is classic Spot use case — fault-tolerant, checkpoint restart handles 2-min warning.

1594. [EC2 Purchasing] A company wants discounts that apply to any EC2 instance family, any region, plus Lambda and Fargate compute. Which option should they choose? - A) Standard Reserved Instances (Regional scope) - B) Convertible Reserved Instances - C) Compute Savings Plans - D) EC2 Instance Savings Plans > **Answer: C**. Compute Savings Plans

cover EC2 (any family, any region, any OS) + Lambda + Fargate. RI = EC2 only. EC2 Instance SP = specific family + region.

1595. [EC2 Purchasing] A financial services company runs Oracle Database on EC2 and needs BYOL (per-socket license). They need to track which physical socket an instance runs on. What purchasing option satisfies this? - A) Dedicated Instance - B) Dedicated Host - C) Reserved Instance with host affinity - D) On-Demand with placement group > **Answer: B**. Dedicated Host provides host-level visibility (sockets, cores, host ID) required for BYOL per-socket Oracle licensing. Dedicated Instance = dedicated hardware but no host visibility.

1596. [EC2 Purchasing] A company buys Standard Reserved Instances for m5.4xlarge. Six months later they need to switch to c5.4xlarge. Is this possible? - A) Yes — Standard RI can be exchanged for any instance type - B) No — Standard RI cannot change family (m5 → c5); requires Convertible RI for family changes - C) Yes — sell on RI Marketplace then buy new RI - D) No — neither Standard nor Convertible RI can be exchanged > **Answer: C**. Standard RI cannot be exchanged for different family. Workaround: sell on RI Marketplace (Convertible RI cannot be sold) and buy new RI. Note: B is also partially correct — but C is the operational path.

1597. [EC2 Purchasing] A company wants to guarantee EC2 capacity in a specific AZ for disaster recovery but does not want to commit to a 1-year RI. What option should they use? - A) Zonal Reserved Instance - B) On-Demand Capacity Reservation - C) Spot Capacity Reservation - D) Compute Savings Plan with zonal scope > **Answer: B**. On-Demand Capacity Reservation reserves capacity in specific AZ, no term commitment, billed at On-Demand rate even if unused.

1598. [EC2 Placement] An HPC cluster requires maximum network bandwidth between 100 EC2 instances for MPI all-reduce operations. All instances are in us-east-1a. Which placement group type should they use? - A) Spread Placement Group - B) Partition Placement Group (1 partition) - C) Cluster Placement Group - D) No placement group — EFA handles bandwidth automatically > **Answer: C**. Cluster PG = all instances on same low-latency high-bandwidth fabric, up to 100 Gbps with ENA. Required for HPC MPI workloads.

1599. [EC2 Placement] A company runs Apache Cassandra with a replication factor of 3 across hundreds of nodes per AZ. They need hardware-fault isolation between replicas. Which placement group type should they use? - A) Cluster Placement Group (3 instances per cluster) - B) Spread Placement Group (max 7 per AZ) - C) Partition Placement Group (3 partitions — 1 replica per partition) - D) No placement group — Cassandra handles replication natively > **Answer: C**. Partition PG with 3 partitions = each partition on separate rack. Place each Cassandra replica in a different partition = no shared hardware failure correlation.

1600. [EC2 Placement] A team needs 5 Zookeeper nodes each placed on completely separate physical hardware (separate power and network). Which placement group satisfies this with the minimum configuration? - A) Cluster Placement Group with 5 instances - B) Partition Placement Group with 5 partitions, 1 node each - C) Spread Placement Group — 5 instances across AZs - D) No placement group with EC2 Auto Scaling multi-AZ > **Answer: C**. Spread PG guarantees each instance on separate rack (distinct power + network). 5 instances well within the 7/AZ limit. Partition PG also works but Spread is simpler for small HA groups.

1601. [EC2 Placement] A Cluster Placement Group is requested spanning us-east-1a, us-east-1b, and us-east-1c. What will happen? - A) AWS will create the group spanning all three AZs - B) Launch will fail — Cluster Placement Group is single-AZ only - C) Only instances in us-east-1a (first AZ) will join the group - D) AWS will automatically convert to Partition Placement Group > **Answer: B**. Cluster PG = single AZ restriction. Any cross-AZ attempt = launch error.

1602. [Lambda] A Lambda function processes 2,000 concurrent requests per second. After memory optimization it still times out after 15 minutes due to a long-running ML inference job. What should the architect recommend? - A) Increase Lambda memory to 10 GB to speed up processing - B) Move the ML inference job to SageMaker Batch Transform or ECS Fargate — beyond Lambda's 15-minute limit - C) Use Lambda Provisioned Concurrency to pre-warm more instances - D) Use Lambda@Edge to distribute processing globally > **Answer: B**. Lambda max timeout = 15 minutes. ML inference taking longer → ECS Fargate task or SageMaker.

1603. [Lambda] Production Lambda functions show high latency on the first request after periods of inactivity. Latency drops to <10ms on subsequent requests. What should the architect enable? - A) Enhanced monitoring via Lambda Insights - B) Provisioned Concurrency to pre-initialize execution environments - C) Reserved Concurrency to guarantee capacity - D) Increase memory — cold starts scale with memory size > **Answer: B**. Cold starts = new execution environment init. Provisioned Concurrency = pre-warmed environments = no cold start latency.

1604. [Lambda] A Lambda function inside a VPC needs to make API calls to DynamoDB. Adding a NAT Gateway seems overly complex. What is a simpler, cheaper approach? - A) Move the Lambda function outside the VPC - B) Enable internet gateway on the Lambda VPC - C) Create a VPC Gateway Endpoint for DynamoDB - D) Use DynamoDB Accelerator (DAX) to route requests > **Answer: C**. VPC Gateway Endpoint for DynamoDB (free) allows VPC Lambda to access DynamoDB without internet routing. NAT Gateway costs $0.045/GB+0.045$/hour.

1605. [Lambda] A Lambda deployment package exceeds 250 MB unzipped (heavy ML dependencies). What deployment mechanism allows Lambda to use this large package? - A) Upload to S3 and increase the Lambda unzipped limit to 500 MB - B) Lambda Layers — split large libraries into up to 5 layers - C) Package as a container image (up to 10 GB) and push to ECR - D) Use Lambda@Edge with CloudFront CDN to distribute the package > **Answer: C**. Lambda container images support up to 10 GB. Layers max 250 MB total unzipped. Container image is the solution for very large dependencies.

1606. [Lambda] A Lambda function triggers other Lambda functions in parallel (fan-out). As traffic increases, the orchestrator Lambda hits concurrency limits and error rates increase. What is the better architectural pattern? - A) Increase Lambda account concurrency limit - B) Replace Lambda orchestrator with Step Functions Parallel state - C) Add SQS queue between orchestrator and worker Lambdas - D) Use EventBridge to fan out to multiple Lambda targets > **Answer: B**. Step Functions Parallel state orchestrates without consuming Lambda concurrency for coordination. Also provides retry, error handling, state visibility — a more robust orchestration pattern.

1607. [App Runner] A developer wants to deploy a Node.js REST API directly from a GitHub repository with zero infrastructure management, automatic HTTPS, and scale-to-zero capability. What is the best AWS service? - A) ECS Fargate with CodePipeline - B) Elastic Beanstalk with auto-scaling - C) AWS App Runner - D) Lambda with API Gateway > **Answer: C**. App Runner deploys directly from GitHub, manages HTTPS, auto-scales, optional scale-to-zero. Zero infrastructure ops.

1608. [App Runner] An App Runner service needs to query a private Amazon RDS database in a VPC. What App Runner feature enables this? - A) App Runner Fargate profile with VPC subnet configuration - B) App Runner VPC Connector - C) App Runner cannot access VPC resources — move to ECS - D) VPC peering between App Runner and the RDS VPC > **Answer: B**. VPC Connector attaches App Runner service to a VPC, enabling access to RDS, ElastiCache, and other private resources.

1609. [Step Functions] A workflow processes an order, then requires a human compliance review before shipping. The review may take 2–3 business days. The workflow must pause and resume when approved. Which Step Functions integration pattern supports this? - A) Request-Response pattern with a polling Lambda - B) Synchronous integration (.sync:2) to a human review Lambda - C) Wait for Task Token — pause until external callback with token - D) Express Workflow with 5-minute timer and retry > **Answer: C**. WaitForTaskToken = workflow pauses, sends token to external system (human portal, email, ticketing). Human approval → callback with token → workflow resumes.

1610. [Step Functions] A financial system runs 500,000 payment validations per second. Each validation takes <1 second. Total execution time < 5 minutes. Which Step Functions workflow type is appropriate? - A) Standard Workflow — for financial data, always use Standard for audit - B) Express Workflow — high throughput (100,000/sec), <5-minute duration - C) Standard Workflow with increased TPS limit - D) Neither — Step Functions cannot handle 500K/sec; use Lambda directly > **Answer: B**. Express Workflow: 100,000 executions/second (vs 2,000 Standard), duration ≤5 min. High-volume short-duration = Express.

1611. [Step Functions] A Step Functions workflow calls a Lambda function that occasionally fails with a transient timeout error. The architect wants automatic retry with exponential backoff, and if all retries fail, route to an error handler state. What is the correct configuration? - A) Configure Lambda with retry policy and add DLQ - B) Add `Retry` block on the Task state with `BackoffRate`, and `Catch` block pointing to error state - C) Wrap Lambda call in another Lambda with try/catch - D) Use EventBridge Dead Letter Queue to catch failed executions > **Answer: B**. Step Functions Task state supports `Retry` (MaxAttempts, IntervalSeconds, BackoffRate) + `Catch` for routing to error handling state. Native to state machine, no Lambda changes needed.

1612. [API Gateway] A company offers a public REST API with three SaaS subscription tiers: Bronze (100 RPS, 10K/day), Silver (500 RPS, 100K/day), Gold (1000 RPS, 1M/day). How should this be implemented in API Gateway? - A) Three separate API Gateways with different throttle settings - B) One REST API with three Usage Plans, each with rate/quota limits, associated with customer API Keys - C) One HTTP API with JWT claims for tier identification - D) Lambda Authorizer reading tier from DynamoDB and applying throttle > **Answer: B**. REST API + Usage Plans = per-customer rate limits and quotas. API Keys associate customers to plans. This is the canonical SaaS tiering pattern.

1613. [API Gateway] A team has a REST API with response caching enabled. A client sends `Cache-Control: max-age=0` in the request. What happens? - A) API Gateway ignores this header - B) API Gateway returns a 400 error - C) API Gateway bypasses cache and calls the backend, then refreshes cache - D) API Gateway clears the entire stage cache > **Answer: C**. `Cache-Control: max-age=0` = cache invalidation request per RFC 7234. API GW calls backend, caches fresh response. Stage cache not fully cleared — just this key.

1614. [API Gateway] An API Gateway REST API serves global users and needs lowest latency worldwide. An ACM certificate is required. In which region must the ACM certificate be provisioned? - A) Same region as the REST API - B) us-west-2

(primary region) - C) us-east-1 (N. Virginia) - D) Any region — API Gateway auto-replicates certificates > **Answer: C**. Edge-Optimized API GW uses CloudFront distribution → cert must be in us-east-1, same as CloudFront origin cert requirement.

1615. [API Gateway] A development team needs an API Gateway that: validates JWT tokens natively, has very low latency, costs as little as possible, but does NOT need response caching or per-customer usage plans. Which type? - A) REST API with Cognito Authorizer - B) HTTP API with JWT Authorizer - C) REST API with Lambda Authorizer - D) WebSocket API with JWT > **Answer: B**. HTTP API = 70% cheaper than REST, lower latency, native JWT Authorizer. No caching and no usage plans needed → HTTP API is optimal.

1616. [API Gateway] A company's API Gateway REST API needs to write directly to a DynamoDB table without any Lambda function in the middle. How is this configured? - A) HTTP Proxy integration pointing to DynamoDB endpoint URL - B) AWS Service Integration with DynamoDB action mapping via VTL template - C) Lambda Proxy integration with DynamoDB SDK call in Lambda - D) Not possible — API Gateway cannot write to DynamoDB directly > **Answer: B**. AWS Service Integration = direct SDK-level call to DynamoDB (PutItem, Query, etc.) with VTL request/response mapping. No Lambda needed.

1617. [API Gateway] An organization runs an internal API exposed only within its VPC. No traffic should ever reach the public internet. Which API Gateway type and configuration achieves this? - A) Regional API with security group blocking public IPs - B) Private API with VPC Interface Endpoint and resource policy restricting to the VPC - C) Edge-Optimized API with WAF rule blocking all non-RFC-1918 traffic - D) HTTP API with VPC Link to an internal ALB > **Answer: B**. Private API = only accessible via VPC Interface Endpoint (PrivateLink). Resource policy restricts to specific VPC. No public routing at all.

1618. [API Gateway] A mobile application uses Cognito User Pools for authentication. The frontend app calls API Gateway REST API endpoints. How should authorization be configured on API Gateway? - A) Lambda Authorizer validates Cognito JWT - B) Cognito User Pool Authorizer — API GW validates Cognito JWT natively - C) IAM Auth with SigV4 signature from mobile client - D) API Key in mobile app headers > **Answer: B**. Cognito User Pool Authorizer = API GW validates JWT from Cognito natively. No Lambda needed. IAM SigV4 from mobile = complex and exposes credentials.

1619. [API Gateway] An API Gateway stage needs to gradually roll out a new deployment to 10% of traffic while monitoring for errors. What feature enables this? - A) Weighted target groups on the backing ALB - B) Canary release on the API Gateway stage - C) Route 53 weighted routing between two API GW endpoints - D) Lambda alias with weighted routing behind the API GW > **Answer: B**. API GW stage canary: route X% to new deployment version, monitor CloudWatch metrics, then promote or roll back.

1620. [Multi-topic] A startup is building an event-driven food ordering system. Orders arrive via API Gateway → Lambda → Step Functions. Steps: validate order (Lambda), charge payment (Lambda), if charge fails → send refund notification (SNS), if success → reserve inventory (DynamoDB), notify customer (SNS). Step Functions workflow must retry payment 3 times with backoff before failure. Which Step Functions workflow type and integration pattern is correct for this short-lived (<2 min) high-volume scenario? - A) Standard Workflow, Request-Response - B) Express Workflow, Synchronous (.sync:2) for each step - C) Express Workflow, with Retry blocks on payment task and Catch routing to SNS notification - D) Standard Workflow, WaitForTaskToken for payment approval > **Answer: C**. Express Workflow = high-volume, <5 min. Retry on payment task with BackoffRate, Catch → SNS notification state on failure.

1621. [Multi-topic] A financial services company needs to migrate an on-premises Oracle Database running on a physical server to AWS. The Oracle license is per-socket (2-socket server). They need to maintain BYOL compliance and control socket placement. Which combination is correct? - A) Dedicated Instance + Reserved Instance - B) Dedicated Host + RDS Oracle BYOL - C) Dedicated Host (known socket count) + EC2 Oracle self-managed - D) Placement Group (Cluster) + Standard Reserved Instance > **Answer: C**. Dedicated Host = specific physical host with known socket/core count = BYOL per-socket compliance. EC2 self-managed Oracle with your license. RDS Oracle BYOL uses Dedicated Instance, not Dedicated Host.

1622. [Multi-topic] A company runs a large Kafka cluster on EC2 in AWS with 150 broker instances. They want hardware-fault isolation between broker groups (3 groups of 50 brokers). Each group should be isolated from the others at the rack level. Which placement strategy? - A) Spread Placement Group — 150 instances across racks - B) Cluster Placement Group — low-latency Kafka inter-broker communication - C) Partition Placement Group — 3 partitions, 50 brokers each - D) No placement group — Kafka handles rack-awareness natively via `broker.rack` config > **Answer: C**. Partition PG with 3 partitions = 3 separate racks. 150 brokers (50 per partition) well within per-AZ limits. Spread PG max 7 instances per AZ — impossible for 150. Use `broker.rack` with partition ID for Kafka rack-awareness.

1623. [Multi-topic] A DevOps team monitors 1,000 EC2 instances. Each instance emits CPU, memory, and disk metrics every 30 seconds. They need to query per-host trends and alert when any metric breaches a threshold over the last 7 days. They want minimum operational overhead. What is the best storage and query solution? - A) CloudWatch Custom

Metrics with CloudWatch Alarms - B) Kinesis → Lambda → DynamoDB with TTL = 7 days - C) Kinesis → Timestream with memory store 1h + magnetic store 7 days + CloudWatch Alarms via Lambda - D) Kinesis → Timestream with memory store + magnetic store + Grafana for dashboards > **Answer: D**. Timestream = purpose-built time-series, auto-tiering memory→magnetic, serverless. Grafana native plugin. CloudWatch Custom Metrics works but Timestream+Grafana is the specialized solution for 1000-host metrics at this scale.

1624. [Multi-topic] A company wants to containerize a legacy web application and deploy it with zero Kubernetes knowledge, automatic HTTPS, GitHub-based deployments, and the ability to connect to a private RDS instance. What is the most appropriate service? - A) EKS with managed node groups and ALB Ingress Controller - B) ECS Fargate with CodePipeline and Application Load Balancer - C) AWS App Runner with VPC Connector - D) Elastic Beanstalk with Docker platform > **Answer: C**. App Runner = GitHub deploy, zero K8s/ECS knowledge, auto HTTPS, VPC Connector for RDS access. Simplest operational path.

1625. [Multi-topic] An e-commerce site uses Aurora Serverless v2 for its product catalog database. A Lambda function that queries the DB via the Data API is experiencing 500ms+ latency per call. The team wants sub-10ms queries. What should they implement? - A) Switch to Aurora Serverless v1 — v1 Data API is faster - B) Use RDS Proxy + direct TCP connection from VPC Lambda instead of Data API - C) Add ElastiCache for Redis as a caching layer in front of Aurora - D) Increase Aurora Serverless v2 maximum ACU > **Answer: C**. Data API adds HTTP overhead → not suitable for sub-10ms. ElastiCache Redis caches hot product catalog = sub-millisecond reads. Or B (RDS Proxy + VPC Lambda) reduces latency but 500ms → 10ms improvement = caching (C).

1626. [Multi-topic] A company runs an EKS cluster with Fargate profiles. A new requirement needs a pod to mount persistent block storage (EBS) for a stateful database container. Will this work with Fargate? - A) Yes — EBS CSI driver supports EKS Fargate with ReadWriteOnce - B) No — EKS Fargate does not support EBS persistent volumes; use EC2 node groups or EFS - C) Yes — use Fargate with `volumeClaimTemplate` in StatefulSet - D) No — EKS Fargate does not support any persistent storage > **Answer: B**. EKS Fargate does not support EBS (block storage). Use EC2 managed node groups with EBS CSI driver for EBS PVs. Fargate DOES support EFS (shared file storage) via EFS CSI.

1627. [Multi-topic] An API Gateway endpoint is hammered by a sudden traffic spike. The Lambda backend gets throttled, causing 429 errors. Usage Plans are not configured. What is the fastest way to protect the backend from being overwhelmed? - A) Enable Lambda Provisioned Concurrency - B) Add an SQS queue between API Gateway and Lambda - C) Configure throttling limits on the API Gateway stage or method level - D) Switch from REST API to HTTP API — lower latency handles more requests > **Answer: C**. Stage/method-level throttling on API GW limits requests before they hit Lambda. Fastest configuration change. SQS (B) = async, breaks synchronous API contract.

1628. [Multi-topic] A startup architect is choosing between Lambda and ECS Fargate for a new microservice that handles HTTP requests. The service receives 10 requests/day on weekdays and 50,000 requests/hour on weekends. Which choice minimizes cost? - A) ECS Fargate — always-on, consistent performance - B) Lambda — scales to zero, pay-per-invocation, ideal for spiky traffic - C) EKS Fargate — per-pod billing, better for bursts - D) App Runner — scale-to-zero, fixed monthly minimum > **Answer: B**. Lambda = scale to zero during weekday idle (10 req/day = essentially free). Weekend 50K/hour handled via concurrency scaling. ECS/EKS Fargate has no scale-to-zero. App Runner has optional scale-to-zero but Lambda is simpler for request-driven workloads.

1629. [Multi-topic] A Step Functions Standard Workflow state machine orchestrates a 3-hour data pipeline (Glue jobs → Lambda → DynamoDB writes). The Glue job takes 2 hours. Which Step Functions integration pattern should be used for the Glue job state? - A) Request-Response — fire Glue job, immediately proceed to next state - B) Synchronous (.sync:2) — wait for Glue job completion before proceeding - C) WaitForTaskToken — send token to Glue, Glue calls back when done - D) Express Workflow — 2-hour Glue job fits within 5-minute limit > **Answer: B**. Synchronous (.sync:2) = Step Functions calls Glue StartJobRun and polls for completion via Glue API. No custom token callback needed. Express Workflow max 5 min → wrong for 2-hour job. Request-Response would not wait for Glue to finish.

1630. [Multi-topic] An organization needs to route traffic from a public API Gateway endpoint to EC2 nodes in a private subnet. The EC2 nodes run different services on different ports. They want path-based routing. What architecture achieves this? - A) API Gateway REST API → VPC Link → NLB → EC2 instances - B) API Gateway REST API → VPC Link → ALB (path-based rules) → EC2 target groups - C) API Gateway HTTP API → Lambda → EC2 via SDK - D) API Gateway → Direct backend integration → EC2 private IPs > **Answer: B**. API GW + VPC Link connects to private ALB. ALB does path-based routing to different EC2 target groups. VPC Link requires NLB or ALB (as VPC Link v2 supports ALB). This is the standard pattern for API GW → private services.

1631. [Aurora Serverless] A company has Aurora Serverless v2 deployed in a single AZ. During a primary AZ outage, the database becomes unavailable for 45 minutes. What should be done to improve availability? - A) Deploy Aurora Serverless v2 with Multi-AZ enabled (Aurora cluster with writer + reader in different AZs) - B) Switch to Aurora Serverless v1 which has built-in AZ redundancy - C) Add a read replica in the same AZ - D) Use RDS Proxy to buffer requests

during AZ outage > **Answer: A**. Aurora Serverless v2 supports Multi-AZ (v1 does not). Deploy writer in AZ-A, reader in AZ-B. Failover < 30s vs 45-min outage.

1632. [EKS] A company needs to run 200 identical parallel processing jobs in EKS. Each job takes 15 minutes. Jobs are triggered by S3 object uploads. The team wants to minimize idle node capacity between jobs. What architecture is most cost-efficient? - A) EKS with Managed Node Groups always running 200 EC2 instances - B) EKS with Fargate Profiles — each job runs as a Fargate pod (no idle EC2 nodes) - C) EKS with Karpenter on Spot instances — Karpenter provisions nodes when jobs arrive, terminates when done - D) ECS Fargate tasks triggered by S3 events via Lambda > **Answer: C**. Karpenter on Spot = lowest cost. Provisions nodes on demand, terminates after job completion. No idle EC2 capacity. Option B (Fargate) also works but Spot is cheaper; C is the cost-optimized answer for EKS.

1633. [API Gateway] An API Gateway deployment is failing canary promotion due to a spike in 5xx errors from the new deployment. How should the team respond? - A) Manually delete the new deployment and redeploy the old one - B) Roll back by setting canary traffic weight to 0% then promote the stable deployment - C) Increase API GW throttle limits to reduce 5xx errors - D) Switch from canary to blue-green by creating a new API GW endpoint > **Answer: B**. Canary rollback: set canary % to 0 → all traffic to stable. Then fix issues. Promote when ready.

1634. [Lambda] A Lambda function processes messages from SQS. Messages occasionally cause poison-pill processing failures (Lambda errors). After Lambda max retries, failed messages should be inspected rather than deleted. What should be configured? - A) Lambda destination for failures pointing to another SQS queue - B) SQS Dead Letter Queue on the SQS source queue; Lambda ESM maxReceiveCount - C) Lambda DLQ (Lambda-level DLQ) configured to SQS - D) Step Functions Catch state around the Lambda invocation > **Answer: B**. SQS DLQ + ESM (Event Source Mapping) maxReceiveCount. After N failures, SQS moves message to DLQ for inspection. Lambda DLQ (C) works for async Lambda invocations, not SQS ESM.

1635. [Multi-topic] A solutions architect must choose the cheapest API Gateway type for a simple CRUD REST API backed by Lambda, using Cognito JWT tokens for auth, no response caching required, and targeting internal microservices with low latency. What is the correct choice? - A) REST API — more features, better for microservices - B) HTTP API — 70% cheaper, native JWT Authorizer, lower latency, all requirements met - C) WebSocket API — bidirectional for microservice communication - D) Private API — zero public exposure for internal services > **Answer: B**. HTTP API meets all requirements (CRUD, Lambda, JWT/Cognito, no caching) at 70% cost savings vs REST API.

1636. [EC2 Purchasing] An Auto Scaling Group uses a mix of On-Demand (base capacity 2) and Spot (above base) instances. During a Spot interruption, the ASG replaces the interrupted instance. Which Spot interruption handling strategy keeps the instance for a period to complete in-flight work? - A) Terminate (default) — immediately replaces the Spot instance - B) Stop — saves instance state, can be resumed if Spot capacity returns - C) Hibernate — saves RAM to EBS, resumes from hibernation - D) Drain — sends interrupt to ELB to deregister before termination > **Answer: B**. Stop strategy for EBS-backed instances = saves instance state to EBS, resumes when Spot capacity available. Hibernate (C) saves RAM too. Drain (D) is ELB connection draining, not a Spot strategy.

1637. [Step Functions] A developer wants to transform data from an S3 file, run it through 3 Lambda functions in sequence, store results in DynamoDB, and send a completion email via SES. The entire workflow should be monitored and retried automatically. Which state machine design is optimal? - A) Single Lambda function that calls all 3 Lambda functions and DynamoDB - B) Step Functions state machine: Task (L1) → Task (L2) → Task (L3) → Task (DynamoDB PutItem) → Task (SES SendEmail), with Retry on each task - C) EventBridge Pipes connecting S3 → Lambda1 → Lambda2 → Lambda3 → DynamoDB - D) SQS FIFO queue chaining each Lambda as consumer/producer > **Answer: B**. Step Functions state machine = visibility, retry per step, error handling, no coordination Lambda needed. Each Task state handles one service call with native AWS SDK integration.

1638. [Multi-topic] A company's engineering team wants a complete serverless web application: React frontend, REST API, authentication, and relational database. They want all components serverless and managed. What AWS services map to each component? - A) S3+CloudFront (frontend) | API Gateway HTTP API (API) | Cognito User Pools (auth) | Aurora Serverless v2 (DB) - B) EC2 + ALB (frontend) | API Gateway REST API (API) | IAM Identity Center (auth) | RDS PostgreSQL (DB) - C) Amplify (frontend) | AppSync GraphQL (API) | Cognito (auth) | DynamoDB (DB) - D) Both A and C are valid architectures depending on API style (REST vs GraphQL) > **Answer: D**. A = standard serverless REST stack. C = serverless GraphQL stack with Amplify hosting. Both valid — choice depends on API style and team preference.

1639. [Lambda] A Lambda function needs persistent, shared file storage accessible by multiple concurrent Lambda invocations and by ECS tasks in the same VPC. What storage solution is appropriate? - A) S3 — object storage, accessible from Lambda and ECS - B) EFS (Elastic File System) mounted to Lambda (VPC required) and ECS tasks - C) EBS volume attached to Lambda - D) ElastiCache Redis as shared key-value store > **Answer: B**. EFS supports concurrent access (ReadWriteMany), mounts to Lambda (needs VPC), and ECS. S3 is correct too (A) but EFS = POSIX filesystem access which Lambda and ECS support natively.

1640. [Multi-topic] An architect reviews a design where API Gateway invokes a Lambda, which calls DynamoDB, which triggers a DynamoDB Stream, which calls another Lambda, which updates an Aurora Serverless DB, which then sends an SNS notification. A new requirement asks to add retry logic between the Aurora Lambda and SNS. What is the most appropriate change? - A) Add try/catch in the Aurora Lambda and call SNS SDK manually - B) Wrap the Aurora Lambda + SNS call in a Step Functions Express Workflow with Retry configuration - C) Add SQS between Lambda and SNS for retry buffering - D) Configure SNS DLQ to retry failed Lambda deliveries > **Answer: B**. Step Functions Express Workflow wraps the Lambda→SNS sequence with built-in Retry + Catch. Clean orchestration without complicating Lambda code.

1641. [API Gateway] A company wants to use API Gateway to expose an existing HTTP endpoint running on an EC2 instance in a private subnet. The EC2 endpoint should not be exposed to the public internet. What is the correct integration? - A) HTTP Proxy integration with EC2 public IP - B) VPC Link to a Network Load Balancer fronting the EC2 instance - C) Lambda Proxy integration where Lambda calls EC2 internally - D) AWS Service Integration pointing to EC2 metadata endpoint > **Answer: B**. API GW VPC Link = private integration. NLB in private subnet → EC2 target. API GW stays public; EC2 EC2 never exposed directly.

1642. [EC2 Placement] A machine learning training job requires 50 p3.16xlarge instances. The job must complete within 4 hours. EFA (Elastic Fabric Adapter) networking is required for distributed training. Which placement type is mandatory? - A) Spread Placement Group — isolates each GPU instance - B) Partition Placement Group — distributes across racks - C) Cluster Placement Group — same AZ, low-latency EFA networking - D) No placement group — EFA works without placement restrictions > **Answer: C**. EFA requires Cluster Placement Group for full bandwidth between instances. Low-latency, high-bandwidth same-AZ network = mandatory for distributed ML training with NCCL/MPI all-reduce.

1643. [Aurora Serverless] Aurora Serverless v2 minimum ACU is set to 0.5. What resource does this always-running minimum capacity represent? - A) 0.5 vCPU only — RAM scales separately - B) Approximately 1 GB RAM + proportional CPU + network, always active - C) 0.5 GB RAM only, no CPU allocated until first query - D) 0.5 ACU = the database storage layer only, no compute > **Answer: B**. 1 ACU ≈ 2 GB RAM + CPU. 0.5 ACU ≈ 1 GB RAM + fractional CPU. Always running, enabling instant response without resume delay.

1644. [Multi-topic] A company wants to replace a complex series of cron-scheduled Lambda functions that call each other via API calls with a managed, monitored, retried, and auditable workflow. Executions can run for up to 30 minutes. Which Step Functions configuration is correct? - A) Express Workflow — 5-minute max, not suitable for 30-minute workflows - B) Standard Workflow — up to 1 year, full execution history, exactly-once semantics - C) Express Workflow with extended timeout setting - D) Standard Workflow with Express sub-workflows for each Lambda > **Answer: B**. Standard Workflow: 1-year max duration, full audit history in CloudWatch Events, exactly-once. 30 minutes well within limit. Express max = 5 min.

1645. [Multi-topic] A SAP-C02 exam scenario: A company wants the lowest possible latency API for their mobile app, supports JWT from a third-party IdP (not Cognito), no response caching needed, and budget is very tight. Which API Gateway type and authorizer combination is cheapest and meets all requirements? - A) REST API + Lambda Authorizer — full features - B) HTTP API + JWT Authorizer — native support, 70% cheaper, low latency - C) HTTP API + Lambda Authorizer — custom JWT validation - D) REST API + Cognito Authorizer — JWT native > **Answer: B**. HTTP API native JWT Authorizer validates JWT from ANY OIDC-compliant IdP (not just Cognito). 70% cheaper than REST API. No caching needed = no need for REST API. Budget constraint → HTTP API.

1646. [Lambda] A Lambda function connected to a VPC needs to call an external HTTPS API on the internet. The VPC has no NAT Gateway. What must be added? - A) Internet Gateway attached to the Lambda subnet - B) NAT Gateway in a public subnet + route from Lambda subnet to NAT GW - C) VPC Peering with AWS's public services VPC - D) Lambda can always reach the internet regardless of VPC configuration > **Answer: B**. VPC Lambda = no public IP. Outbound internet = must route through NAT Gateway (public subnet → IGW). Lambda subnet route table: 0.0.0.0/0 → NAT GW.

1647. [App Runner] A developer wants App Runner to automatically rebuild and redeploy an application whenever code is pushed to a GitHub branch. Is this supported natively? - A) No — App Runner only supports ECR image sources - B) Yes — App Runner has native GitHub source provider with automatic build and deploy on push - C) Partially — requires CodePipeline to bridge GitHub to App Runner - D) Yes — only for Node.js and Python runtimes > **Answer: B**. App Runner natively connects to GitHub via App Runner connections. Auto-deploy on push to configured branch. Supports multiple runtimes (Node.js, Python, Java, Go, .NET, PHP, Ruby).

1648. [EKS] A team needs to give read access to a specific S3 bucket to pods in the `data-processing` namespace with ServiceAccount `s3-reader`. The rest of the cluster should have no S3 access. How is this configured with minimum permissions? - A) Attach S3ReadOnlyAccess to the EC2 node instance profile - B) Create IAM role with S3 read policy; annotate ServiceAccount with IRSA role ARN; only `data-processing/s3-reader` ServiceAccount gets the annotation - C) Create a Kubernetes Secret with AWS access keys and mount into pods - D) Use Kube2IAM with per-pod annotation on all pods in the namespace > **Answer: B**. IRSA = S3-specific IAM role annotated on the specific ServiceAccount. Only

those pods get S3 access. Node instance profile (A) = all pods on node get access (violates least privilege). Secrets with keys (C) = anti-pattern, rotate risk.

1649. [Timestream] A data engineering team migrates from InfluxDB to Amazon Timestream. Their existing queries use `SELECT * FROM measurements WHERE time > now() - 6h`. What Timestream equivalent achieves the same result? - A) `WHERE time > DATE_SUB(NOW(), INTERVAL 6 HOUR)` - B) `WHERE time > ago(6h)` - C) `WHERE time > CURRENT_TIMESTAMP - 6` - D) `WHERE time > DATEADD(hour, -6, GETDATE())` > **Answer: B**. Timestream SQL uses `ago()` function for relative time filters. `ago(6h)` = 6 hours before query time.

1650. [Multi-topic] A company deploys a complete serverless backend for an e-commerce platform. They want to understand AWS Well-Architected cost optimization for this stack: API Gateway + Lambda + Aurora Serverless v2 + Step Functions + SNS. Which combination of cost optimizations is most impactful? - A) Switch REST API to HTTP API; set Aurora v2 min ACU to 0.5; use Express Workflows for high-volume short order steps; use Lambda Graviton2 runtime; SNS standard topic - B) Move Lambda to ECS Fargate; use Provisioned Aurora; use Standard Workflows; add ElastiCache - C) Add CloudFront caching in front of API Gateway; move to RDS Aurora Provisioned; use S3 instead of Lambda for business logic - D) Use API Gateway REST API caching; add DAX for DynamoDB; move to EC2 Reserved Instances > **Answer: A**. HTTP API (70% cheaper) + Aurora v2 min ACU 0.5 + Express Workflows for high-volume short tasks + Graviton2 Lambda (20% cheaper) = comprehensive serverless cost optimization stack.

Chapter 28 Extended — ECS and AWS Fargate

1651. [ECS] A developer deploys a containerized web application using ECS EC2 launch type. The container needs to access S3. A colleague suggests using the EC2 instance profile. A senior architect disagrees. What is the correct IAM approach? - A) EC2 instance profile — ECS inherits permissions from the host - B) Task Role attached to the Task Definition — grants IAM permissions to the container app - C) Task Execution Role — grants app-level permissions to containers - D) Environment variables in Task Definition with hardcoded access keys > **Answer: B**. Task Role = IAM permissions for the application inside the container. Instance profile is too broad (all tasks on that instance inherit it). Task Execution Role is for ECS agent infrastructure (ECR, CloudWatch).

1652. [ECS] An ECS Fargate task is failing to pull its Docker image from a private ECR repository. No NAT Gateway exists in the VPC. What is missing? - A) The task needs a public IP address - B) VPC endpoints for ECR (ecr.api, ecr.dkr) and S3 gateway endpoint are required - C) The Task Execution Role needs IAM permissions and a NAT Gateway is mandatory - D) Fargate requires a public subnet to pull from ECR > **Answer: B**. ECR image pull over PrivateLink: com.amazonaws.region.ecr.api + com.amazonaws.region.ecr.dkr (interface) + com.amazonaws.region.s3 (gateway, for image layers). No NAT Gateway needed.

1653. [ECS] An ECS service runs 4 tasks across 2 AZs. A deployment is triggered. The team wants to ensure at least 2 tasks are always healthy during the rollout, and no more than 6 tasks run simultaneously. What are the correct minimumHealthyPercent and maximumPercent values? - A) minimumHealthyPercent: 25, maximumPercent: 100 - B) minimumHealthyPercent: 50, maximumPercent: 150 - C) minimumHealthyPercent: 100, maximumPercent: 200 - D) minimumHealthyPercent: 75, maximumPercent: 200 > **Answer: B**. Desired = 4. 50% = 2 tasks minimum healthy. 150% = 6 tasks maximum (4 × 1.5 = 6). Exactly meets the requirement.

1654. [ECS] A company uses ECS EC2 launch type. They need to run a log aggregation sidecar on every EC2 container instance exactly once. Which ECS scheduling strategy achieves this? - A) ECS Service with task placement strategy: spread across instances - B) ECS Daemon scheduling — automatically places one task per container instance - C) ECS Fargate — one task per task definition - D) EventBridge Scheduler triggering one task per instance > **Answer: B**. Daemon scheduling = ECS automatically places exactly one task per active EC2 container instance. Used for monitoring agents, log collectors, security tools.

1655. [ECS] A new microservice must communicate with an existing ECS service named `payment-service` without going through an ALB. The team wants DNS-based service-to-service communication. Which ECS feature provides this? - A) ECS Service Connect or AWS Cloud Map integration for private DNS resolution - B) ECS task metadata endpoint for service discovery - C) VPC DNS resolver pointing to payment-service task IP - D) Route 53 public hosted zone for payment-service > **Answer: A**. Cloud Map integration: ECS registers task IPs as DNS A records (payment-service.local). Service Connect provides built-in service mesh with metrics. Both enable service-to-service DNS resolution.

1656. [ECS] An ECS service behind an ALB needs to perform a zero-downtime deployment with instant rollback capability if errors are detected. Which deployment strategy supports this? - A) ECS Rolling Update with minimumHealthyPercent: 100 - B) ECS Blue/Green deployment via AWS CodeDeploy - C) ECS external deployment with manual traffic shifting - D) ECS rolling update with ALB connection draining > **Answer: B**. CodeDeploy Blue/Green for ECS: creates new task set (Green), shifts ALB traffic (linear, canary, or all-at-once), monitors alarms, auto-rolls back if alarms trigger.

1657. [ECS] A container application needs to read a database password at startup without hardcoding it in the Task Definition environment variables. What is the recommended approach? - A) Pass the password via AWS CLI `--overrides` at task launch - B) Reference Secrets Manager secret ARN in the Task Definition `secrets` field — injected at task start - C) Store in S3 and download via entrypoint script - D) Use Lambda to inject the secret at ECS task launch time > **Answer: B**. Task Definition `secrets` field references Secrets Manager ARN or SSM Parameter Store path. ECS agent (Task Execution Role) fetches and injects as environment variable at container start. Never in plaintext env vars.

1658. [Fargate] A batch processing system uses ECS Fargate. Jobs are non-critical and can tolerate interruption. The team wants maximum cost savings. What should they configure? - A) Reserved Capacity for Fargate (1-year) - B) ECS Fargate Spot via capacity provider strategy - C) EC2 Spot instances with ECS EC2 launch type - D) ECS Savings Plans for Fargate compute > **Answer: B**. Fargate Spot = up to 70% discount. Configure capacity provider strategy: FARGATE_SPOT with fallback to FARGATE (base: 1 ensures at least 1 standard Fargate task for critical work).

1659. [Fargate] An ECS Fargate task needs persistent storage that survives task restarts and is shared between multiple concurrent tasks. Which storage option should be used? - A) Increase Fargate ephemeral storage to 200 GB - B) Mount Amazon EFS to the Fargate task via Task Definition volume configuration - C) Attach an EBS volume to the Fargate task - D) Use S3 Sync on task start to restore data from S3 > **Answer: B**. EFS = persistent, shared, ReadWriteMany. Fargate ephemeral storage (A) does not survive task restart. EBS (C) not supported on Fargate.

1660. [Fargate] A Fargate task in a private subnet needs outbound internet access to call a third-party API. What infrastructure must be added? - A) Assign a public IP to the Fargate task - B) NAT Gateway in a public subnet + route 0.0.0.0/0 from private subnet to NAT GW - C) Fargate tasks automatically have outbound internet access via AWS managed networking - D) Internet-facing ALB provides outbound internet routing for tasks > **Answer: B**. Fargate in private subnet has no direct internet route. NAT Gateway (public subnet → IGW) + route table update → outbound internet for private tasks.

1661. [Fargate] An organization wants to run an EKS cluster with serverless pods — no EC2 worker nodes to manage. Pods require 4 vCPU and 8 GB memory. How is Fargate configured for EKS? - A) Set EKS node group type to FARGATE - B) Create EKS Fargate Profile matching pod namespace and label selector - C) Enable Fargate in the EKS add-ons configuration - D) Configure EKS managed node group with Fargate instance type > **Answer: B**. EKS Fargate Profile: define namespace (e.g., `default`) and optional label selector. Any pod matching the profile runs on Fargate. Multiple profiles can be created.

1662. [Fargate] A security audit requires that each microservice container in ECS Fargate have its own Security Group with minimal allowed ports. How is this achieved? - A) Apply SG to the EC2 instance — Fargate uses bridge mode SG - B) awsvpc networking mode (required for Fargate) — each task gets its own ENI and SG - C) ECS Service-level SG applied to all tasks in the service - D) VPC NACL provides per-task security control > **Answer: B**. Fargate requires awsvpc = each task = dedicated ENI = dedicated SG. Task-level security group assignment.

1663. [ECS] An ECS EC2 cluster has 10 instances. A new task requiring 4 vCPU is submitted but fails to place because no instance has sufficient available CPU. What should the team configure to automatically add instances? - A) ECS Service Auto Scaling — scales tasks, not instances - B) EC2 Auto Scaling Group Capacity Provider with managed scaling - C) CloudWatch alarm on CPUUtilization triggering manual instance launch - D) ECS daemon task to monitor and launch new instances > **Answer: B**. EC2 Capacity Provider with managed scaling = ECS automatically scales the EC2 ASG when cluster capacity is insufficient for pending tasks. This is cluster-level auto scaling.

1664. [Fargate] A developer specifies 0.5 vCPU and 512 MB memory for a Fargate task. The task will not launch. Why? - A) Fargate minimum is 1 vCPU - B) 0.5 vCPU requires memory between 1 GB and 4 GB — 512 MB is below the minimum for 0.5 vCPU - C) Fargate does not support fractional vCPU - D) 512 MB memory is too low for any Fargate task > **Answer: B**. Fargate valid combos: 0.5 vCPU requires 1 GB – 4 GB memory. 512 MB is invalid for 0.5 vCPU. Use 0.25 vCPU (0.5 GB – 2 GB range).

1665. [ECS] A microservices application has 5 ECS services each exposed via a different path (/auth, /users, /orders, /products, /notifications). All should use one ALB. Describe the configuration. - A) 5 ALBs, one per service, with Route 53 weighted routing - B) One ALB, one HTTPS listener, 5 path-based rules, each routing to a separate ECS target group - C) One ALB with 5 listeners on different ports (8080-8084) per service - D) API Gateway in front of one ALB routing to all 5 services > **Answer: B**. ALB path-based rules: /auth/* → TG-auth (ECS service A), /users/* → TG-users (ECS service B), etc. One ALB, one listener, one cert — optimal cost + management.

1666. [Multi-topic] A company is choosing between ECS Fargate and Lambda for a container-based microservice that handles HTTP requests. Average request duration is 5 seconds. Traffic is 1,000 requests/day with occasional spikes to 200 requests/minute. Which is more cost-effective? - A) Lambda — scale to zero, per-invocation billing, handles spikes - B) ECS Fargate — containers handle 5s duration better than Lambda - C) Lambda — but only if using Provisioned

Concurrency for spikes - D) ECS Fargate with Fargate Spot — handles duration and cost > **Answer: A**. 1,000/day = low traffic, scale-to-zero saves cost during idle. 5s within 15-min Lambda limit. Lambda scales instantly for spikes. No idle Fargate task cost during 1,000/day periods.

1667. [Multi-topic] A company migrates an on-premises Java application to AWS. The app uses shared NFS storage (multiple app nodes reading/writing simultaneously). Which ECS + storage combination correctly mirrors this architecture? - A) ECS EC2 with EBS volume shared across tasks - B) ECS Fargate with EFS mount (ReadWriteMany) - C) ECS Fargate with Fargate ephemeral storage (200 GB) - D) ECS EC2 with S3 sync on every read/write > **Answer: B**. EFS = NFSv4 protocol, ReadWriteMany, multiple Fargate tasks mount simultaneously — exact NFS equivalent. EBS = ReadWriteOnce per node. Ephemeral = not shared.

1668. [Multi-topic] A solutions architect needs to design a container platform with these requirements: (1) no EC2 management, (2) containers run in the customer VPC, (3) per-container IAM permissions, (4) persistent shared storage, (5) scale to zero when idle. Which combination is correct? - A) ECS Fargate (1,2,3,4) + Fargate Spot (5 — scale to zero when no tasks) - B) EKS Fargate (1,2,3,4) — no scale to zero in EKS - C) ECS Fargate (1,2) + Task Role (3) + EFS (4) + scale-to-zero via Service desired count = 0 on schedule - D) App Runner (1) + VPC Connector (2) — does not support per-container IAM or EFS > **Answer: C**. ECS Fargate: no EC2 (1), awsvpc in customer VPC (2), Task Role per service (3), EFS mount (4), schedule via EventBridge or Application Auto Scaling to 0 (5). True scale-to-zero = Lambda; ECS can scale to 0 desired but not instant.

1669. [Fargate] Which of the following statements about AWS Fargate is TRUE? - A) Fargate supports GPU-based workloads for ML inference - B) Fargate tasks and EKS pods using Fargate run on dedicated kernel infrastructure per customer - C) Fargate requires an EC2 key pair for container access - D) Fargate tasks can use EBS gp3 volumes for persistent storage > **Answer: B**. Fargate = dedicated kernel per task/pod (no shared-host with other customers). No EC2 keys (use ECS Exec for interactive access). No GPU. No EBS.

1670. [Multi-topic] A retail company runs an ECS service handling order processing (EC2 launch type). During peak seasons, the service receives 10× normal load. Auto Scaling is in place. They want to reduce costs during off-peak by using Spot, while maintaining capacity during peak with On-Demand. What architecture achieves this? - A) Two ECS clusters — one with On-Demand EC2, one with Spot EC2 - B) One ECS cluster with two capacity providers: On-Demand ASG (base) + Spot ASG (scale-out), managed via capacity provider strategy - C) ECS Fargate for peak and ECS EC2 Spot for off-peak - D) One On-Demand ASG that auto-replaces instances with Spot during off-peak hours > **Answer: B**. Capacity provider strategy: On-Demand base = guaranteed minimum; Spot = scale-out weight. ECS distributes tasks across both capacity providers, maximizing Spot during normal load, falling back to On-Demand during Spot interruptions.

Chapter 29 Extended — SCPs vs IAM Permission Boundaries

1671. [SCPs] An AWS Organizations management account applies an SCP to the root OU that denies `cloudtrail:DeleteTrail`. A member account administrator with `AdministratorAccess` tries to delete a trail. What happens? - A) The deletion succeeds — AdministratorAccess overrides SCPs - B) The deletion fails — SCP Deny always wins, even over explicit IAM Allow - C) The deletion succeeds — SCPs only apply to IAM users, not roles - D) The deletion fails only if the admin role also has an explicit Deny in its IAM policy > **Answer: B**. SCP Deny = absolute block regardless of IAM policy. Even AdministratorAccess cannot override an SCP Deny.

1672. [SCPs] A company wants to restrict all accounts in the Production OU from launching EC2 instances outside of eu-central-1. Which policy type and where should it be applied? - A) IAM permission boundary on each admin role in each account - B) SCP applied to the Production OU — denies `ec2:RunInstances` with condition `StringNotEquals aws:RequestedRegion eu-central-1` - C) Config rule in each account triggering automatic termination of non-compliant instances - D) GuardDuty threat intelligence rule blocking non-eu-central-1 launches > **Answer: B**. SCP on OU = applies to all current and future accounts in that OU automatically. One policy, organization-wide enforcement.

1673. [Permission Boundaries] A DevOps team is allowed to create Lambda execution roles. Management is concerned they could create a role with AdministratorAccess and escalate privileges. What is the correct mitigation? - A) Remove `iam:CreateRole` from the DevOps policy entirely - B) Attach a Permission Boundary to the DevOps role — require them to attach the same boundary to any role they create (`iam:CreateRole` condition: `iam:PermissionsBoundary = DevOpsBoundaryArn`) - C) Apply an SCP that denies `iam:CreateRole` in the dev accounts - D) Enable IAM Access Analyzer to detect over-privileged roles > **Answer: B**. Permission boundary + IAM condition `iam:PermissionsBoundary` enforces that every role the DevOps team creates must have the boundary attached — caps escalation to DevOpsBoundaryArn.

1674. [Permission Boundaries] A role has an IAM policy allowing `s3:*` on all resources, and a Permission Boundary allowing only `s3:GetObject` and `s3:PutObject`. What actions can the role perform on S3? - A) All S3 actions — IAM policy

allows `s3:*` - B) Only `s3:GetObject` and `s3:PutObject` — effective = IAM ∩ Boundary - C) No S3 actions — boundary blocks all unless SCP allows - D) All S3 actions except delete — boundary only restricts delete actions > **Answer: B**. Effective = Identity Policy ∩ Boundary = {s3:*} ∩ {s3:GetObject, s3:PutObject} = {s3:GetObject, s3:PutObject}.

1675. [SCPs] Which AWS entity is NOT subject to Service Control Policies? - A) IAM users in member accounts - B) IAM roles in member accounts - C) The management (payer) account root user - D) EC2 instance profiles in member accounts > **Answer: C**. Management account root user is exempt from SCPs. All IAM entities (users, roles, including EC2 instance profiles) in member accounts are subject to SCPs.

1676. [Permission Boundaries] A permission boundary is attached to an IAM role. The boundary allows `ec2:*`. The role's identity policy allows `s3:GetObject`. Can the role read from S3? - A) Yes — the identity policy allows s3:GetObject - B) No — effective = IAM policy ∩ Boundary = {s3:GetObject} ∩ {ec2:*} = empty set for S3 - C) Yes — the boundary only restricts EC2 actions - D) No — boundaries always deny all actions not listed in the identity policy > ***Answer: B***. *Boundary allows only ec2:*. Identity policy allows s3:GetObject. Intersection = neither covers the other = s3:GetObject NOT in effective permissions. Result: Denied.

1677. [SCPs] A company has 80 AWS accounts across 5 OUs. They want to prevent any account from disabling AWS Security Hub. What is the most operationally efficient approach? - A) Attach an IAM permission boundary to every administrator role in all 80 accounts - B) Apply one SCP to the root OU denying `securityhub:DisableSecurityHub` — automatically applies to all accounts - C) Create a Config rule in each account triggering re-enablement of Security Hub - D) Use IAM Access Analyzer to alert when Security Hub is disabled > **Answer: B**. SCP on root OU = one policy, all 80 accounts, including any future accounts added. Permission boundaries (A) require per-role attachment in each account — not scalable.

1678. [Multi-topic] An IAM role has: (1) Identity policy: Allow `iam:CreateRole` on *, (2) Permission Boundary: Allow `iam:CreateRole` with condition `iam:PermissionsBoundary = arn:...:DeveloperBoundary`, (3) SCP: Allow `iam:CreateRole` only in us-east-1. The role tries to create a new role in eu-west-1 with the DeveloperBoundary attached. What is the result? - A) Success — identity policy and boundary both allow iam:CreateRole - B) Denied — SCP restricts iam:CreateRole to us-east-1 only - C) Denied — boundary condition requires PermissionsBoundary to match, which it does — but SCP blocks - D) Success — the DeveloperBoundary satisfies all conditions > **Answer: B**. All three must allow. SCP only allows `iam:CreateRole` in us-east-1. Action in eu-west-1 = SCP Deny = blocked regardless of identity policy and boundary.

1679. [Permission Boundaries] A developer has an IAM role with AdministratorAccess and no Permission Boundary. They create a new role with ReadOnlyAccess and attach a Permission Boundary of `PowerUserAccess`. What is the effective permission of the new role? - A) AdministratorAccess — creator's permissions transfer to created role - B) ReadOnlyAccess — the identity policy of the new role - C) ReadOnlyAccess ∩ PowerUserAccess = ReadOnlyAccess (since ReadOnly ⊂ PowerUser) - D) PowerUserAccess — the boundary expands the identity policy > **Answer: C**. New role effective = its own identity policy (ReadOnlyAccess) ∩ its boundary (PowerUserAccess). ReadOnlyAccess is a subset of PowerUserAccess → intersection = ReadOnlyAccess. Creator's AdministratorAccess does not transfer.

1680. [Multi-topic] Which combination correctly describes the difference between SCPs and Permission Boundaries regarding resource-based policies? - A) Both SCPs and permission boundaries restrict resource-based policies - B) SCPs apply to cross-account calls (caller's SCP evaluated); permission boundaries do not affect resource-based policies - C) Permission boundaries restrict resource-based policies; SCPs do not - D) Neither SCPs nor permission boundaries affect resource-based policies > **Answer: B**. SCPs affect cross-account calls — the caller's account SCP is evaluated. For same-account resource-based policies, SCPs don't restrict. Permission boundaries never restrict resource-based policies.

Chapter 29 Extended — AWS Policy Types

1681. [Policy Types] An S3 bucket policy allows `s3:GetObject` for `arn:aws:iam::999:role/AnalyticsRole`. An SCP in Account 999 denies `s3:GetObject` on all S3 resources. Account 999's AnalyticsRole has identity policy allowing `s3:GetObject`. What is the result when AnalyticsRole tries to read from the bucket? - A) Allowed — resource policy and identity policy both allow - B) Denied — SCP in Account 999 blocks the call even cross-account - C) Allowed — SCP only applies within Account 999, not to cross-account calls - D) Denied — cross-account requires explicit Allow in both accounts plus SCPs in both > **Answer: B**. Cross-account: caller's SCP (Account 999) is evaluated. SCP denies s3:GetObject → blocked regardless of identity policy or resource policy.

1682. [Entity vs Principal] A resource policy uses `"Principal": { "AWS": "arn:aws:iam::123:group/Developers" }`. What is the result? - A) All users in the Developers group are granted access - B) Policy is invalid — IAM groups cannot be principals in resource policies - C) Access is granted to the group ARN but only within the same account - D) Policy is

valid but only for S3 bucket policies > **Answer: B**. IAM groups are NOT valid principals in resource policies. Groups cannot make API calls. Use role ARN or user ARN instead.

1683. [Identity-based] A developer has an inline policy allowing `ec2:*` and a customer managed policy allowing `s3:*` attached. What happens if the managed policy is detached? - A) Both policies are deleted - B) Only the managed policy is removed; inline policy with ec2:* remains - C) The managed policy remains in IAM and can be re-attached later - D) Both B and C are true > **Answer: D**. Inline policy stays with the entity. Detached managed policy remains in IAM (it's separate from the entity), can be re-attached to any entity. B and C both describe correct outcomes.

1684. [Resource-based] A Lambda function in Account A needs to be invoked by EventBridge in Account B. What is required? - A) IAM role in Account A with trust policy allowing Account B - B) Lambda resource-based policy granting `lambda:InvokeFunction` to the EventBridge principal in Account B - C) VPC peering between Account A and Account B - D) Cross-account AssumeRole from EventBridge to Account A > **Answer: B**. Lambda resource-based policy = grants cross-account invocation. `"Principal": {"Service": "events.amazonaws.com"}` with `"Condition": {"ArnLike": {"AWS:SourceArn": "arn:aws:events:...:Account-B:..."}}` .

1685. [VPC Endpoint Policy] A company's Lambda function in a private VPC must access only the company's own S3 buckets (never external buckets). What is the correct implementation? - A) Identity policy on Lambda execution role listing only allowed bucket ARNs - B) S3 VPC Gateway Endpoint policy restricting Resource to company bucket ARNs only - C) S3 bucket policy denying all Lambda access - D) VPC NACL blocking all S3 traffic except from allowed IP ranges > **Answer: B**. VPC endpoint policy on the S3 gateway endpoint restricts what buckets any traffic through that endpoint can access — regardless of identity policy. Prevents credential-based access to external S3 buckets via the endpoint.

1686. [Session Policy] A DevOps engineer assumes a role with `AdministratorAccess` identity policy. They pass a session policy allowing only `s3:GetObject` on a specific bucket. What can the session do? - A) Everything AdministratorAccess allows — session policy expands permissions - B) Only `s3:GetObject` on that specific bucket — session policy intersects with role - C) Nothing — session policy conflicts with identity policy and both are blocked - D) Everything except what session policy denies > **Answer: B**. Session policy = intersection. AdministratorAccess ∩ {s3:GetObject on bucket-X} = {s3:GetObject on bucket-X}. Session policy always restricts, never expands.

1687. [AssumeRoleWithSAML] A corporate user authenticates via Okta (SAML 2.0 IdP). The SAML assertion includes the AWS role ARN and session duration of 8 hours. Which STS operation is called? - A) AssumeRole with Okta as principal - B) AssumeRoleWithWebIdentity with SAML token - C) AssumeRoleWithSAML with SAML assertion - D) GetFederationToken with Okta credentials > **Answer: C**. Enterprise SAML 2.0 IdP (Okta, ADFS, Azure AD) → `AssumeRoleWithSAML` . OIDC/JWT → `AssumeRoleWithWebIdentity` . Session duration in SAML assertion is honored.

1688. [AssumeRoleWithWebIdentity] A mobile application allows users to log in with Google. After Google login, the app needs to access a user-specific S3 prefix. Which AWS mechanism is used? - A) AssumeRoleWithSAML using Google SAML provider - B) AssumeRoleWithWebIdentity using Google-issued JWT id_token - C) GetFederationToken using Google OAuth token - D) Cognito Identity Pools (which internally calls AssumeRoleWithWebIdentity) > **Answer: D**. Cognito Identity Pools is the recommended pattern — it handles AssumeRoleWithWebIdentity internally. Both D and B are technically correct (Cognito calls AssumeRoleWithWebIdentity), but D is the AWS-recommended architecture. If forced to choose one raw STS call, B is correct.

1689. [S3 ACL] A company enables cross-account S3 uploads. A partner in Account B uploads objects to a bucket in Account A. The bucket owner (Account A) cannot read the uploaded objects. What is the cause? - A) The bucket policy does not allow Account A to read objects - B) Object Ownership is set to ObjectWriter — Account B retains ownership; Account A cannot access by default - C) S3 ACLs are disabled on the bucket preventing all access - D) Cross-account reads require VPC peering > **Answer: B**. ObjectWriter mode = uploader (Account B) owns the object. Account A (bucket owner) has no automatic access. Fix: set ObjectOwnership to BucketOwnerPreferred or BucketOwnerEnforced, or require `bucket-owner-full-control` canned ACL on upload.

1690. [S3 ACL] A developer uses the `AuthenticatedUsers` predefined S3 ACL grantee to share an object. Who can access it? - A) Only users in the developer's AWS account with valid credentials - B) Any IAM user or role with valid AWS credentials in any AWS account worldwide - C) Only users in the same VPC as the S3 bucket - D) Only authenticated Cognito users in the developer's user pool > **Answer: B**. `AuthenticatedUsers` = ANY authenticated AWS account — the entire AWS customer base. This is a major security mistake. Use bucket policies with specific account ARNs instead.

1691. [NACL] A public subnet has an inbound NACL rule allowing TCP port 443. A user can connect to an EC2 web server, but the response never reaches the user. What is the likely cause? - A) Security group is blocking return traffic - B) NACL is stateless — outbound ephemeral ports (1024–65535) are not allowed in the NACL outbound rules - C) The IGW is blocking outbound traffic - D) EC2 does not have a public IP > **Answer: B**. NACLs are stateless. Client connects from

ephemeral port (e.g., 54321). Server must respond TO that port. Outbound NACL must allow TCP ports 1024–65535 for responses. Security Groups are stateful and would auto-allow return traffic.

1692. [RCP] An organization wants to ensure no S3 bucket in any member account can be made publicly accessible or shared with external accounts, regardless of bucket policies. What policy type achieves this? - A) SCP denying s3:PutBucketPolicy on all accounts - B) Resource Control Policy (RCP) denying s3:* from principals outside the organization - C) S3 Block Public Access enabled per account - D) Config rule detecting and remediating public buckets > **Answer: B**. RCP = org-level resource restriction. Even if a bucket policy allows external access, the RCP overrides it. SCP (A) controls what your principals can do, not what external principals can do to your resources. S3 Block Public Access (C) only prevents public access, not org-scoped external access.

1693. [Permission Boundary] An IAM user has an identity policy allowing `iam:CreateRole` and `iam:AttachRolePolicy`. A permission boundary allows only `s3:*` and `lambda:*`. Can the user create a new role and attach AdministratorAccess to it? - A) Yes — identity policy allows iam:CreateRole and iam:AttachRolePolicy - B) No — permission boundary does not allow iam:CreateRole or iam:AttachRolePolicy - C) Yes — boundary only restricts what actions the created role can perform - D) No — only org admins can create roles with AdministratorAccess > **Answer: B**. Effective = identity policy ∩ boundary = {iam:CreateRole, iam:AttachRolePolicy} ∩ {s3:*, lambda:*} = empty set for IAM actions. User cannot create roles.

1694. [AssumeRole] An IAM role has `MaxSessionDuration` set to 1 hour. A developer calls `AssumeRole` and requests a session duration of 12 hours. What happens? - A) Session is issued for 12 hours — requested duration overrides MaxSessionDuration - B) Session is issued for 1 hour — MaxSessionDuration is the ceiling - C) Call fails — requested duration exceeds MaxSessionDuration - D) Session is issued for 12 hours only for root user calls > **Answer: C**. If requested DurationSeconds > MaxSessionDuration, STS returns an error. Developer must request ≤ MaxSessionDuration or the role's MaxSessionDuration must be increased (up to 12 hours).

1695. [Policy Types] A KMS key was created with a custom key policy that does NOT include the root principal (`arn:aws:iam::123:root`). An IAM policy grants an admin role `kms:*`. Can the admin use the key? - A) Yes — IAM policy `kms:*` grants full access regardless of key policy - B) No — without root in key policy, IAM policies cannot grant access to this key - C) Yes — the admin can update the key policy via IAM permissions - D) No — only AWS Support can grant access to keys without root in key policy > **Answer: B**. KMS key policy is required. Without root principal in key policy, IAM policies alone cannot grant access. No IAM entity can use the key. Recovery requires AWS Support to regain control via special procedure.

1696. [Session Policy] A Cognito Identity Pool issues credentials for unauthenticated guest users. The developer passes a session policy scoping access to `s3:GetObject` on a specific bucket. The role's identity policy allows `s3:*` on all buckets. What can a guest user do? - A) Full s3:* on all buckets — role identity policy applies - B) s3:GetObject on the specific bucket only — session policy intersects - C) Nothing — unauthenticated users cannot use S3 - D) s3:GetObject on all buckets — session policy applies to action only, not resource > **Answer: B**. Cognito Identity Pools issue temporary credentials via AssumeRoleWithWebIdentity. Session policy scopes the session to s3:GetObject on specific bucket only. Role's broader s3:* is intersected down.

1697. [Multi-topic] A Solutions Architect is designing a zero-trust data access model. Which combination of policies correctly implements: (1) only company's own AWS accounts can access S3, (2) only VPC traffic can reach S3, (3) only specific IAM roles can write, (4) even admin cannot delete? - A) (1) RCP with PrincipalOrgID condition, (2) bucket policy aws:SourceVpce, (3) identity policies on specific roles, (4) SCP deny s3:DeleteObject - B) (1) SCP, (2) NACL, (3) resource policy, (4) S3 MFA Delete - C) (1) IAM permission boundary, (2) VPC endpoint policy, (3) inline policy, (4) S3 versioning - D) (1) S3 ACL AuthenticatedUsers, (2) VPC NACL, (3) S3 bucket ACL, (4) permission boundary > **Answer: A**. (1) RCP with PrincipalOrgID = org-level resource access restriction. (2) bucket policy with `aws:SourceVpce` = VPC-only. (3) identity policies on specific roles. (4) SCP deny s3:DeleteObject = even admin cannot delete.

1698. [Multi-topic] Three policies apply to an IAM role: Identity policy allows `ec2:*`, Permission Boundary allows `ec2:DescribeInstances` only, SCP allows `ec2:*`. The role tries to call `ec2:RunInstances`. What is the result? - A) Allowed — SCP and identity policy both allow ec2:RunInstances - B) Denied — Permission Boundary does not include ec2:RunInstances - C) Allowed — SCP supersedes Permission Boundary - D) Denied — must check resource policy too before allowing > **Answer: B**. Effective = Identity ∩ SCP ∩ Boundary = {ec2:*} ∩ {ec2:*} ∩ {ec2:DescribeInstances} = {ec2:DescribeInstances}. ec2:RunInstances NOT in effective permissions → Denied.

1699. [AssumeRoleWithSAML] What must be configured in AWS IAM before AssumeRoleWithSAML can work with an Okta SAML integration? - A) IAM user with Okta-specific policy - B) SAML 2.0 Identity Provider registered in IAM with Okta's metadata XML, and IAM role with trust policy allowing that SAML provider as federated principal - C) Cognito User Pool with Okta as external IdP - D) STS endpoint URL registered in Okta's application settings > **Answer: B**. Two prerequisites: (1) Register Okta as SAML IdP in IAM (upload metadata.xml → creates IAM Identity Provider resource). (2)

Create IAM role with trust policy: `"Principal": {"Federated": "arn:aws:iam::...:saml-provider/Okta"}` and Action `sts:AssumeRoleWithSAML`.

1700. [Multi-topic] A company wants to grant a third-party auditor read-only access to their AWS account without creating IAM users. The auditor uses their own AWS account. The auditor needs to access CloudTrail, Config, and S3 logs. What is the correct architecture? - A) Create IAM user in your account, send credentials to auditor - B) Create IAM role in your account with trust policy allowing auditor's account ARN; auditor assumes role via AssumeRole (cross-account) - C) Share your AWS root credentials with the auditor for read-only access - D) Create SAML federation with auditor's AWS account > **Answer: B**. Cross-account AssumeRole: create role in your account with (1) identity policy: ReadOnlyAccess/AWSSecurityAuditPolicy, (2) trust policy: allow auditor's IAM role ARN from their account to `sts:AssumeRole`. Auditor calls AssumeRole from their account → gets temporary read-only credentials in your account. No IAM user, no long-lived credentials.

Chapter 28 Extended — AWS Outposts, Local Zones, Wavelength

1701. [Outposts] A financial institution must process trading data on servers physically located in their Frankfurt data center to comply with German data residency laws. They want to use AWS EC2, RDS, and their existing VPC networking. What is the correct solution? - A) Deploy resources in AWS eu-central-1 region — Frankfurt is covered - B) Use AWS Outposts — AWS delivers racks to the customer's Frankfurt facility, running EC2/RDS locally within their VPC - C) Use AWS Local Zone Frankfurt — low-latency metro extension - D) Use AWS Snowball Edge for on-premises compute > **Answer: B**. Outposts = AWS-managed rack at customer facility. Data stays physically in the building. Same VPC, EC2, RDS APIs. Local Zone = AWS-managed (not at customer site). eu-central-1 = data in AWS DC, not in customer's building.

1702. [Outposts] An Outposts rack at a customer site loses its Service Link connection to the parent AWS Region. What happens to currently running EC2 instances? - A) All EC2 instances terminate immediately - B) Running EC2 instances continue operating; cannot launch new instances or use management plane - C) Instances migrate to the parent region automatically - D) Instances stop and resume when connectivity is restored > **Answer: B**. Existing running workloads continue on Outpost during connectivity loss (data plane is local). Management plane (launch, terminate, API calls) requires Service Link to region. This is a key Outposts resilience characteristic.

1703. [Outposts] A developer needs to access an on-premises Oracle database from an EC2 instance running on Outposts, without traffic going through the internet or AWS region. Which Outposts networking component enables this? - A) Service Link — routes on-prem traffic to region then back - B) Local Gateway (LGW) — routes traffic directly between Outpost instances and on-premises network - C) Transit Gateway attached to the Outpost - D) VPC Peering between Outpost subnet and on-premises VLAN > **Answer: B**. Local Gateway (LGW) routes traffic from Outpost subnets to the on-premises LAN directly, without traversing the Service Link to the region. Enables sub-millisecond latency to on-prem resources.

1704. [Outposts] Which service is NOT available on AWS Outposts? - A) Amazon RDS (MySQL, PostgreSQL) - B) Amazon EKS - C) AWS Lambda - D) Amazon ElastiCache for Redis > **Answer: C**. Lambda is not available on Outposts. EC2, EBS, ECS, EKS, RDS, ElastiCache (Redis), EMR, S3 on Outposts are supported.

1705. [Outposts] A company wants local object storage on their Outpost for video files that must never leave their facility. What is the correct service? - A) S3 bucket in eu-central-1 with data residency SCP - B) S3 on Outposts — object storage running locally on the Outpost rack - C) EBS volume shared across multiple EC2 instances via NFS - D) EFS mounted to Outpost EC2 instances from the parent region > **Answer: B**. S3 on Outposts = local S3-compatible object storage on the physical Outpost. Data stays on-premises. Standard S3 (A) = data in AWS region, not on-prem. EFS from parent region (D) = data flows over Service Link.

1706. [Local Zones] A video game company's servers are in us-east-1. Players in Los Angeles report 80ms latency. The team wants to reduce this to under 10ms for LA players. What is the most appropriate solution? - A) Deploy EC2 instances in a second AWS Region on the US West Coast (us-west-1) - B) Enable the Los Angeles AWS Local Zone, deploy game servers there within the same us-east-1 VPC - C) Use AWS Wavelength Zone in LA with carrier partnership - D) Deploy CloudFront distribution with Lambda@Edge for game server logic > **Answer: B**. Local Zone LA (us-east-1-lax-1a) = AWS-managed facility in LA, extension of us-east-1 VPC. Same VPC, same APIs, < 10ms to LA users. No second region needed, no carrier dependency.

1707. [Local Zones] A company enables an AWS Local Zone for the first time. A developer tries to launch an EC2 instance in the Local Zone subnet but receives an error. What is the likely cause? - A) Local Zones only support t2 instance types - B) The Local Zone must be explicitly opted-in at the account level before use - C) Local Zones require a Dedicated Host purchase first - D) EC2 in Local Zones requires Direct Connect > **Answer: B**. Local Zones are opt-in. Must go to EC2 → Settings → Zones → Enable the specific Local Zone for the account/region. Default: Local Zones are disabled.

1708. [Local Zones] Which statement correctly describes the relationship between a Local Zone and its parent AWS Region? - A) Local Zone is a separate AWS Region with its own VPC - B) Local Zone is an extension of the parent region — appears as an additional AZ within the same VPC - C) Local Zone has its own independent IAM and billing - D) Local Zone requires a separate AWS account > **Answer: B**. Local Zone subnet = VPC subnet with AvailabilityZone set to Local Zone ID. Same VPC CIDR, same SGs, same route tables. Appears as another "AZ" from the VPC perspective. Same account, same IAM, same billing.

1709. [Local Zones] A media production company needs GPU-accelerated rendering servers in Chicago for their on-site team. There is no AWS Region in Chicago. What AWS infrastructure solves this? - A) AWS Outposts shipped to their Chicago office - B) AWS Local Zone in Chicago — GPU EC2 instances (G4) deployed close to production team - C) AWS Wavelength Zone in Chicago via carrier 5G network - D) EC2 Spot instances in us-east-1 with DirectConnect to Chicago office > **Answer: B**. Local Zone Chicago (if available) = AWS-managed DC in Chicago, GPU EC2 available, < 10ms to local team. Outposts (A) ships hardware to the customer's Chicago office — heavier commitment. Check Local Zone availability per city.

1710. [Wavelength] A mobile gaming company wants to run game server logic with < 5ms latency to 5G smartphone users. Their app uses Verizon 5G network in the US. What AWS infrastructure is designed for this? - A) AWS Local Zones in major cities - B) AWS Wavelength Zones co-located in Verizon 5G infrastructure - C) CloudFront edge locations with Lambda@Edge - D) AWS Outposts at Verizon data centers > **Answer: B**. Wavelength Zones = AWS compute embedded in telco 5G network. Traffic from 5G device → carrier radio network → Wavelength Zone (no internet traverse). Single-digit ms latency to EC2 in Wavelength Zone.

1711. [Wavelength] An application deployed in a Wavelength Zone needs to access DynamoDB. DynamoDB is not available locally. How does it access DynamoDB? - A) DynamoDB is available in all Wavelength Zones - B) The application accesses DynamoDB in the parent region via the VPC backbone (not via carrier network) - C) Data must be cached in ElastiCache in the Wavelength Zone instead - D) The application must use DynamoDB DAX deployed in the Wavelength Zone > **Answer: B**. Wavelength Zone is a VPC subnet extension. Traffic to AWS services not in WZ (S3, DynamoDB, Lambda) routes via VPC backbone to parent region. Not via carrier network — only 5G device → WZ traffic uses the carrier path.

1712. [Wavelength] A developer tries to assign an Elastic IP address to an EC2 instance in a Wavelength Zone. This fails. Why? - A) EIPs are not supported in any VPC subnet extensions - B) Wavelength Zone EC2 instances use carrier IP addresses assigned by the telco, not standard AWS EIPs - C) EIPs require IGW, which is not supported in Wavelength Zones - D) Both B and C are correct > **Answer: D**. Wavelength Zones have no standard IGW (entry is via Carrier Gateway only). EC2 instances get carrier IP addresses from the telco provider, not standard AWS EIPs. EIP = IGW-dependent, not applicable in WZ.

1713. [Wavelength] A Solutions Architect is comparing Local Zones and Wavelength Zones for a new project. The users are enterprise employees working from offices (not on 5G mobile). Which infrastructure is more appropriate? - A) Wavelength Zones — lower latency - B) Local Zones — designed for fixed-network/metro users; Wavelength targets 5G mobile users specifically - C) Outposts — on-premises is always lower latency - D) Standard region — no difference in practice > **Answer: B**. Wavelength = 5G carrier network users. Local Zone = general metro-area low-latency (broadband, office users). For office workers connecting via fixed internet, Local Zone is the correct choice.

1714. [Multi-topic] A company needs to: (1) process factory floor sensor data at < 1ms latency, (2) store raw sensor data locally for GDPR compliance, (3) use standard AWS APIs and the same VPC as their cloud workloads, (4) have AWS manage the infrastructure hardware. Which solution meets ALL requirements? - A) AWS Local Zone in the nearest city - B) AWS Wavelength Zone via carrier network at the factory - C) AWS Outposts rack at the factory facility - D) AWS Region with Direct Connect for low latency > **Answer: C**. Outposts: (1) LAN latency < 1ms to PLCs. (2) S3 on Outposts / EBS — data stays on-prem (GDPR). (3) Same VPC, same AWS APIs. (4) AWS manages hardware. DX to region (D) = 10-50ms round trip, not < 1ms. Local Zone / Wavelength = not at customer's facility.

1715. [Multi-topic] A connected vehicle platform processes telemetry from cars over 5G. The backend needs single-digit millisecond processing and then sends aggregated results to S3 and DynamoDB in the region. What is the correct architecture? - A) EC2 in parent region + CloudFront for caching vehicle data - B) EC2 in Wavelength Zone (telco 5G) for real-time processing → VPC backbone to S3/DynamoDB in region - C) EC2 in Local Zone for metro low-latency processing - D) Lambda@Edge for vehicle telemetry processing > **Answer: B**. Vehicle → 5G network → Wavelength Zone EC2 (ms processing) → parent region S3/DynamoDB via backbone. Wavelength = purpose-built for 5G-connected device processing. Lambda@Edge has no persistent state and can't handle high-throughput telemetry.

1716. [Multi-topic] An architect compares costs for a new media rendering workstation deployment. The team is based in Toronto and the nearest AWS Region is ca-central-1. A Toronto Local Zone exists. Which option is cheapest for daily 8-hour rendering workloads? - A) GPU EC2 in ca-central-1 — cheapest region pricing - B) GPU EC2 in Toronto Local Zone

— slight premium but saves WAN data transfer back to studio - C) GPU EC2 on Outposts at the studio — no region pricing - D) Spot GPU EC2 in ca-central-1 — 90% discount eliminates cost concern > **Answer: D**. Spot in region = up to 90% cheaper than on-demand or Local Zone. For rendering (interruptible, checkpoint-able), Spot is most cost-efficient. Local Zone (B) has a premium + daily large file transfer to studio adds cost. Outposts (C) has hardware commitment cost.

1717. [Outposts] A healthcare company runs imaging analysis software that must process patient MRI scans within the hospital network, with scan data never leaving hospital premises. They want managed infrastructure without hiring hardware engineers. Which AWS service fits? - A) AWS HealthLake in the nearest region - B) AWS Outposts — AWS-managed rack at hospital, EC2 runs imaging software, EBS/S3 on Outposts stores scans locally - C) AWS Snowball Edge — offline compute device at hospital - D) AWS Local Zone in the hospital's city > **Answer: B**. Outposts = AWS-managed hardware at hospital (no hardware engineers needed). EC2 for imaging, S3 on Outposts for local scan storage (HIPAA data residency). Local Zone = AWS-managed DC in city (not hospital-specific). Snowball Edge = offline/migration device, not persistent managed compute.

1718. [Multi-topic] An architect must choose between Outposts, Local Zones, and Wavelength for three different projects: (A) 5G connected robots in a factory, (B) video editing studio in Berlin needing < 5ms to AWS GPU servers, (C) bank requiring EC2+RDS at its own data center in Zurich. Match correctly. - A) A=Wavelength, B=Local Zone, C=Outposts - B) A=Outposts, B=Local Zone, C=Wavelength - C) A=Local Zone, B=Wavelength, C=Outposts - D) A=Wavelength, B=Outposts, C=Local Zone > **Answer: A**. (A) 5G robots = Wavelength (carrier 5G network). (B) Berlin studio low-latency = Local Zone Berlin (AWS-managed metro DC). (C) Bank at own DC = Outposts (AWS hardware at bank's Zurich facility, data stays there).

1719. [Wavelength] Which of the following is TRUE about the Carrier Gateway in an AWS Wavelength Zone? - A) It is equivalent to an Internet Gateway and allows public internet access - B) It routes traffic between the Wavelength Zone subnet and the carrier's mobile network; no standard internet routing - C) It replaces the NAT Gateway for outbound internet from Wavelength EC2 - D) It connects the Wavelength Zone to the parent region's Transit Gateway > **Answer: B**. Carrier Gateway = entry/exit point for carrier mobile network traffic (5G devices). Not an IGW — no standard public internet. Traffic from WZ to region goes via VPC backbone, not Carrier Gateway. Carrier Gateway is only for 5G device ↔ Wavelength traffic.

1720. [Multi-topic] A company runs AWS Outposts in their Amsterdam facility. They want to route traffic from Outpost EC2 instances to their on-premises LDAP server (192.168.1.50) without that traffic leaving the building. What must be configured? - A) VPN from Outpost subnet to LDAP server - B) Local Gateway (LGW) with route for 192.168.0.0/16 pointing to on-premises network; no traffic leaves facility - C) Service Link tunnel from Outpost to region, then VPN from region to Amsterdam - D) VPC peering between Outpost subnet and on-premises VLAN > **Answer: B**. LGW routes Outpost subnet traffic to on-premises network locally. Route table: 192.168.0.0/16 → LGW. Traffic stays in facility LAN — never touches internet or Service Link. Pure local routing.

💰 Chapter 33: AWS Cost Management & Budgets

> **Covers:** AWS Budgets (types, alerts, actions) · Cost Explorer · Cost Allocation Tags · AWS Cost Anomaly Detection · Savings Plans & RI management · Budget enforcement architectures (SCP, Lambda, SNS, EventBridge, CloudWatch) · Billing alarms · Consolidated Billing · Cost-limiting patterns

---

📋 Table of Contents (Chapter 33)

- 33.1 AWS Billing & Cost Fundamentals
- 33.2 AWS Budgets — Deep Dive
- 33.3 AWS Cost Explorer
- 33.4 AWS Cost Anomaly Detection
- 33.5 CloudWatch Billing Alarms
- 33.6 Cost Allocation Tags
- 33.7 Consolidated Billing & AWS Organizations
- 33.8 Savings Plans & Reserved Instance Management
- 33.9 Budget Enforcement Architectures
- 33.10 Well-Architected Framework — Cost Management
- 33.11 SAP-C02 Domains — Cost Management

🧾 33.1 AWS Billing & Cost Fundamentals

📖 33.1.1 Overview

AWS billing is **pay-as-you-go** with no upfront commitments for most services. Understanding the billing model, how costs accumulate, and how to monitor and control them is a core SAP-C02 competency — "Cost Optimization" is one of the six WAF pillars and appears in all four exam domains.

Key billing concepts:

Concept	Definition
Billing cycle	Monthly. Bills generated on the first of the next month.
Billing granularity	Per-second (EC2, Fargate, Lambda duration), per-request (API GW, Lambda invocations), per-GB (storage, data transfer), per-hour (RDS, ElastiCache)
Free Tier	12-month free tier (new accounts) + always-free tier (Lambda 1M requests/month, DynamoDB 25 GB, etc.)
Data transfer costs	Inbound to AWS: free. Same-AZ: free. Cross-AZ: $0.01/GB. Cross-region: varies. Outbound to internet: tiered
Payer account	AWS Organizations management account receives consolidated bill. Member accounts' costs roll up.

🌐 33.1.2 Data Transfer Cost Rules

Transfer Type	Cost
Inbound (internet → AWS)	Free
Same AZ (private IP)	Free
Same AZ (public/EIP)	$0.01/GB each direction
Cross-AZ (same region)	$0.01/GB each direction
Cross-region	$0.02/GB (varies by region pair)
To CloudFront from S3 (origin)	Free
CloudFront → Internet	Tiered, starts $0.0085/GB
PrivateLink / VPC endpoint	Per-hour + per-GB processed

> 🔑 **Exam key:** Multi-AZ architecture saves on data transfer if using private IPs within same AZ. Cross-AZ replication (RDS Multi-AZ, ELB cross-zone) costs data transfer.

📊 33.2 AWS Budgets — Deep Dive

📖 33.2.1 Overview

AWS Budgets lets you **set custom cost, usage, Savings Plans, and RI coverage thresholds** and receive alerts when those thresholds are breached — or forecasted to be breached.

Scope: Account-level or org-level (via AWS Organizations delegated admin). Budgets are visible in the Billing & Cost Management console.

Free tier: First 2 budgets per account are free. $0.02/day per additional budget.

📊 33.2.2 Budget Types

Type	What It Tracks	Use Case
Cost Budget	Actual or forecasted USD spend	"Alert me if I spend > $500 this month"
Usage Budget	Service usage units (EC2 hours, S3 GB, Lambda requests)	"Alert me if I use > 1,000 EC2 vCPU-hours"
Savings Plans Budget	Savings Plans utilization or coverage %	"Alert me if SP utilization < 80%"
RI Budget	Reserved Instance utilization or coverage %	"Alert me if RI coverage drops below 70%"

🔔 33.2.3 Budget Alerts

Each budget can have **up to 10 alert thresholds**. Each alert can notify up to 10 email addresses + 1 SNS topic.

Alert Type	When It Fires
Actual cost	When real spend crosses threshold (e.g., 80% of budget)
Forecasted cost	When projected month-end spend will cross threshold
Actual usage	When actual usage units exceed threshold
Forecasted usage	When projected usage will exceed threshold

```
Budget: $1,000/month
Alert 1: Actual ≥ 50% ($500)  → email finance team
Alert 2: Actual ≥ 80% ($800)  → email + SNS → Lambda → Slack
Alert 3: Forecasted ≥ 100% ($1,000) → email CTO + SNS → Lambda → SCP enforcement
```

⚡ 33.2.4 AWS Budget Actions

Budget Actions automate responses when a budget threshold is breached — no manual intervention required.

Action Type	What It Does	Use Case
Apply IAM Policy	Attaches a deny-all or restrictive IAM policy to users/roles/groups	Prevent further spend by blocking resource creation
Apply SCP	Attaches an SCP to an AWS Organizations OU or account	Org-wide spend lockdown on breach
Target EC2/RDS instances	Stops EC2 instances or RDS instances	Immediately stop running resources consuming budget

Action workflow:

```
flowchart TD
    BUDGET[AWS Budget
Threshold breached] --> ACTION{Budget Action
configured?}
    ACTION -->|Yes — auto execute| AUTO[Execute Action
Automatically]
    ACTION -->|Yes — requires approval| APPROVAL[Send approval request
via SNS]
    APPROVAL --> APPROVE[Approver confirms] --> AUTO
    AUTO -->|Action Type| IAM[Attach IAM Policy
to role/user/group]
    AUTO -->|Action Type| SCP[Attach SCP
to OU/account]
    AUTO -->|Action Type| EC2RDS[Stop EC2/RDS
instances]
```

> 🔑 **Exam key:** Budget Actions = automated cost enforcement. Two execution modes: **automatic** (fires immediately on threshold breach) or **manual** (sends approval notification first). This is the native AWS way to auto-enforce budgets without Lambda.

📋 33.2.5 Budget Scope and Filters

Budgets can be filtered by:

- **Service** (EC2, S3, RDS, Lambda...)
- **Linked Account** (in Organizations — per-account budget)
- **Tag** (cost allocation tags — per-team or per-project budget)
- **Region** (per-region spend)
- **Instance Type** (e.g., budget only for m5 family)
- **Purchase Option** (On-Demand, Spot, Reserved)
- **Usage Type** (e.g., DataTransfer-Out-Bytes)

🔍 33.3 AWS Cost Explorer

📖 33.3.1 Overview

AWS Cost Explorer provides an **interactive visualization and analysis interface** for your AWS costs and usage. It allows drilling down into cost drivers, generating forecasts, and identifying optimization opportunities.

Access: Billing & Cost Management → Cost Explorer. Must be enabled (takes up to 24h for initial data).

Data retention: 13 months of historical data.

📊 33.3.2 Key Features

Feature	Description
Cost & Usage Reports	Granular hourly/daily/monthly costs by service, account, tag, region
Filtering & Grouping	Group by service, account, tag, region, AZ, instance type, purchase type
Forecast	12-month cost forecast using ML model based on historical trends
RI/Savings Plans Recommendations	Recommends optimal RI/SP purchases based on usage patterns
Right-sizing recommendations	Identifies over-provisioned EC2 instances
Cost Explorer API	Programmatic access for custom reporting / FinOps tooling

---

📊 33.3.3 Cost Explorer vs AWS Budgets vs Cost Anomaly Detection

Tool	Purpose	Alerting	Automation
Cost Explorer	Analyze and visualize historical + forecast spend	No (analysis only)	No
AWS Budgets	Set thresholds, alert, and auto-enforce	Yes (email + SNS)	Yes (Budget Actions)
Cost Anomaly Detection	Detect unexpected spend spikes using ML	Yes (SNS)	No
CloudWatch Billing Alarm	Simple threshold alert on estimated charges	Yes (SNS)	Via Lambda

---

🤖 33.4 AWS Cost Anomaly Detection

📖 33.4.1 Overview

Cost Anomaly Detection uses **ML to identify unusual spending patterns** — it learns your normal spend baseline and alerts when actual spend deviates significantly, catching unexpected costs before they appear on the monthly bill.

Key advantage over Budgets: Detects relative anomalies (sudden spike vs. normal baseline) even if absolute spend is below budget. A $50 EC2 spike when you normally spend $5 = anomaly, even if budget is $1,000.

📊 33.4.2 Monitor Types

Monitor Type	Scope
AWS Services	Monitors all AWS service spend, anomalies detected per service
Linked Account	One monitor per linked account (in Organizations)
Cost Category	Custom cost groupings
Cost Allocation Tag	Per tag value (e.g., per team, per project)

Alert thresholds:

- **Absolute dollar amount:** Alert when anomaly causes > $X additional spend
- **Percentage:** Alert when anomaly > X% above expected baseline

Notification: SNS topic → email, Lambda, Slack integration.

🔔 33.5 CloudWatch Billing Alarms

📖 33.5.1 Overview

CloudWatch Billing Alarms monitor the **EstimatedCharges** metric in us-east-1 (billing metrics are global but published only to us-east-1). They trigger SNS notifications when estimated charges exceed a threshold.

Key limitation: Only a single cumulative billing metric — less granular than AWS Budgets. Budgets preferred for most use cases.

Enable: Billing → Preferences → Receive Billing Alerts (must be enabled before metrics appear).

```
CloudWatch Alarm:
  Metric: AWS/Billing → EstimatedCharges
  Dimension: Currency=USD (optionally: ServiceName=AmazonEC2)
  Threshold: > $500
  Action: SNS topic → email/Lambda
```

📊 33.5.2 CloudWatch Billing Alarm vs AWS Budget Alert

Feature	CloudWatch Billing Alarm	AWS Budget Alert
Granularity	Per account total, per service	Per service, account, tag, region, purchase type
Forecast alerts	No	Yes
RI/SP monitoring	No	Yes
Automated actions	Via Lambda subscribed to SNS	Native Budget Actions (SCP, IAM, stop EC2/RDS)
Free	Yes (standard CW pricing)	First 2 budgets free
Region requirement	us-east-1 (billing metrics)	Any region
Recommendation	Simple single-account threshold	Preferred for most cost alerting scenarios

🏷️ 33.6 Cost Allocation Tags

📖 33.6.1 Overview

Cost Allocation Tags allow you to **categorize AWS costs by business dimensions** — team, project, environment, cost center — by tagging resources and then activating those tag keys in the Billing console.

Two types:

Type	Who Creates	Examples
AWS-generated tags	AWS automatically	`aws:createdBy`, `aws:cloudformation:stack-name`
User-defined tags	You	`Team=DataEngineering`, `Project=Phoenix`, `Env=Prod`

Activation: Tags must be activated in Billing → Cost Allocation Tags before they appear in Cost Explorer and CUR (Cost and Usage Reports).

Propagation delay: Up to 24 hours for new tags to appear in billing data.

📐 33.6.2 Tagging Strategy Best Practices

```
Recommended mandatory tags:
  Environment: prod | staging | dev | test
  Team: data-eng | backend | frontend | infra | security
  Project: phoenix | apollo | hermes
  CostCenter: 1234 | 5678
  Owner: alice@company.com

Enforce with:
  - AWS Config rule: required-tags → non-compliant if missing
  - SCP: Deny resource creation if required tags not present
  - Tag Policy (Organizations): standardize tag key casing and values
```

Tag Policy (AWS Organizations):

- Enforces consistent tag key names (e.g., `Team` not `team` or `TEAM`)
- Defines allowed values per key
- Reports non-compliant resources — does NOT block by default (use with SCP for enforcement)

🏢 33.7 Consolidated Billing & AWS Organizations

📖 33.7.1 Overview

Consolidated Billing aggregates all member account usage into the payer (management) account's bill. This enables **volume discounts** because AWS applies tiered pricing based on combined usage across all accounts.

Key benefits:

Benefit	Detail
Volume discounts	S3, EC2 data transfer, CloudFront — combined usage reaches higher discount tiers faster
RI/Savings Plans sharing	RI and SP discounts automatically apply to matching usage in any linked account
Single bill	One invoice for all accounts
Free	No charge for Consolidated Billing itself

📊 33.7.2 RI and Savings Plans Sharing

```
Management Account owns: m5.xlarge Standard RI (us-east-1)

Member Account A: running m5.xlarge On-Demand → RI discount applied automatically
Member Account B: running m5.xlarge On-Demand → RI discount applied if Account A doesn't use it

Result: RI benefit shared across org without manual intervention
```

Disable RI sharing: Account → Settings → Disable RI discount sharing (per-account opt-out). Some teams prefer full isolation for showback/chargeback.

📊 33.7.3 Per-Account Budgets in Organizations

```
Org structure:
  Root
  ├── Production OU
  │     ├── Account A (budget: $10,000/month)
  │     └── Account B (budget: $5,000/month)
  └── Dev OU
        ├── Account C (budget: $500/month → Budget Action: apply SCP)
        └── Account D (budget: $500/month → Budget Action: stop EC2)
```

Each linked account can have its own budget. Payer account can create budgets for linked accounts (with billing access enabled).

💎 33.8 Savings Plans & Reserved Instance Management

📖 33.8.1 Savings Plans Overview

Savings Plans provide **flexible discounts in exchange for a committed spend rate ($/hour) over 1 or 3 years**. Unlike RIs, no specific instance type commitment required (for Compute SP).

Plan Type	Covers	Flexibility	Discount
Compute Savings Plan	EC2 (any family/region/OS/tenancy) + Lambda + Fargate	Maximum — any compute	Up to 66%
EC2 Instance Savings Plan	EC2 — specific family + region, any size/OS/tenancy	Flexible within family	Up to 72%
SageMaker Savings Plan	SageMaker instances	SageMaker only	Up to 64%

Key mechanics:

- Commitment = minimum $/hour spend for 1 or 3 years
- Auto-applied to matching usage in all accounts (Consolidated Billing)
- Unused commitment = still charged (you pay the committed rate regardless)
- Savings Plans queue: most restrictive plan (EC2 Instance SP) applies first, then Compute SP

📊 33.8.2 RI vs Savings Plans Decision

Scenario	Recommendation
Known instance type, region, 3-year stable workload	Standard RI (72% max)
Need to switch instance types in future	Convertible RI or EC2 Instance SP
Want discount across EC2 + Lambda + Fargate	Compute Savings Plan
Unsure of future compute needs	Compute Savings Plan (most flexible)
Oracle/SQL Server BYOL compliance	Dedicated Host RI

🔍 33.8.3 Monitoring Utilization

AWS Budgets RI/SP utilization alerts:

- **Utilization alert:** "Alert me if my SP/RI is < 80% utilized" → wasted committed spend
- **Coverage alert:** "Alert me if < 70% of my EC2 hours are covered by SP/RI" → On-Demand overspend

Cost Explorer → Savings Plans / RI Utilization & Coverage reports show daily/monthly utilization graphs.

🚨 33.9 Budget Enforcement Architectures

This section covers patterns for **automatically limiting or responding to cost overruns** — the most SAP-C02-tested area of cost management.

🔴 33.9.1 Pattern 1 — Budget Action: Attach SCP (Hardest Stop)

```
flowchart TD
    BUDGET[AWS Budget
80% threshold breached] --> ACTION[Budget Action
Execute automatically]
    ACTION --> SCP[Attach SCP to OU/Account
Deny: ec2:RunInstances
Deny: rds:CreateDBInstance
Deny: lambda:CreateFunction]
    SCP --> BLOCK[No new resources
can be created in account]
    SCP --> SNS[SNS Notification
→ email engineering lead]
```

Use case: Dev/test accounts with strict monthly budget. When budget hit → SCP blocks new resource creation for the rest of the month.

SCP example attached by Budget Action:

```
{
  "Effect": "Deny",
  "Action": [
    "ec2:RunInstances",
    "rds:CreateDBInstance",
    "ecs:CreateService",
    "lambda:CreateFunction"
  ],
  "Resource": "*"
}
```

Reverting: Remove SCP at month end (manual or automated via Lambda + EventBridge scheduler).

🟡 33.9.2 Pattern 2 — Budget Action: Stop EC2/RDS Instances

```
flowchart TD
    BUDGET[AWS Budget
Actual spend ≥ 90%] --> ACTION[Budget Action
Stop tagged EC2 instances]
    ACTION --> STOP[Stop: Tag=Env:dev EC2 instances
Stop: RDS dev databases]
    STOP --> NOTIFY[SNS → email team
"Dev instances stopped — budget reached"]
```

Use case: Stop non-critical dev/test EC2 and RDS instances to prevent further spend, while keeping production running.

Tag filter: Budget Actions can target instances by tag (e.g., `Env=dev`) — only stops dev, leaves prod untouched.

🟢 33.9.3 Pattern 3 — Budget SNS → Lambda → Custom Enforcement

```
flowchart TD
    BUDGET[AWS Budget
Forecast ≥ 100%] --> SNS[SNS Topic
Budget alert published]
    SNS --> LAMBDA[Lambda Function
Custom enforcement logic]
    LAMBDA -->|Option A| TAG[Tag all new resources
with shutdown-on-weekend]
    LAMBDA -->|Option B| ASG[Set ASG desired=0
on dev Auto Scaling Groups]
    LAMBDA -->|Option C| NOTIFY[Post to Slack
#finops-alerts channel]
    LAMBDA -->|Option D| JIRA[Create Jira ticket
for budget review]
```

Use case: Custom logic beyond Budget Actions — Slack notifications, ticket creation, partial scale-down, per-service enforcement.

🔵 33.9.4 Pattern 4 — EventBridge + CloudWatch + Lambda Pipeline

```
flowchart TD
    CW[CloudWatch
Custom Metric:
cost per service tag] --> ALARM[CloudWatch Alarm
per-team cost > threshold]
    ALARM --> EB[EventBridge Rule
on alarm state change]
    EB --> LAMBDA[Lambda
enforcement function]
    LAMBDA --> SCPapi[Organizations API
attach SCP to account]
    LAMBDA --> SNS2[SNS
notify team lead]
    LAMBDA --> SLACK[Slack webhook
#team-budget channel]
```

Use case: Fine-grained per-team cost alerting using cost allocation tags → CloudWatch custom metrics → EventBridge → Lambda. More complex than native Budgets but supports arbitrary logic.

🟣 33.9.5 Pattern 5 — Preventive SCP (No Spend Without Tags)

```
// Deny resource creation if required cost tags not present
{
  "Effect": "Deny",
  "Action": [
    "ec2:RunInstances",
    "rds:CreateDBInstance",
    "s3:CreateBucket"
  ],
  "Resource": "*",
  "Condition": {
    "Null": {
      "aws:RequestTag/CostCenter": "true"
    }
  }
}
```

Use case: Preventive cost governance — no resource can be created without a `CostCenter` tag. Enforces attribution before the resource exists (not reactive). Applied via SCP to all accounts.

🟠 33.9.6 Pattern 6 — AWS Config + Auto-Remediation (Reactive Tag Enforcement)

```
Config Rule: required-tags
  → evaluates all EC2, RDS, S3 resources
  → non-compliant if missing CostCenter/Team tags
  → Auto-remediation: invoke SSM Automation (adds default tag + notifies owner)
  OR
  → Auto-remediation: invoke Lambda (terminates untagged dev resources after 24h grace)
```

Use case: Catch resources created without tags — retroactively enforce tagging policy.

🔶 33.9.7 Pattern 7 — Multi-Layer Cost Control Architecture

```
Layer 1 (Preventive):
  SCP → deny resource creation without required tags
  SCP → deny expensive instance types in dev accounts (no p4d, no x2iedn)

Layer 2 (Detective):
  AWS Budgets → per-account monthly cost budget with SNS alerts
  Cost Anomaly Detection → ML anomaly alerts per service/tag
  CloudWatch Billing Alarm → simple total spend alarm

Layer 3 (Responsive):
  Budget Actions → auto-stop dev EC2/RDS at 90% actual
  Budget Actions → attach deny-create SCP at 100% actual
  Lambda via SNS → Slack notification + Jira ticket at 80% forecast

Layer 4 (Corrective):
  Cost Explorer → weekly FinOps review of top cost drivers
  Trusted Advisor → RI/SP utilization, idle resources
  Compute Optimizer → right-sizing recommendations
```

📊 33.9.8 Cost Enforcement Tools Comparison

Tool	Type	Granularity	Automation	Best For
AWS Budgets	Reactive + forecasted	Service/account/tag/region	Budget Actions (native)	Primary cost alerting + enforcement
SCP	Preventive	Account/OU	N/A (always-on policy)	Block expensive services in dev accounts
CloudWatch Alarm	Reactive	Total or per-service	Lambda via SNS	Simple threshold, custom metrics
Cost Anomaly Detection	Reactive (ML)	Service/account/tag	SNS → Lambda	Spike detection
Config Rules	Detective + corrective	Per resource	Auto-remediation	Tag compliance, resource config
Trusted Advisor	Advisory	Per service	None (manual)	RI/SP utilization, idle resources
Compute Optimizer	Advisory	Per resource	None (manual)	Right-sizing EC2/Lambda

---

🏛️ 33.10 Well-Architected Framework — Cost Management

Pillar	Best Practices
Cost Optimization	Use Savings Plans + RI for predictable workloads (up to 72% discount). Spot for fault-tolerant batch. Auto Scaling to match demand. S3 Intelligent-Tiering for unknown access patterns. Budget Actions for automated enforcement. Cost Explorer recommendations for right-sizing. Delete idle resources (Trusted Advisor).
Operational Excellence	Cost Allocation Tags for attribution. Per-team/project budgets with SNS alerts. FinOps review cadence using Cost Explorer. IaC includes required tags (enforce via Config).
Security	SCP to prevent accidental provisioning of expensive services in wrong accounts. Budget Action SCPs as automated cost guardrails.
Reliability	Budgets + Cost Anomaly Detection catch unexpected costs from runaway processes before they become large outages/bills.
Performance	Right-sizing via Compute Optimizer. Graviton2/3 instances: 20% cheaper + 40% better performance/watt.
Sustainability	Graviton = better performance/watt. Spot = uses otherwise-idle AWS capacity. Auto Scaling = zero idle resources. S3 lifecycle policies = move cold data to cheaper tiers.

🎯 33.11 SAP-C02 Domains — Cost Management

Domain 1 — Organizational Complexity: Multi-account FinOps: Consolidated Billing for volume discounts. Per-OU and per-account budgets via Organizations. Shared RI/SP pool at payer account level (auto-applied cross-account). Delegated budget admin to FinOps account. Cost allocation tags enforced via Tag Policy + SCP across all accounts. Chargeback reports per business unit using CUR + Athena + QuickSight.

Domain 2 — New Solutions: Design cost-governance from day one: required tagging SCP, per-account budget with Budget Actions, Cost Anomaly Detection per service, Compute Optimizer integrated in deployment pipeline. New dev environment policy: t3/t4g instance types only (SCP deny expensive types), $500/month budget with auto-stop at 90%.

Domain 3 — Continuous Improvement: Identify cost reduction opportunities: Cost Explorer RI recommendations → purchase Standard RIs for 24/7 workloads. Compute Optimizer right-sizing → reduce m5.2xlarge → m5.xlarge for 20 instances. Migrate On-Demand to Compute Savings Plans after 3-month baseline. Enable S3 Intelligent-Tiering on large buckets with unknown access patterns. Review Cost Anomaly alerts to find runaway Lambda functions.

Domain 4 — Migration: During migration: use Cost Explorer to forecast target state costs. Tag all migrated resources with migration wave and project. Set per-wave budget to compare actual vs estimate. After migration cutover: delete source resources (Config rule + Lambda cleanup). Use Trusted Advisor to find lingering idle resources post-migration.

End of Chapter 33 — AWS Cost Management & Budgets

Chapter 33 — AWS Cost Management & Budgets

1721. [AWS Budgets] A company sets an AWS Budget of $1,000/month for a dev account. At $800 (80%), the team wants an email alert. At $1,000 (100%) the budget is exceeded. What is the MOST cost-effective way to receive both alerts? - A) One budget with one alert at $800 (80%) - B) One budget: alert 1 at $800 (80%), alert 2 at $1,000 (100%) - C) Two separate budgets: one for EC2, one for RDS, each with $1,000 (100%) - D) CloudWatch Billing Alarm at $800 for email; CloudWatch Alarm at $1,000 triggering Lambda to stop instances > **Answer: B**. AWS Budgets supports multiple alerts per budget plus Budget Actions. Budget Action "Stop EC2/RDS instances" can filter by tags (Env=dev) — no Lambda needed.

1722. [AWS Budgets] A Budget Action is configured to automatically attach an SCP when a budget threshold is breached. An engineer sets execution mode to "requires approval." What happens when the threshold is crossed? - A) SCP is immediately attached without any notification - B) Budget Action sends an approval request via SNS; SCP is only attached after approver confirms - C) Budget Action is skipped — approval mode disables automation - D) SCP is attached immediately; approval is needed to remove it > **Answer: B**. "Requires approval" = Budget Action sends SNS

notification with approval request. Approver clicks approve → SCP attached. Prevents accidental lockdown without human review.

1723. [AWS Budgets] A FinOps team wants an alert when their Reserved Instance utilization drops below 70% — meaning they're paying for RIs that aren't being used. Which budget type covers this? - A) Cost Budget with a filter for Reserved Instance purchase type - B) RI Budget with utilization threshold at 70% - C) Usage Budget tracking EC2 instance-hours - D) Savings Plans Budget with coverage alert at 70% > **Answer: B**. RI Budget → Utilization alert fires when RI utilization % falls below configured threshold. Separate from Coverage (what % of usage is RI-covered). Utilization = are you using the RIs you bought.

1724. [Cost Anomaly Detection] A company normally spends $200/day on EC2. On Tuesday, they spend $1,800. Their monthly AWS Budget is $10,000 — the $1,800 spike doesn't breach that budget. How can they detect this anomaly? - A) CloudWatch Billing Alarm with $1,500 daily threshold - B) AWS Cost Anomaly Detection — ML detects spikes vs baseline (200/day), alerts regardless of budget - C) AWS Budgets daily cost budget at $300 - D) Cost Explorer daily view reviewed manually each morning > ** Answer: B **. Cost Anomaly Detection = ML baseline − relative anomaly detection.$1,800 vs $200 baseline = 9× spike = anomaly alert fires even if below monthly budget. This is exactly its design purpose.

1725. [CloudWatch Billing] A developer sets up a CloudWatch Billing Alarm for $500 in us-west-2. The alarm never triggers despite charges exceeding $500. What is wrong? - A) CloudWatch Billing Alarms require an SNS topic to activate - B) Billing metrics are only available in us-east-1 — the alarm must be created there - C) Billing alarms require enabling Cost Explorer first - D) CloudWatch Billing Alarms only work for EC2 charges, not total charges > **Answer: B**. AWS Billing metrics (EstimatedCharges) are published to CloudWatch in us-east-1 only. Billing alarm must be created in us-east-1 regardless of where resources are deployed.

1726. [Cost Allocation Tags] An engineering manager asks why their team's costs are not visible separately in Cost Explorer even though all their EC2 instances have a `Team=Backend` tag. What is the likely cause? - A) Cost Explorer does not support filtering by tags - B) The `Team` tag key has not been activated in the Billing Console → Cost Allocation Tags - C) Tags only appear in Cost Explorer after 90 days of data collection - D) Tag-based cost allocation requires AWS Organizations > **Answer: B**. Tags must be activated as Cost Allocation Tags (Billing → Cost Allocation Tags → Activate) before they appear in Cost Explorer and CUR. Activation can take up to 24 hours.

1727. [Consolidated Billing] A company has 10 AWS accounts in an Organization. Account A purchases a Compute Savings Plan with a $10/hour commitment. Accounts B–J all run EC2 on-demand. What happens to the Savings Plan discount? - A) Only Account A receives the Savings Plan discount - B) The Savings Plan discount automatically applies to matching EC2 usage across all accounts in the Organization - C) Savings Plans can only be shared if explicitly shared via AWS RAM - D) Accounts B–J must individually purchase Savings Plans > **Answer: B**. Consolidated Billing: SP and RI discounts auto-apply to matching usage across all linked accounts under the payer. Account A's SP covers EC2 in any account with matching usage.

1728. [SCP Cost Control] A company wants to prevent any developer in the dev OU from launching EC2 instances larger than xlarge to control costs. Which control achieves this without Lambda or Budgets? - A) AWS Config rule evaluating instance types with auto-termination - B) SCP denying `ec2:RunInstances` when `ec2:InstanceType` matches 2xlarge and larger - C) IAM permission boundary on all developer roles listing only allowed instance types - D) CloudWatch Events rule triggering Lambda to terminate oversized instances > **Answer: B**. SCP with condition on `ec2:InstanceType` is the preventive control. Example: `"Condition": {"StringLike": {"ec2:InstanceType": ["*.2xlarge","*.4xlarge","*.8xlarge","*.12xlarge","*.16xlarge","*.24xlarge","*.metal"]}}` → deny. Applied to dev OU, all accounts, all principals.

1729. [Budget Architecture] A startup wants the simplest possible setup to: (1) receive an email when monthly spend exceeds $200, (2) automatically stop all EC2 instances when spend exceeds $300. What is the minimum AWS configuration? - A) Two CloudWatch Billing Alarms + two Lambda functions - B) One AWS Budget with two alerts: alert 1 (200, email) + Budget Action (300, stop EC2 instances) — no Lambda needed - C) AWS Cost Anomaly Detection monitor + SNS topic - D) EventBridge scheduled rule + Lambda + Budgets API polling > **Answer: B**. One Budget, two actions. Native Budget Actions stop EC2 without Lambda. Simplest possible cost control setup.

1730. [Budget Architecture] A company wants to prevent new resource creation in a dev account when monthly spend exceeds $500 — but allow existing resources to keep running. Which Budget Action achieves this without stopping anything? - A) Budget Action: Stop EC2 instances at 100% actual - B) Budget Action: Attach SCP to account denying `ec2:RunInstances`, `rds:CreateDBInstance`, `s3:CreateBucket` etc. - C) Budget Action: Attach IAM deny-all policy to all developer roles - D) Budget Action: Detach all IAM policies from developer group > **Answer: B**. Attach SCP = prevents NEW resource creation (RunInstances, CreateDBInstance) but does NOT stop running resources. Existing EC2/RDS continues. Perfect for "no new spend" enforcement.

1731. [Budget Architecture] A Budget alert fires and sends to an SNS topic. A Lambda function subscribes to SNS. The Lambda needs to post a message to a Slack webhook and create a Jira ticket. Which IAM permissions does the Lambda execution role need? - A) `sns:Publish` + `jira:CreateIssue` - B) `budgets:ViewBudget` + `sns:Subscribe` - C) Only outbound HTTPS (no AWS permissions needed — Slack and Jira are external APIs) - D) `lambda:InvokeFunction` + `cloudwatch:GetMetricData` > **Answer: C**. Slack webhook = HTTPS POST to external URL. Jira API = HTTPS POST. Both are external HTTP calls from Lambda — no AWS IAM permissions needed for those calls. Lambda needs VPC access (or public subnet) and basic CloudWatch Logs permissions only.

1732. [Cost Explorer] A FinOps analyst wants to see which EC2 instance types are driving the most cost over the last 3 months, broken down by team tag. Which Cost Explorer configuration achieves this? - A) Group by: Service; Filter by: EC2; date range: last 90 days - B) Group by: Instance Type; Filter by: Service=EC2, Tag=Team; date range: last 90 days - C) Group by: Tag (Team); Filter by: Service=EC2; date range: last 90 days — then drill into each team - D) Both B and C are valid approaches depending on whether team or instance type is the primary breakdown > **Answer: D**. Cost Explorer supports multiple grouping dimensions. B = instance type view per team. C = team view then drill to instance type. Both are valid navigation paths to the same insight.

1733. [Tagging] An SCP is configured to deny EC2 RunInstances if the `CostCenter` tag is not present in the request. A developer launches an EC2 instance via the console without specifying the tag. What happens? - A) EC2 launches; Config flags it as non-compliant - B) EC2 launch is denied — SCP condition `Null: aws:RequestTag/CostCenter: true` blocks the request - C) EC2 launches; Budget alert fires - D) EC2 launches with a default CostCenter tag value > **Answer: B**. `"Condition": {"Null": {"aws:RequestTag/CostCenter": "true"}}` = deny if CostCenter tag is missing from the RunInstances request. Preventive control — blocks before resource is created.

1734. [Multi-topic] A company has 50 AWS accounts. They want: (1) no account to spend more than $2,000/month, (2) dev accounts stop EC2 at 9 0500 dev budget, (3) instant Slack notification at 80% for all accounts, (4) all resources must have Project tag. What services/policies are needed? - A) AWS Budgets (per account, $2,000 budget + actions), Budget Actions (stop EC2 at 450), SNS→Lambda→Slack (at 80%), SCP (deny without Project tag) - B) CloudWatch Billing Alarms × 50 accounts, Lambda for EC2 stop, Slack webhooks, Config rules - C) Single Budget in payer account, Lambda for all enforcement, Config for tags - D) Cost Anomaly Detection for all accounts, Budget Actions for dev, SNS for Slack > **Answer: A**. Full multi-layer: per-account Budgets (1, 2), SNS→Lambda→Slack at 80% (3), preventive SCP requiring Project tag (4). CloudWatch approach (B) is more complex per-account. Single payer Budget (C) can't target per-account stop.

1735. [Savings Plans] A company's Compute Savings Plan is at 45% utilization — they committed $10/hour but only use 4.50/hour worth. What does this mean and what should they do? - A) They are saving 45% on their compute costs — this is good - B) They are paying for $5.50/hour of unused commitment — they should review actual usage and potentially reduce commitment at renewal - C) Their coverage is 45 - D) Savings Plans auto-adjust to actual usage — no action needed > **Answer: B**. 45% of 10/hr but only $4.50/hr of actual compute. 5.50/hour wasted commitment. At renewal, reduce commitment to match actual usage. Savings Plans do NOT auto-adjust — commitment is fixed for the term.

1736. [Multi-topic] A Lambda function is accidentally left running in a loop due to a bug, costing $3,000 in 2 hours (Lambda invocation costs). Which AWS service would detect this fastest? - A) AWS Budgets — monthly budget threshold not yet breached - B) AWS Cost Anomaly Detection — detects 3,000/2h vs normal $50/day baseline as anomaly → alerts immediately - C) CloudWatch Billing Alarm — daily metric updated catches it by next day - D) Cost Explorer — analyst notices it in weekly review > **Answer: B**. Cost Anomaly Detection = ML near-real-time anomaly detection. 3,000 in 2 hours vs $50/day baseline = extreme anomaly. Fires immediately. CloudWatch Billing Alarm updates every few hours. Budgets check daily. Cost Explorer = manual.

1737. [Architecture] A company wants to build a self-healing cost control system that: detects cost anomalies, posts to Slack, and if anomaly > $1,000, automatically quarantines the responsible team's account by attaching a deny-all SCP. Which integration achieves this? - A) Cost Anomaly Detection → SNS → Lambda (checks anomaly amount → if >1,000: calls Organizations API to attach SCP + posts to Slack) - B) AWS Budgets → Budget Action → SCP attach (no Lambda needed) - C) CloudWatch Alarm → EventBridge → Lambda → Organizations - D) Cost Explorer API polled by Lambda every hour → threshold check → SCP > **Answer: A**. Cost Anomaly Detection → SNS → Lambda is the correct pattern. Lambda receives anomaly details (amount, service, account), conditionally applies SCP via Organizations API if > $1,000, and posts to Slack. Budget Actions (B) don't have anomaly-based triggers.

1738. [Multi-topic] A Solutions Architect is reviewing a well-architected design for cost optimization. Which combination of controls represents defense-in-depth cost governance? - A) Preventive: SCP (block expensive instances + require tags); Detective: Budgets + Anomaly Detection; Responsive: Budget Actions + Lambda→Slack; Advisory: Cost Explorer +

Trusted Advisor - B) Reactive only: Monthly cost review in Cost Explorer + email alerts - C) Preventive only: SCPs blocking all expensive services - D) Advisory only: Trusted Advisor recommendations reviewed quarterly > **Answer: A**. Multi-layer cost governance mirrors security defense-in-depth. Preventive (SCP), Detective (Budgets, Anomaly), Responsive (Budget Actions, Lambda), Advisory (Cost Explorer, Trusted Advisor). Single-layer (B, C, D) is insufficient for SAP-C02 cost optimization.

1739. [Data Transfer] A company runs a 3-tier app: ALB → EC2 → RDS, all in us-east-1. ALB and EC2 are in different AZs. EC2 and RDS are in different AZs. What data transfer costs apply? - A) No data transfer costs — all within the same region - B) ALB → EC2 cross-AZ: $0.01/GB each direction; EC2 \rightarrow RDS cross-AZ$:0.01/GB each direction - C) Only outbound internet traffic is charged; internal VPC traffic is free - D) Only EC2 → RDS replication traffic is charged ($0.02/GB) > **Answer : B **. Cross-AZ data transfer within same region$ =0.01/GB each direction. ALB distributes to EC2 across AZs → cross-AZ. EC2 connects to RDS in different AZ → cross-AZ. Both directions charged. To reduce: place ALB and EC2 in same AZ (or use ALB cross-zone disabled + zonal architecture).

1740. [Multi-topic] A company uses AWS Organizations with 20 linked accounts. The management account purchases $50,000/year of Standard RIs for m5.xlarge in us-east-1. Seven linked accounts run m5.xlarge On-Demand. Which accounts receive RI discount pricing and in what order? - A) Only the management account - B) Management account first; remaining RI capacity applies to linked accounts in the order they were created - C) All accounts simultaneously — RI discount applies to all matching usage across the org, management account applies first if it has matching usage - D) Only accounts that have enabled RI sharing in their billing settings > **Answer: C**. Consolidated Billing RI sharing: RI applied to matching usage. Management account's own usage is covered first; remaining RI capacity covers linked accounts. All accounts with RI sharing enabled (default) receive benefit. RI sharing is on by default — accounts must explicitly opt out.